



cuSOLVER

Release 13.3

NVIDIA Corporation

Jun 25, 2026

Contents

1	cuSolverDN: Dense LAPACK	3
2	cuSolverSP: Sparse LAPACK	5
3	cuSolverRF: Refactorization	7
4	Naming Conventions	9
5	Asynchronous Execution	11
6	Library Property	13
7	High Precision Package	15
8	Using the CUSOLVER API	17
8.1	General Description	17
8.1.1	Thread Safety	17
8.1.2	Scalar Parameters	17
8.1.3	Parallelism with Streams	17
8.1.4	How to Link cusolver Library	18
8.1.5	Link Third-party LAPACK Library	18
8.1.6	Convention of info	18
8.1.7	Usage of _bufferSize	18
8.1.8	cuSOLVERDn Logging	19
8.1.9	Deterministic Results	19
8.1.10	Floating Point Emulation	20
8.2	cuSolver Types Reference	20
8.2.1	cuSolverDN Types	20
8.2.1.1	cusolverDnHandle_t	21
8.2.1.2	cublasFillMode_t	21
8.2.1.3	cublasOperation_t	21
8.2.1.4	cusolverEigType_t	21
8.2.1.5	cusolverEigMode_t	22
8.2.1.6	cusolverEigComp_t	22
8.2.1.7	cusolverIRSRefinement_t	22
8.2.1.8	cusolverDnIRSParams_t	23
8.2.1.9	cusolverDnIRSInfos_t	23
8.2.1.10	cusolverDnFunction_t	24
8.2.1.11	cusolverAlgMode_t	24
8.2.1.12	cusolverStatus_t	24
8.2.1.13	cusolverDnLoggerCallback_t	24
8.2.1.14	cusolverDeterministicMode_t	24
8.2.1.15	cusolverMathMode_t	25
8.2.1.16	cusolverStorevMode_t	25

8.2.1.17	cusolverDirectMode_t	25
8.2.2	cuSolverSP Types	26
8.2.2.1	cusolverSpHandle_t	26
8.2.2.2	cusparseMatDescr_t	26
8.2.2.3	cusolverStatus_t	26
8.2.3	cuSolverRF Types	27
8.2.3.1	cusolverRfHandle_t	27
8.2.3.2	cusolverRfMatrixFormat_t	27
8.2.3.3	cusolverRfNumericBoostReport_t	28
8.2.3.4	cusolverRfResetValuesFastMode_t	28
8.2.3.5	cusolverRfFactorization_t	28
8.2.3.6	cusolverRfTriangularSolve_t	28
8.2.3.7	cusolverRfUnitDiagonal_t	29
8.2.3.8	cusolverStatus_t	29
8.3	cuSolver Formats Reference	29
8.3.1	Index Base Format	29
8.3.2	Vector (Dense) Format	29
8.3.3	Matrix (Dense) Format	30
8.3.4	Matrix (CSR) Format	30
8.3.5	Matrix (CSC) Format	31
8.4	cuSolverDN: dense LAPACK Function Reference	32
8.4.1	cuSolverDN Helper Function Reference	32
8.4.1.1	cusolverDnCreate()	32
8.4.1.2	cusolverDnDestroy()	33
8.4.1.3	cusolverDnSetStream()	33
8.4.1.4	cusolverDnGetStream()	34
8.4.1.5	cusolverDnLoggerSetCallback()	34
8.4.1.6	cusolverDnLoggerSetFile()	34
8.4.1.7	cusolverDnLoggerOpenFile()	35
8.4.1.8	cusolverDnLoggerSetLevel()	35
8.4.1.9	cusolverDnLoggerSetMask()	36
8.4.1.10	cusolverDnLoggerForceDisable()	36
8.4.1.11	cusolverDnSetDeterministicMode()	36
8.4.1.12	cusolverDnGetDeterministicMode()	37
8.4.1.13	cusolverDnSetMathMode()	37
8.4.1.14	cusolverDnGetMathMode()	38
8.4.1.15	cusolverDnSetEmulationStrategy()	38
8.4.1.16	cusolverDnGetEmulationStrategy()	39
8.4.1.17	cusolverDnSetFixedPointEmulationMantissaControl()	40
8.4.1.18	cusolverDnGetFixedPointEmulationMantissaControl()	40
8.4.1.19	cusolverDnSetFixedPointEmulationMaxMantissaBitCount()	41
8.4.1.20	cusolverDnGetFixedPointEmulationMaxMantissaBitCount()	42
8.4.1.21	cusolverDnSetFixedPointEmulationMantissaBitOffset()	42
8.4.1.22	cusolverDnGetFixedPointEmulationMantissaBitOffset()	43
8.4.1.23	cusolverDnSetEmulationSpecialValuesSupport()	44
8.4.1.24	cusolverDnGetEmulationSpecialValuesSupport()	45
8.4.1.25	cusolverDnCreateSyevjInfo()	45
8.4.1.26	cusolverDnDestroySyevjInfo()	46
8.4.1.27	cusolverDnXsyevjSetTolerance()	46
8.4.1.28	cusolverDnXsyevjSetMaxSweeps()	46
8.4.1.29	cusolverDnXsyevjSetSortEig()	47
8.4.1.30	cusolverDnXsyevjGetResidual()	47
8.4.1.31	cusolverDnXsyevjGetSweeps()	48
8.4.1.32	cusolverDnCreateGesvdjInfo()	48

8.4.1.33	cusolverDnDestroyGesvdjInfo()	49
8.4.1.34	cusolverDnXgesvdjSetTolerance()	49
8.4.1.35	cusolverDnXgesvdjSetMaxSweeps()	49
8.4.1.36	cusolverDnXgesvdjSetSortEig()	50
8.4.1.37	cusolverDnXgesvdjGetResidual()	50
8.4.1.38	cusolverDnXgesvdjGetSweeps()	51
8.4.1.39	cusolverDnIRSPParamsCreate()	51
8.4.1.40	cusolverDnIRSPParamsDestroy()	52
8.4.1.41	cusolverDnIRSPParamsSetSolverPrecisions()	52
8.4.1.42	cusolverDnIRSPParamsSetSolverMainPrecision()	53
8.4.1.43	cusolverDnIRSPParamsSetSolverLowestPrecision()	54
8.4.1.44	cusolverDnIRSPParamsSetRefinementSolver()	55
8.4.1.45	cusolverDnIRSPParamsSetTol()	56
8.4.1.46	cusolverDnIRSPParamsSetTolInner()	57
8.4.1.47	cusolverDnIRSPParamsSetMaxIters()	57
8.4.1.48	cusolverDnIRSPParamsSetMaxItersInner()	58
8.4.1.49	cusolverDnIRSPParamsEnableFallback()	59
8.4.1.50	cusolverDnIRSPParamsDisableFallback()	59
8.4.1.51	cusolverDnIRSPParamsGetMaxIters()	60
8.4.1.52	cusolverDnIRSInfosCreate()	60
8.4.1.53	cusolverDnIRSInfosDestroy()	61
8.4.1.54	cusolverDnIRSInfosGetMaxIters()	61
8.4.1.55	cusolverDnIRSInfosGetNiters()	62
8.4.1.56	cusolverDnIRSInfosGetOuterNiters()	62
8.4.1.57	cusolverDnIRSInfosRequestResidual()	63
8.4.1.58	cusolverDnIRSInfosGetResidualHistory()	63
8.4.1.59	cusolverDnCreateParams()	64
8.4.1.60	cusolverDnDestroyParams()	65
8.4.1.61	cusolverDnSetAdvOptions()	65
8.4.2	Dense Linear Solver Reference (legacy)	66
8.4.2.1	cusolverDn<t>potrf()	66
8.4.2.2	cusolverDn<t>potrs()	68
8.4.2.3	cusolverDn<t>potri()	70
8.4.2.4	cusolverDn<t>getrf()	73
8.4.2.5	cusolverDn<t>getrs()	75
8.4.2.6	cusolverDn<t1><t2>gesv()	78
8.4.2.7	cusolverDnIRSXgesv()	92
8.4.2.8	cusolverDn<t>geqrf()	97
8.4.2.9	cusolverDn<t1><t2>gels()	100
8.4.2.10	cusolverDnIRSXgels()	114
8.4.2.11	cusolverDn<t>ormqr()	119
8.4.2.12	cusolverDn<t>orgqr()	123
8.4.2.13	cusolverDn<t>sytrf()	126
8.4.2.14	cusolverDn<t>potrfBatched()	129
8.4.2.15	cusolverDn<t>potrsBatched()	131
8.4.3	Dense Eigenvalue Solver Reference (legacy)	133
8.4.3.1	cusolverDn<t>gebrd()	134
8.4.3.2	cusolverDn<t>orgbr()	137
8.4.3.3	cusolverDn<t>sytrd()	140
8.4.3.4	cusolverDn<t>ormtr()	143
8.4.3.5	cusolverDn<t>orgtr()	147
8.4.3.6	cusolverDn<t>gesvd()	150
8.4.3.7	cusolverDn<t>gesvdj()	154
8.4.3.8	cusolverDn<t>gesvdjBatched()	159

8.4.3.9	cusolverDn<t>gesvdaStridedBatched()	164
8.4.3.10	cusolverDn<t>syevd()	171
8.4.3.11	cusolverDn<t>syevdx()	174
8.4.3.12	cusolverDn<t>sygvd()	180
8.4.3.13	cusolverDn<t>sygvdx()	185
8.4.3.14	cusolverDn<t>syevj()	191
8.4.3.15	cusolverDn<t>sygvj()	196
8.4.3.16	cusolverDn<t>syevjBatched()	201
8.4.4	Dense Linear Solver Reference (64-bit API)	205
8.4.4.1	cusolverDnXpotrf()	205
8.4.4.2	cusolverDnXpotrs()	208
8.4.4.3	cusolverDnXgetrf()	210
8.4.4.4	cusolverDnXgetrs()	212
8.4.4.5	cusolverDnXgeqrf()	214
8.4.4.6	cusolverDnXsytrs()	217
8.4.4.7	cusolverDnXtrtri()	219
8.4.4.8	cusolverDnXlarft()	221
8.4.5	Dense Eigenvalue Solver Reference (64-bit API)	224
8.4.5.1	cusolverDnXgesvd()	224
8.4.5.2	cusolverDnXgesvdp()	228
8.4.5.3	cusolverDnXgesvdr()	232
8.4.5.4	cusolverDnXsyevd()	236
8.4.5.5	cusolverDnXsyevdx()	239
8.4.5.6	cusolverDnXstedc()	243
8.4.5.7	cusolverDnXsygvd()	246
8.4.5.8	cusolverDnXsygvdx()	250
8.4.5.9	cusolverDnXsyevBatched()	255
8.4.5.10	cusolverDnXgeev()	259
8.4.5.11	cusolverDnXpolar()	264
8.5	cuSolverSP: sparse LAPACK Function Reference [[DEPRECATED]]	267
8.5.1	Helper Function Reference	267
8.5.1.1	cusolverSpCreate() [[DEPRECATED]]	267
8.5.1.2	cusolverSpDestroy() [[DEPRECATED]]	268
8.5.1.3	cusolverSpSetStream() [[DEPRECATED]]	268
8.5.1.4	cusolverSpXcsrissym() [[DEPRECATED]]	269
8.5.2	High Level Function Reference	271
8.5.2.1	cusolverSp<t>csrslsvlu() [[DEPRECATED]]	271
8.5.2.2	cusolverSp<t>csrslsvqr() [[DEPRECATED]]	274
8.5.2.3	cusolverSp<t>csrslsvchol() [[DEPRECATED]]	277
8.5.2.4	cusolverSp<t>csrslsvqr() [[DEPRECATED]]	280
8.5.2.5	cusolverSp<t>csreigvsi() [[DEPRECATED]]	283
8.5.2.6	cusolverSp<t>csreigs() [[DEPRECATED]]	287
8.5.3	Low Level Function Reference	289
8.5.3.1	cusolverSpXcsrsymrcm() [[DEPRECATED]]	289
8.5.3.2	cusolverSpXcsrsymmdq() [[DEPRECATED]]	291
8.5.3.3	cusolverSpXcsrsymamd() [[DEPRECATED]]	293
8.5.3.4	cusolverSpXcsrmetisnd() [[DEPRECATED]]	294
8.5.3.5	cusolverSpXcsrzfd() [[DEPRECATED]]	296
8.5.3.6	cusolverSpXcsrperm() [[DEPRECATED]]	298
8.5.3.7	cusolverSpXcsrqrBatched() [[DEPRECATED]]	300
8.6	cuSolverRF: Refactorization Reference [[DEPRECATED]]	306
8.6.1	cusolverRfAccessBundledFactorsDevice() [[DEPRECATED]]	307
8.6.2	cusolverRfAnalyze() [[DEPRECATED]]	308
8.6.3	cusolverRfSetupHost() [[DEPRECATED]]	311

8.6.4	cusolverRfCreate() [[DEPRECATED]]	313
8.6.5	cusolverRfExtractBundledFactorsHost() [[DEPRECATED]]	314
8.6.6	cusolverRfExtractSplitFactorsHost() [[DEPRECATED]]	315
8.6.7	cusolverRfDestroy() [[DEPRECATED]]	316
8.6.8	cusolverRfGetMatrixFormat() [[DEPRECATED]]	317
8.6.9	cusolverRfGetNumericProperties() [[DEPRECATED]]	317
8.6.10	cusolverRfGetNumericBoostReport() [[DEPRECATED]]	318
8.6.11	cusolverRfGetResetValuesFastMode() [[DEPRECATED]]	319
8.6.12	cusolverRfGetAlgs()	319
8.6.13	cusolverRfRefactor() [[DEPRECATED]]	320
8.6.14	cusolverRfResetValues() [[DEPRECATED]]	321
8.6.15	cusolverRfSetMatrixFormat() [[DEPRECATED]]	322
8.6.16	cusolverRfSetNumericProperties() [[DEPRECATED]]	323
8.6.17	cusolverRfSetResetValuesFastMode() [[DEPRECATED]]	323
8.6.18	cusolverRfSetAlgs() [[DEPRECATED]]	324
8.6.19	cusolverRfSolve() [[DEPRECATED]]	325
8.6.20	cusolverRfBatchSetupHost() [[DEPRECATED]]	326
8.6.21	cusolverRfBatchAnalyze() [[DEPRECATED]]	329
8.6.22	cusolverRfBatchResetValues() [[DEPRECATED]]	330
8.6.23	cusolverRfBatchRefactor() [[DEPRECATED]]	331
8.6.24	cusolverRfBatchSolve() [[DEPRECATED]]	332
8.6.25	cusolverRfBatchZeroPivot() [[DEPRECATED]]	333
9	Using the CUSOLVERMG API	335
9.1	General Description	335
9.1.1	Thread Safety	335
9.1.2	Determinism	335
9.1.3	Tile Strategy	336
9.1.4	Global Matrix Versus Local Matrix	337
9.1.5	Usage of _bufferSize	337
9.1.6	Synchronization	338
9.1.7	Context Switch	338
9.1.8	NVLINK	338
9.2	cuSolverMG Types Reference	338
9.2.1	cuSolverMG Types	338
9.2.2	cusolverMgHandle_t	338
9.2.3	cusolverMgGridMapping_t	339
9.2.4	cudaLibMgGrid_t	339
9.2.5	cudaLibMgMatrixDesc_t	339
9.3	Helper Function Reference	339
9.3.1	cusolverMgCreate() [DEPRECATED]	339
9.3.2	cusolverMgDestroy() [DEPRECATED]	340
9.3.3	cusolverMgDeviceSelect() [DEPRECATED]	340
9.3.4	cusolverMgCreateDeviceGrid() [DEPRECATED]	341
9.3.5	cusolverMgDestroyGrid() [DEPRECATED]	342
9.3.6	cusolverMgCreateMatrixDesc() [DEPRECATED]	342
9.3.7	cusolverMgDestroyMatrixDesc() [DEPRECATED]	343
9.4	Dense Linear Solver Reference	343
9.4.1	cusolverMgPotrf() [DEPRECATED]	343
9.4.2	cusolverMgPotrs() [DEPRECATED]	346
9.4.3	cusolverMgPotri() [DEPRECATED]	349
9.4.4	cusolverMgGetrf() [DEPRECATED]	352
9.4.5	cusolverMgGetrs() [DEPRECATED]	354
9.5	Dense Eigenvalue Solver Reference	358

9.5.1	cusolverMgSyevd() [DEPRECATED]	358
10	Acknowledgements	363
11	Bibliography	365
12	Notices	367
12.1	Notice	367
12.2	OpenCL	368
12.3	Trademarks	368

cuSOLVER API Reference

The API reference guide for cuSOLVER, a GPU accelerated library for decompositions and linear system solutions for both dense and sparse matrices.

The cuSolver library is a high-level package based on the cuBLAS and cuSPARSE libraries. It consists of two modules corresponding to two sets of API:

1. The cuSolver API on a single GPU
2. The cuSolverMG API on a single node multiGPU

Each of these can be used independently or in concert with other toolkit libraries. To simplify the notation, cuSolver denotes single GPU API and cuSolverMg denotes multiGPU API.

The intent of cuSolver is to provide useful LAPACK-like features, such as common matrix factorization and triangular solve routines for dense matrices, a sparse least-squares solver and an eigenvalue solver. In addition cuSolver provides a new refactorization library useful for solving sequences of matrices with a shared sparsity pattern.

cuSolver combines three separate components under a single umbrella. The first part of cuSolver is called cuSolverDN, and deals with dense matrix factorization and solve routines such as LU, QR, SVD and LDLT, as well as useful utilities such as matrix and vector permutations.

Next, cuSolverSP provides a new set of sparse routines based on a sparse QR factorization. Not all matrices have a good sparsity pattern for parallelism in factorization, so the cuSolverSP library also provides a CPU path to handle those sequential-like matrices. For those matrices with abundant parallelism, the GPU path will deliver higher performance. The library is designed to be called from C and C++.

The final part is cuSolverRF, a sparse re-factorization package that can provide very good performance when solving a sequence of matrices where only the coefficients are changed but the sparsity pattern remains the same.

The GPU path of the cuSolver library assumes data is already in the device memory. It is the responsibility of the developer to allocate memory and to copy data between GPU memory and CPU memory using standard CUDA runtime API routines, such as `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`, and `cudaMemcpyAsync()`.

cuSolverMg is GPU-accelerated ScaLAPACK. By now, cuSolverMg supports 1-D column block cyclic layout and provides symmetric eigenvalue solver.

Note: The cuSolver library requires hardware with a CUDA Compute Capability (CC) of 5.0 or higher. Please see the *CUDA C++ Programming Guide* for a list of the [Compute Capabilities](#) corresponding to all NVIDIA GPUs.

Note: cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

Chapter 1. cuSolverDN: Dense LAPACK

The cuSolverDN library was designed to solve dense linear systems of the form

$$Ax = b$$

where the coefficient matrix $A \in R^{n \times n}$, right-hand-side vector $b \in R^n$ and solution vector $x \in R^n$

The cuSolverDN library provides QR factorization and LU with partial pivoting to handle a general matrix A , which may be non-symmetric. Cholesky factorization is also provided for symmetric/Hermitian matrices. For symmetric indefinite matrices, we provide Bunch-Kaufman (LDL) factorization.

The cuSolverDN library also provides a helpful bidiagonalization routine and singular value decomposition (SVD).

The cuSolverDN library targets computationally-intensive and popular routines in LAPACK, and provides an API compatible with LAPACK. The user can accelerate these time-consuming routines with cuSolverDN and keep others in LAPACK without a major change to existing code.

Chapter 2. cuSolverSP: Sparse LAPACK

The cuSolverSP library was mainly designed to solve sparse linear system

$$Ax = b$$

and the least-squares problem

$$x = \operatorname{argmin} \|A * z - b\|$$

where sparse matrix $A \in R^{m \times n}$, right-hand-side vector $b \in R^m$ and solution vector $x \in R^n$. For a linear system, we require $m=n$.

The core algorithm is based on sparse QR factorization. The matrix A is accepted in CSR format. If matrix A is symmetric/Hermitian, the user has to provide a full matrix, ie fill missing lower or upper part.

If matrix A is symmetric positive definite and the user only needs to solve $Ax = b$, Cholesky factorization can work and the user only needs to provide the lower triangular part of A.

On top of the linear and least-squares solvers, the cuSolverSP library provides a simple eigenvalue solver based on shift-inverse power method, and a function to count the number of eigenvalues contained in a box in the complex plane.

Note: cuSolverSp is deprecated and will be removed in a future major release. It is recommended migrating to a new sparse direct solver package, [cuDSS](#), and you can find a transition example in [CUDALibrarySamples/cuSOLVERSp2cuDSS](#) for reference.

Chapter 3. cuSolverRF: Refactorization

The cuSolverRF library was designed to accelerate solution of sets of linear systems by fast re-factorization when given new coefficients in the same sparsity pattern

$$A_i x_i = f_i$$

where a sequence of coefficient matrices $A_i \in R^{n \times n}$, right-hand-sides $f_i \in R^n$ and solutions $x_i \in R^n$ are given for $i=1, \dots, k$.

The cuSolverRF library is applicable when the sparsity pattern of the coefficient matrices A_i as well as the reordering to minimize fill-in and the pivoting used during the LU factorization remain the same across these linear systems. In that case, the first linear system ($i=1$) requires a full LU factorization, while the subsequent linear systems ($i=2, \dots, k$) require only the LU re-factorization. The later can be performed using the cuSolverRF library.

Notice that because the sparsity pattern of the coefficient matrices, the reordering and pivoting remain the same, the sparsity pattern of the resulting triangular factors L_i and U_i also remains the same. Therefore, the real difference between the full LU factorization and LU re-factorization is that the required memory is known ahead of time.

Note: cuSolverRf is deprecated and will be removed in a future major release. It is recommended migrating to a new sparse direct solver package, [cuDSS](#), and you can find a transition example in [CUDALibrarySamples/cuSOLVERSp2cuDSS](#) for reference.

Chapter 4. Naming Conventions

The cuSolverDN library provides two different APIs; `legacy` and `generic`.

The functions in the legacy API are available for data types `float`, `double`, `cuComplex`, and `cuDoubleComplex`. The naming convention for the legacy API is as follows:

`cusolverDn<t><operation>`

where `<t>` can be `S`, `D`, `C`, `Z`, or `X`, corresponding to the data types `float`, `double`, `cuComplex`, `cuDoubleComplex`, and the generic type, respectively. `<operation>` can be Cholesky factorization (`potrf`), LU with partial pivoting (`getrf`), QR factorization (`geqrf`) and Bunch-Kaufman factorization (`sytrf`).

The functions in the generic API provide a single entry point for each routine and support for 64-bit integers to define matrix and vector dimensions. The naming convention for the generic API is data-agnostic and is as follows:

`cusolverDn<operation>`

where `<operation>` can be Cholesky factorization (`potrf`), LU with partial pivoting (`getrf`) and QR factorization (`geqrf`).

The cuSolverSP library functions are available for data types `float`, `double`, `cuComplex`, and `cuDoubleComplex`. The naming convention is as follows:

`cusolverSp[Host]<t>[<matrix data format>]<operation>[<output matrix data format>]<based on>`

where `cuSolverSp` is the GPU path and `cusolverSpHost` is the corresponding CPU path. `<t>` can be `S`, `D`, `C`, `Z`, or `X`, corresponding to the data types `float`, `double`, `cuComplex`, `cuDoubleComplex`, and the generic type, respectively.

The `<matrix data format>` is `csr`, compressed sparse row format.

The `<operation>` can be `ls`, `lsq`, `eig`, `eigs`, corresponding to linear solver, least-square solver, eigenvalue solver and number of eigenvalues in a box, respectively.

The `<output matrix data format>` can be `v` or `m`, corresponding to a vector or a matrix.

`<based on>` describes which algorithm is used. For example, `qr` (sparse QR factorization) is used in linear solver and least-square solver.

All of the functions have the return type `cusolverStatus_t` and are explained in more detail in the chapters that follow.

Table 1: cuSolverSP API

Routine	Data format	Operation	Output format	Based on
csrslsvlu	csr	linear solver (ls)	vector (v)	LU (lu) with partial pivoting
csrslsvqr	csr	linear solver (ls)	vector (v)	QR factorization (qr)
csrslsvchol	csr	linear solver (ls)	vector (v)	Cholesky factorization (chol)
csrslsvqr	csr	least-square solver (lsq)	vector (v)	QR factorization (qr)
csreigvsic	csr	eigenvalue solver (eig)	vector (v)	shift-inverse
csreigs	csr	number of eigenvalues in a box (eigs)		
csrsym-rcm	csr	Symmetric Reverse Cuthill-McKee (symrcm)		

The cuSolverRF library routines are available for data type double. Most of the routines follow the naming convention:

cusolverRf_<operation>_[[Host]](...)

where the trailing optional Host qualifier indicates the data is accessed on the host versus on the device, which is the default. The <operation> can be Setup, Analyze, Refactor, Solve, Reset-Values, AccessBundledFactors and ExtractSplitFactors.

Finally, the return type of the cuSolverRF library routines is `cusolverStatus_t`.

Chapter 5. Asynchronous Execution

The cuSolver library functions prefer to keep asynchronous execution as much as possible. Developers can always use the `cudaDeviceSynchronize()` function to ensure that the execution of a particular cuSolver library routine has completed.

A developer can also use the `cudaMemcpy()` routine to copy data from the device to the host and vice versa, using the `cudaMemcpyDeviceToHost` and `cudaMemcpyHostToDevice` parameters, respectively. In this case there is no need to add a call to `cudaDeviceSynchronize()` because the call to `cudaMemcpy()` with the above parameters is blocking and completes only when the results are ready on the host.

Chapter 6. Library Property

The `libraryPropertyType` data type is an enumeration of library property types (that is, CUDA version X.Y.Z would yield `MAJOR_VERSION=X`, `MINOR_VERSION=Y`, `PATCH_LEVEL=Z`).

```
typedef enum libraryPropertyType_t
{
    MAJOR_VERSION,
    MINOR_VERSION,
    PATCH_LEVEL
} libraryPropertyType;
```

The following code can show the version of cusolver library.

```
int major=-1, minor=-1, patch=-1;
cusolverGetProperty(MAJOR_VERSION, &major);
cusolverGetProperty(MINOR_VERSION, &minor);
cusolverGetProperty(PATCH_LEVEL, &patch);
printf("CUSOLVER Version (Major,Minor,PatchLevel): %d.%d.%d\n", major, minor, patch);
```

Chapter 7. High Precision Package

The `cusolver` library uses high precision for iterative refinement when necessary.

Chapter 8. Using the CUSOLVER API

8.1. General Description

This chapter describes how to use the cuSolver library API. It is not a reference for the cuSolver API data types and functions; that is provided in subsequent chapters.

8.1.1. Thread Safety

The library is thread-safe, and its functions can be called from multiple host threads.

8.1.2. Scalar Parameters

In the cuSolver API, the scalar parameters can be passed by reference on the host.

8.1.3. Parallelism with Streams

If the application performs several small independent computations, or if it makes data transfers in parallel with the computation, then CUDA streams can be used to overlap these tasks.

The application can conceptually associate a stream with each task. To achieve the overlap of computation between the tasks, the developer should:

1. Create CUDA streams using the function `cudaStreamCreate()`, and
2. Set the stream to be used by each individual cuSolver library routine by calling, for example, `cusolverDnSetStream()`, just prior to calling the actual cuSolverDN routine.

The computations performed in separate streams would then be overlapped automatically on the GPU, when possible. This approach is especially useful when the computation performed by a single task is relatively small, and is not enough to fill the GPU with work, or when there is a data transfer that can be performed in parallel with the computation.

8.1.4. How to Link cusolver Library

cusolver library provides dynamic library `libcusolver.so` and static library `libcusolver_static.a`. If the user links the application with `libcusolver.so`, `libcublas.so`, `libcublasLt.so` and `libcusparses.so` are also required. If the user links the application with `libcusolver_static.a`, the following libraries are also needed, `libcudart_static.a`, `libcublasLt.a`, `libcusolver_lapack_static.a`, `libcusolver_mmetis_static.a`, `libcublas_static.a` and `libcusparses_static.a`.

8.1.5. Link Third-party LAPACK Library

Starting with CUDA 10.1 update 2, NVIDIA LAPACK library `libcusolver_lapack_static.a` is a subset of LAPACK and only contains GPU accelerated `stedc` and `bdsqr`. The user has to link `libcusolver_static.a` with `libcusolver_lapack_static.a` in order to build the application successfully. Prior to CUDA 10.1 update 2, the user can replace `libcusolver_lapack_static.a` with a third-party LAPACK library, for example, MKL. In CUDA 10.1 update 2, the third-party LAPACK library no longer affects the behavior of cusolver library, neither functionality nor performance. Furthermore the user cannot use `libcusolver_lapack_static.a` as a standalone LAPACK library because it is only a subset of LAPACK.

- ▶ If you use `libcusolver_static.a`, then you must link with `libcusolver_lapack_static.a` explicitly, otherwise the linker will report missing symbols. There are no symbol conflicts between `libcusolver_lapack_static.a` and other third-party LAPACK libraries, which allows linking the same application to `libcusolver_lapack_static.a` and another third-party LAPACK library.
- ▶ The `libcusolver_lapack_static.a` is built inside `libcusolver.so`. Hence, if you use `libcusolver.so`, then you don't need to specify a LAPACK library. The `libcusolver.so` will not pick up any routines from the third-party LAPACK library even if you link the application with it.

8.1.6. Convention of info

Each LAPACK routine returns an `info` which indicates the position of invalid parameter. If `info = -i`, then *i*-th parameter is invalid. To be consistent with base-1 in LAPACK, cusolver does not report invalid handle into `info`. Instead, cusolver returns `CUSOLVER_STATUS_NOT_INITIALIZED` for invalid handle.

8.1.7. Usage of `_bufferSize`

There is no `cudaMalloc` inside cuSolver library, the user must allocate the device workspace explicitly. The routine `xyz_bufferSize` is to query the size of workspace of the routine `xyz`, for example `xyz = potrf`. To make the API simple, `xyz_bufferSize` follows almost the same signature of `xyz` even it only depends on some parameters, for example, device pointer is not used to decide the size of workspace. In most cases, `xyz_bufferSize` is called in the beginning before actual device data (pointed to by a device pointer) is prepared or before the device pointer is allocated. In such case, the user can pass null pointer to `xyz_bufferSize` without breaking the functionality.

8.1.8. cuSOLVERDn Logging

cuSOLVERDn logging mechanism can be enabled by setting the following environment variables before launching the target application:

- ▶ CUSOLVERDN_LOG_LEVEL=<level> - where <level> is one of the following levels:
 - ▶ 0 - Off - logging is disabled (default)
 - ▶ 1 - Error - only errors will be logged
 - ▶ 2 - Trace - API calls that launch CUDA kernels will log their parameters and important information
 - ▶ 3 - Hints - hints that can potentially improve the application's performance
 - ▶ 4 - Info - provides general information about the library execution, may contain details about heuristic status
 - ▶ 5 - API Trace - API calls will log their parameter and important information
- ▶ CUSOLVERDN_LOG_MASK=<mask> - where mask is a combination of the following masks:
 - ▶ 0 - Off
 - ▶ 1 - Error
 - ▶ 2 - Trace
 - ▶ 4 - Hints
 - ▶ 8 - Info
 - ▶ 16 - API Trace
- ▶ CUSOLVERDN_LOG_FILE=<file_name> - where file name is a path to a log file. File name may contain %i, that will be replaced with the process ID, for example <file_name>_%i.log. If CUSOLVERDN_LOG_FILE is not defined, the log messages are printed to stdout.

Another option is to use the experimental cusolverDn logging API. See: [cusolverDnLoggerSetCallback\(\)](#), [cusolverDnLoggerSetFile\(\)](#), [cusolverDnLoggerOpenFile\(\)](#), [cusolverDnLoggerSetLevel\(\)](#), [cusolverDnLoggerSetMask\(\)](#), [cusolverDnLoggerForceDisable\(\)](#).

8.1.9. Deterministic Results

Throughout this documentation, a function is declared as *deterministic* if it computes the exact same bitwise results for every execution with the same input parameters, hard- and software environment. Conversely, a *non-deterministic* function might compute bitwise different results due to a varying order of floating point operations, e.g., a sum s of four values a, b, c, d can be computed in different orders:

1. $s = (a + b) + (c + d)$
2. $s = (a + (b + c)) + d$
3. $s = a + (b + (c + d))$
4. ...

Due to the non-associativity of floating point arithmetic, all results might be bitwise different.

By default, cuSolverDN computes deterministic results. For improved performance of some functions, it is possible to allow non-deterministic results with `cusolverDnSetDeterministicMode()`.

8.1.10. Floating Point Emulation

cuSOLVERDn leverages techniques for floating point emulation as described in [cuBLAS 1.5 Floating Point Emulation](#) for improved performance. Please note that all `cusolverDn{Get, Set}{FixedPoint}Emulation*` APIs only affect execution, if a math mode, which allows floating point emulated math, is enabled. However, the corresponding configuration options, which affect floating point emulated math, can be set, regardless of the current math mode. Example:

```
cusolverDnSetMathMode(handle, CUSOLVER_DEFAULT_MATH);
cusolverDnSetEmulationStrategy(handle, EMULATION_STRATEGY_EAGER);
cusolverDnXgetrf(handle, ...); // default math
cusolverDnSetMathMode(handle, CUSOLVER_FP32_EMULATED_BF16X9_MATH);
cusolverDnXgetrf(handle, ...); // FP32 emulated math with eager emulation strategy
```

Note: Please note that cuBLAS environment variables, enabling floating point emulated math, won't affect the cuSOLVERDn internal cuBLAS handles.

Note: Please note that the workspace sizes returned by `*_bufferSize` APIs may depend on the math mode and emulation strategy.

8.2. cuSolver Types Reference

8.2.1. cuSolverDN Types

The `float`, `double`, `cuComplex`, and `cuDoubleComplex` data types are supported. The first two are standard C data types, while the last two are exported from `cuComplex.h`. In addition, `cuSolverDN` uses some familiar types from `cuBLAS`.

8.2.1.1 cusolverDnHandle_t

This is a pointer type to an opaque cuSolverDN context, which the user must initialize by calling *cusolverDnCreate()* prior to calling any other library function. An uninitialized Handle object will lead to unexpected behavior, including crashes of cuSolverDN. The handle created and returned by *cusolverDnCreate()* must be passed to every cuSolverDN function.

8.2.1.2 cublasFillMode_t

The type indicates which part (lower or upper) of the dense matrix was filled and consequently should be used by the function.

Value	Meaning
CUBLAS_FILL_MODE_LOWER	The lower part of the matrix is filled.
CUBLAS_FILL_MODE_UPPER	The upper part of the matrix is filled.
CUBLAS_FILL_MODE_FULL	The full matrix is filled.

Notice that BLAS implementations often use Fortran characters 'L' or 'l' (lower) and 'U' or 'u' (upper) to describe which part of the matrix is filled.

8.2.1.3 cublasOperation_t

The cublasOperation_t type indicates which operation needs to be performed with the dense matrix.

Value	Meaning
CUBLAS_OP_N	The non-transpose operation is selected.
CUBLAS_OP_T	The transpose operation is selected.
CUBLAS_OP_C	The conjugate transpose operation is selected.

Notice that BLAS implementations often use Fortran characters 'N' or 'n' (non-transpose), 'T' or 't' (transpose) and 'C' or 'c' (conjugate transpose) to describe which operations need to be performed with the dense matrix.

8.2.1.4 cusolverEigType_t

The cusolverEigType_t type indicates which type of eigenvalue the solver is.

Value	Meaning
CUSOLVER_EIG_TYPE_1	$A*x = \text{lambda}*B*x$
CUSOLVER_EIG_TYPE_2	$A*B*x = \text{lambda}*x$
CUSOLVER_EIG_TYPE_3	$B*A*x = \text{lambda}*x$

Notice that LAPACK implementations often use Fortran integer 1 ($A^*x = \text{lambda} * B^*x$), 2 ($A^*B^*x = \text{lambda} * x$), 3 ($B^*A^*x = \text{lambda} * x$) to indicate which type of eigenvalue the solver is.

8.2.1.5 cusolverEigMode_t

The `cusolverEigMode_t` type indicates whether or not eigenvectors are computed.

Value	Meaning
CUSOLVER_EIG_MODE_NOVECTOR	Only eigenvalues are computed.
CUSOLVER_EIG_MODE_VECTOR	Both eigenvalues and eigenvectors are computed.

Notice that LAPACK implementations often use Fortran character 'N' (only eigenvalues are computed), 'V' (both eigenvalues and eigenvectors are computed) to indicate whether or not eigenvectors are computed.

8.2.1.6 cusolverEigComp_t

The `cusolverEigComp_t` type indicates the computational mode for eigen routines that compute eigenvalues and optionally eigenvectors, analogous to the `compz` argument in LAPACK.

Value	Meaning
CU-SOLVER_EIG_COMP_N	Only eigenvalues are computed.
CU-SOLVER_EIG_COMP_I	The unitary matrix is initialized to the identity matrix.
CU-SOLVER_EIG_COMP_V	The product of the matrix given on entry with the orthogonal transformations accumulated during the computational routine is returned.

The values of `cusolverEigComp_t` correspond directly to the LAPACK `compz` argument. In LAPACK, `compz` is specified as a single character: 'N' means no eigenvectors are computed (eigenvalues only); 'I' means the eigenvector matrix is initialized to the identity and then updated by the routine; 'V' means the routine uses the matrix provided on entry and overwrites it with the product of that matrix and the orthogonal (or unitary) transformations accumulated during the computation. The `cuSolver` enumerants `CUSOLVER_EIG_COMP_N`, `CUSOLVER_EIG_COMP_I`, and `CUSOLVER_EIG_COMP_V` map to LAPACK 'N', 'I', and 'V' respectively.

8.2.1.7 cusolverIRSRefinement_t

The `cusolverIRSRefinement_t` type indicates which solver type would be used for the specific `cu-solver` function. Most of our experimentation shows that `CUSOLVER_IRS_REFINE_GMRES` is the best option.

More details about the refinement process can be found in Azzam Haidar, Stanimire Tomov, Jack Dongarra, and Nicholas J. Higham. 2018. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. In Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC '18). IEEE Press, Piscataway, NJ, USA, Article 47, 11 pages.

CUSOLVER_IRS_REFINE_NOT_SET

Solver is not set; this value is what is set when creating the params structure. IRS solver will return an error.

CUSOLVER_IRS_REFINE_NONE

No refinement solver, the IRS solver performs a factorization followed by a solve without any refinement. For example if the IRS solver was `cusolverDnIRSXgesv()`, this is equivalent to a `Xgesv` routine without refinement and where the factorization is carried out in the lowest precision. If for example the main precision was `CUSOLVER_R_64F` and the lowest was `CUSOLVER_R_64F` as well, then this is equivalent to a call to `cusolverDnDgesv()`.

CUSOLVER_IRS_REFINE_CLASSICAL

Classical iterative refinement solver. Similar to the one used in LAPACK routines.

CUSOLVER_IRS_REFINE_GMRES

GMRES (Generalized Minimal Residual) based iterative refinement solver. In recent study, the GMRES method has drawn the scientific community attention for its ability to be used as refinement solver that outperforms the classical iterative refinement method. Based on our experimentation, we recommend this setting.

CUSOLVER_IRS_REFINE_CLASSICAL_GMRES

Classical iterative refinement solver that uses the GMRES (Generalized Minimal Residual) internally to solve the correction equation at each iteration. We call the *classical refinement iteration* the outer iteration while the *GMRES* is called inner iteration. Note that if the tolerance of the inner GMRES is set very low, lets say to machine precision, then the outer *classical refinement iteration* will performs only one iteration and thus this option will behave like `CUSOLVER_IRS_REFINE_GMRES`.

CUSOLVER_IRS_REFINE_GMRES_GMRES

Similar to `CUSOLVER_IRS_REFINE_CLASSICAL_GMRES` which consists of classical refinement process that uses GMRES to solve the inner correction system; here it is a GMRES (Generalized Minimal Residual) based iterative refinement solver that uses another GMRES internally to solve the preconditioned system.

8.2.1.8 cusolverDnIRSParams_t

This is a pointer type to an opaque `cusolverDnIRSParams_t` structure, which holds parameters for the iterative refinement linear solvers such as `cusolverDnXgesv()`. Use corresponding helper functions described below to either Create/Destroy this structure or Set/Get solver parameters.

8.2.1.9 cusolverDnIRSInfos_t

This is a pointer type to an opaque `cusolverDnIRSInfos_t` structure, which holds information about the performed call to an iterative refinement linear solver (such as `cusolverDnXgesv()`). Use corresponding helper functions described below to either Create/Destroy this structure or retrieve solve information.

8.2.1.10 cusolverDnFunction_t

The `cusolverDnFunction_t` type indicates which routine needs to be configured by `cusolverDnSetAdvOptions()`. The value `CUSOLVERDN_GETRF` corresponds to the routine `Getrf`.

Value	Meaning
CUSOLVERDN_GETRF	Corresponds to <code>Getrf</code> .

8.2.1.11 cusolverAlgMode_t

The `cusolverAlgMode_t` type indicates which algorithm is selected by `cusolverDnSetAdvOptions()`. The set of algorithms supported for each routine is described in detail along with the routine's documentation.

The default algorithm is `CUSOLVER_ALG_0`. The user can also provide `NULL` to use the default algorithm.

8.2.1.12 cusolverStatus_t

This is the same as *cusolverStatus_t* in the sparse LAPACK section.

8.2.1.13 cusolverDnLoggerCallback_t

cusolverDnLoggerCallback_t is a callback function pointer type.

Parameters

Parameter	Memory	In/out	Description
<code>logLevel</code>		output	See <i>cuSOLVERDn Logging</i>
<code>functionName</code>		output	The name of the API that logged this message.
<code>message</code>		output	The log message.

Use the below function to set the callback function: *cusolverDnLoggerSetCallback()*.

8.2.1.14 cusolverDeterministicMode_t

The `cusolverDeterministicMode_t` type indicates whether multiple `cuSolver` function executions with the same input have the same bitwise equal result (deterministic) or might have bitwise different results (non-deterministic). In comparison to *cublasAtomicsMode_t*, which only includes the usage of atomic functions, `cusolverDeterministicMode_t` includes all non-deterministic programming patterns. The deterministic mode can be set and queried using `cusolverDnSetDeterministicMode()` and `cusolverDnGetDeterministicMode()` routines, respectively.

Value	Meaning
CUSOLVER_DETERMINISTIC_RESULTS	Compute deterministic results.
CUSOLVER_ALLOW_NON_DETERMINISTIC_RESULTS	Allow non-deterministic results.

8.2.1.15 cusolverMathMode_t

The `cusolverMathMode_t` type is used in `cusolverDnSetMathMode()` to choose compute precision modes as defined in the following table:

Value	Meaning
CUSOLVER_DEFAULT_MATH	This is the default math mode. Tensor Cores will be used whenever possible.
CUSOLVER_FP32_EMULATED_BF16X9_MATH	Use FP32 emulation according to the configured emulation strategy (see <code>cusolverDnSetEmulationStrategy()</code>).
CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH	Use FP64 emulation according to the configured emulation strategy (see <code>cusolverDnSetEmulationStrategy()</code>).
CUSOLVER_FP32_FP64_EMULATED_MATH	Combination of CUSOLVER_FP32_EMULATED_BF16X9_MATH and CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH.

The following combinations of `cusolverMathMode_t` using the bitwise OR operator are allowed:

- `CUSOLVER_FP32_FP64_EMULATED_MATH = CUSOLVER_FP32_EMULATED_BF16X9_MATH | CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH.`

8.2.1.16 cusolverStorevMode_t

Specifies how the vectors which define the elementary reflectors are stored.

Value	Meaning
CUBLAS_STOREV_COLUMNWISE	Columnwise.
CUBLAS_STOREV_ROWWISE	Rowwise.

8.2.1.17 cusolverDirectMode_t

Specifies the order in which the elementary reflectors are multiplied to form the block reflector.

Value	Meaning
CUBLAS_DIRECT_FORWARD	Forward.
CUBLAS_DIRECT_BACKWARD	Backward.

8.2.2. cuSolverSP Types

The float, double, cuComplex, and cuDoubleComplex data types are supported. The first two are standard C data types, while the last two are exported from `cuComplex.h`.

8.2.2.1 cusolverSpHandle_t

This is a pointer type to an opaque cuSolverSP context, which the user must initialize by calling `cusolverSpCreate()` prior to calling any other library function. An uninitialized Handle object will lead to unexpected behavior, including crashes of cuSolverSP. The handle created and returned by `cusolverSpCreate()` must be passed to every cuSolverSP function.

8.2.2.2 cusparseMatDescr_t

We have chosen to keep the same structure as exists in cuSPARSE to describe the shape and properties of a matrix. This enables calls to either cuSPARSE or cuSOLVER using the same matrix description.

```
typedef struct {
    cusparseMatrixType_t MatrixType;
    cusparseFillMode_t FillMode;
    cusparseDiagType_t DiagType;
    cusparseIndexBase_t IndexBase;
} cusparseMatDescr_t;
```

Please read documentation of the cuSPARSE Library to understand each field of `cusparseMatDescr_t`.

8.2.2.3 cusolverStatus_t

This is a status type returned by the library functions and it can have the following values.

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The cuSolver library was not initialized. This is usually caused by the lack of a prior call, an error in the CUDA Runtime API called by the cuSolver routine, or an error in the hardware setup.

To correct: call `cusolverDnCreate()` prior to the function call; and check that the hardware, an appropriate version of the driver, and the cuSolver library are correctly installed.

CUSOLVER_STATUS_ALLOC_FAILED

Resource allocation failed inside the cuSolver library. This is usually caused by a `cudaMalloc()` failure.

To correct: prior to the function call, deallocate previously allocated memory as much as possible.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed to the function (a negative vector size, for example).

To correct: ensure that all the parameters being passed have valid values.

CUSOLVER_STATUS_ARCH_MISMATCH

The function requires a feature absent from the device architecture; usually caused by the lack of support for atomic operations or double precision.

To correct: compile and run the application on a device with compute capability 5.0 or above.

CUSOLVER_STATUS_EXECUTION_FAILED

The GPU program failed to execute. This is often caused by a launch failure of the kernel on the GPU, which can be caused by multiple reasons.

To correct: check that the hardware, an appropriate version of the driver, and the cuSolver library are correctly installed.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal cuSolver operation failed. This error is usually caused by a `cudaMemcpyAsync()` failure.

To correct: check that the hardware, an appropriate version of the driver, and the cuSolver library are correctly installed. Also, check that the memory passed as a parameter to the routine is not being deallocated prior to the routine's completion.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported by this function. This is usually caused by passing an invalid matrix descriptor to the function.

To correct: check that the fields in `descrA` were set correctly.

CUSOLVER_STATUS_NOT_SUPPORTED

The parameter combination is not supported, for example batched version is not supported or $M < N$ is not supported.

To correct: consult the documentation, and use a supported configuration.

8.2.3. cuSolverRF Types

cuSolverRF only supports double.

8.2.3.1 cusolverRfHandle_t

The `cusolverRfHandle_t` is a pointer to an opaque data structure that contains the cuSolverRF library handle. The user must initialize the handle by calling `cusolverRfCreate()` prior to any other cuSolverRF library calls. The handle is passed to all other cuSolverRF library calls.

8.2.3.2 cusolverRfMatrixFormat_t

The `cusolverRfMatrixFormat_t` is an enum that indicates the input/output matrix format assumed by the `cusolverRfSetupDevice()`, `cusolverRfSetupHost()`, `cusolverRfResetValues()`, `cusolverRfExtractBundledFactorsHost()` and `cusolverRfExtractSplitFactorsHost()` routines.

Value	Meaning
CUSOLVER_MATRIX_FORMAT_CSR	Matrix format CSR is assumed. (default)
CUSOLVER_MATRIX_FORMAT_CSC	Matrix format CSC is assumed.

8.2.3.3 cusolverRfNumericBoostReport_t

The `cusolverRfNumericBoostReport_t` is an enum that indicates whether numeric boosting (of the pivot) was used during the `cusolverRfRefactor()` and `cusolverRfSolve()` routines. The numeric boosting is disabled by default.

Value	Meaning
CUSOLVER_NUMERIC_BOOST_NOT_USED	Numeric boosting not used. (default)
CUSOLVER_NUMERIC_BOOST_USED	Numeric boosting used.

8.2.3.4 cusolverRfResetValuesFastMode_t

The `cusolverRfResetValuesFastMode_t` is an enum that indicates the mode used for the `cusolverRfResetValues()` routine. The fast mode requires extra memory and is recommended only if very fast calls to `cusolverRfResetValues()` are needed.

Value	Meaning
CUSOLVER_RESET_VALUES_FAST_MODE_OFF	Fast mode disabled. (default)
CUSOLVER_RESET_VALUES_FAST_MODE_ON	Fast mode enabled.

8.2.3.5 cusolverRfFactorization_t

The `cusolverRfFactorization_t` is an enum that indicates which (internal) algorithm is used for refactorization in the `cusolverRfRefactor()` routine.

Value	Meaning
CUSOLVER_FACTORIZATION_ALG0	Algorithm 0. (default)
CUSOLVER_FACTORIZATION_ALG1	Algorithm 1.
CUSOLVER_FACTORIZATION_ALG2	Algorithm 2. Domino-based scheme.

8.2.3.6 cusolverRfTriangularSolve_t

The `cusolverRfTriangularSolve_t` is an enum that indicates which (internal) algorithm is used for triangular solve in the `cusolverRfSolve()` routine.

Value	Meaning
CUSOLVER_TRIANGULAR_SOLVE_ALG1	Algorithm 1. (default)
CUSOLVER_TRIANGULAR_SOLVE_ALG2	Algorithm 2. Domino-based scheme.
CUSOLVER_TRIANGULAR_SOLVE_ALG3	Algorithm 3. Domino-based scheme.

8.2.3.7 cusolverRfUnitDiagonal_t

The `cusolverRfUnitDiagonal_t` is an enum that indicates whether and where the unit diagonal is stored in the input/output triangular factors in the `cusolverRfSetupDevice()`, `cusolverRfSetupHost()` and `cusolverRfExtractSplitFactorsHost()` routines.

Value	Meaning
CUSOLVER_UNIT_DIAGONAL_STORED_L	Unit diagonal is stored in lower triangular factor (default).
CUSOLVER_UNIT_DIAGONAL_STORED_U	Unit diagonal is stored in upper triangular factor.
CUSOLVER_UNIT_DIAGONAL_ASSUMED_L	Unit diagonal is assumed in lower triangular factor.
CUSOLVER_UNIT_DIAGONAL_ASSUMED_U	Unit diagonal is assumed in upper triangular factor.

8.2.3.8 cusolverStatus_t

The `cusolverStatus_t` is an enum that indicates success or failure of the cuSolverRF library call. It is returned by all the cuSolver library routines, and it uses the same enumerated values as the sparse and dense Lapack routines.

8.3. cuSolver Formats Reference

8.3.1. Index Base Format

Both one-based and zero-based indexing are supported in cuSolver.

8.3.2. Vector (Dense) Format

The vectors are assumed to be stored linearly in memory. For example, the vector

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

is represented as

$$(x_1 \quad x_2 \quad \dots \quad x_n)$$

8.3.3. Matrix (Dense) Format

The dense matrices are assumed to be stored in column-major order in memory. The sub-matrix can be accessed using the leading dimension of the original matrix. For example, the $m \times n$ (sub-)matrix

$$\begin{pmatrix} a_{1,1} & \cdots & a_{1,n} \\ a_{2,1} & \cdots & a_{2,n} \\ \vdots & & \\ a_{m,1} & \cdots & a_{m,n} \end{pmatrix}$$

is represented as

$$\begin{pmatrix} a_{1,1} & \cdots & a_{1,n} \\ a_{2,1} & \cdots & a_{2,n} \\ \vdots & \ddots & \vdots \\ a_{m,1} & \cdots & a_{m,n} \\ \vdots & \ddots & \vdots \\ a_{lda,1} & \cdots & a_{lda,n} \end{pmatrix}$$

with its elements arranged linearly in memory as

$$(a_{1,1} \ a_{2,1} \ \cdots \ a_{m,1} \ \cdots \ a_{lda,1} \ \cdots \ a_{1,n} \ a_{2,n} \ \cdots \ a_{m,n} \ \cdots \ a_{lda,n})$$

where $lda \geq m$ is the leading dimension of A.

8.3.4. Matrix (CSR) Format

In CSR format the matrix is represented by the following parameters:

Parameter	Type	Size	Meaning
n	(int)		The number of rows (and columns) in the matrix.
nnz	(int)		The number of non-zero elements in the matrix.
csr-RowPtr	(int *)	n+1	The array of offsets corresponding to the start of each row in the arrays <code>csrColInd</code> and <code>csrVal</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix.
csr-ColInd	(int *)	nnz	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row.
csrVal	(S D C Z)	nnz	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row.

Note that in our CSR format, sparse matrices are assumed to be stored in row-major order, in other words, the index arrays are first sorted by row indices and then within each row by column indices. Also it is assumed that each pair of row and column indices appears only once.

For example, the 4x4 matrix

$$A = \begin{pmatrix} 1.0 & 3.0 & 0.0 & 0.0 \\ 0.0 & 4.0 & 6.0 & 0.0 \\ 2.0 & 5.0 & 7.0 & 8.0 \\ 0.0 & 0.0 & 0.0 & 9.0 \end{pmatrix}$$

is represented as

$$\begin{aligned} csrRowPtr &= (0 \quad 2 \quad 4 \quad 8 \quad 9) \\ csrColInd &= (0 \quad 1 \quad 1 \quad 2 \quad 0 \quad 1 \quad 2 \quad 3 \quad 3) \\ csrVal &= (1.0 \quad 3.0 \quad 4.0 \quad 6.0 \quad 2.0 \quad 5.0 \quad 7.0 \quad 8.0 \quad 9.0) \end{aligned}$$

8.3.5. Matrix (CSC) Format

In CSC format the matrix is represented by the following parameters:

Parameter	Type	Size	Meaning
n	(int)		The number of rows (and columns) in the matrix.
nnz	(int)		The number of non-zero elements in the matrix.
cscColPtr	(int *)	n+1	The array of offsets corresponding to the start of each column in the arrays cscRowInd and cscVal. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix.
cscRowInd	(int *)	nnz	The array of row indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by column and by row within each column.
cscVal	(S D C Z)*	nnz	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by column and by row within each column.

Note that in our CSC format, sparse matrices are assumed to be stored in column-major order, in other words, the index arrays are first sorted by column indices and then within each column by row indices. Also it is assumed that each pair of row and column indices appears only once.

For example, the 4x4 matrix

$$A = \begin{pmatrix} 1.0 & 3.0 & 0.0 & 0.0 \\ 0.0 & 4.0 & 6.0 & 0.0 \\ 2.0 & 5.0 & 7.0 & 8.0 \\ 0.0 & 0.0 & 0.0 & 9.0 \end{pmatrix}$$

is represented as

$$\begin{aligned} cscColPtr &= (0 \quad 2 \quad 5 \quad 7 \quad 9) \\ cscRowInd &= (0 \quad 2 \quad 0 \quad 1 \quad 2 \quad 1 \quad 2 \quad 2 \quad 3) \\ cscVal &= (1.0 \quad 2.0 \quad 3.0 \quad 4.0 \quad 5.0 \quad 6.0 \quad 7.0 \quad 8.0 \quad 9.0) \end{aligned}$$

8.4. cuSolverDN: dense LAPACK Function Reference

This section describes the API of cuSolverDN, which provides a subset of dense LAPACK functions.

8.4.1. cuSolverDN Helper Function Reference

The cuSolverDN helper functions are described in this section.

8.4.1.1 cusolverDnCreate()

```
cusolverStatus_t
cusolverDnCreate(cusolverDnHandle_t *handle);
```

This function initializes the cuSolverDN library and creates a handle on the cuSolverDN context. It must be called before any other cuSolverDN API function is invoked. It allocates hardware resources necessary for accessing the GPU. This function allocates 4 MiB or 32 MiB of memory (for GPUs with Compute Capability of 9.0 and higher), which will be used as the cuBLAS workspace for the first user-defined stream on which *cusolverDnSetStream()* is called. For the default stream and in all the other cases, cuBLAS will manage its own workspace.

Parameter	Memory	In/out	Meaning
handle	host	output	The pointer to the handle to the cuSolverDN context.

Status Returned

CUSOLVER_STATUS_SUCCESS

The initialization succeeded.

CUSOLVER_STATUS_NOT_INITIALIZED

The CUDA Runtime initialization failed.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

8.4.1.2 cusolverDnDestroy()

```
cusolverStatus_t
cusolverDnDestroy(cusolverDnHandle_t handle);
```

This function releases CPU-side resources used by the cuSolverDN library.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The shutdown succeeded.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.4.1.3 cusolverDnSetStream()

```
cusolverStatus_t
cusolverDnSetStream(cusolverDnHandle_t handle, cudaStream_t streamId)
```

This function sets the stream to be used by the cuSolverDN library to execute its routines.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
streamId	host	input	The stream to be used by the library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The stream was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.4.1.4 `cusolverDnGetStream()`

```
cusolverStatus_t
cusolverDnGetStream(cusolverDnHandle_t handle, cudaStream_t *streamId)
```

This function queries the stream to be used by the cuSolverDN library to execute its routines.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
streamId	host	output	The stream which is used by handle.

Status Returned

CUSOLVER_STATUS_SUCCESS

The stream was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.4.1.5 `cusolverDnLoggerSetCallback()`

```
cusolverStatus_t cusolverDnLoggerSetCallback(cusolverDnLoggerCallback_t callback);
```

This function sets the logging callback function.

Parameters

Parameter	Memory	In/out	Meaning
callback		input	Pointer to a callback function. See cusolverDnLoggerCallback_t .

Status Returned

CUSOLVER_STATUS_SUCCESS

If the callback function was successfully set.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.6 `cusolverDnLoggerSetFile()`

```
cusolverStatus_t cusolverDnLoggerSetFile(FILE* file);
```

This function sets the logging output file. Note: once registered using this function call, the provided file handle must not be closed unless the function is called again to switch to a different file handle.

Parameters

Parameter	Memory	In/out	Meaning
file		input	Pointer to an open file. File should have write permission.

Status Returned

CUSOLVER_STATUS_SUCCESS

If logging file was successfully set.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.7 cusolverDnLoggerOpenFile()

```
cusolverStatus_t cusolverDnLoggerOpenFile(const char* logFile);
```

This function opens a logging output file in the given path.

Parameters

Parameter	Memory	In/out	Meaning
logFile		input	Path of the logging output file.

Status Returned**CUSOLVER_STATUS_SUCCESS**

If the logging file was successfully opened.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.8 cusolverDnLoggerSetLevel()

```
cusolverStatus_t cusolverDnLoggerSetLevel(int level);
```

This function sets the value of the logging level.

Parameters

Parameter	Memory	In/out	Meaning
level		input	Value of the logging level. See cuSOLVERDn Logging .

Status Returned**CUSOLVER_STATUS_INVALID_VALUE**

If the value was not a valid logging level. See [cuSOLVERDn Logging](#).

CUSOLVER_STATUS_SUCCESS

If the logging level was successfully set.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.9 cusolverDnLoggerSetMask()

```
cusolverStatus_t cusolverDnLoggerSetMask(int mask);
```

This function sets the value of the logging mask.

Parameters

Parameter	Memory	In/out	Meaning
mask		input	Value of the logging mask. See cuSOLVERDn Logging .

Status Returned

CUSOLVER_STATUS_SUCCESS

If the logging mask was successfully set.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.10 cusolverDnLoggerForceDisable()

```
cusolverStatus_t cusolverDnLoggerForceDisable();
```

This function disables logging for the entire run.

Status Returned

CUSOLVER_STATUS_SUCCESS

If logging was successfully disabled.

See [cusolverStatus_t](#) for a complete list of valid return codes.

8.4.1.11 cusolverDnSetDeterministicMode()

```
cusolverStatus_t
cusolverDnSetDeterministicMode(cusolverDnHandle_t handle, cusolverDeterministicMode_t
↪ mode)
```

This function sets the deterministic mode of all cuSolverDN functions for handle. For improved performance, non-deterministic results can be allowed. Affected functions are `cusolverDn<t>geqrf()`, `cusolverDn<t>syevd()`, `cusolverDn<t>syevdx()`, `cusolverDn<t>gesvd()` (if $m > n$), `cusolverDn<t>gesvdj()`, `cusolverDnXgeqrf()`, `cusolverDnXsyevd()`, `cusolverDnXsyevdx()`, `cusolverDnXgesvd()` (if $m > n$), `cusolverDnXgesvdr()` and `cusolverDnXgesvdp()`.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
mode	host	input	The deterministic mode to be used with handle.

Status Returned

CUSOLVER_STATUS_SUCCESS

The modes were set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

8.4.1.12 cusolverDnGetDeterministicMode()

```
cusolverStatus_t
cusolverDnGetDeterministicMode(cusolverDnHandle_t handle, cusolverDeterministicMode_t* mode)
```

This function queries the deterministic mode which is set for handle.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
mode	host	output	The deterministic mode of handle.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The modes were queried successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

mode is a NULL pointer.

8.4.1.13 cusolverDnSetMathMode()

```
cusolverStatus_t cusolverDnSetMathMode(cusolverDnHandle_t handle, cusolverMathMode_t modes);
```

This function sets the math modes of all cuSolverDN functions for handle. For more information about the effects of the corresponding math modes, please refer to [cusolverMathMode_t](#). Note that math modes can be combined, e.g., `cusolverDnSetMathMode(handle, CUSOLVER_FP32_EMULATED_BF16X9_MATH | CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH)`. Please see [cusolverMathMode_t](#) for allowed combinations.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
modes	host	input	The math modes to be used with handle.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The mode was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An invalid mode was given.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

Note: Please note that the workspace sizes returned by *_bufferSize APIs may depend on the math mode.

8.4.1.14 cusolverDnGetMathMode()

```
cusolverStatus_t cusolverDnGetMathMode(cusolverDnHandle_t handle, cusolverMathMode_t*  
↪ modes);
```

This function queries the math modes which are set for handle. Note that math modes can be combined, e.g., `cusolverDnSetMathMode(handle, CUSOLVER_FP32_EMULATED_BF16X9_MATH | CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH)`. Please see [cusolverMathMode_t](#) for allowed combinations.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
modes	host	output	The math modes of handle.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The mode was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

modes is a NULL pointer.

8.4.1.15 cusolverDnSetEmulationStrategy()

```
cusolverStatus_t cusolverDnSetEmulationStrategy(cusolverHandle_t handle,  
↪ cudaEmulationStrategy_t strategy);
```

This function sets the emulation strategy of all cuSolverDN functions for handle. For more information about the effects of the corresponding strategies, please refer to the analogous definition of [cublasEmulationStrategy_t](#).

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
strategy	host	input	The emulation strategy to be used with handle.

The emulation strategy set by this API only has an effect, **once** one of the following math modes is enabled (see also [cusolverMathMode_t](#)):

- ▶ CUSOLVER_FP32_EMULATED_BF16X9_MATH
- ▶ CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- ▶ CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned**CUSOLVER_STATUS_SUCCESS**

The strategy was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

strategy was not a supported emulation strategy

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

Note: Please note that the workspace sizes returned by *_bufferSize APIs may depend on the emulation strategy.

8.4.1.16 cusolverDnGetEmulationStrategy()

```
cusolverStatus_t cusolverDnGetEmulationStrategy(cusolverHandle_t handle,
↪ cudaEmulationStrategy_t* emulationStrategy);
```

This function queries the emulation strategy which is set for handle.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
strategy	host	output	The emulation strategy of handle.

The emulation strategy returned by this API only has an effect, **once** one of the following math modes is enabled (see also *[cusolverMathMode_t](#)*):

- ▶ CUSOLVER_FP32_EMULATED_BF16X9_MATH
- ▶ CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- ▶ CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned**CUSOLVER_STATUS_SUCCESS**

The strategy was queried successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

emulationStrategy is a NULL pointer.

8.4.1.17 cusolverDnSetFixedPointEmulationMantissaControl()

```
cusolverStatus_t cusolverDnSetFixedPointEmulationMantissaControl(
    cusolverDnHandle_t handle,
    cudaEmulationMantissaControl_t control);
```

This function sets how the number of mantissa bits is determined for fixed point FP64 emulation. For more information about the effects of the corresponding control modes, please refer to [cudaEmulationMantissaControl_t](#).

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
control	host	input	The mantissa control mode to be used with handle.

The mantissa control set by this API only has an effect, **once** one of the following math modes is enabled (see also [cusolverMathMode_t](#)):

- ▶ CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- ▶ CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned

CUSOLVER_STATUS_SUCCESS

The mantissa control was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

control is not a valid cudaEmulationMantissaControl_t value.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

8.4.1.18 cusolverDnGetFixedPointEmulationMantissaControl()

```
cusolverStatus_t cusolverDnGetFixedPointEmulationMantissaControl(
    cusolverDnHandle_t handle,
    cudaEmulationMantissaControl_t* control);
```

This function queries how the number of mantissa bits is determined for fixed point FP64 emulation.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
control	host	output	The mantissa control mode of handle.

The mantissa control returned by this API only has an effect, **once** one of the following math modes is enabled (see also [cusolverMathMode_t](#)):

- ▶ CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- ▶ CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned**CUSOLVER_STATUS_SUCCESS**

The mantissa control was queried successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

`control` is a NULL pointer.

8.4.1.19 `cusolverDnSetFixedPointEmulationMaxMantissaBitCount()`

```
cusolverStatus_t cusolverDnSetFixedPointEmulationMaxMantissaBitCount(
    cusolverDnHandle_t handle,
    int mantissaBitCount);
```

This function sets the maximum number of mantissa bits for fixed point FP64 emulation.

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>mantissaBitCount</code>	host	input	The number of mantissa bits to be used. Setting <code>mantissaBitCount = 0</code> resets to the default configuration as described in cuBLAS defaults .

The maximum mantissa bit count set by this API only has an effect, **once** one of the following math modes is enabled (see also [cusolverMathMode_t](#)):

- CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned**CUSOLVER_STATUS_SUCCESS**

The mantissa bit count was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

`mantissaBitCount` is less than 0.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

8.4.1.20 `cusolverDnGetFixedPointEmulationMaxMantissaBitCount()`

```
cusolverStatus_t cusolverDnGetFixedPointEmulationMaxMantissaBitCount(
    cusolverDnHandle_t handle,
    int* mantissaBitCount);
```

This function queries the maximum number of mantissa bits used for fixed point FP64 emulation.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
mantissaBitCount	host	output	The maximum number of mantissa bits used.

The mantissa bit count returned by this API only has an effect, **once** one of the following math modes is enabled (see also `cusolverMathMode_t`):

- CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- CUSOLVER_FP32_FP64_EMULATED_MATH

Status Returned

CUSOLVER_STATUS_SUCCESS

The mantissa bit count was queried successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

mantissaBitCount is a NULL pointer.

8.4.1.21 `cusolverDnSetFixedPointEmulationMantissaBitOffset()`

```
cusolverStatus_t cusolverDnSetFixedPointEmulationMantissaBitOffset(
    cusolverDnHandle_t handle,
    int mantissaBitOffset);
```

This function sets the mantissa bit offset for fixed point FP64 emulation in case of dynamic mantissa control mode.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
mantissaBitOffset	host	input	The mantissa bit offset (default = 0) to be used when the mantissa control is CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC.

The mantissa bit offset, which is set by this API only has an effect, **once** one of the following math modes is enabled (see also `cusolverMathMode_t`):

- CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- CUSOLVER_FP32_FP64_EMULATED_MATH

And the following mantissa control is enabled (see also [cusolverDnSetFixedPointEmulationMantissaControl\(\)](#)):

- CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC

You may tune values for `mantissaBitOffset` based on your accuracy and performance requirements, e.g., choose negative values -8, -16, ... for better performance while fewer mantissa bits may reduce accuracy.

Please note that values of `mantissaBitOffset` which are unequal to zero require the mantissa control to be equal to `CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC`. Otherwise, the computational cuSOLVER routines will return `CUSOLVER_STATUS_INVALID_VALUE`, indicating an unsupported handle state.

Status Returned

CUSOLVER_STATUS_SUCCESS

The mantissa bit offset was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

8.4.1.22 cusolverDnGetFixedPointEmulationMantissaBitOffset()

```
cusolverStatus_t cusolverDnGetFixedPointEmulationMantissaBitOffset(
    cusolverDnHandle_t handle,
    int* mantissaBitOffset);
```

This function queries the mantissa bit offset for fixed point FP64 emulation in case of dynamic mantissa control mode.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
mantissaBitOffset	host	output	The mantissa bit offset used when <code>CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC</code> is in use.

The mantissa bit offset, which is returned by this API only has an effect, **once** one of the following math modes is enabled (see also [cusolverMathMode_t](#)):

- CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH
- CUSOLVER_FP32_FP64_EMULATED_MATH

And the following mantissa control is enabled (see also [cusolverDnSetFixedPointEmulationMantissaControl\(\)](#)):

- CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC

Please note that values of `mantissaBitOffset` which are unequal to zero require the mantissa control to be equal to `CUDA_EMULATION_MANTISSA_CONTROL_DYNAMIC`. Otherwise, the computational cuSOLVER routines will return `CUSOLVER_STATUS_INVALID_VALUE`, indicating an unsupported handle state.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The mantissa bit offset was queried successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

`mantissaBitOffset` is a NULL pointer.

8.4.1.23 `cusolverDnSetEmulationSpecialValuesSupport()`

```
cusolverStatus_t cusolverDnSetEmulationSpecialValuesSupport(  
    cusolverDnHandle_t handle,  
    cudaEmulationSpecialValuesSupport_t mask);
```

This function sets the handling of special floating point values for `handle`, which is used **once** floating point emulation is allowed.

Param-eter	Mem-ory	In/out	Meaning
<code>handle</code>	host	in-put	Handle to the cuSolverDN library context.
<code>mask</code>	host	in-put	If set to <code>CUDA_EMULATION_SPECIAL_VALUE_SUPPORT_DEFAULT</code> , values are propagated as expected. Performance of floating point emulated math may improve if set to <code>CUDA_EMULATION_SPECIAL_VALUES_SUPPORT_NONE</code> for which the propagation of special values is undefined.

The special value support set by this API only has an effect, **once** one of the following math modes is enabled (see also [`cusolverMathMode\_t`](#)):

- ▶ `CUSOLVER_FP32_EMULATED_BF16X9_MATH`
- ▶ `CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH`
- ▶ `CUSOLVER_FP32_FP64_EMULATED_MATH`

Status Returned**CUSOLVER_STATUS_SUCCESS**

The special value support was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred.

8.4.1.24 `cusolverDnGetEmulationSpecialValuesSupport()`

```
cusolverStatus_t cusolverDnGetEmulationSpecialValuesSupport(
    cusolverDnHandle_t handle,
    cudaEmulationSpecialValuesSupport_t* mask);
```

This function queries the special floating point value support which is set for `handle` if floating point emulation is allowed.

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>mask</code>	host	output	The special value support of <code>handle</code> . Please see <code>cudaEmulationSpecialValuesSupport_t</code> for more information about the allowed values of <code>mask</code> .

The special floating point value support returned by this API only has an effect, **once** one of the following math modes is enabled (see also `cusolverMathMode_t`):

- `CUSOLVER_FP32_EMULATED_BF16X9_MATH`
- `CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH`
- `CUSOLVER_FP32_FP64_EMULATED_MATH`

Otherwise, special floating point values are handled as expected.

Status Returned

`CUSOLVER_STATUS_SUCCESS`

The special value handling was queried successfully.

`CUSOLVER_STATUS_NOT_INITIALIZED`

The library was not initialized.

`CUSOLVER_STATUS_INVALID_VALUE`

`mask` is a NULL pointer.

8.4.1.25 `cusolverDnCreateSyevjInfo()`

```
cusolverStatus_t
cusolverDnCreateSyevjInfo(
    syevjInfo_t *info);
```

This function creates and initializes the structure of `syevj`, `syevjBatched` and `sygvj` to default values.

Parameter	Memory	In/out	Meaning
<code>info</code>	host	output	The pointer to the structure of <code>syevj</code> .

Status Returned

`CUSOLVER_STATUS_SUCCESS`

The structure was initialized successfully.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

8.4.1.26 cusolverDnDestroySyevjInfo()

```
cusolverStatus_t  
cusolverDnDestroySyevjInfo(  
    syevjInfo_t info);
```

This function destroys and releases any memory required by the structure.

Parameter	Memory	In/out	Meaning
info	host	input	The structure of syevj.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The resources were released successfully.

8.4.1.27 cusolverDnXsyevjSetTolerance()

```
cusolverStatus_t  
cusolverDnXsyevjSetTolerance(  
    syevjInfo_t info,  
    double tolerance)
```

This function configures tolerance of syevj.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of syevj.
tolerance	host	input	Accuracy of numerical eigenvalues.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

8.4.1.28 cusolverDnXsyevjSetMaxSweeps()

```
cusolverStatus_t  
cusolverDnXsyevjSetMaxSweeps(  
    syevjInfo_t info,  
    int max_sweeps)
```

This function configures maximum number of sweeps in syevj. The default value is 100.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of syevj.
max_sweeps	host	input	Maximum number of sweeps.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

8.4.1.29 cusolverDnXsyevjSetSortEig()

```
cusolverStatus_t
cusolverDnXsyevjSetSortEig(
    syevjInfo_t info,
    int sort_eig)
```

If sort_eig is zero, the eigenvalues are not sorted. This function only works for syevjBatched. syevj and sygvj always sort eigenvalues in ascending order. By default, eigenvalues are always sorted in ascending order.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of syevj.
sort_eig	host	input	If sort_eig is zero, the eigenvalues are not sorted.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

8.4.1.30 cusolverDnXsyevjGetResidual()

```
cusolverStatus_t
cusolverDnXsyevjGetResidual(
    cusolverDnHandle_t handle,
    syevjInfo_t info,
    double *residual)
```

This function reports residual of syevj or sygvj. It does not support syevjBatched. If the user calls this function after syevjBatched, the error CUSOLVER_STATUS_NOT_SUPPORTED is returned.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
info	host	input	The pointer to the structure of syevj.
residual	host	output	Residual of syevj.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_SUPPORTED

Does not support batched version.

8.4.1.31 cusolverDnXsyevjGetSweeps()

```
cusolverStatus_t  
cusolverDnXsyevjGetSweeps(  
    cusolverDnHandle_t handle,  
    syevjInfo_t info,  
    int *executed_sweeps)
```

This function reports number of executed sweeps of syevj or sygvj. It does not support syevjBatched. If the user calls this function after syevjBatched, the error CUSOLVER_STATUS_NOT_SUPPORTED is returned.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
info	host	input	The pointer to the structure of syevj.
executed_sweeps	host	output	Number of executed sweeps.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_SUPPORTED

Does not support batched version.

8.4.1.32 cusolverDnCreateGesvdjInfo()

```
cusolverStatus_t  
cusolverDnCreateGesvdjInfo(  
    gesvdjInfo_t *info);
```

This function creates and initializes the structure of gesvdj and gesvdjBatched to default values.

Parameter	Memory	In/out	Meaning
info	host	output	The pointer to the structure of gesvdj.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The structure was initialized successfully.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

8.4.1.33 cusolverDnDestroyGesvdjInfo()

```
cusolverStatus_t
cusolverDnDestroyGesvdjInfo(
    gesvdjInfo_t info);
```

This function destroys and releases any memory required by the structure.

Parameter	Memory	In/out	Meaning
info	host	input	The structure of gesvdj.

Status Returned

CUSOLVER_STATUS_SUCCESS

The resources were released successfully.

8.4.1.34 cusolverDnXgesvdjSetTolerance()

```
cusolverStatus_t
cusolverDnXgesvdjSetTolerance(
    gesvdjInfo_t info,
    double tolerance)
```

This function configures tolerance of gesvdj.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of gesvdj.
tolerance	host	input	Accuracy of numerical singular values.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

8.4.1.35 cusolverDnXgesvdjSetMaxSweeps()

```
cusolverStatus_t
cusolverDnXgesvdjSetMaxSweeps(
    gesvdjInfo_t info,
    int max_sweeps)
```

This function configures the maximum number of sweeps in gesvdj. The default value is 100.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of gesvdj.
max_sweeps	host	input	Maximum number of sweeps.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

8.4.1.36 cusolverDnXgesvdjSetSortEig()

```
cusolverStatus_t
cusolverDnXgesvdjSetSortEig(
    gesvdjInfo_t info,
    int sort_svd)
```

If sort_svd is zero, the singular values are not sorted. This function only works for gesvdjBatched. gesvdj always sorts singular values in descending order. By default, singular values are always sorted in descending order.

Parameter	Memory	In/out	Meaning
info	host	in/out	The pointer to the structure of gesvdj.
sort_svd	host	input	If sort_svd is zero, the singular values are not sorted.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

8.4.1.37 cusolverDnXgesvdjGetResidual()

```
cusolverStatus_t
cusolverDnXgesvdjGetResidual(
    cusolverDnHandle_t handle,
    gesvdjInfo_t info,
    double *residual)
```

This function reports the Frobenius norm of the internal residual returned by gesvdj. Note that this is *not* the Frobenius norm of the exact residual calculated as

$$\|S - U^H * A * V\|_F$$

This function does not support gesvdjBatched. If the user calls this function after gesvdjBatched, the error CUSOLVER_STATUS_NOT_SUPPORTED is returned.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
info	host	input	The pointer to the structure of gesvdj.
residual	host	output	Residual of gesvdj.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_SUPPORTED

Does not support batched version

8.4.1.38 cusolverDnXgesvdjGetSweeps()

```
cusolverStatus_t
cusolverDnXgesvdjGetSweeps(
    cusolverDnHandle_t handle,
    gesvdjInfo_t info,
    int *executed_sweeps)
```

This function reports number of executed sweeps of gesvdj. It does not support gesvdjBatched. If the user calls this function after gesvdjBatched, the error CUSOLVER_STATUS_NOT_SUPPORTED is returned.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
info	host	input	The pointer to the structure of gesvdj.
executed_sweeps	host	output	Number of executed sweeps.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_SUPPORTED

Does not support batched version

8.4.1.39 cusolverDnIRSPParamsCreate()

```
cusolverStatus_t
cusolverDnIRSPParamsCreate(cusolverDnIRSPParams_t *params);
```

This function creates and initializes the structure of parameters for an IRS solver such as the `cusolverDnIRSXgesv()` or the `cusolverDnIRSXgels()` functions to default values. The params structure created by this function can be used by one or more call to the same or to a different IRS solver. Note that in CUDA 10.2, the behavior was different and a new params structure was needed to be created per each call to an IRS solver. Also note that the user can also change configurations of the params and then call a new IRS instance, but be careful that the previous call was done because any change to the configuration before the previous call was done could affect it.

Parameter	Memory	In/out	Meaning
params	host	output	Pointer to the cusolverDnIRSPParams_t Params structure

Status Returned**CUSOLVER_STATUS_SUCCESS**

The structure was created and initialized successfully.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

8.4.1.40 cusolverDnIRSParamsDestroy()

```
cusolverStatus_t
cusolverDnIRSParamsDestroy(cusolverDnIRSParams_t params);
```

This function destroys and releases any memory required by the Params structure.

Parameter	Memory	In/out	Meaning
params	host	input	The cusolverDnIRSParams_t Params structure.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The resources were released successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

CUSOLVER_STATUS_IRS_INFOS_NOT_DESTROYED

Not all the Infos structure associated with this Params structure have been destroyed yet.

8.4.1.41 cusolverDnIRSParamsSetSolverPrecisions()

```
cusolverStatus_t
cusolverDnIRSParamsSetSolverPrecisions(
    cusolverDnIRSParams_t params,
    cusolverPrecType_t solver_main_precision,
    cusolverPrecType_t solver_lowest_precision );
```

This function sets both the main and the lowest precision for the Iterative Refinement Solver (IRS). By main precision, we mean the precision of the Input and Output datatype. By lowest precision, we mean the solver is allowed to use as lowest computational precision during the LU factorization process. Note that the user has to set both the main and lowest precision before the first call to the IRS solver because they are NOT set by default with the params structure creation, as it depends on the Input Output data type and user request. It is a wrapper to both `cusolverDnIRSParamsSetSolverMainPrecision()` and `cusolverDnIRSParamsSetSolverLowestPrecision()`. All possible combinations of main/lowest precision are described in the table below. Usually the lowest precision defines the speedup that can be achieved. The ratio of the performance of the lowest precision over the main precision (e.g., Inputs/Outputs datatype) define the upper bound of the speedup that could be obtained. More precisely, it depends on many factors, but for large matrices sizes, it is the ratio of the matrix-matrix rank-k product (e.g., GEMM where K is 256 and M=N=size of the matrix) that define the possible speedup. For instance, if the inout precision is real double precision CUSOLVER_R_64F and the lowest precision is CUSOLVER_R_32F, then we can expect a speedup of at most 2X for large problem sizes. If the lowest precision was CUSOLVER_R_16F, then we can expect 3X-4X. A reasonable strategy should take the number of right-hand sides, the size of the matrix as well as the convergence rate into account.

Parameter	Memory	In/out	Meaning
params	host	in/out	The cusolverDnIRSParams_t Params structure.
solver_main_precision	host	input	Allowed Inputs/Outputs datatype (for example CUSOLVER_R_FP64 for a real double precision data). See the table below for the supported precisions.
solver_lowest_precision	host	input	Allowed lowest compute type (for example CUSOLVER_R_16F for half precision computation). See the table below for the supported precisions.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

Table 1: **Supported Inputs/Outputs data type and lower precision for the IRS solver**

Inputs/Outputs Data Type (e.g., main precision)	Supported values for the lowest precision
CUSOLVER_C_64F	CUSOLVER_C_64F, CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_C_32F	CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_R_64F	CUSOLVER_R_64F, CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32
CUSOLVER_R_32F	CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32

8.4.1.42 cusolverDnIRSParamsSetSolverMainPrecision()

```
cusolverStatus_t
cusolverDnIRSParamsSetSolverMainPrecision(
    cusolverDnIRSParams_t params,
    cusolverPrecType_t solver_main_precision);
```

This function sets the main precision for the Iterative Refinement Solver (IRS). By main precision, we mean, the type of the Input and Output data. Note that the user has to set both the main and lowest precision before a first call to the IRS solver because they are NOT set by default with the params structure creation, as it depends on the Input Output data type and user request. user can set it by either calling this function or by calling `cusolverDnIRSParamsSetSolverPrecisions()` which set both the main and the lowest precision together. All possible combinations of main/lowest precision are described in the [table](#) in the `cusolverDnIRSParamsSetSolverPrecisions()` section above.

Parameter	Mem-ory	In/out	Meaning
params	host	in/out	The cusolverDnIRSPParams_t Params structure.
solver_main_precision	host	input	Allowed Inputs/Outputs datatype (for example CUSOLVER_R_FP64 for a real double precision data). See the table in the cusolverDnIRSPParamsSetSolverPrecisions() section above for the supported precisions.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.43 cusolverDnIRSPParamsSetSolverLowestPrecision()

```
cusolverStatus_t
cusolverDnIRSPParamsSetSolverLowestPrecision(
    cusolverDnIRSPParams_t params,
    cusolverPrecType_t lowest_precision_type);
```

This function sets the lowest precision that will be used by Iterative Refinement Solver. By lowest precision, we mean the solver is allowed to use as lowest computational precision during the LU factorization process. Note that the user has to set both the main and lowest precision before a first call to the IRS solver because they are NOT set by default with the params structure creation, as it depends on the Input Output data type and user request. Usually the lowest precision defines the speedup that can be achieved. The ratio of the performance of the lowest precision over the main precision (e.g., Inputs/Outputs datatype) define somehow the upper bound of the speedup that could be obtained. More precisely, it depends on many factors, but for large matrices sizes, it is the ratio of the matrix-matrix rank-k product (e.g., GEMM where K is 256 and M=N=size of the matrix) that define the possible speedup. For instance, if the inout precision is real double precision CUSOLVER_R_64F and the lowest precision is CUSOLVER_R_32F, then we can expect a speedup of at most 2X for large problem sizes. If the lowest precision was CUSOLVER_R_16F, then we can expect 3X-4X. A reasonable strategy should take the number of right-hand sides, the size of the matrix as well as the convergence rate into account.

Parameter	Mem-ory	In/out	Meaning
params	host	in/out	The cusolverDnIRSPParams_t Params structure.
lowest_precision_type	host	input	Allowed lowest compute type (for example CUSOLVER_R_16F for half precision computation). See the table in the cusolverDnIRSPParamsSetSolverPrecisions() section above for the supported precisions.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.44 cusolverDnIRSParamsSetRefinementSolver()

```
cusolverStatus_t
cusolverDnIRSParamsSetRefinementSolver(
    cusolverDnIRSParams_t params,
    cusolverIRSRefinement_t solver);
```

This function sets the refinement solver to be used in the Iterative Refinement Solver functions such as the `cusolverDnIRSXgesv()` or the `cusolverDnIRSXgels()` functions. Note that the user has to set the refinement algorithm before a first call to the IRS solver because it is NOT set by default with the creating of params. Details about values that can be set to and theirs meaning are described in the table below.

Parameter	Memory	In/out	Meaning
params	host	in/out	The <code>cusolverDnIRSParams_tParams</code> structure
solver	host	input	Type of the refinement solver to be used by the IRS solver such as <code>cusolverDnIRSXgesv()</code> or <code>cusolverDnIRSXgels()</code> .

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

CUSOLVER_IRS_REFINE_NOT_SET

Solver is not set, this value is what is set when creating the params structure. IRS solver will return an error.

CUSOLVER_IRS_REFINE_NONE

No refinement solver; the IRS solver performs a factorization followed by a solve without any refinement. For example, if the IRS solver was `cusolverDnIRSXgesv()`, this is equivalent to a Xgesv routine without refinement and where the factorization is carried out in the lowest precision. If for example the main precision was `CUSOLVER_R_64F` and the lowest was `CUSOLVER_R_64F` as well, then this is equivalent to a call to `cusolverDnDgesv()`.

CUSOLVER_IRS_REFINE_CLASSICAL

Classical iterative refinement solver. Similar to the one used in LAPACK routines.

CUSOLVER_IRS_REFINE_GMRES

GMRES (Generalized Minimal Residual) based iterative refinement solver. In recent study, the GMRES method has drawn the scientific community attention for its ability to be used as refinement solver that outperforms the classical iterative refinement method. Based on our experimentation, we recommend this setting.

CUSOLVER_IRS_REFINE_CLASSICAL_GMRES

Classical iterative refinement solver that uses the GMRES (Generalized Minimal Residual) internally to solve the correction equation at each iteration. We call the *classical refinement iteration* the outer iteration while the *GMRES* is called inner iteration. Note that if the tolerance of the inner GMRES is set very low, let say to machine precision, then the outer *classical refinement iteration* will performs only one iteration and thus this option will behaves like CUSOLVER_IRS_REFINE_GMRES.

CUSOLVER_IRS_REFINE_GMRES_GMRES

Similar to CUSOLVER_IRS_REFINE_CLASSICAL_GMRES which consists of classical refinement process that uses GMRES to solve the inner correction system, here it is a GMRES (Generalized Minimal Residual) based iterative refinement solver that uses another GMRES internally to solve the preconditioned system.

8.4.1.45 cusolverDnIRSParamsSetTol()

```
cusolverStatus_t
cusolverDnIRSParamsSetTol(
    cusolverDnIRSParams_t params,
    double val );
```

This function sets the tolerance for the refinement solver. By default it is such that all the RHS satisfy:

$RNRM < \sqrt{N} * XNRM * ANRM * EPS * BWDMAX$ where

- RNRM is the infinity-norm of the residual
- XNRM is the infinity-norm of the solution
- ANRM is the infinity-operator-norm of the matrix A
- EPS is the machine epsilon for the Inputs/Outputs datatype that matches LAPACK <X>LAMCH('Epsilon')
- BWDMAX, the value BWDMAX is fixed to 1.0

The user can use this function to change the tolerance to a lower or higher value. Our goal is to give the user more control such a way he can investigate and control every detail of the IRS solver. Note that the tolerance value is always in *real double precision* whatever the Inputs/Outputs datatype is.

Parameter	Memory	In/out	Meaning
params	host	in/out	The cusolverDnIRSParams_t Params structure.
val	host	input	Double precision real value to which the refinement tolerance will be set.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.46 cusolverDnIRSParamsSetTolInner()

```
cusolverStatus_t
cusolverDnIRSParamsSetTolInner(
    cusolverDnIRSParams_t params,
    double val );
```

This function sets the tolerance for the inner refinement solver when the refinement solver consists of two-levels solver (for example, CUSOLVER_IRS_REFINE_CLASSICAL_GMRES or CUSOLVER_IRS_REFINE_GMRES_GMRES cases). It is not referenced in case of one level refinement solver such as CUSOLVER_IRS_REFINE_CLASSICAL or CUSOLVER_IRS_REFINE_GMRES. It is set to 1e-4 by default. This function set the tolerance for the inner solver (e.g. the inner GMRES). For example, if the Refinement Solver was set to CUSOLVER_IRS_REFINE_CLASSICAL_GMRES, setting this tolerance mean that the inner GMRES solver will converge to that tolerance at each outer iteration of the classical refinement solver. Our goal is to give the user more control such a way he can investigate and control every detail of the IRS solver. Note the, the tolerance value is always in *real double precision* whatever the Inputs/Outputs datatype is.

Parameter	Memory	In/out	Meaning
params	host	in/out	The cusolverDnIRSParams_t Params structure.
val	host	input	Double precision real value to which the tolerance of the inner refinement solver will be set.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.47 cusolverDnIRSParamsSetMaxIters()

```
cusolverStatus_t
cusolverDnIRSParamsSetMaxIters(
    cusolverDnIRSParams_t params,
    int max_iters);
```

This function sets the total number of allowed refinement iterations after which the solver will stop. Total means any iteration which means the sum of the outer and the inner iterations (inner is meaningful when two-levels refinement solver is set). Default value is set to 50. Our goal is to give the user more control such a way he can investigate and control every detail of the IRS solver.

Parameter	Memory	In/out	Meaning
params	host	in/out	The cusolverDnIRSParams_t Params structure.
max_iters	host	input	Maximum total number of iterations allowed for the refinement solver.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.48 cusolverDnIRSParamsSetMaxItersInner()

```
cusolverStatus_t  
    cusolverDnIRSParamsSetMaxItersInner(  
        cusolverDnIRSParams_t params,  
        cusolver_int_t maxiters_inner );
```

This function sets the maximal number of iterations allowed for the inner refinement solver. It is not referenced in case of one level refinement solver such as CUSOLVER_IRS_REFINE_CLASSICAL or CUSOLVER_IRS_REFINE_GMRES. The inner refinement solver will stop after reaching either the inner tolerance or the MaxItersInner value. By default, it is set to 50. Note that this value could not be larger than the MaxIters since MaxIters is the total number of allowed iterations. Note that if the user calls cusolverDnIRSParamsSetMaxIters after calling this function, SetMaxIters has priority and will overwrite MaxItersInner to the minimum value of (MaxIters, MaxItersInner).

Parameter	Mem-ory	In/out	Meaning
params	host	in/ out	The cusolverDnIRSParams_t Params structure
max- iters_inner	host	in- put	Maximum number of allowed inner iterations for the inner refinement solver. Meaningful when the refinement solver is a two-levels solver such as CUSOLVER_IRS_REFINE_CLASSICAL_GMRES or CUSOLVER_IRS_REFINE_GMRES_GMRES. Value should be less or equal to MaxIters.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

CUSOLVER_STATUS_IRS_PARAMS_INVALID

If the value was larger than MaxIters.

8.4.1.49 cusolverDnIRSParamsEnableFallback()

```
cusolverStatus_t
cusolverDnIRSParamsEnableFallback(
    cusolverDnIRSParams_t params );
```

This function enable the fallback to the main precision in case the Iterative Refinement Solver (IRS) failed to converge. In other term, if the IRS solver failed to converge, the solver will return a no convergence code (e.g., `niter < 0`), but can either return the non-convergent solution as it is (e.g., disable fallback) or can fallback (e.g., enable fallback) to the main precision (which is the precision of the Inputs/Outputs data) and solve the problem from scratch returning the good solution. This is the behavior by default, and it will guarantee that the IRS solver always provide the good solution. This function is provided because we provided `cusolverDnIRSParamsDisableFallback` which allows the user to disable the fallback and thus this function allow the user to re-enable it.

Parameter	Memory	In/out	Meaning
params	host	in/out	The <code>cusolverDnIRSParams_t</code> Params structure

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.50 cusolverDnIRSParamsDisableFallback()

```
cusolverStatus_t
cusolverDnIRSParamsDisableFallback(
    cusolverDnIRSParams_t params );
```

This function disables the fallback to the main precision in case the Iterative Refinement Solver (IRS) failed to converge. In other term, if the IRS solver failed to converge, the solver will return a no convergence code (e.g., `niter < 0`), but can either return the non-convergent solution as it is (e.g., disable fallback) or can fallback (e.g., enable fallback) to the main precision (which is the precision of the Inputs/Outputs data) and solve the problem from scratch returning the good solution. This function disables the fallback and the returned solution is whatever the refinement solver was able to reach before it returns. Disabling fallback does not guarantee that the solution is the good one. However, if users want to keep getting the solution of the lower precision in case the IRS did not converge after certain number of iterations, they need to disable the fallback. The user can re-enable it by calling `cusolverDnIRSParamsEnableFallback`.

Parameter	Memory	In/out	Meaning
params	host	in/out	The <code>cusolverDnIRSParams_t</code> Params structure

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.51 cusolverDnIRSParmsGetMaxIters()

```
cusolverStatus_t
cusolverDnIRSParmsGetMaxIters(
    cusolverDnIRSParms_t params,
    cusolver_int_t *maxiters );
```

This function returns the current setting in the params structure for the maximal allowed number of iterations (for example, either the default MaxIters, or the one set by the user in case he set it using cusolverDnIRSParmsSetMaxIters). Note that this function returns the current setting in the params configuration and not to be confused with the cusolverDnIRSInfosGetMaxIters which return the maximal allowed number of iterations for a particular call to an IRS solver. To be clearer, the params structure can be used for many calls to an IRS solver. A user can change the allowed MaxIters between calls while the Infos structure in cusolverDnIRSInfosGetMaxIters contains information about a particular call and cannot be reused for different calls, and thus, cusolverDnIRSInfosGetMaxIters returns the allowed MaxIters for that call.

Parameter	Memory	In/out	Meaning
params	host	in	The cusolverDnIRSParms_t Params structure.
maxiters	host	output	The maximal number of iterations that is currently set.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The Params structure was not created.

8.4.1.52 cusolverDnIRSInfosCreate()

```
cusolverStatus_t
cusolverDnIRSInfosCreate(
    cusolverDnIRSInfos_t* infos )
```

This function creates and initializes the Infos structure that will hold the refinement information of an Iterative Refinement Solver (IRS) call. Such information includes the total number of iterations that was needed to converge (Niters), the outer number of iterations (meaningful when two-levels preconditioner such as CUSOLVER_IRS_REFINE_CLASSICAL_GMRES is used), the maximal number of iterations that was allowed for that call, and a pointer to the matrix of the convergence history residual norms. The Infos structure needs to be created before a call to an IRS solver. The Infos structure is valid for only one call to an IRS solver, since it holds info about that solve and thus each solve will requires its own Infos structure.

Parameter	Memory	In/out	Meaning
info	host	output	Pointer to the cusolverDnIRSInfos_t Infos structure.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The structure was initialized successfully.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

8.4.1.53 cusolverDnIRSInfosDestroy()

```
cusolverStatus_t
cusolverDnIRSInfosDestroy(
    cusolverDnIRSInfos_t infos );
```

This function destroys and releases any memory required by the Infos structure. This function destroys all the information (for example, Niters performed, OuterNiters performed, residual history etc.) about a solver call; thus, this function should only be called after the user is finished with the information.

Parameter	Memory	In/out	Meaning
infos	host	in/out	The cusolverDnIRSInfos_t Infos structure.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The resources were released successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created.

8.4.1.54 cusolverDnIRSInfosGetMaxIters()

```
cusolverStatus_t
cusolverDnIRSInfosGetMaxIters(
    cusolverDnIRSInfos_t infos,
    cusolver_int_t *maxiters );
```

This function returns the maximal allowed number of iterations that was set for the corresponding call to the IRS solver. Note that this function returns the setting that was set when that call happened and is not to be confused with the `cusolverDnIRSParamsGetMaxIters` which returns the current setting in the params configuration structure. To be clearer, the params structure can be used for many calls to an IRS solver. A user can change the allowed MaxIters between calls while the Infos structure in `cusolverDnIRSInfosGetMaxIters` contains information about a particular call and cannot be reused for different calls, thus `cusolverDnIRSInfosGetMaxIters` returns the allowed MaxIters for that call.

Parameter	Memory	In/out	Meaning
infos	host	in	The cusolverDnIRSInfos_t Infos structure.
maxiters	host	output	The maximal number of iterations that is currently set.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created.

8.4.1.55 cusolverDnIRSInfosGetNiters()

```
cusolverStatus_t cusolverDnIRSInfosGetNiters(
    cusolverDnIRSInfos_t infos,
    cusolver_int_t *niters );
```

This function returns the total number of iterations performed by the IRS solver. If it was negative, it means that the IRS solver did not converge and if the user did not disable the fallback to full precision, then the fallback to a full precision solution happened and solution is good. Please refer to the description of negative niters values in the corresponding IRS linear solver functions such as `cusolverDnXgesv()` or `cusolverDnXgels()`.

Parameter	Memory	In/out	Meaning
infos	host	in	The <code>cusolverDnIRSInfos_t</code> Infos structure.
niters	host	output	The total number of iterations performed by the IRS solver.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created.

8.4.1.56 cusolverDnIRSInfosGetOuterNiters()

```
cusolverStatus_t
    cusolverDnIRSInfosGetOuterNiters(
        cusolverDnIRSInfos_t infos,
        cusolver_int_t *outer_niters );
```

This function returns the number of iterations performed by the outer refinement loop of the IRS solver. When the refinement solver consists of a one-level solver such as `CUSOLVER_IRS_REFINE_CLASSICAL` or `CUSOLVER_IRS_REFINE_GMRES`, it is the same as `Niters`. When the refinement solver consists of a two-levels solver such as `CUSOLVER_IRS_REFINE_CLASSICAL_GMRES` or `CUSOLVER_IRS_REFINE_GMRES_GMRES`, it is the number of iterations of the outer loop. Refer to the description of the [*cusolverIRSRefinement_t*](#) for more details.

Parameter	Memory	In/out	Meaning
infos	host	in	The <code>cusolverDnIRSInfos_t</code> Infos structure.
outer_niters	host	output	The number of iterations of the outer refinement loop of the IRS solver.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created.

8.4.1.57 cusolverDnIRSInfosRequestResidual()

```
cusolverStatus_t cusolverDnIRSInfosRequestResidual(
    cusolverDnIRSInfos_t infos );
```

This function tells the IRS solver to store the convergence history (residual norms) of the refinement phase in a matrix that can be accessed via a pointer returned by the `cusolverDnIRSInfosGetResidualHistory()` function.

Parameter	Memory	In/out	Meaning
infos	host	in	The <code>cusolverDnIRSInfos_t</code> Infos structure

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created.

8.4.1.58 cusolverDnIRSInfosGetResidualHistory()

```
cusolverStatus_t
cusolverDnIRSInfosGetResidualHistory(
    cusolverDnIRSInfos_t infos,
    void **residual_history );
```

If the user called `cusolverDnIRSInfosRequestResidual()` before the call to the IRS function, then the IRS solver will store the convergence history (residual norms) of the refinement phase in a matrix that can be accessed via a pointer returned by this function. The datatype of the residual norms depends on the input and output data type. If the Inputs/Outputs datatype is double precision real or complex (CUSOLVER_R_FP64 or CUSOLVER_C_FP64), this residual will be of type real double precision (FP64) *double*, otherwise if the Inputs/Outputs datatype is single precision real or complex (CUSOLVER_R_FP32 or CUSOLVER_C_FP32), this residual will be real single precision FP32 *float*.

The residual history matrix consists of two columns (even for the multiple right-hand side case NRHS) of `MaxIters+1` row, thus a matrix of size `(MaxIters+1, 2)`. Only the first `OuterNits+1` rows contains the residual norms the other (e.g., `OuterNits+2:Maxiters+1`) are garbage. On the first column, each row “*i*” specify the total number of iterations happened till this outer iteration “*i*” and on the second columns the residual norm corresponding to this outer iteration “*i*”. Thus, the first row (e.g., outer iteration “0”) consists of the initial residual (e.g., the residual before the refinement loop start) then the consecutive rows are the residual obtained at each outer iteration of the refinement loop. Note, it only consists of the history of the outer loop.

If the refinement solver was CUSOLVER_IRS_REFINE_CLASSICAL or CUSOLVER_IRS_REFINE_GMRES, then OuterNiters=Niters (Niters is the total number of iterations performed) and there is Niters+1 rows of norms that correspond to the Niters outer iterations.

If the refinement solver was CUSOLVER_IRS_REFINE_CLASSICAL_GMRES or CUSOLVER_IRS_REFINE_GMRES_GMRES, then OuterNiters <= Niters corresponds to the outer iterations performed by the outer refinement loop. Thus, there is OuterNiters+1 residual norms where row “i” correspond to the outer iteration “i” and the first column specify the total number of iterations (outer and inner) that were performed till this step the second columns correspond to the residual norm at this step.

For example, let’s say the user specifies CUSOLVER_IRS_REFINE_CLASSICAL_GMRES as a refinement solver and say it needed 3 outer iterations to converge and 4,3,3 inner iterations at each outer, respectively. This consists of 10 total iterations. Row 0 corresponds to the first residual before the refinement start, so it has 0 in its first column. On row 1 which corresponds to the outer iteration 1, it will be 4 (4 is the total number of iterations that were performed till now), on row 2 it will be 7, and on row 3 it will be 10.

In summary, let’s define `ldh=Maxiters+1`, the leading dimension of the residual matrix. then `residual_history[i]` shows the total number of iterations performed at the outer iteration “i” and `residual_history[i+ldh]` corresponds to the norm of the residual at this outer iteration.

Parameter	Memory	In/out	Meaning
infos	host	in	The <code>cusolverDnIRSInfos_t</code> Infos structure.
residual_history	host	output	Returns a void pointer to the matrix of the convergence history residual norms. See the description above for the relation between the residual norm datatype and the inout datatype.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The Infos structure was not created

CUSOLVER_STATUS_INVALID_VALUE

This function was called without calling `cusolverDnIRSInfosRequestResidual()` in advance.

8.4.1.59 cusolverDnCreateParams()

```
cusolverStatus_t
cusolverDnCreateParams(
    cusolverDnParams_t *params);
```

This function creates and initializes the structure of 64-bit API to default values.

Parameter	Memory	In/out	Meaning
params	host	output	The pointer to the structure of 64-bit API.

Status Returned

CUSOLVER_STATUS_SUCCESS

The structure was initialized successfully.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

8.4.1.60 cusolverDnDestroyParams()

```
cusolverStatus_t
cusolverDnDestroyParams(
    cusolverDnParams_t params);
```

This function destroys and releases any memory required by the structure.

Parameter	Memory	In/out	Meaning
params	host	input	The structure of 64-bit API.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The resources were released successfully.

8.4.1.61 cusolverDnSetAdvOptions()

```
cusolverStatus_t
cusolverDnSetAdvOptions (
    cusolverDnParams_t params,
    cusolverDnFunction_t function,
    cusolverAlgMode_t algo );
```

This function configures algorithm algo of function, a 64-bit API routine.

Parameter	Memory	In/out	Meaning
params	host	in/out	The pointer to the structure of 64-bit API.
function	host	input	The routine to be configured.
algo	host	input	The algorithm to be configured.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

Wrong combination of function and algo.

8.4.2. Dense Linear Solver Reference (legacy)

This section describes linear solver API of cuSolverDN, including Cholesky factorization, LU with partial pivoting, QR factorization and Bunch-Kaufman (LDLT) factorization.

8.4.2.1 cusolverDn<t>potrf()

These helper functions calculate the necessary size of work buffers.

```
cusolverStatus_t
cusolverDnSpotrf_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           float *A,
                           int lda,
                           int *Lwork );

cusolverStatus_t
cusolverDnDpotrf_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           double *A,
                           int lda,
                           int *Lwork );

cusolverStatus_t
cusolverDnCpotrf_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           cuComplex *A,
                           int lda,
                           int *Lwork );

cusolverStatus_t
cusolverDnZpotrf_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           cuDoubleComplex *A,
                           int lda,
                           int *Lwork);
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSpotrf(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 float *A,
                 int lda,
                 float *Workspace,
                 int Lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnDpotrf(cusolverDnHandle_t handle,
```

(continues on next page)

(continued from previous page)

```

cublasFillMode_t uplo,
int n,
double *A,
int lda,
double *Workspace,
int Lwork,
int *devInfo );

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCpotrf(cusolverDnHandle_t handle,
cublasFillMode_t uplo,
int n,
cuComplex *A,
int lda,
cuComplex *Workspace,
int Lwork,
int *devInfo );

cusolverStatus_t
cusolverDnZpotrf(cusolverDnHandle_t handle,
cublasFillMode_t uplo,
int n,
cuDoubleComplex *A,
int lda,
cuDoubleComplex *Workspace,
int Lwork,
int *devInfo );

```

This function computes the Cholesky factorization of a Hermitian positive-definite matrix.

A is an $n \times n$ Hermitian matrix, only the lower or upper part is meaningful. The input parameter uplo indicates which part of the matrix is used. The function will leave the other part untouched.

If input parameter uplo is CUBLAS_FILL_MODE_LOWER, only the lower triangular part of A is processed, and replaced by the lower triangular Cholesky factor L.

$$A = L * L^H$$

If input parameter uplo is CUBLAS_FILL_MODE_UPPER, only upper triangular part of A is processed, and replaced by upper triangular Cholesky factor U.

$$A = U^H * U$$

The user has to provide working space which is pointed by input parameter Workspace. The input parameter Lwork is size of the working space, and it is returned by potrf_bufferSize().

If Cholesky factorization failed, i.e. some leading minor of A is not positive definite, or equivalently some diagonal elements of L or U is not a real number. The output parameter devInfo would indicate smallest leading minor of A which is not positive definite.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

Table 2: API of potrf

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored; the other part is not referenced.
n	host	input	Number of rows and columns of matrix A.
A	device	in/out	<type> array of dimension lda * n with lda is not less than max(1, n).
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
Workspace	device	in/out	Working space, <type> array of size Lwork.
Lwork	host	input	Size of Workspace, returned by potrf_bufferSize.
devInfo	device	output	If devInfo = 0, the Cholesky factorization is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle). if devInfo = i, the leading minor of order i is not positive definite.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.2 cusolverDn<t>potrs()

```
cusolverStatus_t
cusolverDnSpotrs(cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    const float *A,
    int lda,
    float *B,
    int ldb,
    int *devInfo);

cusolverStatus_t
```

(continues on next page)

(continued from previous page)

```

cusolverDnDpotrs(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 int nrhs,
                 const double *A,
                 int lda,
                 double *B,
                 int ldb,
                 int *devInfo);

cusolverStatus_t
cusolverDnCpotrs(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 int nrhs,
                 const cuComplex *A,
                 int lda,
                 cuComplex *B,
                 int ldb,
                 int *devInfo);

cusolverStatus_t
cusolverDnZpotrs(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 int nrhs,
                 const cuDoubleComplex *A,
                 int lda,
                 cuDoubleComplex *B,
                 int ldb,
                 int *devInfo);

```

This function solves a system of linear equations

$$A * X = B$$

where A is an $n \times n$ Hermitian matrix, only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used. The function will leave the other part untouched.

The user has to call `potrf` first to factorize matrix A. If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, A is lower triangular Cholesky factor L corresponding to $A = L * L^H$. If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, A is upper triangular Cholesky factor U corresponding to $A = U^H * U$.

The operation is in-place, i.e. matrix X overwrites matrix B with the same leading dimension `ldb`.

If output parameter `devInfo` = -i (less than zero), the i-th parameter is wrong (not counting handle).

Table 3: **API of potrs**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.

continues on next page

Table 3 – continued from previous page

Parameter	Memory	In/out	Meaning
n	host	input	Number of rows and columns of matrix A.
nrhs	host	input	Number of columns of matrix X and B.
A	device	input	<type> array of dimension lda * n with lda is not less than max(1,n). A is either lower Cholesky factor L or upper Cholesky factor U.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
B	device	in/out	<type> array of dimension ldb * nrhs. ldb is not less than max(1,n). As an input, B is right hand side matrix. As an output, B is the solution matrix.
devInfo	device	output	If devInfo = 0, the Cholesky factorization is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n<0, nrhs<0, lda<max(1,n) or ldb<max(1,n)).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.3 cusolverDn<t>potri()

These helper functions calculate the necessary size of work buffers.

```

cusolverStatus_t
cusolverDnSpotri_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           float *A,
                           int lda,
                           int *Lwork );

cusolverStatus_t
cusolverDnDpotri_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           double *A,
                           int lda,

```

(continues on next page)

(continued from previous page)

```

        int *Lwork );

cusolverStatus_t
cusolverDnCpotri_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           cuComplex *A,
                           int lda,
                           int *Lwork );

cusolverStatus_t
cusolverDnZpotri_bufferSize(cusolverDnHandle_t handle,
                           cublasFillMode_t uplo,
                           int n,
                           cuDoubleComplex *A,
                           int lda,
                           int *Lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSpotri(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 float *A,
                 int lda,
                 float *Workspace,
                 int Lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnDpotri(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 double *A,
                 int lda,
                 double *Workspace,
                 int Lwork,
                 int *devInfo );

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCpotri(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 cuComplex *A,
                 int lda,
                 cuComplex *Workspace,
                 int Lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnZpotri(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 cuDoubleComplex *A,

```

(continues on next page)

(continued from previous page)

```

int lda,
cuDoubleComplex *Workspace,
int Lwork,
int *devInfo );

```

This function computes the inverse of a positive-definite matrix A using the Cholesky factorization

$$A = L * L^H = U^H * U$$

computed by `potrf()`.

A is an $n \times n$ matrix containing the triangular factor L or U computed by the Cholesky factorization. Only lower or upper part is meaningful and the input parameter `uplo` indicates which part of the matrix is used. The function would leave the other part untouched.

If the input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, only lower triangular part of A is processed, and replaced the by lower triangular part of the inverse of A.

If the input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, only upper triangular part of A is processed, and replaced by the upper triangular part of the inverse of A.

The user has to provide the working space which is pointed to by input parameter `Workspace`. The input parameter `Lwork` is the size of the working space, returned by `potri_bufferSize()`.

If the computation of the inverse fails, i.e. some leading minor of L or U, is null, the output parameter `devInfo` would indicate the smallest leading minor of L or U which is not positive definite.

If the output parameter `devInfo` = -i (less than zero), the i-th parameter is wrong (not counting the handle).

Table 4: **API of potri**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.
n	host	input	Number of rows and columns of matrix A.
A	device	in/out	<type> array of dimension <code>lda * n</code> where <code>lda</code> is not less than <code>max(1, n)</code> .
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
Workspace	device	in/out	Working space, <type> array of size <code>Lwork</code> .
Lwork	host	input	Size of Workspace, returned by <code>potri_bufferSize</code> .
devInfo	device	output	If <code>devInfo</code> = 0, the computation of the inverse is successful. if <code>devInfo</code> = -i, the i-th parameter is wrong (not counting handle). if <code>devInfo</code> = i, the leading minor of order i is zero.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.4 cusolverDn<t>getrf()

These helper functions calculate the size of work buffers needed.

Please visit [cuSOLVER Library Samples - getrf](#) for a code example.

```
cusolverStatus_t
cusolverDnSgetrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    float *A,
    int lda,
    int *Lwork );

cusolverStatus_t
cusolverDnDgetrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    double *A,
    int lda,
    int *Lwork );

cusolverStatus_t
cusolverDnCgetrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    cuComplex *A,
    int lda,
    int *Lwork );

cusolverStatus_t
cusolverDnZgetrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,
    int *Lwork );
```

The S and D data types are real single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSgetrf(cusolverDnHandle_t handle,
    int m,
```

(continues on next page)

(continued from previous page)

```

    int n,
    float *A,
    int lda,
    float *Workspace,
    int *devI piv,
    int *devInfo );

cusolverStatus_t
cusolverDnDgetrf(cusolverDnHandle_t handle,
    int m,
    int n,
    double *A,
    int lda,
    double *Workspace,
    int *devI piv,
    int *devInfo );

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCgetrf(cusolverDnHandle_t handle,
    int m,
    int n,
    cuComplex *A,
    int lda,
    cuComplex *Workspace,
    int *devI piv,
    int *devInfo );

cusolverStatus_t
cusolverDnZgetrf(cusolverDnHandle_t handle,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,
    cuDoubleComplex *Workspace,
    int *devI piv,
    int *devInfo );

```

This function computes the LU factorization of an $m \times n$ matrix

$$P * A = L * U$$

where A is an $m \times n$ matrix, P is a permutation matrix, L is a lower triangular matrix with unit diagonal, and U is an upper triangular matrix.

The user has to provide working space which is pointed by input parameter Workspace. The input parameter Lwork is size of the working space, and it is returned by `getrf_bufferSize()`.

If LU factorization failed, i.e. matrix A (U) is singular, The output parameter devInfo=i indicates $U(i, i) = 0$.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

If devI piv is null, no pivoting is performed. The factorization is $A=L*U$, which is not numerically stable.

No matter LU factorization failed or not, the output parameter devI piv contains pivoting sequence, row i is interchanged with row devI piv(i).

The user can combine `getrf` and `getrs` to complete a linear solver.

Remark: `getrf` uses fastest implementation with large workspace of size $m*n$. The user can choose the legacy implementation with minimal workspace by `Getrf` and `cusolverDnSetAdvOptions(params, CUSOLVERDN_GETRF, CUSOLVER_ALG_1)`.

Table 5: **API of `getrf`**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
A	device	in/out	<type> array of dimension $lda * n$ with lda is not less than $\max(1, m)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
Workspace	device	in/out	Working space, <type> array of size Lwork.
devI piv	device	output	Array of size at least $\min(m, n)$, containing pivot indices.
devInfo	device	output	If <code>devInfo = 0</code> , the LU factorization is successful. if <code>devInfo = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle). if <code>devInfo = i</code> , the $U(i, i) = 0$.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.5 `cusolverDn<t>getrs()`

Please visit [cuSOLVER Library Samples - `getrf`](#) for a code example.

```
cusolverStatus_t
cusolverDnSgetrs(cusolverDnHandle_t handle,
                 cublasOperation_t trans,
                 int n,
                 int nrhs,
                 const float *A,
```

(continues on next page)

(continued from previous page)

```

    int lda,
    const int *devIpiv,
    float *B,
    int ldb,
    int *devInfo );

cusolverStatus_t
cusolverDnDgetrs(cusolverDnHandle_t handle,
    cublasOperation_t trans,
    int n,
    int nrhs,
    const double *A,
    int lda,
    const int *devIpiv,
    double *B,
    int ldb,
    int *devInfo );

cusolverStatus_t
cusolverDnCgetrs(cusolverDnHandle_t handle,
    cublasOperation_t trans,
    int n,
    int nrhs,
    const cuComplex *A,
    int lda,
    const int *devIpiv,
    cuComplex *B,
    int ldb,
    int *devInfo );

cusolverStatus_t
cusolverDnZgetrs(cusolverDnHandle_t handle,
    cublasOperation_t trans,
    int n,
    int nrhs,
    const cuDoubleComplex *A,
    int lda,
    const int *devIpiv,
    cuDoubleComplex *B,
    int ldb,
    int *devInfo );

```

This function solves a linear system of multiple right-hand sides

$$op(A) * X = B$$

where A is an $n \times n$ matrix, and was LU-factored by `getrf`, that is, lower triangular part of A is L, and upper triangular part (including diagonal elements) of A is U. B is an $n \times nrhs$ right-hand side matrix.

The input parameter `trans` is defined by

$$op(A) = \begin{cases} A & \text{if } trans == \text{CUBLAS_OP_N} \\ A^T & \text{if } trans == \text{CUBLAS_OP_T} \\ A^H & \text{if } trans == \text{CUBLAS_OP_C} \end{cases}$$

The input parameter `devI piv` is an output of `getrf`. It contains pivot indices, which are used to permute right-hand sides.

If output parameter `devInfo = -i` (less than zero), the *i*-th parameter is wrong (not counting handle).

The user can combine `getrf` and `getrs` to complete a linear solver.

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
trans	host	input	Operation $op(A)$ that is non- or (conj.) transpose.
n	host	input	Number of rows and columns of matrix A.
nrhs	host	input	Number of right-hand sides.
A	device	input	<type> array of dimension $lda * n$ with lda is not less than $\max(1, n)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
devI piv	device	input	Array of size at least n, containing pivot indices.
B	device	output	<type> array of dimension $ldb * nrhs$ with ldb is not less than $\max(1, n)$.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B.
devInfo	device	output	If <code>devInfo = 0</code> , the operation is successful. if <code>devInfo = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle).

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$ or $ldb < \max(1, n)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.6 cusolverDn<t1><t2>gesv()

These functions are modelled after functions DSGESV and ZCGESV from LAPACK. They compute the solution of a system of linear equations with one or multiple right hand sides using mixed precision iterative refinement techniques based on the LU factorization Xgesv. These functions are similar in term of functionalities to the full precision LU solver (Xgesv, where X denotes Z,C,D,S) but it uses lower precision internally in order to provide faster time to solution, from here comes the name mixed precision. Mixed precision iterative refinement techniques means that the solver compute an LU factorization in lower precision and then iteratively refine the solution to achieve the accuracy of the Inputs/Outputs datatype precision. The <t1> corresponds to the Inputs/Outputs datatype precision while <t2> represent the internal lower precision at which the factorization will be carried on.

$$A \times X = B$$

Where A is n-by-n matrix and X and B are n-by-nrhs matrices.

Functions API are designed to be as close as possible to LAPACK API to be considered as a quick and easy drop-in replacement. Parameters and behavior are mostly the same as LAPACK counterparts. Description of these functions and differences from LAPACK is given below. <t1><t2>gesv() functions are designated by two floating point precisions The <t1> corresponds to the main precision (e.g., Inputs/Outputs datatype precision) and the <t2> represent the internal lower precision at which the factorization will be carried on. cusolver<t1><t2>gesv() first attempts to factorize the matrix in lower precision and use this factorization within an iterative refinement procedure to obtain a solution with same normwise backward error as the main precision <t1>. If the approach fails to converge, then the method fallback to the main precision factorization and solve (Xgesv) such a way that there is always a good solution at the output of these functions. If <t2> is equal to <t1>, then it is not a mixed precision process but rather a full one precision factorization, solve and refinement within the same main precision.

The iterative refinement process is stopped if

$$\text{ITER} > \text{ITERMAX}$$

or for all the RHS we have:

$$\text{RNRM} < \text{SQRT}(N) * \text{XNRM} * \text{ANRM} * \text{EPS} * \text{BWDMAX}$$

where

- ▶ ITER is the number of the current iteration in the iterative refinement process
- ▶ RNRM is the infinity-norm of the residual
- ▶ XNRM is the infinity-norm of the solution
- ▶ ANRM is the infinity-operator-norm of the matrix A
- ▶ EPS is the machine epsilon that matches LAPACK <t1>LAMCH('Epsilon')

The value ITERMAX and BWDMAX are fixed to 50 and 1.0 respectively.

The function returns value describes the results of the solving process. A CUSOLVER_STATUS_SUCCESS indicates that the function finished with success otherwise, it indicates if one of the API arguments is incorrect, or if the function did not finish with success. More details about the error will be in the niters and the dinfo API parameters. See their description below for more details. User should provide the required workspace allocated on device memory. The amount of bytes required can be queried by calling the respective function <t1><t2>gesv_bufferSize().

Note that in addition to the two mixed precision functions available in LAPACK (for example, dsgesv and zcgsv), we provide a large set of mixed precision functions that include half, bfloat and tensorfloat as a lower precision as well as same precision functions (, main and lowest precision are equal

<t2> is equal to <t1>). The following table specifies which precisions will be used for which interface function.

Tensor Float (TF32), introduced with NVIDIA Ampere Architecture GPUs, is the most robust tensor core accelerated compute mode for the iterative refinement solver. It is able to solve the widest range of problems in HPC arising from different applications and provides up to 4X and 5X speedup for real and complex systems, respectively. On Volta and Turing architecture GPUs, half precision tensor core acceleration is recommended. In cases where the iterative refinement solver fails to converge to the desired accuracy (main precision, INOUT data precision), it is recommended to use main precision as internal lowest precision (i.e., `cusolverDn[DD, ZZ]gesv` for the FP64 case).

Table 6: **Supported combinations of floating point precisions for `cusolver <t1><t2>gesv()` functions**

Interface function	Main precision (matrix, rhs and solution datatype)	Lowest precision allowed to be used internally
<code>cusolverDnZZgesv</code>	<code>cuDoubleComplex</code>	double complex
<code>cusolverDnZCgesv*</code>	<code>cuDoubleComplex</code>	single complex
<code>cusolverDnZKgesv</code>	<code>cuDoubleComplex</code>	half complex
<code>cusolverDnZEgesv</code>	<code>cuDoubleComplex</code>	bfloat complex
<code>cusolverDnZYgesv</code>	<code>cuDoubleComplex</code>	tensorfloat complex
<code>cusolverDnCCgesv</code>	<code>cuComplex</code>	single complex
<code>cusolverDnCKgesv</code>	<code>cuComplex</code>	half complex
<code>cusolverDnCEgesv</code>	<code>cuComplex</code>	bfloat complex
<code>cusolverDnCYgesv</code>	<code>cuComplex</code>	tensorfloat complex
<code>cusolverDnDDgesv</code>	<code>double</code>	double
<code>cusolverDnDSgesv*</code>	<code>double</code>	single
<code>cusolverDnDHgesv</code>	<code>double</code>	half
<code>cusolverDnDBgesv</code>	<code>double</code>	bfloat
<code>cusolverDnDXgesv</code>	<code>double</code>	tensorfloat
<code>cusolverDnSSgesv</code>	<code>float</code>	single
<code>cusolverDnSHgesv</code>	<code>float</code>	half
<code>cusolverDnSBgesv</code>	<code>float</code>	bfloat
<code>cusolverDnSXgesv</code>	<code>float</code>	tensorfloat

* Has LAPACK counterparts

`cusolverDn<t1><t2>gesv_bufferSize()` functions will return workspace buffer size in bytes required for the corresponding `cusolverDn<t1><t2>gesv()` function.

```
cusolverStatus_t
cusolverDnZZgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
```

(continues on next page)

(continued from previous page)

```

    int
    cuDoubleComplex      *   nrhs,
    int                  *   dA,
    int                  *   ldda,
    int                  *   dipiv,
    cuDoubleComplex      *   dB,
    int                  *   lddb,
    cuDoubleComplex      *   dX,
    int                  *   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZCgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    int                   dipiv,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZKgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    int                   dipiv,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZEgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    int                   dipiv,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverDnZYGesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuDoubleComplex       *dA,
    int                   ldda,
    int                   *dipiv,
    cuDoubleComplex       *dB,
    int                   lddb,
    cuDoubleComplex       *dX,
    int                   lddx,
    void                  *dwork,
    size_t                 lwork_bytes);

```

```

cusolverStatus_t
cusolverDnCCgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuComplex             *dA,
    int                   ldda,
    int                   *dipiv,
    cuComplex             *dB,
    int                   lddb,
    cuComplex             *dX,
    int                   lddx,
    void                  *dwork,
    size_t                 lwork_bytes);

```

```

cusolverStatus_t
cusolverDnCKgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuComplex             *dA,
    int                   ldda,
    int                   *dipiv,
    cuComplex             *dB,
    int                   lddb,
    cuComplex             *dX,
    int                   lddx,
    void                  *dwork,
    size_t                 lwork_bytes);

```

```

cusolverStatus_t
cusolverDnCEgesv_bufferSize(
    cusolverHandle_t      handle,
    int                   n,
    int                   nrhs,
    cuComplex             *dA,
    int                   ldda,
    int                   *dipiv,
    cuComplex             *dB,
    int                   lddb,
    cuComplex             *dX,

```

(continues on next page)

(continued from previous page)

```

    int          lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnCYgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    cuComplex      * dA,
    int ldda,
    int dipiv,
    cuComplex      * dB,
    int lddb,
    cuComplex      * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDDgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    double         * dA,
    int ldda,
    int dipiv,
    double         * dB,
    int lddb,
    double         * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDSgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    double         * dA,
    int ldda,
    int dipiv,
    double         * dB,
    int lddb,
    double         * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDHgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    double         * dA,
    int ldda,

```

(continues on next page)

(continued from previous page)

```

    int          * dipiv,
    double        * dB,
    int          * lddb,
    double        * dX,
    int          * lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDBgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    double * dA,
    int * ldda,
    int * dipiv,
    double * dB,
    int * lddb,
    double * dX,
    int * lddx,
    void * dwork,
    size_t * lwork_bytes);

cusolverStatus_t
cusolverDnDXgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    double * dA,
    int * ldda,
    int * dipiv,
    double * dB,
    int * lddb,
    double * dX,
    int * lddx,
    void * dwork,
    size_t * lwork_bytes);

cusolverStatus_t
cusolverDnSSgesv_bufferSize(
    cusolverHandle_t handle,
    int n,
    int nrhs,
    float * dA,
    int * ldda,
    int * dipiv,
    float * dB,
    int * lddb,
    float * dX,
    int * lddx,
    void * dwork,
    size_t * lwork_bytes);

cusolverStatus_t
cusolverDnSHgesv_bufferSize(
    cusolverHandle_t handle,
```

(continues on next page)

(continued from previous page)

```

    int          n,
    int          nrhs,
    float        * dA,
    int          * ldda,
    int          * dipiv,
    float        * dB,
    int          * lddb,
    float        * dX,
    int          * lddx,
    void         * dwork,
    size_t       * lwork_bytes);

cusolverStatus_t
cusolverDnSBgesv_bufferSize(
    cusolverHandle_t handle,
    int          n,
    int          nrhs,
    float        * dA,
    int          * ldda,
    int          * dipiv,
    float        * dB,
    int          * lddb,
    float        * dX,
    int          * lddx,
    void         * dwork,
    size_t       * lwork_bytes);

cusolverStatus_t
cusolverDnSXgesv_bufferSize(
    cusolverHandle_t handle,
    int          n,
    int          nrhs,
    float        * dA,
    int          * ldda,
    int          * dipiv,
    float        * dB,
    int          * lddb,
    float        * dX,
    int          * lddx,
    void         * dwork,
    size_t       * lwork_bytes);

```

Table 7: Parameters of `cusolverDn<T1><T2>gesv_bufferSize()` functions

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDN library context.
n	host	input	Number of rows and columns of square matrix A. Should be non-negative.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative.
dA	device	None	Matrix A with size n-by-n. Can be NULL.
ldda	host	input	Leading dimension of two-dimensional array used to store matrix A. $ldda \geq n$.
dipiv	device	None	Pivoting sequence. Not used and can be NULL.
dB	device	None	Set of right hand sides B of size n-by-nrhs. Can be NULL.
lddb	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. $lddb \geq n$.
dX	device	None	Set of solution vectors X of size n-by-nrhs. Can be NULL.
lddx	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. $lddx \geq n$.
dwork	device	none	Pointer to device workspace. Not used and can be NULL.
lwork_bytes	host	output	Pointer to a variable where required size of temporary workspace in bytes will be stored. Can't be NULL.

```

cusolverStatus_t cusolverDnZZgesv(
    cusolverDnHandle_t    handle,
    int                    n,
    int                    nrhs,
    cuDoubleComplex        * dA,
    int                    ldda,
    int                    * dipiv,
    cuDoubleComplex        * dB,
    int                    lddb,
    cuDoubleComplex        * dX,
    int                    lddx,
    void                   * dWorkspace,
    size_t                 lwork_bytes,
    int                    * niter,
    int                    * dinfo);

cusolverStatus_t cusolverDnZCgesv(
    cusolverDnHandle_t    handle,

```

(continues on next page)

(continued from previous page)

```

    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    int * dipiv,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnZKgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    int * dipiv,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnZEgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    int * dipiv,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnZYgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    int * dipiv,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,

```

(continues on next page)

(continued from previous page)

```

    int          lddx,
    void          * dWorkspace,
    size_t        lwork_bytes,
    int           * niter,
    int           * dinfo);

cusolverStatus_t cusolverDnCCgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuComplex * dA,
    int ldda,
    int * dipiv,
    cuComplex * dB,
    int lddb,
    cuComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnCKgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuComplex * dA,
    int ldda,
    int * dipiv,
    cuComplex * dB,
    int lddb,
    cuComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnCEgesv(
    cusolverDnHandle_t handle,
    int n,
    int nrhs,
    cuComplex * dA,
    int ldda,
    int * dipiv,
    cuComplex * dB,
    int lddb,
    cuComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnCYgesv(
    cusolverDnHandle_t handle,

```

(continues on next page)

(continued from previous page)

```

    int          n,
    int          nrhs,
    cuComplex    * dA,
    int          ldda,
    int          * dipiv,
    cuComplex    * dB,
    int          lddb,
    cuComplex    * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnDDgesv(
    cusolverDnHandle_t handle,
    int          n,
    int          nrhs,
    double       * dA,
    int          ldda,
    int          * dipiv,
    double       * dB,
    int          lddb,
    double       * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnDSgesv(
    cusolverDnHandle_t handle,
    int          n,
    int          nrhs,
    double       * dA,
    int          ldda,
    int          * dipiv,
    double       * dB,
    int          lddb,
    double       * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnDHgesv(
    cusolverDnHandle_t handle,
    int          n,
    int          nrhs,
    double       * dA,
    int          ldda,
    int          * dipiv,
    double       * dB,
    int          lddb,
    double       * dX,

```

(continues on next page)

(continued from previous page)

```

        int          lddx,
        void          * dWorkspace,
        size_t        lwork_bytes,
        int           * niter,
        int           * dinfo);

cusolverStatus_t cusolverDnDBgesv(
    cusolverDnHandle_t handle,
    int                n,
    int                nrhs,
    double             * dA,
    int                ldda,
    int                * dipiv,
    double             * dB,
    int                lddb,
    double             * dX,
    int                lddx,
    void               * dWorkspace,
    size_t             lwork_bytes,
    int                * niter,
    int                * dinfo);

cusolverStatus_t cusolverDnDXgesv(
    cusolverDnHandle_t handle,
    int                n,
    int                nrhs,
    double             * dA,
    int                ldda,
    int                * dipiv,
    double             * dB,
    int                lddb,
    double             * dX,
    int                lddx,
    void               * dWorkspace,
    size_t             lwork_bytes,
    int                * niter,
    int                * dinfo);

cusolverStatus_t cusolverDnSSgesv(
    cusolverDnHandle_t handle,
    int                n,
    int                nrhs,
    float              * dA,
    int                ldda,
    int                * dipiv,
    float              * dB,
    int                lddb,
    float              * dX,
    int                lddx,
    void               * dWorkspace,
    size_t             lwork_bytes,
    int                * niter,
    int                * dinfo);

cusolverStatus_t cusolverDnSHgesv(
    cusolverDnHandle_t handle,

```

(continues on next page)

(continued from previous page)

```

    int          n,
    int          nrhs,
    float        * dA,
    int          ldda,
    int          * dipiv,
    float        * dB,
    int          lddb,
    float        * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnSBgesv(
    cusolverDnHandle_t handle,
    int          n,
    int          nrhs,
    float        * dA,
    int          ldda,
    int          * dipiv,
    float        * dB,
    int          lddb,
    float        * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnSXgesv(
    cusolverDnHandle_t handle,
    int          n,
    int          nrhs,
    float        * dA,
    int          ldda,
    int          * dipiv,
    float        * dB,
    int          lddb,
    float        * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

```

Table 8: Parameters of cusolverDn<T1><T2>gesv() functions

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDN library context.
n	host	input	Number of rows and columns of square matrix A. Should be non-negative.

continues on next page

Table 8 – continued from previous page

Parameter	Memory	In/out	Meaning
nrhs	host	input	Number of right hand sides to solve. Should be non-negative.
dA	device	in/out	Matrix A with size n-by-n. Can't be NULL. On return - unchanged if the iterative refinement process converged. If not - will contains the factorization of the matrix A in the main precision <T1> ($A = P * L * U$, where P - permutation matrix defined by vector ipiv, L and U - lower and upper triangular matrices).
ldda	host	input	Leading dimension of two-dimensional array used to store matrix A. $lda \geq n$.
dipiv	device	output	Vector that defines permutation for the factorization - row i was interchanged with row ipiv[i]
dB	device	input	Set of right hand sides B of size n-by-nrhs. Can't be NULL.
lddb	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. $ldb \geq n$.
dX	device	output	Set of solution vectors X of size n-by-nrhs. Can't be NULL.
lddx	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. $ldx \geq n$.
dWorkspace	device	input	Pointer to an allocated workspace in device memory of size lwork_bytes.
lwork_bytes	host	input	Size of the allocated device workspace. Should be at least what was returned by <code>cusolverDn<T1><T2>gesv_bufferSize()</code> function.
niters	host	output	If iter is ▶ <0 : iterative refinement has failed, main precision (Inputs/Outputs precision) factorization has been performed ▶ -1 : taking into account machine parameters, n, nrhs, it is a priori not worth working in lower precision ▶ -2 : overflow of an entry when moving from main to lower precision ▶ -3 : failure during the factorization ▶ -5 : overflow occurred during computation ▶ -50: solver stopped the iterative refinement after reaching maximum allowed iterations ▶ >0 : iter is a number of iterations solver performed to reach convergence criteria

continues on next page

Table 8 – continued from previous page

Parameter	Memory	In/out	Meaning
dinfo	device	output	Status of the IRS solver on the return. If 0 - solve was successful. If dinfo = -i then i-th argument is not valid. If dinfo = i, then U(i,i) computed in main precision is exactly zero. The factorization has been completed, but the factor U is exactly singular, so the solution could not be computed.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed, for example:

- ▶ $n < 0$
- ▶ $lda < \max(1, n)$
- ▶ $ldb < \max(1, n)$
- ▶ $ldx < \max(1, n)$

CUSOLVER_STATUS_ARCH_MISMATCH

The IRS solver supports compute capability 7.0 and above. The lowest precision options CUSOLVER_[CR]_16BF and CUSOLVER_[CR]_TF32 are only available on compute capability 8.0 and above.

CUSOLVER_STATUS_INVALID_WORKSPACE

lwork_bytes is smaller than the required workspace.

CUSOLVER_STATUS_IRS_OUT_OF_RANGE

Numerical error related to niters < 0 , see niters description for more details.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred, check the dinfo and the niters arguments for more details.

8.4.2.7 cusolverDnIRSXgesv()

This function is designed to perform same functionality as `cusolverDn<T1><T2>gesv()` functions, but wrapped in a more generic and expert interface that gives user more control to parametrize the function as well as it provides more information on output. `cusolverDnIRSXgesv()` allows additional control of the solver parameters such as setting:

- ▶ the main precision (Inputs/Outputs precision) of the solver
- ▶ the lowest precision to be used internally by the solver
- ▶ the refinement solver type
- ▶ the maximum allowed number of iterations in the refinement phase
- ▶ the tolerance of the refinement solver
- ▶ the fallback to main precision

► and more

through the configuration parameters structure `gesv_irs_params` and its helper functions. For more details about what configuration can be set and its meaning please refer to all the functions in the cuSolverDN Helper Function Section that start with `cusolverDnIRSParamsxxxx()`. Moreover, `cusolverDnIRSXgesv()` provides additional information on the output such as the convergence history (e.g., the residual norms) at each iteration and the number of iterations needed to converge. For more details about what information can be retrieved and its meaning please refer to all the functions in the cuSolverDN Helper Function Section that start with `cusolverDnIRSInfosxxxx()`.

The function returns value describes the results of the solving process. A `CUSOLVER_STATUS_SUCCESS` indicates that the function finished with success otherwise, it indicates if one of the API arguments is incorrect, or if the configurations of `params/infos` structure is incorrect or if the function did not finish with success. More details about the error can be found by checking the `niters` and the `dinfo` API parameters. See their description below for further details. User should provide the required workspace allocated on device for the `cusolverDnIRSXgesv()` function. The amount of bytes required for the function can be queried by calling the respective function `cusolverDnIRSXgesv_bufferSize()`. Note that, if the user would like a particular configuration to be set via the `params` structure, it should be set before the call to `cusolverDnIRSXgesv_bufferSize()` to get the size of the required workspace.

Tensor Float (TF32), introduced with NVIDIA Ampere architecture GPUs, is the most robust tensor core accelerated compute mode for the iterative refinement solver. It is able to solve the widest range of problems in HPC arising from different applications and provides up to 4X and 5X speedup for real and complex systems, respectively. On Volta and Turing architecture GPUs, half precision tensor core acceleration is recommended. In cases where the iterative refinement solver fails to converge to the desired accuracy (main precision, INOUT data precision), it is recommended to use main precision as internal lowest precision.

The following table provides all possible combinations values for the lowest precision corresponding to the Inputs/Outputs data type. Note that if the lowest precision matches the Inputs/Outputs datatype, then the main precision factorization will be used.

Table 9: **Supported Inputs/Outputs data type and lower precision for the IRS solver**

Inputs/Outputs Data Type (e.g., main precision)	Supported values for the lowest precision
CUSOLVER_C_64F	CUSOLVER_C_64F, CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_C_32F	CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_R_64F	CUSOLVER_R_64F, CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32
CUSOLVER_R_32F	CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32

The `cusolverDnIRSXgesv_bufferSize()` function returns the required workspace buffer size in bytes for the corresponding `cusolverDnXgesv()` call with the given `gesv_irs_params` configuration.

```
cusolverStatus_t
cusolverDnIRSXgesv_bufferSize(
```

(continues on next page)

(continued from previous page)

```

cusolverDnHandle_t      handle,
cusolverDnIRSParams_t   gesv_irs_params,
cusolver_int_t          n,
cusolver_int_t          nrhs,
size_t                  * lwork_bytes);

```

Table 10: **Parameters of cusolverDnIRSXgesv_bufferSize() functions**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDn library context.
params	host	input	Xgesv configuration parameters
n	host	input	Number of rows and columns of the square matrix A. Should be non-negative.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative. Note that nrhs is limited to 1 if the selected IRS refinement solver is CUSOLVER_IRS_REFINE_GMRES, CUSOLVER_IRS_REFINE_GMRES_GMRES, CUSOLVER_IRS_REFINE_CLASSICAL_GMRES.
lwork_bytes	host	out	Pointer to a variable, where the required size in bytes, of the workspace will be stored after a call to cusolverDnIRSXgesv_bufferSize. Can't be NULL.

```

cusolverStatus_t cusolverDnIRSXgesv(
    cusolverDnHandle_t      handle,
    cusolverDnIRSParams_t   gesv_irs_params,
    cusolverDnIRSInfos_t     gesv_irs_infos,
    int                     n,
    int                     nrhs,
    void                    * dA,
    int                     ldda,
    void                    * dB,
    int                     lddb,
    void                    * dX,
    int                     lddx,
    void                    * dWorkspace,
    size_t                  lwork_bytes,
    int                     * dinfo);

```

Table 11: **Parameters of cusolverDnIRSXgesv() functions**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDn library context.
gesv_irs_params	host	input	Configuration parameters structure, can serve one or more calls to any IRS solver

continues on next page

Table 11 – continued from previous page

Parameter	Memory	In/out	Meaning
gesv_irs_infos	host	in/out	Info structure, where information about a particular solve will be stored. The <code>gesv_irs_infos</code> structure correspond to a particular call. Thus different calls requires different <code>gesv_irs_infos</code> structure otherwise, it will be overwritten.
n	host	input	Number of rows and columns of square matrix A. Should be non-negative.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative. Note that, <code>nrhs</code> is limited to 1 if the selected IRS refinement solver is <code>CUSOLVER_IRS_REFINE_GMRES</code> , <code>CUSOLVER_IRS_REFINE_GMRES_GMRES</code> , <code>CUSOLVER_IRS_REFINE_CLASSICAL_GMRES</code> .
dA	device	in/out	Matrix A with size n-by-n. Can't be NULL. On return - will contain the factorization of the matrix A in the main precision ($A = P * L * U$, where P - permutation matrix defined by vector <code>ipiv</code> , L and U - lower and upper triangular matrices) if the iterative refinement solver was set to <code>CUSOLVER_IRS_REFINE_NONE</code> and the lowest precision is equal to the main precision (Inputs/Outputs datatype), or if the iterative refinement solver did not converge and the fallback to main precision was enabled (fallback enabled is the default setting); unchanged otherwise.
ldda	host	input	Leading dimension of two-dimensional array used to store matrix A. <code>lda</code> $\geq$ n.
dB	device	input	Set of right hand sides B of size n-by-nrhs. Can't be NULL.
lddb	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. <code>ldb</code> $\geq$ n.
dX	device	output	Set of solution vectors X of size n-by-nrhs. Can't be NULL.
lddx	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. <code>ldx</code> $\geq$ n.
dWorkspace	device	input	Pointer to an allocated workspace in device memory of size <code>lwork_bytes</code> .
lwork_bytes	host	input	Size of the allocated device workspace. Should be at least what was returned by <code>cusolverDnIRSXgesv_bufferSize()</code> function

continues on next page

Table 11 – continued from previous page

Parameter	Memory	In/out	Meaning
niters	host	output	If iter is ▶ <0 : iterative refinement has failed, main precision (Inputs/Outputs precision) factorization has been performed if fallback is enabled. ▶ -1 : taking into account machine parameters, n, nrhs, it is a priori not worth working in lower precision ▶ -2 : overflow of an entry when moving from main to lower precision ▶ -3 : failure during the factorization ▶ -5 : overflow occurred during computation ▶ -maxiter: solver stopped the iterative refinement after reaching maximum allowed iterations. ▶ >0 : iter is a number of iterations solver performed to reach convergence criteria
dinfo	device	output	Status of the IRS solver on the return. If 0 - solve was successful. If dinfo = -i then i-th argument is not valid. If dinfo = i, then U(i, i) computed in main precision is exactly zero. The factorization has been completed, but the factor U is exactly singular, so the solution could not be computed.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed, for example:

- ▶ $n < 0$
- ▶ $lda < \max(1, n)$
- ▶ $ldb < \max(1, n)$
- ▶ $ldx < \max(1, n)$

CUSOLVER_STATUS_ARCH_MISMATCH

The IRS solver supports compute capability 7.0 and above. The lowest precision options CUSOLVER_[CR]_16BF and CUSOLVER_[CR]_TF32 are only available on compute capability 8.0 and above.

CUSOLVER_STATUS_INVALID_WORKSPACE

lwork_bytes is smaller than the required workspace. Could happen if the users called cusolverDnIRSxgesv_bufferSize() function, then changed some of the configurations setting such as the lowest precision.

CUSOLVER_STATUS_IRS_OUT_OF_RANGE

Numerical error related to niters <0, see niters description for more details.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred, check the dinfo and the niters arguments for more details.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The configuration parameter gesv_irs_params structure was not created.

CUSOLVER_STATUS_IRS_PARAMS_INVALID

One of the configuration parameter in the gesv_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_PREC

The main and/or the lowest precision configuration parameter in the gesv_irs_params structure is not valid, check the table above for the supported combinations.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_MAXITER

The maxiter configuration parameter in the gesv_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_REFINE

The refinement solver configuration parameter in the gesv_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_NOT_SUPPORTED

One of the configuration parameter in the gesv_irs_params structure is not supported. For example if nrhs > 1, and refinement solver was set to CUSOLVER_IRS_REFINE_GMRES.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The information structure gesv_irs_infos was not created.

CUSOLVER_STATUS_ALLOC_FAILED

CPU memory allocation failed, most likely during the allocation of the residual array that store the residual norms.

8.4.2.8 cusolverDn<t>geqrf()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSgeqrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    float *A,
    int lda,
    int *Lwork );

cusolverStatus_t
cusolverDnDgeqrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    double *A,
    int lda,
    int *Lwork );

cusolverStatus_t
cusolverDnCgeqrf_bufferSize(cusolverDnHandle_t handle,
    int m,
    int n,
    cuComplex *A,
    int lda,
    int *Lwork );
```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverDnZgeqrf_bufferSize(cusolverDnHandle_t handle,
                            int m,
                            int n,
                            cuDoubleComplex *A,
                            int lda,
                            int *Lwork );

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSgeqrf(cusolverDnHandle_t handle,
                 int m,
                 int n,
                 float *A,
                 int lda,
                 float *TAU,
                 float *Workspace,
                 int Lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnDgeqrf(cusolverDnHandle_t handle,
                 int m,
                 int n,
                 double *A,
                 int lda,
                 double *TAU,
                 double *Workspace,
                 int Lwork,
                 int *devInfo );

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCgeqrf(cusolverDnHandle_t handle,
                 int m,
                 int n,
                 cuComplex *A,
                 int lda,
                 cuComplex *TAU,
                 cuComplex *Workspace,
                 int Lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnZgeqrf(cusolverDnHandle_t handle,
                 int m,
                 int n,
                 cuDoubleComplex *A,
                 int lda,
                 cuDoubleComplex *TAU,
                 cuDoubleComplex *Workspace,
                 int Lwork,
                 int *devInfo );

```

This function computes the QR factorization of an $m \times n$ matrix

$$A = Q * R$$

where A is an $m \times n$ matrix, Q is an $m \times n$ matrix, and R is a $n \times n$ upper triangular matrix.

The user has to provide working space which is pointed by input parameter Workspace. The input parameter Lwork is size of the working space, and it is returned by `geqrf_bufferSize()`.

The matrix R is overwritten in upper triangular part of A, including diagonal elements.

The matrix Q is not formed explicitly, instead, a sequence of householder vectors are stored in lower triangular part of A. The leading nonzero element of householder vector is assumed to be 1 such that output parameter TAU contains the scaling factor τ . If v is original householder vector, q is the new householder vector corresponding to τ , satisfying the following relation

$$I - 2 * v * v^H = I - \tau * q * q^H$$

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

Table 12: API of `geqrf`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
A	device	in/out	<type> array of dimension lda * n with lda is not less than max(1, m).
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
TAU	device	output	<type> array of dimension at least min(m, n).
Workspace	device	in/out	Working space, <type> array of size Lwork.
Lwork	host	input	Size of working array Workspace.
devInfo	device	output	If devInfo = 0, the LU factorization is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n < 0 or lda < max(1, m)).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.9 cusolverDn<t1><t2>gels()

These functions compute the solution of a system of linear equations with one or multiple right hand sides using mixed precision iterative refinement techniques based on the QR factorization Xgels. These functions are similar in term of functionalities to the full precision LAPACK QR (least squares) solver (Xgels, where X denotes Z,C,D,S) but it uses lower precision internally in order to provide faster time to solution, from here comes the name mixed precision. Mixed precision iterative refinement techniques means that the solver compute an QR factorization in lower precision and then iteratively refine the solution to achieve the accuracy of the Inputs/Outputs datatype precision. The <t1> corresponds to the Inputs/Outputs datatype precision while <t2> represent the internal lower precision at which the factorization will be carried on.

$$A \times X = B$$

Where A is m-by-n matrix and X is n-by-nrhs and B is m-by-nrhs matrices.

Functions API are designed to be as close as possible to LAPACK API to be considered as a quick and easy drop-in replacement. Description of these functions is given below. <t1><t2>gels() functions are designated by two floating point precisions The <t1> corresponds to the main precision (e.g., Inputs/Outputs datatype precision) and the <t2> represent the internal lower precision at which the factorization will be carried on. cusolver<t1><t2>gels() first attempts to factorize the matrix in lower precision and use this factorization within an iterative refinement procedure to obtain a solution with same normwise backward error as the main precision <t1>. If the approach fails to converge, then the method fallback to the main precision factorization and solve (Xgels) such a way that there is always a good solution at the output of these functions. If <t2> is equal to <t1>, then it is not a mixed precision process but rather a full one precision factorization, solve and refinement within the same main precision.

The iterative refinement process is stopped if:

ITER > ITERMAX

or for all the RHS we have:

RNRM < SQRT(N)*XNRM*ANRM*EPS*BWDMAX

where

- ▶ ITER is the number of the current iteration in the iterative refinement process
- ▶ RNRM is the infinity-norm of the residual
- ▶ XNRM is the infinity-norm of the solution
- ▶ ANRM is the infinity-operator-norm of the matrix A
- ▶ EPS is the machine epsilon that matches LAPACK<t1>LAMCH('Epsilon')

The values ITERMAX and BWDMAX are fixed to 50 and 1.0 respectively.

The function returns value describes the results of the solving process. A CUSOLVER_STATUS_SUCCESS indicates that the function finished with success otherwise, it indicates if one of the API arguments is incorrect, or if the function did not finish with success. More details about the error will be in the niters and the dinfo API parameters. See their description below for more details. User should provide the required workspace allocated on device memory. The amount of bytes required can be queried by calling the respective function <t1><t2>gels_bufferSize().

We provide a large set of mixed precision functions that include half, bfloat and tensorfloat as a lower precision as well as same precision functions (e.g., main and lowest precision are equal <t2> is equal to <t1>). The following table specifies which precisions will be used for which interface function:

Tensor Float (TF32), introduced with NVIDIA Ampere Architecture GPUs, is the most robust tensor core accelerated compute mode for the iterative refinement solver. It is able to solve the widest range of problems in HPC arising from different applications and provides up to 4X and 5X speedup for real and complex systems, respectively. On Volta and Turing architecture GPUs, half precision tensor core acceleration is recommended. In cases where the iterative refinement solver fails to converge to the desired accuracy (main precision, INOUT data precision), it is recommended to use main precision as internal lowest precision (i.e., `cusolverDn[DD, ZZ]gels` for the FP64 case).

Table 13: **Supported combinations of floating point precisions for `cusolver <t1><t2>gels()` functions**

Interface function	Main precision (matrix, rhs and solution datatype)	Lowest precision allowed to be used internally
<code>cusolverDnZZgels</code>	<code>cuDoubleComplex</code>	double complex
<code>cusolverDnZCgels</code>	<code>cuDoubleComplex</code>	single complex
<code>cusolverDnZKgels</code>	<code>cuDoubleComplex</code>	half complex
<code>cusolverDnZegels</code>	<code>cuDoubleComplex</code>	bfloat complex
<code>cusolverDnZYgels</code>	<code>cuDoubleComplex</code>	tensorfloat complex
<code>cusolverDnCCgels</code>	<code>cuComplex</code>	single complex
<code>cusolverDnCKgels</code>	<code>cuComplex</code>	half complex
<code>cusolverDnCEgels</code>	<code>cuComplex</code>	bfloat complex
<code>cusolverDnCYgels</code>	<code>cuComplex</code>	tensorfloat complex
<code>cusolverDnDDgels</code>	double	double
<code>cusolverDnDSgels</code>	double	single
<code>cusolverDnDHgels</code>	double	half
<code>cusolverDnDBGels</code>	double	bfloat
<code>cusolverDnDXgels</code>	double	tensorfloat
<code>cusolverDnSSgels</code>	float	single
<code>cusolverDnSHgels</code>	float	half
<code>cusolverDnSBgels</code>	float	bfloat
<code>cusolverDnSXgels</code>	float	tensorfloat

`cusolverDn<t1><t2>gels_bufferSize()` functions will return workspace buffer size in bytes required for the corresponding `cusolverDn<t1><t2>gels()` function.

```
cusolverStatus_t
cusolverDnZZgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
```

(continues on next page)

(continued from previous page)

```

    cuDoubleComplex      *   dA,
    int                  *   ldda,
    cuDoubleComplex      *   dB,
    int                  *   lddb,
    cuDoubleComplex      *   dX,
    int                  *   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZCgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZKgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZEgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    cuDoubleComplex      *   dA,
    int                   ldda,
    cuDoubleComplex      *   dB,
    int                   lddb,
    cuDoubleComplex      *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnZYgels_bufferSize(

```

(continues on next page)

(continued from previous page)

```

cusolverHandle_t      handle,
int                  m,
int                  n,
int                  nrhs,
cuDoubleComplex      * dA,
int                  ldda,
cuDoubleComplex      * dB,
int                  lddb,
cuDoubleComplex      * dX,
int                  lddx,
void                 * dwork,
size_t               * lwork_bytes);

cusolverStatus_t
cusolverDnCCgels_bufferSize(
    cusolverHandle_t      handle,
    int                  m,
    int                  n,
    int                  nrhs,
    cuComplex             * dA,
    int                  ldda,
    cuComplex             * dB,
    int                  lddb,
    cuComplex             * dX,
    int                  lddx,
    void                 * dwork,
    size_t               * lwork_bytes);

cusolverStatus_t
cusolverDnCKgels_bufferSize(
    cusolverHandle_t      handle,
    int                  m,
    int                  n,
    int                  nrhs,
    cuComplex             * dA,
    int                  ldda,
    cuComplex             * dB,
    int                  lddb,
    cuComplex             * dX,
    int                  lddx,
    void                 * dwork,
    size_t               * lwork_bytes);

cusolverStatus_t
cusolverDnCEgels_bufferSize(
    cusolverHandle_t      handle,
    int                  m,
    int                  n,
    int                  nrhs,
    cuComplex             * dA,
    int                  ldda,
    cuComplex             * dB,
    int                  lddb,
    cuComplex             * dX,
    int                  lddx,
    void                 * dwork,
    size_t               * lwork_bytes);

```

(continues on next page)

(continued from previous page)

```

    size_t                *    lwork_bytes);

cusolverStatus_t
cusolverDnCYgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    cuComplex             *    dA,
    int                   ldda,
    cuComplex             *    dB,
    int                   lddb,
    cuComplex             *    dX,
    int                   lddx,
    void                  *    dwork,
    size_t                *    lwork_bytes);

cusolverStatus_t
cusolverDnDDgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    double                *    dA,
    int                   ldda,
    double                *    dB,
    int                   lddb,
    double                *    dX,
    int                   lddx,
    void                  *    dwork,
    size_t                *    lwork_bytes);

cusolverStatus_t
cusolverDnDSgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    double                *    dA,
    int                   ldda,
    double                *    dB,
    int                   lddb,
    double                *    dX,
    int                   lddx,
    void                  *    dwork,
    size_t                *    lwork_bytes);

cusolverStatus_t
cusolverDnDHgels_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    double                *    dA,
    int                   ldda,
    double                *    dB,

```

(continues on next page)

(continued from previous page)

```

    int          lddb,
    double        * dX,
    int          lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDBgels_bufferSize(
    cusolverHandle_t handle,
    int m,
    int n,
    int nrhs,
    double        * dA,
    int ldda,
    double        * dB,
    int lddb,
    double        * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnDXgels_bufferSize(
    cusolverHandle_t handle,
    int m,
    int n,
    int nrhs,
    double        * dA,
    int ldda,
    double        * dB,
    int lddb,
    double        * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnSSgels_bufferSize(
    cusolverHandle_t handle,
    int m,
    int n,
    int nrhs,
    float         * dA,
    int ldda,
    float         * dB,
    int lddb,
    float         * dX,
    int lddx,
    void          * dwork,
    size_t        * lwork_bytes);

cusolverStatus_t
cusolverDnSHgels_bufferSize(
    cusolverHandle_t handle,
    int m,
    int n,

```

(continues on next page)

(continued from previous page)

```

    int          nrhs,
    float        *   dA,
    int          ldda,
    float        *   dB,
    int          lddb,
    float        *   dX,
    int          lddx,
    void         *   dwork,
    size_t       *   lwork_bytes);

cusolverStatus_t
cusolverDnSBgejs_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    float                 *   dA,
    int                   ldda,
    float                 *   dB,
    int                   lddb,
    float                 *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

cusolverStatus_t
cusolverDnSXgejs_bufferSize(
    cusolverHandle_t      handle,
    int                   m,
    int                   n,
    int                   nrhs,
    float                 *   dA,
    int                   ldda,
    float                 *   dB,
    int                   lddb,
    float                 *   dX,
    int                   lddx,
    void                 *   dwork,
    size_t               *   lwork_bytes);

```

Table 14: **Parameters of `cusolverDn<T1><T2>gejs_bufferSize()` functions**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDN library context.
m	host	input	Number of rows of the matrix A. Should be non-negative and $n \leq m$.
n	host	input	Number of columns of the matrix A. Should be non-negative and $n \leq m$.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative.

continues on next page

Table 14 – continued from previous page

Parameter	Memory	In/out	Meaning
dA	device	None	Matrix A with size m-by-n. Can be NULL.
ldda	host	input	Leading dimension of two-dimensional array used to store matrix A. $ldda \geq m$.
dB	device	None	Set of right hand sides B of size m-by-nrhs. Can be NULL.
lddb	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. $lddb \geq \max(1, m)$.
dX	device	None	Set of solution vectors X of size n-by-nrhs. Can be NULL.
lddx	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. $lddx \geq \max(1, n)$.
dwork	device	none	Pointer to device workspace. Not used and can be NULL.
lwork_bytes	host	output	Pointer to a variable where required size of temporary workspace in bytes will be stored. Can't be NULL.

```

cusolverStatus_t cusolverDnZZgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

```

```

cusolverStatus_t cusolverDnZCgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,

```

(continues on next page)

(continued from previous page)

```

        int * dinfo);

cusolverStatus_t cusolverDnZKgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnZEgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnZYgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuDoubleComplex * dA,
    int ldda,
    cuDoubleComplex * dB,
    int lddb,
    cuDoubleComplex * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnCCgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    cuComplex * dA,

```

(continues on next page)

(continued from previous page)

```

    int          ldda,
    cuComplex    *   dB,
    int          lddb,
    cuComplex    *   dX,
    int          lddx,
    void         *   dWorkspace,
    size_t       lwork_bytes,
    int          *   niter,
    int          *   dinfo);

cusolverStatus_t cusolverDnCKgels(
    cusolverDnHandle_t handle,
    int          m,
    int          n,
    int          nrhs,
    cuComplex    *   dA,
    int          ldda,
    cuComplex    *   dB,
    int          lddb,
    cuComplex    *   dX,
    int          lddx,
    void         *   dWorkspace,
    size_t       lwork_bytes,
    int          *   niter,
    int          *   dinfo);

cusolverStatus_t cusolverDnCEgels(
    cusolverDnHandle_t handle,
    int          m,
    int          n,
    int          nrhs,
    cuComplex    *   dA,
    int          ldda,
    cuComplex    *   dB,
    int          lddb,
    cuComplex    *   dX,
    int          lddx,
    void         *   dWorkspace,
    size_t       lwork_bytes,
    int          *   niter,
    int          *   dinfo);

cusolverStatus_t cusolverDnCYgels(
    cusolverDnHandle_t handle,
    int          m,
    int          n,
    int          nrhs,
    cuComplex    *   dA,
    int          ldda,
    cuComplex    *   dB,
    int          lddb,
    cuComplex    *   dX,
    int          lddx,
    void         *   dWorkspace,
    size_t       lwork_bytes,
    int          *   niter,

```

(continues on next page)

(continued from previous page)

```

        int * dinfo);

cusolverStatus_t cusolverDnDDgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    double * dA,
    int ldda,
    double * dB,
    int lddb,
    double * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int niter,
    int * dinfo);

cusolverStatus_t cusolverDnDSgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    double * dA,
    int ldda,
    double * dB,
    int lddb,
    double * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int niter,
    int * dinfo);

cusolverStatus_t cusolverDnDHgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    double * dA,
    int ldda,
    double * dB,
    int lddb,
    double * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int niter,
    int * dinfo);

cusolverStatus_t cusolverDnDBgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    double * dA,

```

(continues on next page)

(continued from previous page)

```

    int          ldda,
    double       * dB,
    int          lddb,
    double       * dX,
    int          lddx,
    void         * dWorkspace,
    size_t       lwork_bytes,
    int          * niter,
    int          * dinfo);

cusolverStatus_t cusolverDnDXgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    double * dA,
    int ldda,
    double * dB,
    int lddb,
    double * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnSSgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    float * dA,
    int ldda,
    float * dB,
    int lddb,
    float * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnSHgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    float * dA,
    int ldda,
    float * dB,
    int lddb,
    float * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

```

(continues on next page)

(continued from previous page)

```

    int * dinfo);

cusolverStatus_t cusolverDnSBgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    float * dA,
    int ldda,
    float * dB,
    int lddb,
    float * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

cusolverStatus_t cusolverDnSXgels(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int nrhs,
    float * dA,
    int ldda,
    float * dB,
    int lddb,
    float * dX,
    int lddx,
    void * dWorkspace,
    size_t lwork_bytes,
    int * niter,
    int * dinfo);

```

Table 15: Parameters of `cusolverDn<T1><T2>gels()` functions

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDN library context.
m	host	input	Number of rows of the matrix A. Should be non-negative and $n \leq m$.
n	host	input	Number of columns of the matrix A. Should be non-negative and $n \leq m$.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative.
dA	device	in/out	Matrix A with size m -by- n . Can't be NULL. On return - unchanged if the lowest precision is not equal to the main precision and the iterative refinement solver converged, - garbage otherwise.
ldda	host	input	Leading dimension of two-dimensional array used to store matrix A. $ldda \geq m$.

continues on next page

Table 15 – continued from previous page

Parameter	Memory	In/out	Meaning
dB	device	input	Set of right hand sides B of size m-by-nrhs. Can't be NULL.
lddb	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. $lddb \geq \max(1, m)$.
dX	device	output	Set of solution vectors X of size n-by-nrhs. Can't be NULL.
lddx	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. $lddx \geq \max(1, n)$.
dWorkspace	device	input	Pointer to an allocated workspace in device memory of size lwork_bytes.
lwork_bytes	host	input	Size of the allocated device workspace. Should be at least what was returned by <code>cusolverDn<T1><T2>gels_bufferSize()</code> function
niters	host	output	If iter is ▶ <0 : iterative refinement has failed, main precision (Inputs/Outputs precision) factorization has been performed. ▶ -1 : taking into account machine parameters, n, nrhs, it is a priori not worth working in lower precision ▶ -2 : overflow of an entry when moving from main to lower precision ▶ -3 : failure during the factorization ▶ -5 : overflow occurred during computation ▶ -50: solver stopped the iterative refinement after reaching maximum allowed iterations. ▶ >0 : iter is a number of iterations solver performed to reach convergence criteria
dinfo	device	output	Status of the IRS solver on the return. If 0 - solve was successful. If <code>dinfo = -i</code> then <i>i</i> -th argument is not valid.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed, for example:

- ▶ $n < 0$
- ▶ $ldda < \max(1, m)$

- ▶ `lddb < max(1, m)`
- ▶ `lddx < max(1, n)`

CUSOLVER_STATUS_ARCH_MISMATCH

The IRS solver supports compute capability 7.0 and above. The lowest precision options CUSOLVER_[CR]_16BF and CUSOLVER_[CR]_TF32 are only available on compute capability 8.0 and above.

CUSOLVER_STATUS_INVALID_WORKSPACE

`lwork_bytes` is smaller than the required workspace.

CUSOLVER_STATUS_IRS_OUT_OF_RANGE

Numerical error related to `niters < 0`, see `niters` description for more details.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred; check the `dinfo` and the `niters` arguments for more details.

8.4.2.10 `cusolverDnIRSXgels()`

This function is designed to perform same functionality as `cusolverDn<T1><T2>gels()` functions, but wrapped in a more generic and expert interface that gives user more control to parametrize the function as well as it provides more information on output. `cusolverDnIRSXgels()` allows additional control of the solver parameters such as setting:

- ▶ the main precision (Inputs/Outputs precision) of the solver,
- ▶ the lowest precision to be used internally by the solver,
- ▶ the refinement solver type
- ▶ the maximum allowed number of iterations in the refinement phase
- ▶ the tolerance of the refinement solver
- ▶ the fallback to main precision
- ▶ and others

through the configuration parameters structure `gels_irs_params` and its helper functions. For more details about what configuration can be set and its meaning please refer to all the functions in the `cuSolverDN Helper Function Section` that start with `cusolverDnIRSParamsxxxx()`. Moreover, `cusolverDnIRSXgels()` provides additional information on the output such as the convergence history (e.g., the residual norms) at each iteration and the number of iterations needed to converge. For more details about what information can be retrieved and its meaning please refer to all the functions in the `cuSolverDN Helper Function Section` that start with `cusolverDnIRSInfosxxxx()`.

The function returns value describes the results of the solving process. A `CUSOLVER_STATUS_SUCCESS` indicates that the function finished with success otherwise, it indicates if one of the API arguments is incorrect, or if the configurations of `params/infos` structure is incorrect or if the function did not finish with success. More details about the error can be found by checking the `niters` and the `dinfo` API parameters. See their description below for further details. Users should provide the required workspace allocated on device for the `cusolverDnIRSXgels()` function. The amount of bytes required for the function can be queried by calling the respective function `cusolverDnIRSXgels_bufferSize()`. Note that, if the user would like a particular configuration to be set via the `params` structure, it should be set before the call to `cusolverDnIRSXgels_bufferSize()` to get the size of the required workspace.

The following table provides all possible combinations values for the lowest precision corresponding to the Inputs/Outputs data type. Note that if the lowest precision matches the Inputs/Outputs datatype, then main precision factorization will be used

Tensor Float (TF32), introduced with NVIDIA Ampere Architecture GPUs, is the most robust tensor core accelerated compute mode for the iterative refinement solver. It is able to solve the widest range of problems in HPC arising from different applications and provides up to 4X and 5X speedup for real and complex systems, respectively. On Volta and Turing architecture GPUs, half precision tensor core acceleration is recommended. In cases where the iterative refinement solver fails to converge to the desired accuracy (main precision, INOUT data precision), it is recommended to use main precision as internal lowest precision.

Table 16: **Supported Inputs/Outputs data type and lower precision for the IRS solver**

Inputs/Outputs Data Type (e.g., main precision)	Supported values for the lowest precision
CUSOLVER_C_64F	CUSOLVER_C_64F, CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_C_32F	CUSOLVER_C_32F, CUSOLVER_C_16F, CUSOLVER_C_16BF, CUSOLVER_C_TF32
CUSOLVER_R_64F	CUSOLVER_R_64F, CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32
CUSOLVER_R_32F	CUSOLVER_R_32F, CUSOLVER_R_16F, CUSOLVER_R_16BF, CUSOLVER_R_TF32

The `cusolverDnIRSxgels_bufferSize()` function return the required workspace buffer size in bytes for the corresponding `cusolverDnXgels()` call with given `gels_irs_params` configuration.

```
cusolverStatus_t
cusolverDnIRSxgels_bufferSize(
    cusolverDnHandle_t      handle,
    cusolverDnIRSParms_t    gels_irs_params,
    cusolver_int_t          m,
    cusolver_int_t          n,
    cusolver_int_t          nrhs,
    size_t                  * lwork_bytes);
```

Table 17: **Parameters of cusolverDnIRSXgels_bufferSize() functions**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDn library context.
params	host	input	Xgels configuration parameters
m	host	input	Number of rows of the matrix A. Should be non-negative and $n \leq m$
n	host	input	Number of columns of the matrix A. Should be non-negative and $n \leq m$.
nrhs	host	input	Number of right hand sides to solve. Should be non-negative. Note that, nrhs is limited to 1 if the selected IRS refinement solver is CUSOLVER_IRS_REFINE_GMRES, CUSOLVER_IRS_REFINE_GMRES_GMRES, CUSOLVER_IRS_REFINE_CLASSICAL_GMRES.
lwork_bytes	host	out	Pointer to a variable, where the required size in bytes, of the workspace will be stored after a call to cusolverDnIRSXgels_bufferSize. Can't be NULL.

```

cusolverStatus_t cusolverDnIRSXgels(
    cusolverDnHandle_t      handle,
    cusolverDnIRSParams_t   gels_irs_params,
    cusolverDnIRSInfos_t    gels_irs_infos,
    int                     m,
    int                     n,
    int                     nrhs,
    void                    * dA,
    int                     ldda,
    void                    * dB,
    int                     lddb,
    void                    * dX,
    int                     lddx,
    void                    * dWorkspace,
    size_t                  lwork_bytes,
    int                     dinfo);

```

Table 18: **Parameters of cusolverDnIRSXgels() functions**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cusolverDn library context.
gels_irs_params	host	input	Configuration parameters structure, can serve one or more calls to any IRS solver

continues on next page

Table 18 – continued from previous page

Parameter	Memory	In/out	Meaning
<code>gels_irs_infos</code>	host	in/out	Info structure, where information about a particular solve will be stored. The <code>gels_irs_infos</code> structure correspond to a particular call. Thus different calls requires different <code>gels_irs_infos</code> structure otherwise, it will be overwritten.
<code>m</code>	host	input	Number of rows of the matrix A. Should be non-negative and $n \leq m$
<code>n</code>	host	input	Number of columns of the matrix A. Should be non-negative and $n \leq m$.
<code>nrhs</code>	host	input	Number of right hand sides to solve. Should be non-negative. Note that, <code>nrhs</code> is limited to 1 if the selected IRS refinement solver is <code>CUSOLVER_IRS_REFINE_GMRES</code> , <code>CUSOLVER_IRS_REFINE_GMRES_GMRES</code> , <code>CUSOLVER_IRS_REFINE_CLASSICAL_GMRES</code> .
<code>dA</code>	device	in/out	Matrix A with size <code>m-by-n</code> . Can't be NULL. On return - unchanged if the lowest precision is not equal to the main precision and the iterative refinement solver converged, - garbage otherwise.
<code>ldda</code>	host	input	Leading dimension of two-dimensional array used to store matrix A. <code>ldda</code> $\geq m$.
<code>dB</code>	device	input	Set of right hand sides B of size <code>m-by-nrhs</code> . Can't be NULL.
<code>lddb</code>	host	input	Leading dimension of two-dimensional array used to store matrix of right hand sides B. <code>lddb</code> $\geq \max(1, m)$.
<code>dX</code>	device	output	Set of solution vectors X of size <code>n-by-nrhs</code> . Can't be NULL.
<code>lddx</code>	host	input	Leading dimension of two-dimensional array used to store matrix of solution vectors X. <code>lddx</code> $\geq \max(1, n)$.
<code>dWorkspace</code>	device	input	Pointer to an allocated workspace in device memory of size <code>lwork_bytes</code> .
<code>lwork_bytes</code>	host	input	Size of the allocated device workspace. Should be at least what was returned by <code>cusolverDnIRSxgels_bufferSize()</code> function.

continues on next page

Table 18 – continued from previous page

Parameter	Memory	In/out	Meaning
niters	host	output	If iter is ▶ <0 : iterative refinement has failed, main precision (Inputs/Outputs precision) factorization has been performed if fallback is enabled ▶ -1 : taking into account machine parameters, n, nrhs, it is a priori not worth working in lower precision ▶ -2 : overflow of an entry when moving from main to lower precision ▶ -3 : failure during the factorization ▶ -5 : overflow occurred during computation ▶ -maxiter: solver stopped the iterative refinement after reaching maximum allowed iterations ▶ >0 : iter is a number of iterations solver performed to reach convergence criteria
dinfo	device	output	Status of the IRS solver on the return. If 0 - solve was successful. If dinfo = -i then i-th argument is not valid.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed, for example:

- ▶ $n < 0$
- ▶ $ldda < \max(1, m)$
- ▶ $lddb < \max(1, m)$
- ▶ $lddx < \max(1, n)$

CUSOLVER_STATUS_ARCH_MISMATCH

The IRS solver supports compute capability 7.0 and above. The lowest precision options CUSOLVER_[CR]_16BF and CUSOLVER_[CR]_TF32 are only available on compute capability 8.0 and above.

CUSOLVER_STATUS_INVALID_WORKSPACE

lwork_bytes is smaller than the required workspace. Could happen if the users called cusolverDnIRSxgels_bufferSize() function, then changed some of the configurations setting such as the lowest precision.

CUSOLVER_STATUS_IRS_OUT_OF_RANGE

Numerical error related to niters < 0; see niters description for more details.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal error occurred, check the dinfo and the niters arguments for more details.

CUSOLVER_STATUS_IRS_PARAMS_NOT_INITIALIZED

The configuration parameter gels_irs_params structure was not created.

CUSOLVER_STATUS_IRS_PARAMS_INVALID

One of the configuration parameter in the gels_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_PREC

The main and/or the lowest precision configuration parameter in the gels_irs_params structure is not valid, check the table above for the supported combinations.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_MAXITER

The maxiter configuration parameter in the gels_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_PARAMS_INVALID_REFINE

The refinement solver configuration parameter in the gels_irs_params structure is not valid.

CUSOLVER_STATUS_IRS_NOT_SUPPORTED

One of the configuration parameter in the gels_irs_params structure is not supported. For example if nrhs > 1, and refinement solver was set to CUSOLVER_IRS_REFINE_GMRES.

CUSOLVER_STATUS_IRS_INFOS_NOT_INITIALIZED

The information structure gels_irs_infos was not created.

CUSOLVER_STATUS_ALLOC_FAILED

CPU memory allocation failed, most likely during the allocation of the residual array that store the residual norms.

8.4.2.11 cusolverDn<t>ormqr()

These helper functions calculate the size of work buffers needed. Please visit [cuSOLVER Library Samples - ormqr](#) for a code example.

```
cusolverStatus_t
cusolverDnSormqr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const float *A,
    int lda,
    const float *tau,
    const float *C,
    int ldc,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDormqr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const double *A,
    int lda,
    const double *tau,
```

(continues on next page)

(continued from previous page)

```

    const double *C,
    int ldc,
    int *lwork);

cusolverStatus_t
cusolverDnCunmqr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    const cuComplex *C,
    int ldc,
    int *lwork);

cusolverStatus_t
cusolverDnZunmqr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    const cuDoubleComplex *C,
    int ldc,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSormqr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const float *A,
    int lda,
    const float *tau,
    float *C,
    int ldc,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDormqr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,

```

(continues on next page)

(continued from previous page)

```

cublasOperation_t trans,
int m,
int n,
int k,
const double *A,
int lda,
const double *tau,
double *C,
int ldc,
double *work,
int lwork,
int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCunmqr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    cuComplex *C,
    int ldc,
    cuComplex *work,
    int lwork,
    int *devInfo);

```

```

cusolverStatus_t
cusolverDnZunmqr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasOperation_t trans,
    int m,
    int n,
    int k,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    cuDoubleComplex *C,
    int ldc,
    cuDoubleComplex *work,
    int lwork,
    int *devInfo);

```

This function overwrites $m \times n$ matrix C by

$$C = \begin{cases} op(Q) * C & \text{if } side == \text{CUBLAS_SIDE_LEFT} \\ C * op(Q) & \text{if } side == \text{CUBLAS_SIDE_RIGHT} \end{cases}$$

The operation of Q is defined by

$$\text{op}(q) = \begin{cases} Q & \text{if } \text{transa} == \text{CUBLAS_OP_N} \\ Q^T & \text{if } \text{transa} == \text{CUBLAS_OP_T} \\ Q^H & \text{if } \text{transa} == \text{CUBLAS_OP_C} \end{cases}$$

Q is a unitary matrix formed by a sequence of elementary reflection vectors from QR factorization (geqrf) of A.

$Q = H(1)H(2) \dots H(k)$

Q is of order m if side = CUBLAS_SIDE_LEFT and of order n if side = CUBLAS_SIDE_RIGHT.

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by geqrf_bufferSize() or ormqr_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

The user can combine geqrf, ormqr and trsm to complete a linear solver or a least-square solver.

Table 19: **API of ormqr**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDn library context.
side	host	input	Indicates if matrix Q is on the left or right of C.
trans	host	input	Operation op(Q) that is non- or (conj.) trans- pose.
m	host	input	Number of rows of matrix C.
n	host	input	Number of columns of matrix C.
k	host	input	Number of elementary reflections whose product defines the matrix Q. If side is CUBLAS_SIDE_LEFT, $m \geq k \geq 0$; if side is CUBLAS_SIDE_RIGHT, $n \geq k \geq 0$.
A	device	in	<type> array of dimension lda * k with lda is not less than max(1,m). The matrix A is from geqrf, so i-th column contains elementary re- flection vector.
lda	host	input	Leading dimension of two-dimensional ar- ray used to store matrix A. If side is CUBLAS_SIDE_LEFT, $lda \geq \max(1,m)$; if side is CUBLAS_SIDE_RIGHT, $lda \geq \max(1,n)$.
tau	device	input	<type> array of dimension k. The vector tau is from geqrf, so tau(i) is the scalar of the i-th elementary reflection vector.
C	device	in/out	<type> array of size ldc * n. On exit, C is over- written by op(Q)*C or C*op(Q).

continues on next page

Table 19 – continued from previous page

Parameter	Memory	In/out	Meaning
ldc	host	input	Leading dimension of two-dimensional array of matrix C. ldc $\geq$ max(1,m).
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of working array work.
devInfo	device	output	If devInfo = 0, the ormqr is successful. If devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m,n,k<0, k > m if CUBLAS_SIDE_LEFT, k > n if CUBLAS_SIDE_RIGHT, wrong lda or ldc or invalid trans).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.12 cusolverDn<t>orgqr()

These helper functions calculate the size of work buffers needed. Please visit [cuSOLVER Library Samples - orgqr](#) for a code example.

```
cusolverStatus_t
cusolverDnSorgqr_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const float *A,
    int lda,
    const float *tau,
    int *lwork);

cusolverStatus_t
cusolverDnDorgqr_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const double *A,
    int lda,
    const double *tau,
```

(continues on next page)

(continued from previous page)

```

    int *lwork);

cusolverStatus_t
cusolverDnCungqr_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    int *lwork);

cusolverStatus_t
cusolverDnZungqr_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSorgqr(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    float *A,
    int lda,
    const float *tau,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDorgqr(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    double *A,
    int lda,
    const double *tau,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCungqr(
    cusolverDnHandle_t handle,

```

(continues on next page)

(continued from previous page)

```

    int m,
    int n,
    int k,
    cuComplex *A,
    int lda,
    const cuComplex *tau,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZungqr(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int k,
    cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    cuDoubleComplex *work,
    int lwork,
    int *devInfo);

```

This function overwrites $m \times n$ matrix A by

$$Q = H(1) * H(2) * \dots * H(k)$$

where Q is a unitary matrix formed by a sequence of elementary reflection vectors stored in A.

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by `orgqr_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

The user can combine `geqrf`, `orgqr` to complete orthogonalization.

Table 20: API of `orgqr`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
m	host	input	Number of rows of matrix Q. $m \geq 0$;
n	host	input	Number of columns of matrix Q. $m \geq n \geq 0$;
k	host	input	Number of elementary reflections whose product defines the matrix Q. $n \geq k \geq 0$;
A	device	in/out	<type> array of dimension <code>lda * n</code> with <code>lda</code> is not less than <code>max(1, m)</code> . i-th column of A contains elementary reflection vector.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. <code>lda</code> $\geq \max(1, m)$.

continues on next page

Table 20 – continued from previous page

Parameter	Memory	In/out	Meaning
tau	device	input	<type> array of dimension k. tau(i) is the scalar of i-th elementary reflection vector.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of working array work.
devInfo	device	output	If info = 0, the orgqr is successful. if info = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n, k < 0, n > m, k > n or lda < m).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.13 cusolverDn<t>sytrf()

These helper functions calculate the size of the needed buffers.

```
cusolverStatus_t
cusolverDnSsytrf_bufferSize(cusolverDnHandle_t handle,
    int n,
    float *A,
    int lda,
    int *lwork );

cusolverStatus_t
cusolverDnDsytrf_bufferSize(cusolverDnHandle_t handle,
    int n,
    double *A,
    int lda,
    int *lwork );

cusolverStatus_t
cusolverDnCsytrf_bufferSize(cusolverDnHandle_t handle,
    int n,
    cuComplex *A,
    int lda,
    int *lwork );

cusolverStatus_t
```

(continues on next page)

(continued from previous page)

```
cusolverDnZsytrf_bufferSize(cusolverDnHandle_t handle,
                            int n,
                            cuDoubleComplex *A,
                            int lda,
                            int *lwork );
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSsytrf(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 float *A,
                 int lda,
                 int *devIpiv,
                 float *work,
                 int lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnDsytrf(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 double *A,
                 int lda,
                 int *devIpiv,
                 double *work,
                 int lwork,
                 int *devInfo );
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCsytrf(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 cuComplex *A,
                 int lda,
                 int *devIpiv,
                 cuComplex *work,
                 int lwork,
                 int *devInfo );

cusolverStatus_t
cusolverDnZsytrf(cusolverDnHandle_t handle,
                 cublasFillMode_t uplo,
                 int n,
                 cuDoubleComplex *A,
                 int lda,
                 int *devIpiv,
                 cuDoubleComplex *work,
                 int lwork,
                 int *devInfo );
```

This function computes the factorization of a symmetric indefinite matrix using the Bunch-Kaufman diagonal pivoting.

A is a $n \times n$ symmetric matrix, only lower or upper part is meaningful. The input parameter uplo

indicates which part of the matrix is used. If devI piv is null, no pivoting is performed, which is not numerically stable.

If input parameter uplo is CUBLAS_FILL_MODE_LOWER, only lower triangular part of A is processed, and replaced by lower triangular factor L and block diagonal matrix D. Each block of D is either 1x1 or 2x2 block, depending on pivoting.

$$P * A * P^T = L * D * L^T$$

If input parameter uplo is CUBLAS_FILL_MODE_UPPER, only upper triangular part of A is processed, and replaced by upper triangular factor U and block diagonal matrix D.

$$P * A * P^T = U * D * U^T$$

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by sytrf_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork. When no pivoting is performed, the other triangular part of the input matrix A is used as workspace.

If Bunch-Kaufman factorization failed, i.e. A is singular. The output parameter devInfo = i would indicate D(i, i)=0.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

The output parameter devI piv contains pivoting sequence. If devI piv(i) = k > 0, D(i, i) is 1x1 block, and i-th row/column of A is interchanged with k-th row/column of A. If uplo is CUBLAS_FILL_MODE_UPPER and devI piv(i-1) = devI piv(i) = -m < 0, D(i-1:i, i-1:i) is a 2x2 block, and (i-1)-th row/column is interchanged with m-th row/column. If uplo is CUBLAS_FILL_MODE_LOWER and devI piv(i+1) = devI piv(i) = -m < 0, D(i:i+1, i:i+1) is a 2x2 block, and (i+1)-th row/column is interchanged with m-th row/column.

Table 21: API of sytrf

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.
n	host	input	Number of rows and columns of matrix A.
A	device	in/out	<type> array of dimension lda * n with lda is not less than max(1, n).
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
devI piv	device	output	Array of size at least n, containing pivot indices. To perform no pivoting, set the argument NULL.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of working space work.
devInfo	device	output	If devInfo = 0, the LU factorization is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle). if devInfo = i, the D(i, i) = 0.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.14 cusolverDn<t>potrfBatched()

The S and D data types are real valued single and double precision, respectively. Please visit [cuSOLVER Library Samples - potrfBatched](#) for a code example.

```
cusolverStatus_t
cusolverDnSpotrfBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    float *Aarray[],
    int lda,
    int *infoArray,
    int batchSize);
```

```
cusolverStatus_t
cusolverDnDpotrfBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    double *Aarray[],
    int lda,
    int *infoArray,
    int batchSize);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCpotrfBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    cuComplex *Aarray[],
    int lda,
    int *infoArray,
    int batchSize);
```

```
cusolverStatus_t
cusolverDnZpotrfBatched(
```

(continues on next page)

(continued from previous page)

```

cusolverDnHandle_t handle,
cublasFillMode_t uplo,
int n,
cuDoubleComplex *Aarray[],
int lda,
int *infoArray,
int batchSize);

```

This function computes the Cholesky factorization of a sequence of Hermitian positive-definite matrices.

Each `Aarray[i]` for $i=0,1,\dots, \text{batchSize}-1$ is a $n \times n$ Hermitian matrix, only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, only lower triangular part of A is processed, and replaced by lower triangular Cholesky factor L .

$$A = L * L^H$$

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, only upper triangular part of A is processed, and replaced by upper triangular Cholesky factor U .

$$A = U^H * U$$

If Cholesky factorization failed, i.e. some leading minor of A is not positive definite, or equivalently some diagonal elements of L or U is not a real number. The output parameter `infoArray` would indicate smallest leading minor of A which is not positive definite.

`infoArray` is an integer array of size `batchsize`. If `potrfBatched` returns `CUSOLVER_STATUS_INVALID_VALUE`, `infoArray[0] = -i` (less than zero), meaning that the i -th parameter is wrong (not counting `handle`). If `potrfBatched` returns `CUSOLVER_STATUS_SUCCESS` but `infoArray[i] = k` is positive, then i -th matrix is not positive definite and the Cholesky factorization failed at row k .

Remark: the other part of A is used as a workspace. For example, if `uplo` is `CUBLAS_FILL_MODE_UPPER`, upper triangle of A contains Cholesky factor U and lower triangle of A is destroyed after `potrfBatched`.

Table 22: API of `potrfBatched`

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the <code>cuSolverDN</code> library context.
<code>uplo</code>	host	input	Indicates if lower or upper part is stored; the other part is used as a workspace.
<code>n</code>	host	input	Number of rows and columns of matrix A .
<code>Aarray</code>	device	in/out	Array of pointers to <type> array of dimension <code>lda * n</code> with <code>lda</code> is not less than $\max(1, n)$.
<code>lda</code>	host	input	Leading dimension of two-dimensional array used to store each matrix <code>Aarray[i]</code> .

continues on next page

Table 22 – continued from previous page

Parameter	Memory	In/out	Meaning
infoArray	device	output	Array of size batchSize. infoArray[i] contains information of factorization of Aarray[i]. if potrfBatched returns CUSOLVER_STATUS_INVALID_VALUE, infoArray[0] = -i (less than zero) means the i-th parameter is wrong (not counting handle). if potrfBatched returns CUSOLVER_STATUS_SUCCESS, infoArray[i] = 0 means the Cholesky factorization of i-th matrix is successful, and infoArray[i] = k means the leading submatrix of order k of i-th matrix is not positive definite.
batchSize	host	input	Number of pointers in Aarray.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$ or $batchSize < 1$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.2.15 cusolverDn<t>potrsBatched()

```
cusolverStatus_t
cusolverDnSpotrsBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    float *Aarray[],
    int lda,
    float *Barray[],
    int ldb,
    int *info,
    int batchSize);
```

```
cusolverStatus_t
cusolverDnDpotrsBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    double *Aarray[],
    int lda,
```

(continues on next page)

(continued from previous page)

```

    double *Barray[],
    int ldb,
    int *info,
    int batchSize);

cusolverStatus_t
cusolverDnCpotrsBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    cuComplex *Aarray[],
    int lda,
    cuComplex *Barray[],
    int ldb,
    int *info,
    int batchSize);

cusolverStatus_t
cusolverDnZpotrsBatched(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    cuDoubleComplex *Aarray[],
    int lda,
    cuDoubleComplex *Barray[],
    int ldb,
    int *info,
    int batchSize);

```

This function solves a sequence of linear systems

$$A[i] * X[i] = B[i]$$

where each `Aarray[i]` for `i=0,1,..., batchSize-1` is a $n \times n$ Hermitian matrix, only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used.

The user has to call `potrfBatched` first to factorize matrix `Aarray[i]`. If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, `A` is lower triangular Cholesky factor `L` corresponding to $A = L * L^H$. If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, `A` is upper triangular Cholesky factor `U` corresponding to $A = U^H * U$.

The operation is in-place, i.e. matrix `X` overwrites matrix `B` with the same leading dimension `ldb`.

The output parameter `info` is a scalar. If `info = -i` (less than zero), the `i`-th parameter is wrong (not counting `handle`).

Remark 1: only `nrhs=1` is supported.

Remark 2: `infoArray` from `potrfBatched` indicates if the matrix is positive definite. `info` from `potrsBatched` only shows which input parameter is wrong (not counting `handle`).

Remark 3: the other part of `A` is used as a workspace. For example, if `uplo` is `CUBLAS_FILL_MODE_UPPER`, upper triangle of `A` contains Cholesky factor `U` and lower triangle of `A` is destroyed after `potrsBatched`.

Please visit [cuSOLVER Library Samples - potrfBatched](#) for a code example.

Table 23: API of potrsBatched

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored.
n	host	input	Number of rows and columns of matrix A.
nrhs	host	input	Number of columns of matrix X and B.
Aarray	device	in/out	Array of pointers to <type> array of dimension $lda * n$ with lda is not less than $\max(1, n)$. Aarray[i] is either lower Cholesky factor L or upper Cholesky factor U.
lda	host	input	Leading dimension of two-dimensional array used to store each matrix Aarray[i].
Barray	device	in/out	Array of pointers to <type> array of dimension $ldb * nrhs$. ldb is not less than $\max(1, n)$. As an input, Barray[i] is right hand side matrix. As an output, Barray[i] is the solution matrix.
ldb	host	input	Leading dimension of two-dimensional array used to store each matrix Barray[i].
info	device	output	If $info = 0$, all parameters are correct. if $info = -i$, the i -th parameter is wrong (not counting handle).
batchSize	host	input	Number of pointers in Aarray.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, $nrhs < 0$, $lda < \max(1, n)$, $ldb < \max(1, n)$ or $batchSize < 0$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3. Dense Eigenvalue Solver Reference (legacy)

This chapter describes eigenvalue solver API of cuSolverDN, including bidiagonalization and SVD.

8.4.3.1 cusolverDn<t>gebrd()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSgebrd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *Lwork );

cusolverStatus_t
cusolverDnDgebrd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *Lwork );

cusolverStatus_t
cusolverDnCgebrd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *Lwork );

cusolverStatus_t
cusolverDnZgebrd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *Lwork );
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSgebrd(cusolverDnHandle_t handle,
    int m,
    int n,
    float *A,
    int lda,
    float *D,
    float *E,
    float *TAUQ,
    float *TAUP,
    float *Work,
    int Lwork,
    int *devInfo );

cusolverStatus_t
cusolverDnDgebrd(cusolverDnHandle_t handle,
    int m,
    int n,
    double *A,
    int lda,
    double *D,
    double *E,
    double *TAUQ,
    double *TAUP,
```

(continues on next page)

(continued from previous page)

```
double *Work,
int Lwork,
int *devInfo );
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCgebrd(cusolverDnHandle_t handle,
    int m,
    int n,
    cuComplex *A,
    int lda,
    float *D,
    float *E,
    cuComplex *TAUQ,
    cuComplex *TAUP,
    cuComplex *Work,
    int Lwork,
    int *devInfo );

cusolverStatus_t
cusolverDnZgebrd(cusolverDnHandle_t handle,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,
    double *D,
    double *E,
    cuDoubleComplex *TAUQ,
    cuDoubleComplex *TAUP,
    cuDoubleComplex *Work,
    int Lwork,
    int *devInfo );
```

This function reduces a general $m \times n$ matrix A to a real upper or lower bidiagonal form B by an orthogonal transformation: $Q^H * A * P = B$

If $m \geq n$, B is upper bidiagonal; if $m < n$, B is lower bidiagonal.

The matrix Q and P are overwritten into matrix A in the following sense:

- if $m \geq n$, the diagonal and the first superdiagonal are overwritten with the upper bidiagonal matrix B ; the elements below the diagonal, with the array $TAUQ$, represent the orthogonal matrix Q as a product of elementary reflectors, and the elements above the first superdiagonal, with the array $TAUP$, represent the orthogonal matrix P as a product of elementary reflectors.
- if $m < n$, the diagonal and the first subdiagonal are overwritten with the lower bidiagonal matrix B ; the elements below the first subdiagonal, with the array $TAUQ$, represent the orthogonal matrix Q as a product of elementary reflectors, and the elements above the diagonal, with the array $TAUP$, represent the orthogonal matrix P as a product of elementary reflectors.

The user has to provide working space which is pointed by input parameter `Work`. The input parameter `Lwork` is size of the working space, and it is returned by `gebrd_bufferSize()`.

If output parameter `devInfo` = $-i$ (less than zero), the i -th parameter is wrong (not counting handle).

Remark: `gebrd` only supports $m \geq n$.

Table 24: API of `gebrd`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
A	device	in/out	<type> array of dimension $lda * n$ with lda is not less than $\max(1, n)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
D	device	output	Real array of dimension $\min(m, n)$. The diagonal elements of the bidiagonal matrix B: $D(i) = A(i, i)$.
E	device	output	Real array of dimension $\min(m, n)$. The off-diagonal elements of the bidiagonal matrix B: if $m \geq n$, $E(i) = A(i, i+1)$ for $i = 1, 2, \dots, n-1$; if $m < n$, $E(i) = A(i+1, i)$ for $i = 1, 2, \dots, m-1$.
TAUQ	device	output	<type> array of dimension $\min(m, n)$. The scalar factors of the elementary reflectors which represent the orthogonal matrix Q.
TAUP	device	output	<type> array of dimension $\min(m, n)$. The scalar factors of the elementary reflectors which represent the orthogonal matrix P.
Work	device	in/out	Working space, <type> array of size Lwork.
Lwork	host	input	Size of Work, returned by <code>gebrd_bufferSize</code> .
devInfo	device	output	If <code>devInfo = 0</code> , the operation is successful. if <code>devInfo = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$, or $lda < \max(1, m)$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.2 cusolverDn<t>orgbr()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSorgbr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    const float *A,
    int lda,
    const float *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDorgbr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    const double *A,
    int lda,
    const double *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnCungbr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnZungbr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    int *lwork);
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSorgbr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
```

(continues on next page)

(continued from previous page)

```

    int m,
    int n,
    int k,
    float *A,
    int lda,
    const float *tau,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDorgbr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    double *A,
    int lda,
    const double *tau,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCungbr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    cuComplex *A,
    int lda,
    const cuComplex *tau,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZungbr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    int m,
    int n,
    int k,
    cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    cuDoubleComplex *work,
    int lwork,
    int *devInfo);

```

This function generates one of the unitary matrices Q or P**H determined by gebrd when reducing a matrix A to bidiagonal form: $Q^H * A * P = B$

Q and P**H are defined as products of elementary reflectors H(i) or G(i) respectively.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is size of the working space, and it is returned by `orgbr_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo` = `-i` (less than zero), the `i`-th parameter is wrong (not counting handle).

Table 25: **API of orgbr**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
side	host	input	If <code>side</code> = <code>CUBLAS_SIDE_LEFT</code> , generate <code>Q</code> . if <code>side</code> = <code>CUBLAS_SIDE_RIGHT</code> , generate <code>P**T</code> .
m	host	input	Number of rows of matrix <code>Q</code> or <code>P**T</code> .
n	host	input	If <code>side</code> = <code>CUBLAS_SIDE_LEFT</code> , <code>m</code> >= <code>n</code> >= <code>min(m,k)</code> . if <code>side</code> = <code>CUBLAS_SIDE_RIGHT</code> , <code>n</code> >= <code>m</code> >= <code>min(n,k)</code> .
k	host	input	If <code>side</code> = <code>CUBLAS_SIDE_LEFT</code> , the number of columns in the original <code>m</code> -by- <code>k</code> matrix reduced by <code>gebrd</code> . if <code>side</code> = <code>CUBLAS_SIDE_RIGHT</code> , the number of rows in the original <code>k</code> -by- <code>n</code> matrix reduced by <code>gebrd</code> .
A	device	in/out	<type> array of dimension <code>lda * n</code> On entry, the vectors which define the elementary reflectors, as returned by <code>gebrd</code> . On exit, the <code>m</code> -by- <code>n</code> matrix <code>Q</code> or <code>P**T</code> .
lda	host	input	Leading dimension of two-dimensional array used to store matrix <code>A</code> . <code>lda</code> >= <code>max(1,m)</code> ;
tau	device	input	<type> array of dimension <code>min(m,k)</code> if <code>side</code> is <code>CUBLAS_SIDE_LEFT</code> ; of dimension <code>min(n,k)</code> if <code>side</code> is <code>CUBLAS_SIDE_RIGHT</code> ; <code>tau(i)</code> must contain the scalar factor of the elementary reflector <code>H(i)</code> or <code>G(i)</code> , which determines <code>Q</code> or <code>P**T</code> , as returned by <code>gebrd</code> in its array argument <code>TAUQ</code> or <code>TAUP</code> .
work	device	in/out	Working space, <type> array of size <code>lwork</code> .
lwork	host	input	Size of working array <code>work</code> .
devInfo	device	output	If <code>info</code> = <code>0</code> , the <code>ormqr</code> is successful. if <code>info</code> = <code>-i</code> , the <code>i</code> -th parameter is wrong (not counting handle).

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n < 0 or wrong lda).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.3 cusolverDn<t>sytrd()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSsytrd_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *d,
    const float *e,
    const float *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDsytrd_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *d,
    const double *e,
    const double *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnChetrd_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const float *d,
    const float *e,
    const cuComplex *tau,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnZhetrd_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *d,
```

(continues on next page)

(continued from previous page)

```
const double *e,
const cuDoubleComplex *tau,
int *lwork);
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSsytrd(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *d,
    float *e,
    float *tau,
    float *work,
    int lwork,
    int *devInfo);
```

```
cusolverStatus_t
cusolverDnDsytrd(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *d,
    double *e,
    double *tau,
    double *work,
    int lwork,
    int *devInfo);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnChetrd(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    float *d,
    float *e,
    cuComplex *tau,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t CUDENSEAPI cusolverDnZhetrd(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    double *d,
```

(continues on next page)

(continued from previous page)

```
double *e,
cuDoubleComplex *tau,
cuDoubleComplex *work,
int lwork,
int *devInfo);
```

This function reduces a general symmetric (Hermitian) $n \times n$ matrix A to real symmetric tridiagonal form T by an orthogonal transformation: $Q^H * A * Q = T$

As an output, A contains T and householder reflection vectors. If `uplo = CUBLAS_FILL_MODE_UPPER`, the diagonal and first superdiagonal of A are overwritten by the corresponding elements of the tridiagonal matrix T, and the elements above the first superdiagonal, with the array tau, represent the orthogonal matrix Q as a product of elementary reflectors; If `uplo = CUBLAS_FILL_MODE_LOWER`, the diagonal and first subdiagonal of A are overwritten by the corresponding elements of the tridiagonal matrix T, and the elements below the first subdiagonal, with the array tau, represent the orthogonal matrix Q as a product of elementary reflectors.

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by `sytrd_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo = -i` (less than zero), the *i*-th parameter is wrong (not counting handle). Note that the problem size *n* is limited by a condition $n * lda < INT32_MAX$ primarily due to the current implementation constraints.

Table 26: API of `sytrd`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Specifies which part of A is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A is stored.
n	host	input	Number of rows (columns) of matrix A.
A	device	in/out	<type> array of dimension <code>lda * n</code> with <code>lda</code> is not less than <code>max(1,n)</code> . If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading <i>n</i> -by- <i>n</i> upper triangular part of A contains the upper triangular part of the matrix A, and the strictly lower triangular part of A is not referenced. If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading <i>n</i> -by- <i>n</i> lower triangular part of A contains the lower triangular part of the matrix A, and the strictly upper triangular part of A is not referenced. On exit, A is overwritten by T and householder reflection vectors.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. <code>lda >= max(1,n)</code> .
D	device	output	Real array of dimension <i>n</i> . The diagonal elements of the tridiagonal matrix T: $D(i) = A(i, i)$.

continues on next page

Table 26 – continued from previous page

Parameter	Memory	In/out	Meaning
E	device	output	Real array of dimension (n-1). The off-diagonal elements of the tridiagonal matrix T: if uplo = CUBLAS_FILL_MODE_UPPER, $E(i) = A(i, i+1)$. if uplo = CUBLAS_FILL_MODE_LOWER, $E(i) = A(i+1, i)$.
tau	device	output	<type> array of dimension (n-1). The scalar factors of the elementary reflectors which represent the orthogonal matrix Q.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of work, returned by sytrd_bufferSize.
devInfo	device	output	If devInfo = 0, the operation is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n<0, or lda<max(1,n), or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.4 cusolverDn<t>ormtr()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSormtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    const float *A,
    int lda,
    const float *tau,
    const float *C,
    int ldc,
    int *lwork);
```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverDnDormtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    const double *A,
    int lda,
    const double *tau,
    const double *C,
    int ldc,
    int *lwork);

```

```

cusolverStatus_t
cusolverDnCunmtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    const cuComplex *C,
    int ldc,
    int *lwork);

```

```

cusolverStatus_t
cusolverDnZunmtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    const cuDoubleComplex *C,
    int ldc,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSormtr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    float *A,

```

(continues on next page)

(continued from previous page)

```

    int lda,
    float *tau,
    float *C,
    int ldc,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDormtr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    double *A,
    int lda,
    double *tau,
    double *C,
    int ldc,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCunmtr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    cuComplex *A,
    int lda,
    cuComplex *tau,
    cuComplex *C,
    int ldc,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZunmtr(
    cusolverDnHandle_t handle,
    cublasSideMode_t side,
    cublasFillMode_t uplo,
    cublasOperation_t trans,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,
    cuDoubleComplex *tau,
    cuDoubleComplex *C,
    int ldc,

```

(continues on next page)

(continued from previous page)

```
cuDoubleComplex *work,
int lwork,
int *devInfo);
```

This function overwrites $m \times n$ matrix C by

$$C = \begin{cases} op(Q) * C & \text{if } side == \text{CUBLAS_SIDE_LEFT} \\ C * op(Q) & \text{if } side == \text{CUBLAS_SIDE_RIGHT} \end{cases}$$

where Q is a unitary matrix formed by a sequence of elementary reflection vectors from `sytrd`.

The operation on Q is defined by

$$op(q) = \begin{cases} Q & \text{if } transa == \text{CUBLAS_OP_N} \\ Q^T & \text{if } transa == \text{CUBLAS_OP_T} \\ Q^H & \text{if } transa == \text{CUBLAS_OP_C} \end{cases}$$

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is size of the working space, and it is returned by `ormtr_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo = -i` (less than zero), the *i*-th parameter is wrong (not counting handle).

Table 27: **API of ormtr**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
side	host	input	side = CUBLAS_SIDE_LEFT, apply Q or Q**T from the Left; side = CUBLAS_SIDE_RIGHT, apply Q or Q**T from the Right.
uplo	host	input	uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A contains elementary reflectors from <code>sytrd</code> . uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A contains elementary reflectors from <code>sytrd</code> .
trans	host	input	Operation <code>op(Q)</code> that is non- or (conj.) transpose.
m	host	input	Number of rows of matrix C.
n	host	input	Number of columns of matrix C.
A	device	in/out	<type> array of dimension <code>lda * m</code> if side = CUBLAS_SIDE_LEFT; <code>lda * n</code> if side = CUBLAS_SIDE_RIGHT. The matrix A from <code>sytrd</code> contains the elementary reflectors.

continues on next page

Table 27 – continued from previous page

Parameter	Memory	In/out	Meaning
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. if side is CUBLAS_SIDE_LEFT, lda $\geq \max(1,m)$; if side is CUBLAS_SIDE_RIGHT, lda $\geq \max(1,n)$.
tau	device	output	<type> array of dimension (m-1) if side is CUBLAS_SIDE_LEFT; of dimension (n-1) if side is CUBLAS_SIDE_RIGHT; The vector tau is from sytrd, so tau(i) is the scalar of i-th elementary reflection vector.
C	device	in/out	<type> array of size ldc * n. On exit, C is overwritten by $op(Q)*C$ or $C*op(Q)$.
ldc	host	input	Leading dimension of two-dimensional array of matrix C. ldc $\geq \max(1,m)$.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of working array work.
devInfo	device	output	If devInfo = 0, the ormqr is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n < 0 or wrong lda or ldc).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.5 cusolverDn<t>orgtr()

These helper functions calculate the size of work buffers needed.

```
cusolverStatus_t
cusolverDnSorgtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *tau,
    int *lwork);
```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverDnDorgtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *tau,
    int *lwork);

cusolverStatus_t
cusolverDnCungtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const cuComplex *tau,
    int *lwork);

cusolverStatus_t
cusolverDnZungtr_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSorgtr(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    const float *tau,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDorgtr(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    const double *tau,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCungtr(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    const cuComplex *tau,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZungtr(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *tau,
    cuDoubleComplex *work,
    int lwork,
    int *devInfo);
```

This function generates a unitary matrix Q which is defined as the product of n-1 elementary reflectors of order n, as returned by sytrd:

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by orgtr_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle).

Table 28: API of orgtr

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A contains elementary reflectors from sytrd. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A contains elementary reflectors from sytrd.
n	host	input	Number of rows (columns) of matrix Q.
A	device	in/out	<type> array of dimension lda * n On entry, matrix A from sytrd contains the elementary reflectors. On exit, matrix A contains the n-by-n orthogonal matrix Q.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda >= max(1,n).

continues on next page

Table 28 – continued from previous page

Parameter	Memory	In/out	Meaning
tau	device	input	<type> array of dimension (n-1) tau(i) is the scalar of i-th elementary reflection vector.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of working array work.
devInfo	device	output	If devInfo = 0, the orgtr is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n<0 or wrong lda).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.6 cusolverDn<t>gesvd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSgesvd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *lwork );

cusolverStatus_t
cusolverDnDgesvd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *lwork );

cusolverStatus_t
cusolverDnCgesvd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *lwork );

cusolverStatus_t
```

(continues on next page)

(continued from previous page)

```
cusolverDnZgesvd_bufferSize(
    cusolverDnHandle_t handle,
    int m,
    int n,
    int *lwork );
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSgesvd (
    cusolverDnHandle_t handle,
    signed char jobu,
    signed char jobvt,
    int m,
    int n,
    float *A,
    int lda,
    float *S,
    float *U,
    int ldu,
    float *VT,
    int ldvt,
    float *work,
    int lwork,
    float *rwork,
    int *devInfo);
```

```
cusolverStatus_t
cusolverDnDgesvd (
    cusolverDnHandle_t handle,
    signed char jobu,
    signed char jobvt,
    int m,
    int n,
    double *A,
    int lda,
    double *S,
    double *U,
    int ldu,
    double *VT,
    int ldvt,
    double *work,
    int lwork,
    double *rwork,
    int *devInfo);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCgesvd (
    cusolverDnHandle_t handle,
    signed char jobu,
    signed char jobvt,
    int m,
    int n,
    cuComplex *A,
    int lda,
```

(continues on next page)

(continued from previous page)

```

    float *S,
    cuComplex *U,
    int ldu,
    cuComplex *VT,
    int ldvt,
    cuComplex *work,
    int lwork,
    float *rwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZgesvd (
    cusolverDnHandle_t handle,
    signed char jobu,
    signed char jobvt,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,
    double *S,
    cuDoubleComplex *U,
    int ldu,
    cuDoubleComplex *VT,
    int ldvt,
    cuDoubleComplex *work,
    int lwork,
    double *rwork,
    int *devInfo);

```

This function computes the singular value decomposition (SVD) of an $m \times n$ matrix A and corresponding the left and/or right singular vectors. The SVD is written

$$A = U * \Sigma * V^H$$

where Σ is an $m \times n$ matrix which is zero except for its $\min(m, n)$ diagonal elements, U is an $m \times m$ unitary matrix, and V is an $n \times n$ unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is size of the working space, and it is returned by `gesvd_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo = -i` (less than zero), the `i`-th parameter is wrong (not counting `handle`). If `bdsqr` did not converge, `devInfo` specifies how many superdiagonals of an intermediate bidiagonal form did not converge to zero.

The `rwork` is real array of dimension $(\min(m, n) - 1)$. If `devInfo > 0` and `rwork` is not NULL, `rwork` contains the unconverged superdiagonal elements of an upper bidiagonal matrix. This is slightly different from LAPACK which puts unconverged superdiagonal elements in `work` if type is `real`; in `rwork` if type is `complex`. `rwork` can be a NULL pointer if the user does not want the information from superdiagonal.

Please visit [cuSOLVER Library Samples - gesvd](#) for a code example.

Remark 1: `gesvd` only supports $m \geq n$.

Remark 2: the routine returns V^H , not V .

Table 29: API of gesvd

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobu	host	input	Specifies options for computing all or part of the matrix U: = 'A': all m columns of U are returned in array U; = 'S': the first min(m,n) columns of U (the left singular vectors) are returned in the array U; = 'O': the first min(m,n) columns of U (the left singular vectors) are overwritten on the array A; = 'N': no columns of U (no left singular vectors) are computed.
jobvt	host	input	Specifies options for computing all or part of the matrix V**T: = 'A': all N rows of V**T are returned in the array VT; = 'S': the first min(m,n) rows of V**T (the right singular vectors) are returned in the array VT; = 'O': the first min(m,n) rows of V**T (the right singular vectors) are overwritten on the array A; = 'N': no rows of V**T (no right singular vectors) are computed.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
A	device	in/out	<type> array of dimension lda * n with lda is not less than max(1, m). On exit, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
S	device	output	Real array of dimension min(m, n). The singular values of A, sorted so that S(i) >= S(i+1).
U	device	output	<type> array of dimension ldu * m with ldu is not less than max(1, m). U contains the $m \times m$ unitary matrix U.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U.
VT	device	output	<type> array of dimension ldvt * n with ldvt is not less than max(1, n). VT contains the $n \times n$ unitary matrix V**T.
ldvt	host	input	Leading dimension of two-dimensional array used to store matrix Vt.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of work, returned by gesvd_bufferSize.
rwork	device	input	Real array of dimension min(m,n)-1. It contains the unconverged superdiagonal elements of an upper bidiagonal matrix if devInfo > 0.

continues on next page

Table 29 – continued from previous page

Parameter	Memory	In/out	Meaning
devInfo	device	output	If devInfo = 0, the operation is successful. if devInfo = -i, the i-th parameter is wrong (not counting handle). If devInfo > 0, devInfo indicates how many superdiagonals of an intermediate bidiagonal form did not converge to zero.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n < 0 or lda < max(1, m) or ldu < max(1, m) or ldvt < max(1, n)).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.7 cusolverDn<t>gesvdj()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSgesvdj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    const float *A,
    int lda,
    const float *S,
    const float *U,
    int ldu,
    const float *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);
```

```
cusolverStatus_t
cusolverDnDgesvdj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
```

(continues on next page)

(continued from previous page)

```

    const double *A,
    int lda,
    const double *S,
    const double *U,
    int ldu,
    const double *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);

cusolverStatus_t
cusolverDnGgesvdj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    const cuComplex *A,
    int lda,
    const float *S,
    const cuComplex *U,
    int ldu,
    const cuComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);

cusolverStatus_t
cusolverDnZgesvdj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *S,
    const cuDoubleComplex *U,
    int ldu,
    const cuDoubleComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSgesvdj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    float *A,
    int lda,
    float *S,
    float *U,

```

(continues on next page)

(continued from previous page)

```

    int ldu,
    float *V,
    int ldv,
    float *work,
    int lwork,
    int *info,
    gesvdjInfo_t params);

cusolverStatus_t
cusolverDnDgesvdj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    double *A,
    int lda,
    double *S,
    double *U,
    int ldu,
    double *V,
    int ldv,
    double *work,
    int lwork,
    int *info,
    gesvdjInfo_t params);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCgesvdj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    cuComplex *A,
    int lda,
    float *S,
    cuComplex *U,
    int ldu,
    cuComplex *V,
    int ldv,
    cuComplex *work,
    int lwork,
    int *info,
    gesvdjInfo_t params);

cusolverStatus_t
cusolverDnZgesvdj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int econ,
    int m,
    int n,
    cuDoubleComplex *A,
    int lda,

```

(continues on next page)

(continued from previous page)

```

double *S,
cuDoubleComplex *U,
int ldu,
cuDoubleComplex *V,
int ldv,
cuDoubleComplex *work,
int lwork,
int *info,
gesvdjInfo_t params);

```

This function computes the singular value decomposition (SVD) of an $m \times n$ matrix A and corresponding the left and/or right singular vectors. The SVD is written:

$$A = U * \Sigma * V^H$$

where Σ is an $m \times n$ matrix which is zero except for its $\min(m, n)$ diagonal elements, U is an $m \times m$ unitary matrix, and V is an $n \times n$ unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

gesvdj has the same functionality as gesvd. The difference is that gesvd uses QR algorithm and gesvdj uses Jacobi method. The parallelism of Jacobi method gives GPU better performance on small and medium size matrices. Moreover the user can configure gesvdj to perform approximation up to certain accuracy.

gesvdj iteratively generates a sequence of unitary matrices to transform matrix A to the following form

$$U^H * A * V = S + E$$

where S is diagonal and diagonal of E is zero.

During the iterations, the Frobenius norm of E decreases monotonically. As E goes down to zero, S is the set of singular values. In practice, Jacobi method stops if

$$\|E\|_F \leq \text{eps} * \|A\|_F$$

where eps is given tolerance. Note that if the real residual norm

$$\|S - U^H * A * V\|_F$$

is computed, it will differ from $\|E\|_F$ up to roundoff errors of order $N = \max(m, n)$, to still have the standard SVD accuracy expectation

$$\frac{\|S - U^H * A * V\|_F}{O(N) * \|A\|_F} \leq \frac{\|E\|_F}{\|A\|_F} \leq \text{eps}$$

$O(N)$ is typically N , but the constant depends on the number of sweeps, which gives an upper roundoff error bound of $\text{sweeps} * N$.

gesvdj has two parameters to control the accuracy. First parameter is tolerance (eps). The default value is machine accuracy but The user can use function `cusolverDnXgesvdjSetTolerance` to set a priori tolerance. The second parameter is maximum number of sweeps which controls number of iterations of Jacobi method. The default value is 100 but the user can use function `cusolverDnXgesvdjSetMaxSweeps` to set a proper bound. The experiments show 15 sweeps are good enough to converge to machine accuracy. gesvdj stops either tolerance is met or maximum number of sweeps is met.

Jacobi method has quadratic convergence, so the accuracy is not proportional to number of sweeps. To guarantee certain accuracy, the user should configure tolerance only.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is the size of the working space, and it is returned by `gesvdj_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `info = min(m,n)+1`, `gesvdj` does not converge under given tolerance and maximum sweeps.

If the user sets an improper tolerance, `gesvdj` may not converge. For example, tolerance should not be smaller than machine accuracy.

Please visit [cuSOLVER Library Samples - gesvdj](#) for a code example.

Remark 1: `gesvdj` supports any combination of m and n .

Remark 2: the routine returns V , not V^H . This is different from `gesvd`.

Table 30: **API of gesvdj**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobz	host	input	Specifies options to either compute singular value only or singular vectors as well: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute singular values only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute singular values and singular vectors.
econ	host	input	econ = 1 for economy size for U and V.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
A	device	in/out	<type> array of dimension <code>lda * n</code> with <code>lda</code> is not less than <code>max(1, m)</code> . On exit, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
S	device	output	Real array of dimension <code>min(m, n)</code> . The singular values of A, sorted so that <code>S(i) >= S(i+1)</code> .
U	device	output	<type> array of dimension <code>ldu * m</code> if econ is zero. If econ is nonzero, the dimension is <code>ldu * min(m, n)</code> . U contains the left singular vectors.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U. <code>ldu</code> is not less than <code>max(1, m)</code> .
V	device	output	<type> array of dimension <code>ldv * n</code> if econ is zero. If econ is nonzero, the dimension is <code>ldv * min(m, n)</code> . V contains the right singular vectors.

continues on next page

Table 30 – continued from previous page

Parameter	Memory	In/out	Meaning
ldv	host	input	Leading dimension of two-dimensional array used to store matrix V. ldv is not less than $\max(1, n)$.
work	device	in/out	<type> array of size lwork, working space.
lwork	host	input	Size of work, returned by gesvdj_bufferSize.
info	device	output	If info = 0, the operation is successful. if info = -i, the i-th parameter is wrong (not counting handle). if info = $\min(m, n)+1$, gesvdj dose not converge under given tolerance and maximum sweeps.
params	host	input	Structure filled with parameters of Jacobi algorithm and results of gesvdj. params can be destroyed after the host thread exits the routine.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$ or $ldu < \max(1, m)$ or $ldv < \max(1, n)$ or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.8 cusolverDn<t>gesvdjBatched()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```

cusolverStatus_t
cusolverDnSgesvdjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    const float *A,
    int lda,
    const float *S,
    const float *U,
    int ldu,
    const float *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverDnDgesvdjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    const double *A,
    int lda,
    const double *S,
    const double *U,
    int ldu,
    const double *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

```

```

cusolverStatus_t
cusolverDnCgesvdjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    const cuComplex *A,
    int lda,
    const float *S,
    const cuComplex *U,
    int ldu,
    const cuComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

```

```

cusolverStatus_t
cusolverDnZgesvdjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *S,
    const cuDoubleComplex *U,
    int ldu,
    const cuDoubleComplex *V,
    int ldv,
    int *lwork,
    gesvdjInfo_t params,
    int batchSize);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSgesvdjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,

```

(continues on next page)

(continued from previous page)

```

    int m,
    int n,
    float *A,
    int lda,
    float *S,
    float *U,
    int ldu,
    float *V,
    int ldv,
    float *work,
    int lwork,
    int *info,
    gesvdjInfo_t params,
    int batchSize);

cusolverStatus_t
cusolverDnDgesvdjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    double *A,
    int lda,
    double *S,
    double *U,
    int ldu,
    double *V,
    int ldv,
    double *work,
    int lwork,
    int *info,
    gesvdjInfo_t params,
    int batchSize);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCgesvdjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int m,
    int n,
    cuComplex *A,
    int lda,
    float *S,
    cuComplex *U,
    int ldu,
    cuComplex *V,
    int ldv,
    cuComplex *work,
    int lwork,
    int *info,
    gesvdjInfo_t params,
    int batchSize);

cusolverStatus_t
cusolverDnZgesvdjBatched(

```

(continues on next page)

(continued from previous page)

```

cusolverDnHandle_t handle,
cusolverEigMode_t jobz,
int m,
int n,
cuDoubleComplex *A,
int lda,
double *S,
cuDoubleComplex *U,
int ldu,
cuDoubleComplex *V,
int ldv,
cuDoubleComplex *work,
int lwork,
int *info,
gesvdjInfo_t params,
int batchSize);

```

This function computes singular values and singular vectors of a sequence of general $m \times n$ matrices

$$A_j = U_j * \Sigma_j * V_j^H$$

where Σ_j is a real $m \times n$ diagonal matrix which is zero except for its $\min(m, n)$ diagonal elements. U_j (left singular vectors) is an $m \times m$ unitary matrix and V_j (right singular vectors) is a $n \times n$ unitary matrix. The diagonal elements of Σ_j are the singular values of A_j in either descending order or non-sorting order.

gesvdjBatched performs gesvdj on each matrix. It requires that all matrices are of the same size m, n no greater than 32 and are packed in contiguous way,

$$A = \begin{pmatrix} A_0 & A_1 & \dots \end{pmatrix}$$

Each matrix is column-major with leading dimension lda , so the formula for random access is $A_k(i, j) = A[i + lda * j + lda * n * k]$.

The parameter S also contains singular values of each matrix in contiguous way,

$$S = \begin{pmatrix} S_0 & S_1 & \dots \end{pmatrix}$$

The formula for random access of S is $S_k(j) = S[j + \min(m, n) * k]$.

Except for tolerance and maximum sweeps, gesvdjBatched can either sort the singular values in descending order (default) or chose as-is (without sorting) by the function cusolverDnXgesvdjSetSortEig. If the user packs several tiny matrices into diagonal blocks of one matrix, non-sorting option can separate singular values of those tiny matrices.

gesvdjBatched cannot report residual and executed sweeps by function cusolverDnXgesvdjGetResidual and cusolverDnXgesvdjGetSweeps. Any call of the above two returns CUSOLVER_STATUS_NOT_SUPPORTED. The user needs to compute residual explicitly.

The user has to provide working space pointed by input parameter work. The input parameter lwork is the size of the working space, and it is returned by gesvdjBatched_bufferSize(). Please note that the size in bytes of the working space is equal to sizeof(<type>) * lwork.

The output parameter info is an integer array of size batchSize. If the function returns CUSOLVER_STATUS_INVALID_VALUE, the first element info[0] = -i (less than zero) indicates i-th parameter is wrong (not counting handle). Otherwise, if info[i] = min(m, n)+1, gesvdjBatched does not converge on i-th matrix under given tolerance and maximum sweeps.

Please visit [cuSOLVER Library Samples - gesvdjBatched](#) for a code example.

Table 31: **API of gesvdjBatched**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobz	host	input	Specifies options to either compute singular value only or singular vectors as well: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute singular values only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute singular values and singular vectors.
m	host	input	Number of rows of matrix A_j . m is no greater than 32.
n	host	input	Number of columns of matrix A_j . n is no greater than 32.
A	device	in/out	<type> array of dimension $lda * n * batchSize$ with lda is not less than $\max(1, n)$. on Exit: the contents of A_j are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A_j .
S	device	output	Array of dimension $\min(m, n) * batchSize$. It stores the singular values of A_j in descending order or non-sorting order.
U	device	output	<type> array of dimension $ldu * m * batchSize$. U_j contains the left singular vectors of A_j .
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U_j . ldu is not less than $\max(1, m)$.
V	device	output	<type> array of dimension $ldv * n * batchSize$. V_j contains the right singular vectors of A_j .
ldv	host	input	Leading dimension of two-dimensional array used to store matrix V_j . ldv is not less than $\max(1, n)$.
work	device	in/out	<type> array of size $lwork$, working space.
lwork	host	input	Size of work, returned by gesvdjBatched_bufferSize.

continues on next page

Table 31 – continued from previous page

Parameter	Memory	In/out	Meaning
info	device	output	An integer array of dimension batchSize. If CUSOLVER_STATUS_INVALID_VALUE is returned, info[0] = -i (less than zero) indicates i-th parameter is wrong (not counting handle). Otherwise, if info[i] = 0, the operation is successful. if info[i] = min(m,n)+1, gesvdjBatched dose not converge on i-th matrix under given tolerance and maximum sweeps.
params	host	input	Structure filled with parameters of Jacobi algorithm. params can be destroyed after the host thread exits the routine.
batchSize	host	input	Number of matrices.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$ or $ldu < \max(1, m)$ or $ldv < \max(1, n)$ or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or batchSize < 0).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.9 cusolverDn<t>gesvdaStridedBatched()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```

cusolverStatus_t
cusolverDnSgesvdaStridedBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const float *A,
    int lda,
    long long int strideA,
    const float *S,
    long long int strideS,
    const float *U,
    int ldu,
    long long int strideU,
    const float *V,
    int ldv,
    long long int strideV,

```

(continues on next page)

(continued from previous page)

```

    int *lwork,
    int batchSize);

cusolverStatus_t
cusolverDnDgesvdaStridedBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const double *A,
    int lda,
    long long int strideA,
    const double *S,
    long long int strideS,
    const double *U,
    int ldu,
    long long int strideU,
    const double *V,
    int ldv,
    long long int strideV,
    int *lwork,
    int batchSize);

cusolverStatus_t
cusolverDnCgesvdaStridedBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const cuComplex *A,
    int lda,
    long long int strideA,
    const float *S,
    long long int strideS,
    const cuComplex *U,
    int ldu,
    long long int strideU,
    const cuComplex *V,
    int ldv,
    long long int strideV,
    int *lwork,
    int batchSize);

cusolverStatus_t
cusolverDnZgesvdaStridedBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const cuDoubleComplex *A,
    int lda,
    long long int strideA,
    const double *S,

```

(continues on next page)

(continued from previous page)

```

long long int strideS,
const cuDoubleComplex *U,
int ldu,
long long int strideU,
const cuDoubleComplex *V,
int ldv,
long long int strideV,
int *lwork,
int batchSize);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSgesvdaStridedBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const float *A,
    int lda,
    long long int strideA,
    float *S,
    long long int strideS,
    float *U,
    int ldu,
    long long int strideU,
    float *V,
    int ldv,
    long long int strideV,
    float *work,
    int lwork,
    int *info,
    double *h_R_nrmF,
    int batchSize);

```

```

cusolverStatus_t
cusolverDnDgesvdaStridedBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const double *A,
    int lda,
    long long int strideA,
    double *S,
    long long int strideS,
    double *U,
    int ldu,
    long long int strideU,
    double *V,
    int ldv,
    long long int strideV,
    double *work,
    int lwork,
    int *info,

```

(continues on next page)

(continued from previous page)

```
double *h_R_nrmF,
int batchSize);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnGgesvdaStridedBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const cuComplex *A,
    int lda,
    long long int strideA,
    float *S,
    long long int strideS,
    cuComplex *U,
    int ldu,
    long long int strideU,
    cuComplex *V,
    int ldv,
    long long int strideV,
    cuComplex *work,
    int lwork,
    int *info,
    double *h_R_nrmF,
    int batchSize);

cusolverStatus_t
cusolverDnZgesvdaStridedBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    int rank,
    int m,
    int n,
    const cuDoubleComplex *A,
    int lda,
    long long int strideA,
    double *S,
    long long int strideS,
    cuDoubleComplex *U,
    int ldu,
    long long int strideU,
    cuDoubleComplex *V,
    int ldv,
    long long int strideV,
    cuDoubleComplex *work,
    int lwork,
    int *info,
    double *h_R_nrmF,
    int batchSize);
```

This function gesvda (a stands for approximate) approximates the singular value decomposition of a tall skinny $m \times n$ matrix A and corresponding the left and right singular vectors. The economy form of

SVD is written by

$$A = U * \Sigma * V^H$$

where Σ is an $n \times n$ matrix. U is an $m \times n$ unitary matrix, and V is an $n \times n$ unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. U and V are the left and right singular vectors of A .

gesvda computes eigenvalues of $A^{**T}A$, or $A^{**H}A$ (if A is complex), to approximate singular values and singular vectors. It generates matrices U and V and transforms the matrix A to the following form

$$U^H * A * V = S + E$$

where S is diagonal and E depends on rounding errors. To certain conditions, U , V and S approximate singular values and singular vectors up to machine zero of single precision. In general, V is unitary, S is more accurate than U . If singular value is far from zero, then left singular vector U is accurate. In other words, the accuracy of singular values and left singular vectors depend on the distance between singular value and zero. Since the computation of $A^{**T}A$, or $A^{**H}A$ can greatly amplify errors, it is recommended to use gesvda only with well-conditioned data.

The input parameter `rank` decides the number of singular values and singular vectors are computed in parameter S , U and V .

The output parameter `h_RnormF` computes Frobenius norm of residual. To compute `h_RnormF`, `info != NULL` is required.

$$A - U * S * V^H$$

if the parameter `rank` is equal `n`. Otherwise, `h_RnormF` reports

$$\|U * S * V^H\| - \|S\|$$

in Frobenius norm sense, that is, how far U is from unitary.

gesvdaStridedBatched performs gesvda on each matrix. It requires that all matrices are of the same size `m`, `n` and are packed in a contiguous way,

$$A = \begin{pmatrix} A_0 & A_1 & \dots \end{pmatrix}$$

Each matrix is column-major with leading dimension `lda`, so the formula for random access is $A_k(i,j) = A[i + lda * j + strideA * k]$. Similarly, the formula for random access of S is $S_k(j) = S[j + StrideS * k]$, the formula for random access of U is $U_k(i,j) = U[i + ldu * j + strideU * k]$ and the formula for random access of V is $V_k(i,j) = V[i + ldv * j + strideV * k]$.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is the size of the working space, and it is returned by `gesvdaStridedBatched_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

The output parameter `info` is an integer array of size `batchSize`. If the function returns `CUSOLVER_STATUS_INVALID_VALUE`, the first element `info[0] = -i` (less than zero) indicates `i`-th parameter is wrong (not counting handle). Otherwise, if `info[i] = min(m,n)+1`, `gesvdaStridedBatched` did not converge on the `i`-th matrix. If $0 < info[i] < min(m,n)+1$, `gesvdaStridedBatched` could not compute an SVD of the `i`-th matrix fully; the leading singular values $S_i[k]$, $0 \leq k \leq info[i]-1$, and corresponding singular vectors may still be useful. In this case, if `h_RnormF` is requested, `h_RnormF` reports the residual as if `rank` was set to `info[i]-1`.

Note that the problem size is limited by the condition `batchSize*stride{A/S/U/V} <= INT32_MAX` primarily due to the current implementation constraints.

Please visit [cuSOLVER Library Samples - gesvdaStridedBatched](#) for a code example.

Remark 1: The routine returns V , not V^H . This is different from gesvd.

Remark 2: The routine only supports $m \geq n$.

Remark 3: It is recommended to use an FP64 data type, that is DgesvdaStridedBatched or ZgesvdaStridedBatched.

Remark 4: If the user is confident on the accuracy of singular values and singular vectors, for example, certain conditions hold (required singular value is far from zero), then the performance can be improved by passing a null pointer to h_RnrmF, i.e. no computation of the residual norm.

Table 32: API of gesvdaStridedBatched

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobz	host	input	Specifies options to either compute singular value only or singular vectors as well: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute singular values only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute singular values and singular vectors.
rank	host	input	Number of singular values (from largest to smallest).
m	host	input	Number of rows of matrix A_j .
n	host	input	Number of columns of matrix A_j .
A	device	input	<type> array of dimension strideA * batchSize with lda is not less than max(1,m). A_j is of dimension $m \times n$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A_j .
strideA	host	input	Value of type long long int that gives the address offset between $A[i]$ and $A[i+1]$. strideA is not less than lda*n.
S	device	output	A real array of dimension strideS*batchSize. It stores the singular values of A_j in descending order. S_j is of dimension rank * 1
strideS	host	input	Value of type long long int that gives the address offset between $S[i]$ and $S[i+1]$. strideS is not less than rank.
U	device	output	<type> array of dimension strideU * batchSize. U_j contains the left singular vectors of A_j . U_j is of dimension $m \times \text{rank}$.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U_j . ldu is not less than max(1,m).

continues on next page

Table 32 – continued from previous page

Parameter	Memory	In/out	Meaning
strideU	host	input	Value of type long long int that gives the address offset between $U[i]$ and $U[i+1]$. strideU is not less than $ldu \times rank$.
V	device	output	<type> array of dimension $strideV \times batchSize$. V_j contains the right singular vectors of A_j . V_j is of dimension $n \times rank$.
ldv	host	input	Leading dimension of two-dimensional array used to store matrix V_j . ldv is not less than $\max(1, n)$.
strideV	host	input	Value of type long long int that gives the address offset between $V[i]$ and $V[i+1]$. strideV is not less than $ldv \times rank$.
work	device	in/out	<type> array of size lwork, working space.
lwork	host	input	Size of work, returned by gesvdaStridedBatched_bufferSize.
info	device	output	An integer array of dimension batchSize. If CUSOLVER_STATUS_INVALID_VALUE is returned, $info[0] = -i$ (less than zero) indicates i -th parameter is wrong (not counting handle). Otherwise, if $info[i] = 0$, the operation is successful. If $info[i] = \min(m, n) + 1$, gesvdaStridedBatched did not converge on the i -th matrix. If $0 < info[i] < \min(m, n) + 1$, gesvdaStridedBatched computed only a partial SVD of the i -th matrix.
h_RnrmF	host	output	<double> array of size batchSize. $h_RnrmF[i]$ is norm of residual of i -th matrix. If $0 < info[i] < \min(m, n) + 1$, $h_RnrmF[i]$ reports the residual as if rank of the i -th matrix had been set to $info[i] - 1$.
batchSize	host	input	Number of matrices. batchSize is not less than 1.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $batchSize \times stride\{A/S/U/V\} > INT32_MAX$ or $lda < \max(1, m)$ or $ldu < \max(1, m)$ or $ldv < \max(1, n)$ or $strideA < lda \times n$ or $strideS < rank$ or $strideU < ldu \times rank$ or $strideV < ldv \times rank$ or $batchSize < 1$ or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.10 cusolverDn<t>syevd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsyevd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDsyevd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnCsyevd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const float *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnZsyevd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *W,
    int *lwork);
```

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSsyevd(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
```

(continues on next page)

(continued from previous page)

```

    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *W,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDsyevd(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *W,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCHeevd(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    float *W,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZheevd(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    double *W,
    cuDoubleComplex *work,
    int lwork,
    int *devInfo);

```

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A . The standard symmetric eigenvalue problem is

$$A * V = V * \Lambda$$

where Λ is a real $n \times n$ diagonal matrix. V is an $n \times n$ unitary matrix. The diagonal elements of Λ are the eigenvalues of A in ascending order.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is size of the working space, and it is returned by `syevd_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `devInfo = i` (greater than zero), i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, `A` contains the orthonormal eigenvectors of the matrix `A`. The eigenvectors are computed by a divide and conquer algorithm.

Please visit [cuSOLVER Library Samples - syevd](#) for a code example.

Table 33: **API of syevd**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>jobz</code>	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
<code>uplo</code>	host	input	Specifies which part of <code>A</code> is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of <code>A</code> is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of <code>A</code> is stored.
<code>n</code>	host	input	Number of rows (or columns) of matrix <code>A</code> .
<code>A</code>	device	in/out	<code><type></code> array of dimension <code>lda * n</code> with <code>lda</code> is not less than <code>max(1,n)</code> . If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading n -by- n upper triangular part of <code>A</code> contains the upper triangular part of the matrix <code>A</code> . If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading n -by- n lower triangular part of <code>A</code> contains the lower triangular part of the matrix <code>A</code> . On exit, if <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> , and <code>devInfo = 0</code> , <code>A</code> contains the orthonormal eigenvectors of the matrix <code>A</code> . If <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> , the contents of <code>A</code> are destroyed.
<code>lda</code>	host	input	Leading dimension of two-dimensional array used to store matrix <code>A</code> .
<code>W</code>	device	output	A real array of dimension n . The eigenvalue values of <code>A</code> , in ascending order ie, sorted so that <code>W(i) <= W(i+1)</code> .
<code>work</code>	device	in/out	Working space, <code><type></code> array of size <code>lwork</code> .
<code>Lwork</code>	host	input	Size of work, returned by <code>syevd_bufferSize</code> .

continues on next page

Table 33 – continued from previous page

Parameter	Memory	In/out	Meaning
devInfo	device	output	If devInfo = 0, the operation is successful. If devInfo = -i, the i-th parameter is wrong (not counting handle). If devInfo = i (> 0), devInfo indicates i off-diagonal elements of an intermediate tridiagonal form did not converge to zero;

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.11 cusolverDn<t>syevdx()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsyevdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    const float *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDsyevdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
```

(continues on next page)

(continued from previous page)

```

    int n,
    const double *A,
    int lda,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    const double *W,
    int *lwork);

cusolverStatus_t
cusolverDnCheevdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    const float *W,
    int *lwork);

cusolverStatus_t
cusolverDnZheevdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    const double *W,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSsyevdx(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float vl,

```

(continues on next page)

(continued from previous page)

```

    float vu,
    int il,
    int iu,
    int *h_meig,
    float *W,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDsyejvdx(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    double *W,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCheevdx(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    float *W,
    cuComplex *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnZheevdx(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,

```

(continues on next page)

(continued from previous page)

```

int lda,
double vl,
double vu,
int il,
int iu,
int *h_meig,
double *W,
cuDoubleComplex *work,
int lwork,
int *devInfo);

```

This function computes all or selection of the eigenvalues and optionally eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A . The standard symmetric eigenvalue problem is:

$$A * V = V * \Lambda$$

where Λ is a real $n \times h_meig$ diagonal matrix. V is an $n \times h_meig$ unitary matrix. h_meig is the number of eigenvalues/eigenvectors computed by the routine, h_meig is equal to n when the whole spectrum (e.g., `range = CUSOLVER_EIG_RANGE_ALL`) is requested. The diagonal elements of Λ are the eigenvalues of A in ascending order.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is size of the working space, and it is returned by `syevdx_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `devInfo = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `devInfo = i` (greater than zero), i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors of the matrix A . The eigenvectors are computed by a divide and conquer algorithm.

Please visit [cuSOLVER Library Samples - syevdx](#) for a code example.

Table 34: API of `syevdx`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.

continues on next page

Table 34 – continued from previous page

Parameter	Memory	In/out	Meaning
range	host	input	Specifies options to which selection of eigenvalues and optionally eigenvectors that need to be computed: range = CUSOLVER_EIG_RANGE_ALL : all eigenvalues/eigenvectors will be found, will become the classical syevd/heevd routine; range = CUSOLVER_EIG_RANGE_V : all eigenvalues/eigenvectors in the half-open interval (vl,vu] will be found; range = CUSOLVER_EIG_RANGE_I : the il-th through iu-th eigenvalues/eigenvectors will be found;
uplo	host	input	Specifies which part of A is stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A is stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A is stored.
n	host	input	Number of rows (or columns) of matrix A.
A	device	in/out	<type> array of dimension lda * n with lda is not less than max(1,n). If uplo = CUBLAS_FILL_MODE_UPPER, the leading n-by-n upper triangular part of A contains the upper triangular part of the matrix A. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n-by-n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and devInfo = 0, A contains the orthonormal eigenvectors of the matrix A. If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.lda is not less than max(1,n).

continues on next page

Table 34 – continued from previous page

Parameter	Memory	In/out	Meaning
v1, vu	host	input	Real values float or double for (C, S) or (Z, D) precision respectively. If range = CUSOLVER_EIG_RANGE_V, the lower and upper bounds of the interval to be searched for eigenvalues. $v1 > vu$. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_I. Note that, if eigenvalues are very close to each other, it is well known that two different eigenvalues routines might find slightly different number of eigenvalues inside the same interval. This is due to the fact that different eigenvalue algorithms, or even same algorithm but different run might find eigenvalues within some rounding error close to the machine precision. Thus, if the user wants to be sure not to miss any eigenvalue within the interval bound, we suggest that the user subtract/add epsilon (machine precision) to the interval bound such as ($v1=v1-eps$, $vu=vu+eps$). This suggestion is valid for any selective routine from cuSolver or LAPACK.
il, iu	host	input	Integer. If range = CUSOLVER_EIG_RANGE_I, the indices (in ascending order) of the smallest and largest eigenvalues to be returned. $1 \leq il \leq iu \leq n$, if $n > 0$; $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_V.
h_meig	host	output	Integer. The total number of eigenvalues found. $0 \leq h_meig \leq n$. If range = CUSOLVER_EIG_RANGE_ALL, $h_meig = n$, and if range = CUSOLVER_EIG_RANGE_I, $h_meig = iu - il + 1$.
W	device	output	A real array of dimension n. The eigenvalue values of A, in ascending order ie, sorted so that $W(i) \leq W(i+1)$.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of work, returned by syevdx_bufferSize.
devInfo	device	output	If devInfo = 0, the operation is successful. If devInfo = -i, the i-th parameter is wrong (not counting handle). If devInfo = i (> 0), devInfo indicates i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or `jobz` is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or `range` is not `CUSOLVER_EIG_RANGE_ALL` or `CUSOLVER_EIG_RANGE_V` or `CUSOLVER_EIG_RANGE_I`, or `uplo` is not `CUBLAS_FILL_MODE_LOWER` or `CUBLAS_FILL_MODE_UPPER`).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.12 cusolverDn<t>sygvd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsygvd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *B,
    int ldb,
    const float *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDsygvd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *B,
    int ldb,
    const double *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnC hegvd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const cuComplex *B,
```

(continues on next page)

(continued from previous page)

```

    int ldb,
    const float *W,
    int *lwork);

cusolverStatus_t
cusolverDnZhegyd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *B,
    int ldb,
    const double *W,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSsygvd(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *B,
    int ldb,
    float *W,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDsygvd(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *B,
    int ldb,
    double *W,
    double *work,
    int lwork,
    int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnC hegvd(
    cusolverDnHandle_t handle,

```

(continues on next page)

(continued from previous page)

```

cusolverEigType_t itype,
cusolverEigMode_t jobz,
cublasFillMode_t uplo,
int n,
cuComplex *A,
int lda,
cuComplex *B,
int ldb,
float *W,
cuComplex *work,
int lwork,
int *devInfo);

cusolverStatus_t
cusolverDnZhegv(
cusolverDnHandle_t handle,
cusolverEigType_t itype,
cusolverEigMode_t jobz,
cublasFillMode_t uplo,
int n,
cuDoubleComplex *A,
int lda,
cuDoubleComplex *B,
int ldb,
double *W,
cuDoubleComplex *work,
int lwork,
int *devInfo);

```

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix-pair (A,B). The generalized symmetric-definite eigenvalue problem is

$$\text{eig}(A, B) = \begin{cases} A * V = B * V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1} \\ A * B * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_2} \\ B * A * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

where the matrix B is positive definite. Λ is a real $n \times n$ diagonal matrix. The diagonal elements of Λ are the eigenvalues of (A, B) in ascending order. V is an $n \times n$ orthogonal matrix. The eigenvectors are normalized as follows:

$$\begin{cases} V^H * B * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1}, \text{CUSOLVER_EIG_TYPE_2} \\ V^H * inv(B) * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by `sygvd_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle). If devInfo = i (i > 0 and i ≤ n) and jobz = CUSOLVER_EIG_MODE_NOVECTOR, i off-diagonal elements of an intermediate tridiagonal form did not converge to zero. If devInfo = N + i (i > 0), then the leading minor of order i of B is not positive definite. The factorization of B could not be completed and no eigenvalues or eigenvectors were computed.

if `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthogonal eigenvectors of the matrix A. The eigenvectors are computed by divide and conquer algorithm.

Please visit [cuSOLVER Library Samples - sygv](#) for a code example.

Table 35: **API of sygv**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
itype	host	input	Specifies the problem type to be solved: ▶ <code>itype=CUSOLVER_EIG_TYPE_1</code>: $A^*x = (\text{lambda}) * B^*x$. ▶ <code>itype=CUSOLVER_EIG_TYPE_2</code>: $A^*B^*x = (\text{lambda}) * x$. ▶ <code>itype=CUSOLVER_EIG_TYPE_3</code>: $B^*A^*x = (\text{lambda}) * x$.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
uplo	host	input	Specifies which part of A and B are stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A and B are stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A and B are stored.
n	host	input	Number of rows (or columns) of matrix A and B.
A	device	in/out	<type> array of dimension <code>lda * n</code> with <code>lda</code> is not less than $\max(1, n)$. If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading <code>n</code> -by- <code>n</code> upper triangular part of A contains the upper triangular part of the matrix A. If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading <code>n</code> -by- <code>n</code> lower triangular part of A contains the lower triangular part of the matrix A. On exit, if <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> , and <code>devInfo = 0</code> , A contains the orthonormal eigenvectors of the matrix A. If <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> , the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. <code>lda</code> is not less than $\max(1, n)$.

continues on next page

Table 35 – continued from previous page

Parameter	Memory	In/out	Meaning
B	device	in/out	<type> array of dimension $ldb * n$. If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading n -by- n upper triangular part of B contains the upper triangular part of the matrix B. If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading n -by- n lower triangular part of B contains the lower triangular part of the matrix B. On exit, if <code>devInfo</code> is less than n , B is overwritten by triangular factor U or L from the Cholesky factorization of B.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B. <code>ldb</code> is not less than $\max(1, n)$.
W	device	output	A real array of dimension n . The eigenvalue values of A, sorted so that $W(i) \geq W(i+1)$.
work	device	in/out	Working space, <type> array of size <code>lwork</code> .
Lwork	host	input	Size of work, returned by <code>sygvd_bufferSize</code> .
devInfo	device	output	If <code>devInfo = 0</code> , the operation is successful. If <code>devInfo = -i</code> , the i -th parameter is wrong (not counting handle). If <code>devInfo = i</code> (> 0), <code>devInfo</code> indicates either <code>potrf</code> or <code>syevd</code> is wrong.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $ldb < \max(1, n)$, or `itype` is not 1, 2 or 3, or `jobz` is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or `uplo` is not `CUBLAS_FILL_MODE_LOWER` or `CUBLAS_FILL_MODE_UPPER`).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.13 cusolverDn<t>sygvdx()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsygvdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *B,
    int ldb,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    const float *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnDsygvdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *B,
    int ldb,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    const double *W,
    int *lwork);
```

```
cusolverStatus_t
cusolverDnCsygvdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const cuComplex *B,
    int ldb,
    float vl,
    float vu,
```

(continues on next page)

(continued from previous page)

```

    int il,
    int iu,
    int *h_meig,
    const float *W,
    int *lwork);

cusolverStatus_t
cusolverDnZhegvdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const cuDoubleComplex *B,
    int ldb,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    const double *W,
    int *lwork);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSsygvdx(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *B,
    int ldb,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    float *W,
    float *work,
    int lwork,
    int *devInfo);

cusolverStatus_t
cusolverDnDsygvdx(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,

```

(continues on next page)

(continued from previous page)

```

int n,
double *A,
int lda,
double *B,
int ldb,
double vl,
double vu,
int il,
int iu,
int *h_meig,
double *W,
double *work,
int lwork,
int *devInfo);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnChegvdx(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    cuComplex *B,
    int ldb,
    float vl,
    float vu,
    int il,
    int iu,
    int *h_meig,
    float *W,
    cuComplex *work,
    int lwork,
    int *devInfo);

```

```

cusolverStatus_t
cusolverDnZhegvdx(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    cuDoubleComplex *B,
    int ldb,
    double vl,
    double vu,
    int il,
    int iu,
    int *h_meig,
    double *W,

```

(continues on next page)

(continued from previous page)

```
cuDoubleComplex *work,
int lwork,
int *devInfo);
```

This function computes all or selection of the eigenvalues and optionally eigenvectors of a symmetric (Hermitian) $n \times n$ matrix-pair (A,B). The generalized symmetric-definite eigenvalue problem is

$$\text{eig}(A, B) = \begin{cases} A * V = B * V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1} \\ A * B * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_2} \\ B * A * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

where the matrix B is positive definite. Λ is a real $n \times h_meig$ diagonal matrix. The diagonal elements of Λ are the eigenvalues of (A, B) in ascending order. V is an $n \times h_meig$ orthogonal matrix. h_meig is the number of eigenvalues/eigenvectors computed by the routine, h_meig is equal to n when the whole spectrum (for example, range = CUSOLVER_EIG_RANGE_ALL) is requested. The eigenvectors are normalized as follows:

$$\begin{cases} V^H * B * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1}, \text{CUSOLVER_EIG_TYPE_2} \\ V^H * inv(B) * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

The user has to provide working space which is pointed by input parameter work. The input parameter lwork is size of the working space, and it is returned by `sygvdx_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter devInfo = -i (less than zero), the i-th parameter is wrong (not counting handle). If devInfo = i (i > 0 and i ≤ n) and jobz = CUSOLVER_EIG_MODE_NOVECTOR, i off-diagonal elements of an intermediate tridiagonal form did not converge to zero. If devInfo = n + i (i > 0), then the leading minor of order i of B is not positive definite. The factorization of B could not be completed and no eigenvalues or eigenvectors were computed.

If jobz = CUSOLVER_EIG_MODE_VECTOR, A contains the orthogonal eigenvectors of the matrix A. The eigenvectors are computed by divide and conquer algorithm.

Please visit [cuSOLVER Library Samples - sygvdx](#) for a code example.

Table 36: API of sygvdx

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
itype	host	input	Specifies the problem type to be solved: ▶ itype = CUSOLVER_EIG_TYPE_1: $A * x = (\text{lambda}) * B * x$ ▶ itype = CUSOLVER_EIG_TYPE_2: $A * B * x = (\text{lambda}) * x$ ▶ itype = CUSOLVER_EIG_TYPE_3: $B * A * x = (\text{lambda}) * x$

continues on next page

Table 36 – continued from previous page

Parameter	Memory	In/out	Meaning
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute eigenvalues only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute eigenvalues and eigenvectors.
range	host	input	Specifies options to which selection of eigenvalues and optionally eigenvectors that need to be computed: range = CUSOLVER_EIG_RANGE_ALL : all eigenvalues/eigenvectors will be found, will becomes the classical syevd/heevd routine; range = CUSOLVER_EIG_RANGE_V : all eigenvalues/eigenvectors in the half-open interval $(v_l, v_u]$ will be found; range = CUSOLVER_EIG_RANGE_I : the i_l -th through i_u -th eigenvalues/eigenvectors will be found;
uplo	host	input	Specifies which part of A and B are stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A and B are stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A and B are stored.
n	host	input	Number of rows (or columns) of matrix A and B.
A	device	in/out	<type> array of dimension $lda * n$ with lda is not less than $\max(1, n)$. If uplo = CUBLAS_FILL_MODE_UPPER, the leading n -by- n upper triangular part of A contains the upper triangular part of the matrix A. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n -by- n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and devInfo = 0, A contains the orthonormal eigenvectors of the matrix A. If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda is not less than $\max(1, n)$.

continues on next page

Table 36 – continued from previous page

Parameter	Memory	In/out	Meaning
B	device	in/out	<type> array of dimension $ldb * n$. If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading n -by- n upper triangular part of B contains the upper triangular part of the matrix B. If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading n -by- n lower triangular part of B contains the lower triangular part of the matrix B. On exit, if <code>devInfo</code> is less than n , B is overwritten by triangular factor U or L from the Cholesky factorization of B.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B. <code>ldb</code> is not less than $\max(1, n)$.
v1, vu	host	input	Real values float or double for (C, S) or (Z, D) precision respectively. If <code>range = CUSOLVER_EIG_RANGE_V</code> , the lower and upper bounds of the interval to be searched for eigenvalues. $v1 > vu$. Not referenced if <code>range = CUSOLVER_EIG_RANGE_ALL</code> or <code>range = CUSOLVER_EIG_RANGE_I</code> . Note that, if eigenvalues are very close to each other, it is well known that two different eigenvalues routines might find slightly different number of eigenvalues inside the same interval. This is due to the fact that different eigenvalue algorithms, or even same algorithm but different run might find eigenvalues within some rounding error close to the machine precision. Thus, if the user want to be sure not to miss any eigenvalue within the interval bound, we suggest that, the user subtract/add epsilon (machine precision) to the interval bound such as $(v1=v1-eps, vu=vu+eps]$. this suggestion is valid for any selective routine from cuSolver or LAPACK.
il, iu	host	input	Integer. If <code>range = CUSOLVER_EIG_RANGE_I</code> , the indices (in ascending order) of the smallest and largest eigenvalues to be returned. $1 \leq il \leq iu \leq n$, if $n > 0$; $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if <code>range = CUSOLVER_EIG_RANGE_ALL</code> or <code>range = CUSOLVER_EIG_RANGE_V</code> .
h_meig	host	output	Integer. The total number of eigenvalues found. $0 \leq h_meig \leq n$. If <code>range = CUSOLVER_EIG_RANGE_ALL</code> , $h_meig = n$, and if <code>range = CUSOLVER_EIG_RANGE_I</code> , $h_meig = iu - il + 1$.

continues on next page

Table 36 – continued from previous page

Parameter	Memory	In/out	Meaning
W	device	output	A real array of dimension n. The eigenvalue values of A, sorted so that $W(i) \geq W(i+1)$.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of work, returned by sygvdx_bufferSize.
devInfo	device	output	If devInfo = 0, the operation is successful. If devInfo = -i, the i-th parameter is wrong (not counting handle). If devInfo = i (> 0), devInfo indicates either potrf or syevd is wrong.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $ldb < \max(1, n)$, or itype is not CUSOLVER_EIG_TYPE_1 or CUSOLVER_EIG_TYPE_2 or CUSOLVER_EIG_TYPE_3 or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTORL, or range is not CUSOLVER_EIG_RANGE_ALL or CUSOLVER_EIG_RANGE_V or CUSOLVER_EIG_RANGE_I, or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.14 cusolverDn<t>syevj()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsyevj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params);
```

```
cusolverStatus_t
cusolverDnDsyevj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
```

(continues on next page)

(continued from previous page)

```

    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params);

cusolverStatus_t
cusolverDnCheevj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params);

cusolverStatus_t
cusolverDnZheevj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSsyevj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *W,
    float *work,
    int lwork,
    int *info,
    syevjInfo_t params);

cusolverStatus_t
cusolverDnDsyevj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *W,

```

(continues on next page)

(continued from previous page)

```
double *work,
int lwork,
int *info,
syevjInfo_t params);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCheevj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    float *W,
    cuComplex *work,
    int lwork,
    int *info,
    syevjInfo_t params);
```

```
cusolverStatus_t
cusolverDnZheevj(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    double *W,
    cuDoubleComplex *work,
    int lwork,
    int *info,
    syevjInfo_t params);
```

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A. The standard symmetric eigenvalue problem is

$$A * Q = Q * \Lambda$$

where Λ is a real $n \times n$ diagonal matrix. Q is an $n \times n$ unitary matrix. The diagonal elements of Λ are the eigenvalues of A in ascending order.

syevj has the same functionality as syevd. The difference is that syevd uses QR algorithm and syevj uses Jacobi method. The parallelism of Jacobi method gives GPU better performance on small and medium size matrices. Moreover the user can configure syevj to perform approximation up to certain accuracy.

How does it work?

syevj iteratively generates a sequence of unitary matrices to transform matrix A to the following form

$$V^H * A * V = W + E$$

where W is diagonal and E is symmetric without diagonal.

During the iterations, the Frobenius norm of E decreases monotonically. As E goes down to zero, W is the set of eigenvalues. In practice, Jacobi method stops if

$$\|E\|_F \leq \text{eps} * \|A\|_F$$

where `eps` is the given tolerance.

`syevj` has two parameters to control the accuracy. First parameter is tolerance (`eps`). The default value is machine accuracy but The user can use function `cusolverDnXsyevjSetTolerance` to set a priori tolerance. The second parameter is maximum number of sweeps which controls number of iterations of Jacobi method. The default value is 100 but the user can use function `cusolverDnXsyevjSetMaxSweeps` to set a proper bound. The experiments show 15 sweeps are good enough to converge to machine accuracy. `syevj` stops either tolerance is met or maximum number of sweeps is met.

The Jacobi method has quadratic convergence, so the accuracy is not proportional to number of sweeps. To guarantee certain accuracy, the user should configure tolerance only.

After `syevj`, the user can query residual by function `cusolverDnXsyevjGetResidual` and number of executed sweeps by function `cusolverDnXsyevjGetSweeps`. However the user needs to be aware that residual is the Frobenius norm of E , not accuracy of individual eigenvalue, i.e.

$$residual = \|E\|_F = \|\Lambda - W\|_F$$

The same as `syevd`, the user has to provide working space pointed by input parameter `work`. The input parameter `lwork` is the size of the working space, and it is returned by `syevj_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `info = n+1`, `syevj` does not converge under given tolerance and maximum sweeps.

If the user sets an improper tolerance, `syevj` may not converge. For example, tolerance should not be smaller than machine accuracy.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors V .

Please visit [cuSOLVER Library Samples - syevj](#) for a code example.

Table 37: **API of syevj**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>jobz</code>	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
<code>uplo</code>	host	input	Specifies which part of A is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A is stored.
<code>n</code>	host	input	Number of rows (or columns) of matrix A .

continues on next page

Table 37 – continued from previous page

Parameter	Memory	In/out	Meaning
A	device	in/out	<type> array of dimension $lda * n$ with lda is not less than $\max(1, n)$. If $uplo = CUBLAS_FILL_MODE_UPPER$, the leading n -by- n upper triangular part of A contains the upper triangular part of the matrix A. If $uplo = CUBLAS_FILL_MODE_LOWER$, the leading n -by- n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if $jobz = CUSOLVER_EIG_MODE_VECTOR$, and $info = 0$, A contains the orthonormal eigenvectors of the matrix A. If $jobz = CUSOLVER_EIG_MODE_NOVECTOR$, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
W	device	output	A real array of dimension n . The eigenvalue values of A, in ascending order ie, sorted so that $W(i) \leq W(i+1)$.
work	device	in/out	Working space, <type> array of size $lwork$.
lwork	host	input	Size of work, returned by <code>syevj_bufferSize</code> .
info	device	output	If $info = 0$, the operation is successful. If $info = -i$, the i -th parameter is wrong (not counting handle). If $info = n+1$, <code>syevj</code> does not converge under given tolerance and maximum sweeps.
params	host	in/out	Structure filled with parameters of Jacobi algorithm and results of <code>syevj</code> .

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $jobz$ is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or $uplo$ is not `CUBLAS_FILL_MODE_LOWER` or `CUBLAS_FILL_MODE_UPPER`).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.15 cusolverDn<t>sygvj()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsygvj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *B,
    int ldb,
    const float *W,
    int *lwork,
    syevjInfo_t params);
```

```
cusolverStatus_t
cusolverDnDsygvj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *B,
    int ldb,
    const double *W,
    int *lwork,
    syevjInfo_t params);
```

```
cusolverStatus_t
cusolverDnCsygvj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const cuComplex *B,
    int ldb,
    const float *W,
    int *lwork,
    syevjInfo_t params);
```

```
cusolverStatus_t
cusolverDnZsygvj_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
```

(continues on next page)

(continued from previous page)

```

const cuDoubleComplex *B,
int ldb,
const double *W,
int *lwork,
syevjInfo_t params);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnSsygvj(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *B,
    int ldb,
    float *W,
    float *work,
    int lwork,
    int *info,
    syevjInfo_t params);

```

```

cusolverStatus_t
cusolverDnDsygvj(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *B,
    int ldb,
    double *W,
    double *work,
    int lwork,
    int *info,
    syevjInfo_t params);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverDnCgsvj(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    cuComplex *B,
    int ldb,
    float *W,
    cuComplex *work,

```

(continues on next page)

(continued from previous page)

```

    int lwork,
    int *info,
    syevjInfo_t params);

cusolverStatus_t
cusolverDnZhegvj(
    cusolverDnHandle_t handle,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuDoubleComplex *A,
    int lda,
    cuDoubleComplex *B,
    int ldb,
    double *W,
    cuDoubleComplex *work,
    int lwork,
    int *info,
    syevjInfo_t params);

```

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix-pair (A,B). The generalized symmetric-definite eigenvalue problem is

$$\text{eig}(A, B) = \begin{cases} A * V = B * V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1} \\ A * B * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_2} \\ B * A * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

where the matrix B is positive definite. Λ is a real $n \times n$ diagonal matrix. The diagonal elements of Λ are the eigenvalues of (A, B) in ascending order. V is an $n \times n$ orthogonal matrix. The eigenvectors are normalized as follows:

$$\begin{cases} V^H * B * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1}, \text{CUSOLVER_EIG_TYPE_2} \\ V^H * \text{inv}(B) * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

This function has the same functionality as `sygvj` except that `syevd` in `sygvj` is replaced by `syevj` in `sygvj`. Therefore, `sygvj` inherits properties of `syevj`, the user can use `cusolverDnXsyevjSetTolerance` and `cusolverDnXsyevjSetMaxSweeps` to configure tolerance and maximum sweeps.

However the meaning of residual is different from `syevj`. `sygvj` first computes Cholesky factorization of matrix B,

$$B = L * L^H$$

transform the problem to standard eigenvalue problem, then calls `syevj`.

For example, the standard eigenvalue problem of type I is

$$M * Q = Q * \Lambda$$

where matrix M is symmetric

$$M = L^{-1} * A * L^{-H}$$

The residual is the result of `syevj` on matrix `M`, not `A`.

The user has to provide working space which is pointed by input parameter `work`. The input parameter `lwork` is the size of the working space, and it is returned by `sygvj_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

If output parameter `info = -i` (less than zero), the `i`-th parameter is wrong (not counting handle). If `info = i` ($i > 0$ and $i \leq n$), `B` is not positive definite, the factorization of `B` could not be completed and no eigenvalues or eigenvectors were computed. If `info = n+1`, `syevj` does not converge under given tolerance and maximum sweeps. In this case, the eigenvalues and eigenvectors are still computed because non-convergence comes from improper tolerance of maximum sweeps.

if `jobz = CUSOLVER_EIG_MODE_VECTOR`, `A` contains the orthogonal eigenvectors `V`.

Please visit [cuSOLVER Library Samples - sygvj](#) for a code example.

Table 38: API of `sygvj`

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>itype</code>	host	input	Specifies the problem type to be solved: <code>itype=CUSOLVER_EIG_TYPE_1</code> : $A^*x = (\text{lambda}) * B^*x$. <code>itype=CUSOLVER_EIG_TYPE_2</code> : $A^*B^*x = (\text{lambda}) * x$. <code>itype=CUSOLVER_EIG_TYPE_3</code> : $B^*A^*x = (\text{lambda}) * x$.
<code>jobz</code>	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
<code>uplo</code>	host	input	Specifies which part of <code>A</code> and <code>B</code> are stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of <code>A</code> and <code>B</code> are stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of <code>A</code> and <code>B</code> are stored.
<code>n</code>	host	input	Number of rows (or columns) of matrix <code>A</code> and <code>B</code> .
<code>A</code>	device	in/out	<code><type></code> array of dimension <code>lda * n</code> with <code>lda</code> is not less than $\max(1, n)$. If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading <code>n</code> -by- <code>n</code> upper triangular part of <code>A</code> contains the upper triangular part of the matrix <code>A</code> . If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading <code>n</code> -by- <code>n</code> lower triangular part of <code>A</code> contains the lower triangular part of the matrix <code>A</code> . On exit, if <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> , and <code>info = 0</code> , <code>A</code> contains the orthonormal eigenvectors of the matrix <code>A</code> . If <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> , the contents of <code>A</code> are destroyed.

continues on next page

Table 38 – continued from previous page

Parameter	Memory	In/out	Meaning
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda is not less than $\max(1, n)$.
B	device	in/out	<type> array of dimension ldb * n. If uplo = CUBLAS_FILL_MODE_UPPER, the leading n-by-n upper triangular part of B contains the upper triangular part of the matrix B. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n-by-n lower triangular part of B contains the lower triangular part of the matrix B. On exit, if info is less than n, B is overwritten by triangular factor U or L from the Cholesky factorization of B.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B. ldb is not less than $\max(1, n)$.
W	device	output	A real array of dimension n. The eigenvalue values of A, sorted so that $W(i) \geq W(i+1)$.
work	device	in/out	Working space, <type> array of size lwork.
lwork	host	input	Size of work, returned by sygvj_bufferSize.
info	device	output	If info = 0, the operation is successful. if info = -i, the i-th parameter is wrong (not counting handle). If info = i (> 0), info indicates either B is not positive definite or syevj (called by sygvj) does not converge.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $ldb < \max(1, n)$, or itype is not 1, 2 or 3, or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.3.16 cusolverDn<t>syevjBatched()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnSsyevjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const float *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
);
```

```
cusolverStatus_t
cusolverDnDsyevjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const double *A,
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
);
```

```
cusolverStatus_t
cusolverDnCsyevjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuComplex *A,
    int lda,
    const float *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
);
```

```
cusolverStatus_t
cusolverDnZsyevjBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    const cuDoubleComplex *A,
    int lda,
    const double *W,
    int *lwork,
    syevjInfo_t params,
    int batchSize
);
```

(continues on next page)

(continued from previous page)

);

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnSsyevjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    float *A,
    int lda,
    float *W,
    float *work,
    int lwork,
    int *info,
    syevjInfo_t params,
    int batchSize
);
```

```
cusolverStatus_t
cusolverDnDsyevjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    double *A,
    int lda,
    double *W,
    double *work,
    int lwork,
    int *info,
    syevjInfo_t params,
    int batchSize
);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverDnCcheevjBatched(
    cusolverDnHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int n,
    cuComplex *A,
    int lda,
    float *W,
    cuComplex *work,
    int lwork,
    int *info,
    syevjInfo_t params,
    int batchSize
);
```

```
cusolverStatus_t
cusolverDnZcheevjBatched(
    cusolverDnHandle_t handle,
```

(continues on next page)

(continued from previous page)

```

cusolverEigMode_t jobz,
cublasFillMode_t uplo,
int n,
cuDoubleComplex *A,
int lda,
double *W,
cuDoubleComplex *work,
int lwork,
int *info,
syevjInfo_t params,
int batchSize
);

```

This function computes eigenvalues and eigenvectors of a sequence of symmetric (Hermitian) $n \times n$ matrices

$$A_j * Q_j = Q_j * \Lambda_j$$

where Λ_j is a real $n \times n$ diagonal matrix. Q_j is an $n \times n$ unitary matrix. The diagonal elements of Λ_j are the eigenvalues of A_j in either ascending order or non-sorting order.

`syevjBatched` performs `syevj` on each matrix. It requires that all matrices are of the same size n and are packed in contiguous way,

$$A = \begin{pmatrix} A_0 & A_1 & \dots \end{pmatrix}$$

Each matrix is column-major with leading dimension `lda`, so the formula for random access is $A_k(i, j) = A[i + lda * j + lda * n * k]$.

The parameter `W` also contains eigenvalues of each matrix in contiguous way,

$$W = \begin{pmatrix} W_0 & W_1 & \dots \end{pmatrix}$$

The formula for random access of `W` is $W_k(j) = W[j + n * k]$.

Except for tolerance and maximum sweeps, `syevjBatched` can either sort the eigenvalues in ascending order (default) or chose as-is (without sorting) by the function `cusolverDnXsyevjSetSortEig`. If the user packs several tiny matrices into diagonal blocks of one matrix, non-sorting option can separate spectrum of those tiny matrices.

`syevjBatched` cannot report residual and executed sweeps by function `cusolverDnXsyevjGetResidual` and `cusolverDnXsyevjGetSweeps`. Any call of the above two returns `CUSOLVER_STATUS_NOT_SUPPORTED`. The user needs to compute residual explicitly.

The user has to provide working space pointed by input parameter `work`. The input parameter `lwork` is the size of the working space, and it is returned by `syevjBatched_bufferSize()`. Please note that the size in bytes of the working space is equal to `sizeof(<type>) * lwork`.

The output parameter `info` is an integer array of size `batchSize`. If the function returns `CUSOLVER_STATUS_INVALID_VALUE`, the first element `info[0] = -i` (less than zero) indicates i -th parameter is wrong (not counting handle). Otherwise, if `info[i] = n+1`, `syevjBatched` does not converge on i -th matrix under given tolerance and maximum sweeps.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A_j contains the orthonormal eigenvectors V_j .

Please visit [cuSOLVER Library Samples - syevjBatched](#) for a code example.

Table 39: API of syevjBatched

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute eigenvalues only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute eigenvalues and eigenvectors.
uplo	host	input	Specifies which part of A_j is stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A_j is stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A_j is stored.
n	host	input	Number of rows (or columns) of matrix each A_j .
A	device	in/out	<type> array of dimension $lda * n * batchSize$ with lda is not less than $\max(1, n)$. If uplo = CUBLAS_FILL_MODE_UPPER, the leading n -by- n upper triangular part of A_j contains the upper triangular part of the matrix A_j . If uplo = CUBLAS_FILL_MODE_LOWER, the leading n -by- n lower triangular part of A_j contains the lower triangular part of the matrix A_j . On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and $info[j] = 0$, A_j contains the orthonormal eigenvectors of the matrix A_j . If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A_j are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A_j .
W	device	output	A real array of dimension $n * batchSize$. It stores the eigenvalues of A_j in ascending order or non-sorting order.
work	device	in/out	<type> array of size lwork, workspace.
lwork	host	input	Size of work, returned by syevjBatched_bufferSize.
info	device	output	An integer array of dimension batchSize. If CUSOLVER_STATUS_INVALID_VALUE is returned, $info[0] = -i$ (less than zero) indicates i -th parameter is wrong (not counting handle). Otherwise, if $info[i] = 0$, the operation is successful. If $info[i] = n+1$, syevjBatched does not converge on i -th matrix under given tolerance and maximum sweeps.
params	host	in/out	Structure filled with parameters of Jacobi algorithm.

continues on next page

Table 39 – continued from previous page

Parameter	Memory	In/out	Meaning
batchSize	host	input	Number of matrices.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER), or batchSize < 0 .

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4. Dense Linear Solver Reference (64-bit API)

This section describes linear solver 64-bit API of cuSolverDN, including Cholesky factorization, LU with partial pivoting and QR factorization.

8.4.4.1 cusolverDnXpotrf()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXpotrf_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t
cusolverDnXpotrf(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
```

(continues on next page)

(continued from previous page)

```

int64_t lda,
cudaDataType computeType,
void *bufferOnDevice,
size_t workspaceInBytesOnDevice,
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int *info )

```

computes the Cholesky factorization of a Hermitian positive-definite matrix using the generic API interface.

A is a $n \times n$ Hermitian matrix, only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used. The function will leave the other part untouched.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, only lower triangular part of A is processed, and replaced by lower triangular Cholesky factor L.

$$A = L * L^H$$

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, only upper triangular part of A is processed, and replaced by upper triangular Cholesky factor U.

$$A = U^H * U$$

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXpotrf_bufferSize()`.

If Cholesky factorization failed, i.e. some leading minor of A is not positive definite, or equivalently some diagonal elements of L or U is not a real number. The output parameter `info` would indicate smallest leading minor of A which is not positive definite.

If output parameter `info` = -i (less than zero), the i-th parameter is wrong (not counting handle).

Currently, `cusolverDnXpotrf` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xpotrf](#) for a code example.

Algorithms supported by `cusolverDnXpotrf`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXpotrf_bufferSize` and `cusolverDnXpotrf`:

Table 40: **API of potrf**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.

continues on next page

Table 40 – continued from previous page

Parameter	Memory	In/out	Meaning
n	host	input	Number of rows and columns of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension $lda * n$ with lda is not less than $\max(1, n)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXpotrf_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceInBytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXpotrf_bufferSize</code> .
info	device	output	If <code>info = 0</code> , the Cholesky factorization is successful. If <code>info = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle). If <code>info = i</code> , the leading minor of order <i>i</i> is not positive definite.

The generic API has two different types, `dataTypeA` is data type of the matrix A, `computeType` is compute type of the operation. `cusolverDnXpotrf` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SPOTRF
CUDA_R_64F	CUDA_R_64F	DPOTRF
CUDA_C_32F	CUDA_C_32F	CPOTRF
CUDA_C_64F	CUDA_C_64F	ZPOTRF

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.2 cusolverDnXpotrs()

```
cusolverStatus_t
cusolverDnXpotrs(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasFillMode_t uplo,
    int64_t n,
    int64_t nrhs,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t dataTypeB,
    void *B,
    int64_t ldb,
    int *info)
```

This function solves a system of linear equations

$$A * X = B$$

where A is a $n \times n$ Hermitian matrix, only lower or upper part is meaningful using the generic API interface. The input parameter `uplo` indicates which part of the matrix is used. The function will leave the other part untouched.

The user has to call `cusolverDnXpotrf` first to factorize matrix A. If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, A is lower triangular Cholesky factor L corresponding to $A = L * L^H$. If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, A is upper triangular Cholesky factor U corresponding to $A = U^H * U$.

The operation is in-place, i.e. matrix X overwrites matrix B with the same leading dimension `ldb`.

If output parameter `info` = -i (less than zero), the i-th parameter is wrong (not counting handle).

Currently, `cusolverDnXpotrs` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xpotrf](#) for a code example.

Algorithms supported by cusolverDnXpotrs

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXpotrs`:

Table 41: API of potrs

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.
n	host	input	Number of rows and columns of matrix A.

continues on next page

Table 41 – continued from previous page

Parameter	Memory	In/out	Meaning
nrhs	host	input	Number of columns of matrix X and B.
dataTypeA	host	in	Data type of array A.
A	device	input	Array of dimension $lda * n$ with lda is not less than $\max(1, n)$. A is either lower Cholesky factor L or upper Cholesky factor U.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeB	host	in	Data type of array B.
B	device	in/out	Array of dimension $ldb * nrhs$. ldb is not less than $\max(1, n)$. As an input, B is right hand side matrix. As an output, B is the solution matrix.
info	device	output	If $info = 0$, the Cholesky factorization is successful. if $info = -i$, the i -th parameter is wrong (not counting handle).

The generic API has two different types, `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix B. `cusolverDnXpotrs` only supports the following four combinations.

Valid combination of data type and compute type

dataTypeA	dataTypeB	Meaning
CUDA_R_32F	CUDA_R_32F	SPOTRS
CUDA_R_64F	CUDA_R_64F	DPOTRS
CUDA_C_32F	CUDA_C_32F	CPOTRS
CUDA_C_64F	CUDA_C_64F	ZPOTRS

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, $nrhs < 0$, $lda < \max(1, n)$ or $ldb < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.3 cusolverDnXgetrf()

The helper function below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t cusolverDnXgetrf_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    int64_t m,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The function below

```
cusolverStatus_t
cusolverDnXgetrf(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    int64_t m,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    int64_t *ipiv,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info )
```

computes the LU factorization of a $m \times n$ matrix

$$P * A = L * U$$

where A is a $m \times n$ matrix, P is a permutation matrix, L is a lower triangular matrix with unit diagonal, and U is an upper triangular matrix using the generic API interface.

If LU factorization failed, i.e. matrix A (U) is singular, The output parameter `info=i` indicates $U(i, i) = 0$.

If output parameter `info = -i` (less than zero), the *i*-th parameter is wrong (not counting handle).

If `ipiv` is null, no pivoting is performed. The factorization is $A=L*U$, which is not numerically stable.

No matter LU factorization failed or not, the output parameter `ipiv` contains pivoting sequence, row *i* is interchanged with row `ipiv(i)`.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXgetrf_bufferSize()`.

The user can combine `cusolverDnXgetrf` and `cusolverDnGetrs` to complete a linear solver.

Currently, `cusolverDnXgetrf` supports two algorithms. To select legacy implementation, the user has to call `cusolverDnSetAdvOptions`.

Please visit [cuSOLVER Library Samples - Xgetrf](#) for a code example.

Algorithms supported by cusolverDnXgetrf

CUSOLVER_ALG_0 NULL	or	Default algorithm. The fastest, requires a large workspace of $m \times n$ elements.
CUSOLVER_ALG_1		Legacy implementation

List of input arguments for `cusolverDnXgetrf_bufferSize` and `cusolverDnXgetrf`:

Table 42: API of `cusolverDnXgetrf`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	<type> array of dimension $lda * n$ with lda is not less than $\max(1, m)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
ipiv	device	output	Array of size at least $\min(m, n)$, containing pivot indices.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXgetrf_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceInBytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXgetrf_bufferSize</code> .
info	device	output	If <code>info = 0</code> , the LU factorization is successful. If <code>info = -i</code> , the i -th parameter is wrong (not counting handle). If <code>info = i</code> , the $U(i, i) = 0$.

The generic API has two different types, `dataTypeA` is data type of the matrix A, `computeType` is compute type of the operation. `cusolverDnXgetrf` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SGETRF
CUDA_R_64F	CUDA_R_64F	DGETRF
CUDA_C_32F	CUDA_C_32F	CGETRF
CUDA_C_64F	CUDA_C_64F	ZGETRF

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.4 cusolverDnXgetrs()

```
cusolverStatus_t
cusolverDnXgetrs(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasOperation_t trans,
    int64_t n,
    int64_t nrhs,
    cudaDataType dataTypeA,
    const void *A,
    int64_t lda,
    const int64_t *ipiv,
    cudaDataType dataTypeB,
    void *B,
    int64_t ldb,
    int *info )
```

This function solves a linear system of multiple right-hand sides

$$op(A) * X = B$$

where A is an $n \times n$ matrix, and was LU-factored by `cusolverDnXgetrf`, that is, lower triangular part of A is L, and upper triangular part (including diagonal elements) of A is U. B is an $n \times nrhs$ right-hand side matrix using the generic API interface.

The input parameter `trans` is defined by

$$op(A) = \begin{cases} A & \text{if } trans == \text{CUBLAS_OP_N} \\ A^T & \text{if } trans == \text{CUBLAS_OP_T} \\ A^H & \text{if } trans == \text{CUBLAS_OP_C} \end{cases}$$

The input parameter `ipiv` is an output of `cusolverDnXgetrf`. It contains pivot indices, which are used to permute right-hand sides.

If output parameter `info` = `-i` (less than zero), the `i`-th parameter is wrong (not counting handle).

The user can combine `cusolverDnXgetrf` and `cusolverDnXgetrs` to complete a linear solver.

Currently, `cusolverDnXgetrs` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xgetrf](#) for a code example.

Algorithms supported by `cusolverDnXgetrs`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXgetrs`:

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>trans</code>	host	input	Operation $\text{op}(A)$ that is non- or (conj.) transpose.
<code>n</code>	host	input	Number of rows and columns of matrix A.
<code>nrhs</code>	host	input	Number of right-hand sides.
<code>dataTypeA</code>	host	in	Data type of array A.
<code>A</code>	device	input	Array of dimension $\text{lda} * n$ with lda is not less than $\max(1, n)$.
<code>lda</code>	host	input	Leading dimension of two-dimensional array used to store matrix A.
<code>ipiv</code>	device	input	Array of size at least n , containing pivot indices.
<code>dataTypeB</code>	host	in	Data type of array B.
<code>B</code>	device	output	<type> array of dimension $\text{ldb} * \text{nrhs}$ with ldb is not less than $\max(1, n)$.
<code>ldb</code>	host	input	Leading dimension of two-dimensional array used to store matrix B.
<code>info</code>	device	output	If <code>info</code> = 0, the operation is successful. If <code>info</code> = <code>-i</code> , the <code>i</code> -th parameter is wrong (not counting handle).

The generic API has two different types: `dataTypeA` is data type of the matrix A and `dataTypeB` is data type of the matrix B. `cusolverDnXgetrs` only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	dataTypeB	Meaning
CUDA_R_32F	CUDA_R_32F	SGETRS
CUDA_R_64F	CUDA_R_64F	DGETRS
CUDA_C_32F	CUDA_C_32F	CGETRS
CUDA_C_64F	CUDA_C_64F	ZGETRS

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$ or $ldb < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.5 cusolverDnXgeqrf()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXgeqrf_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    int64_t m,
    int64_t n,
    cudaDataType dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType dataTypeTau,
    const void *tau,
    cudaDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t cusolverDnXgeqrf(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    int64_t m,
    int64_t n,
    cudaDataType dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType dataTypeTau,
    void *tau,
    cudaDataType computeType,
    void *bufferOnDevice,
```

(continues on next page)

(continued from previous page)

```

size_t workspaceInBytesOnDevice,
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int *info )

```

computes the QR factorization of a $m \times n$ matrix

$$A = Q * R$$

where A is an $m \times n$ matrix, Q is a $m \times n$ matrix, and R is an $n \times n$ upper triangular matrix using the generic API interface.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXgeqrf_bufferSize()`.

The matrix R is overwritten in upper triangular part of A, including diagonal elements.

The matrix Q is not formed explicitly, instead, a sequence of householder vectors are stored in lower triangular part of A. The leading nonzero element of householder vector is assumed to be 1 such that output parameter TAU contains the scaling factor τ . If v is original householder vector, q is the new householder vector corresponding to τ , satisfying the following relation

$$I - 2 * v * v^H = I - \tau * q * q^H$$

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle).

Currently, `cusolverDnXgeqrf` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xgeqrf](#) for a code example.

Algorithms supported by `cusolverDnXgeqrf`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXgeqrf_bufferSize` and `cusolverDnXgeqrf`:

Table 44: API of `geqrf`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension $lda * n$ with lda is not less than $\max(1, m)$.

continues on next page

Table 44 – continued from previous page

Parameter	Memory	In/out	Meaning
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeTau	host	in	Data type of array tau.
tau	device	output	Array of dimension at least $\min(m, n)$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXgeqrf_bufferSize.
bufferOnHost	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceInBytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXgeqrf_bufferSize.
info	device	output	If <code>info = 0</code> , the QR factorization is successful. If <code>info = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle).

The generic API has two different types, `dataTypeA` is data type of the matrix A, `dataTypeTau` is data type of the array tau and `computeType` is compute type of the operation. `cusolverDnXgeqrf` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SGEQRF
CUDA_R_64F	CUDA_R_64F	DGEQRF
CUDA_C_32F	CUDA_C_32F	CGEQRF
CUDA_C_64F	CUDA_C_64F	ZGEQRF

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.6 cusolverDnXsytrs()

The helper functions below can calculate the sizes needed for pre-allocated buffers.

```
cusolverStatus_t
cusolverDnXsytrs_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int64_t n,
    int64_t nrhs,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    const int64_t *devIpiv,
    cudaDataType_t dataTypeB,
    void *B,
    int64_t ldb,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost);
```

The following routine:

```
cusolverStatus_t cusolverDnXsytrs(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    int64_t n,
    int64_t nrhs,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    const int64_t *devIpiv,
    cudaDataType_t dataTypeB,
    void *B,
    int64_t ldb,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *devInfo);
```

solves a system of linear equations using the generic API interface.

A contains the factorization from `cusolverDn<t>sytrf()`, only lower or upper part is meaningful, the other part is not touched.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, the details of the factorization are stores as:

$$A = L * D * L^T$$

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, the details of the factorization are stores as:

$$A = U * D * U^T$$

The user has to provide the pivot indices that can be obtained by `cusolverDn<t>sytrf()` as well as device and host work spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` and `workspaceInBytesOnHost` are sizes in bytes of the device and host work spaces, and they are returned by `cusolverDnXsytrs_bufferSize()`. To factorize and solve the symmetric system without pivoting, the user should set `devIpiv = NULL` when calling `cusolverDn<t>sytrf` and `cusolverDnXsytrs`.

If output parameter `devInfo = -i` (less than zero), the *i*-th parameter is wrong (not counting handle).

List of input arguments for `cusolverDnXsytrs_bufferSize` and `cusolverDnXsytrs`:

Table 45: **API of sytrs**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.
n	host	input	Number of rows and columns of matrix A.
nrhs	host	input	Number of right-hand sides.
dataTypeA	host	in	Data type of array A.
A	device	input	Array of dimension $lda * n$ with lda is not less than $\max(1, n)$.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
devI piv	device	input	Array of size at least n , containing pivot indices. To solve the system without pivoting, set the argument NULL.
dataTypeB	host	in	Data type of array B.
B	device	in/out	Array of dimension $ldb * nrhs$ with ldb is not less than $\max(1, nrhs)$.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B.
bufferOnDe- vice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceIn- BytesOnDe- vice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXsytrs_bufferSize</code> .
bufferOn- Host	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceIn- BytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXsytrs_bufferSize</code> .
devInfo	device	output	If <code>devInfo = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle). Otherwise, it sets <code>devInfo = 0</code> indicating all parameters are valid.

The generic API has two different types: `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix A. `cusolverDnXsytrs` only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeB	Meaning
CUDA_R_32F	CUDA_R_32F	SSYTRS
CUDA_R_64F	CUDA_R_64F	DSYTRS
CUDA_C_32F	CUDA_C_32F	CSYTRS
CUDA_C_64F	CUDA_C_64F	ZSYTRS

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

Data type is not supported.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.7 cusolverDnXtrtri()

The helper functions below can calculate the sizes needed for pre-allocated buffers.

```
cusolverStatus_t
cusolverDnXtrtri_bufferSize(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    cublasDiagType_t diag,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost);
```

The following routine:

```
cusolverStatus_t
cusolverDnXtrtri(
    cusolverDnHandle_t handle,
    cublasFillMode_t uplo,
    cublasDiagType_t diag,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
```

(continues on next page)

(continued from previous page)

```
size_t workspaceInBytesOnHost,
int *info);
```

computes the inverse of a triangular matrix using the generic API interface.

A is an $n \times n$ triangular matrix, only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used. The function will leave the other part untouched.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, only lower triangular part of A is processed, and replaced by lower triangular inverse.

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, only upper triangular part of A is processed, and replaced by upper triangular inverse.

The user has to provide device and host work spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` and `workspaceInBytesOnHost` are sizes in bytes of the device and host work spaces, and they are returned by `cusolverDnXtrtri_bufferSize()`.

If matrix inversion fails, the output parameter `info = i` shows $A(i, i) = 0$.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle).

Please visit [cuSOLVER Library Samples - Xtrtri](#) for a code example.

List of input arguments for `cusolverDnXtrtri_bufferSize` and `cusolverDnXtrtri`:

Table 46: **API of trtri**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>uplo</code>	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced.
<code>diag</code>	host	input	The enumerated unit diagonal type.
<code>n</code>	host	input	Number of rows and columns of matrix A.
<code>dataTypeA</code>	host	in	Data type of array A.
<code>A</code>	device	in/out	Array of dimension <code>lda * n</code> with <code>lda</code> is not less than $\max(1, n)$.
<code>lda</code>	host	input	Leading dimension of two-dimensional array used to store matrix A.
<code>bufferOnDevice</code>	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
<code>workspaceInBytesOnDevice</code>	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXtrtri_bufferSize</code> .
<code>bufferOnHost</code>	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
<code>workspaceInBytesOnHost</code>	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXtrtri_bufferSize</code> .

continues on next page

Table 46 – continued from previous page

Parameter	Memory	In/out	Meaning
info	device	output	If info = 0, the matrix inversion succeeded. If info = -i, the i-th parameter is wrong (not counting handle). If info = i, $A(i,i) = 0$.

Valid data types

DataTypeA	Meaning
CUDA_R_32F	STRTRI
CUDA_R_64F	DTRTRI
CUDA_C_32F	CTRTRI
CUDA_C_64F	ZTRTRI

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_NOT_SUPPORTED

Data type is not supported.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$ or $lda < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.4.8 cusolverDnXlarft()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t cusolverDnXlarft_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverDirectMode_t direct,
    cusolverStorevMode_t storev,
    int64_t n,
    int64_t k,
    cudaDataType dataTypeV,
    const void *V,
    int64_t ldv,
    cudaDataType dataTypeTau,
    const void *tau,
    cudaDataType dataTypeT,
    void *T,
    int64_t ldt,
    cudaDataType computeType,
```

(continues on next page)

(continued from previous page)

```
size_t *workspaceInBytesOnDevice,
size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t cusolverDnXlarft(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverDirectMode_t direct,
    cusolverStorevMode_t storev,
    int64_t n,
    int64_t k,
    cudaDataType_t dataTypeV,
    const void *V,
    int64_t ldv,
    cudaDataType_t dataTypeTau,
    const void *tau,
    cudaDataType_t dataTypeT,
    void *T,
    int64_t ldt,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost)
```

forms the triangular factor T of a real block reflector H of order n , which is defined as a product of k elementary reflectors. If:

- `direct == CUBLAS_DIRECT_FORWARD`: $H = H(1)H(2)\dots H(k)$ and T is upper triangular;
- `direct == CUBLAS_DIRECT_BACKWARD`: $H = H(k)\dots H(2)H(1)$ and T is lower triangular.

Only `storev == CUBLAS_STOREV_COLUMNWISE` is supported, which indicates that the vector defining the elementary reflector $H(i)$ is stored in the i -th column of the array V , and $H = I - V * T * V^T$ ($H = I - V * T * V^H$ for complex types).

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXlarft_bufferSize()`.

Currently, only $n \geq k$ scenario is supported.

Table 47: **API of larft**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>direct</code>	host	input	Specifies the order in which the elementary reflectors are multiplied to form the block reflector.

continues on next page

Table 47 – continued from previous page

Parameter	Memory	In/out	Meaning
storev	host	input	Specifies how the vectors which define the elementary reflectors are stored.
n	host	input	The order of the block reflector H. $n \geq 0$.
k	host	input	The order of the triangular factor T (= the number of elementary reflectors). $k \geq 1$.
dataTypeV	host	input	Data type of array V.
V	device	input	The matrix V of dimension $lda * k$.
ldv	host	input	Leading dimension of the array V. $ldv \geq \max(1, n)$.
dataTypeTau	host	input	Data type of array tau.
tau	device	input	Dimension k. $\tau(i)$ must contain the scalar factor of the elementary reflector $H(i)$.
dataTypeT	host	input	Data type of array T.
T	device	output	Dimension $ldt * k$. The $k \times k$ triangular factor T of the block reflector. If <code>direct == CUBLAS_DIRECT_FORWARD</code> , T is upper triangular; if <code>direct == CUBLAS_DIRECT_BACKWARD</code> , T is lower triangular.
ldt	host	input	The leading dimension of the array T. $ldt \geq k$.
computeType	host	input	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXlarft_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceInBytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXlarft_bufferSize</code> .

The generic API has four different types:

- `dataTypeV` is data type of the array V
- `dataTypeTau` is data type of the array tau
- `dataTypeT` is data type of the array T
- `computeType` is compute type of the operation

`cusolverDnXlarft` only supports the following four combinations.

Valid combinations of data types and compute types

DataTypeV	DataTypeTau	DataTypeT	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SLARFT
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DLARFT
CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CLARFT
CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZLARFT

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n == 0$, $k > n$, or $storev == \text{CUBLAS_STOREV_ROWWISE}$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5. Dense Eigenvalue Solver Reference (64-bit API)

This section describes eigenvalue solver API of cuSolverDN, including bidiagonalization and SVD.

8.4.5.1 cusolverDnXgesvd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```

cusolverStatus_t
cusolverDnXgesvd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    signed char jobu,
    signed char jobvt,
    int64_t m,
    int64_t n,
    cudaDataType dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType dataTypeS,
    const void *S,
    cudaDataType dataTypeU,
    const void *U,
    int64_t ldu,
    cudaDataType dataTypeVT,
    const void *VT,
    int64_t ldvt,
    cudaDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)

```


The following routine:

```
cusolverStatus_t
cusolverDnXgesvd(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    signed char jobu,
    signed char jobvt,
    int64_t m,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType_t dataTypeS,
    void *S,
    cudaDataType_t dataTypeU,
    void *U,
    int64_t ldu,
    cudaDataType_t dataTypeVT,
    void *VT,
    int64_t ldvt,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info)
```

This function computes the singular value decomposition (SVD) of an $m \times n$ matrix A and corresponding the left and/or right singular vectors. The SVD is written

$$A = U * \Sigma * V^H$$

where Σ is an $m \times n$ matrix which is zero except for its $\min(m, n)$ diagonal elements, U is an $m \times m$ unitary matrix, and V is an $n \times n$ unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXgesvd_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `bdsqr` did not converge, `info` specifies how many superdiagonals of an intermediate bidiagonal form did not converge to zero.

Currently, `cusolverDnXgesvd` supports only the default algorithm.

Algorithms supported by `cusolverDnXgesvd`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

Please visit [cuSOLVER Library Samples - Xgesvd](#) for a code example.

Remark 1: `gesvd` only supports $m \geq n$.

Remark 2: the routine returns V^H , not V .

List of input arguments for `cusolverDnXgesvd_bufferSize` and `cusolverDnXgesvd`:

Table 48: **API of `cusolverDnXgesvd`**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
jobu	host	input	Specifies options for computing all or part of the matrix U: = 'A': all m columns of U are returned in array U; = 'S': the first min(m,n) columns of U (the left singular vectors) are returned in the array U; = 'O': the first min(m,n) columns of U (the left singular vectors) are overwritten on the array A; = 'N': no columns of U (no left singular vectors) are computed.
jobvt	host	input	Specifies options for computing all or part of the matrix V**T: = 'A': all N rows of V**T are returned in the array VT; = 'S': the first min(m,n) rows of V**T (the right singular vectors) are returned in the array VT; = 'O': the first min(m,n) rows of V**T (the right singular vectors) are overwritten on the array A; = 'N': no rows of V**T (no right singular vectors) are computed.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
dataTypeA	host	input	Data type of array A.
A	device	in/out	Array of dimension $lda * n$ with lda is not less than $\max(1, m)$. On exit, if <code>jobu</code> = 'O', A is overwritten with U; if <code>jobvt</code> = 'O', A is overwritten with VT; otherwise, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeS	host	input	Data type of array S.
S	device	output	Real array of dimension $\min(m, n)$. The singular values of A, sorted so that $S(i) \geq S(i+1)$.
dataTypeU	host	input	Data type of array U.
U	device	output	Array of dimension $ldu * m$ with ldu is not less than $\max(1, m)$. If <code>jobu</code> = 'A', U contains the $m \times m$ unitary matrix U. If <code>jobu</code> = 'S', U contains the first min(m,n) columns of U. If <code>jobu</code> = 'N' or 'O', U is not referenced.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U. If <code>jobu</code> = 'A' or 'S', $ldu \geq \max(1, m)$. Otherwise, $ldu \geq 1$.

continues on next page

Table 48 – continued from previous page

Parameter	Memory	In/out	Meaning
dataTypeVT	host	input	Data type of array VT.
VT	device	output	Array of dimension $ldvt * n$ with $ldvt$ is not less than $\max(1, n)$. If $jobvt = 'A'$, VT contains the $n \times n$ unitary matrix $V^{*}T$. If $jobvt = 'S'$, VT contains the first $\min(m, n)$ rows of $V^{*}T$. If $jobvt = 'N'$ or $'O'$, VT is not referenced
ldvt	host	input	Leading dimension of two-dimensional array used to store matrix VT. If $jobvt = 'A'$, $ldvt \geq \max(1, n)$. If $jobvt = 'S'$, $ldvt \geq \max(1, \min(m, n))$. Otherwise, $ldvt \geq 1$.
computeType	host	input	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXgesvd_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceInBytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXgesvd_bufferSize</code> .
info	device	output	If $info = 0$, the operation is successful. If $info = -i$, the i -th parameter is wrong (not counting handle). If $info > 0$, $info$ indicates how many superdiagonals of an intermediate bidiagonal form did not converge to zero.

The generic API has three different types, `dataTypeA` is data type of the matrix A, `dataTypeS` is data type of the vector S and `dataTypeU` is data type of the matrix U, `dataTypeVT` is data type of the matrix VT, `computeType` is compute type of the operation. `cusolverDnXgesvd` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	DataTypeS	DataTypeU	DataTypeVT	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SGESVD
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DGESVD
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CGESVD
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZGESVD

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$, or $lda < \max(1, m)$, or $ldu < 1$, or if `jobu` is 'S' or 'A' and $ldu < m$, or $ldvt < 1$, or if `jobvt` is 'A' and $ldvt < n$, or if `jobvt` is 'S' and $ldvt < \min(m, n)$, or `jobu, jobvt` are none of 'N', 'O', 'S', 'A', or `jobu = jobvt = 'O'`).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.2 cusolverDnXgesvdp()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXgesvdp_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    int econ,
    int64_t m,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t dataTypeS,
    const void *S,
    cudaDataType_t dataTypeU,
    const void *U,
    int64_t ldu,
    cudaDataType_t dataTypeV,
    const void *V,
    int64_t ldv,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The routine below:

```
cusolverStatus_t
cusolverDnXgesvdp(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    int econ,
    int64_t m,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType_t dataTypeS,
    void *S,
    cudaDataType_t dataTypeU,
    void *U,
    int64_t ldu,
    cudaDataType_t dataTypeV,
```

(continues on next page)

(continued from previous page)

```

void *V,
int64_t ldv,
cudaDataType computeType,
void *bufferOnDevice,
size_t workspaceInBytesOnDevice,
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int *d_info,
double *h_err_sigma)

```

This function computes the singular value decomposition (SVD) of an $m \times n$ matrix A and corresponding the left and/or right singular vectors. The SVD is written

$$A = U * \Sigma * V^H$$

where Σ is an $m \times n$ matrix which is zero except for its $\min(m, n)$ diagonal elements, U is an $m \times m$ unitary matrix, and V is an $n \times n$ unitary matrix. The diagonal elements of Σ are the singular values of A ; they are real and non-negative, and are returned in descending order. The first $\min(m, n)$ columns of U and V are the left and right singular vectors of A .

`cusolverDnXgesvdp` combines polar decomposition in [14] and `cusolverDnXsyevd` to compute SVD. It is much faster than `cusolverDnXgesvd` which is based on QR algorithm. However polar decomposition in [14] may not deliver a full unitary matrix when the matrix A has a singular value close to zero. To workaround the issue when the singular value is close to zero, we add a small perturbation so polar decomposition can deliver the correct result. The consequence is inaccurate singular values shifted by this perturbation. The output parameter `h_err_sigma` is the magnitude of this perturbation. In other words, `h_err_sigma` shows the accuracy of SVD.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXgesvdp_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle).

Currently, `cusolverDnXgesvdp` supports only the default algorithm.

Algorithms supported by `cusolverDnXgesvdp`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

Please visit [cuSOLVER Library Samples - Xgesvdp](#) for a code example.

Remark 1: `gesvdp` supports $n \geq m$ as well.

Remark 2: the routine returns V , not V^H

List of input arguments for `cusolverDnXgesvdp_bufferSize` and `cusolverDnXgesvdp`:

Table 49: **API of `cusolverDnXgesvdp`**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the <code>cuSolverDN</code> library context.

continues on next page

Table 49 – continued from previous page

Parameter	Memory	In/out	Meaning
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
jobz	host	input	Specifies options to either compute singular values only or compute singular vectors as well: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute singular values only. <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute singular values and singular vectors.
econ	host	input	<code>econ = 1</code> for economy size for U and V.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
dataTypeA	host	input	Data type of array A.
A	device	in/out	Array of dimension <code>lda * n</code> with <code>lda</code> is not less than <code>max(1,m)</code> . On exit, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeS	host	input	Data type of array S.
S	device	output	Real array of dimension <code>min(m,n)</code> . The singular values of A, sorted so that <code>S(i) >= S(i+1)</code> .
dataTypeU	host	input	Data type of array U.
U	device	output	Array of dimension <code>ldu * m</code> with <code>ldu</code> is not less than <code>max(1,m)</code> . U contains the $m \times m$ unitary matrix U. If <code>econ=1</code> , only reports first <code>min(m,n)</code> columns of U.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U.
dataTypeV	host	input	Data type of array V.
V	device	output	Array of dimension <code>ldv * n</code> with <code>ldv</code> is not less than <code>max(1,n)</code> . V contains the $n \times n$ unitary matrix V. if <code>econ=1</code> , only reports first <code>min(m,n)</code> columns of V.
ldv	host	input	Leading dimension of two-dimensional array used to store matrix V.
computeType	host	input	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type <code>void</code> of size <code>workspaceInBytesOnDevice</code> bytes.

continues on next page

Table 49 – continued from previous page

Parameter	Memory	In/out	Meaning
workspaceIn-BytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXgesvdp_bufferSize.
bufferOn-Host	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceIn-BytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXgesvdp_bufferSize.
info	device	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle).
h_err_sigma	host	output	Magnitude of the perturbation, showing the accuracy of SVD.

The generic API has three different types, dataTypeA is data type of the matrix A, dataTypeS is data type of the vector S and dataTypeU is data type of the matrix U, dataTypeV is data type of the matrix V, computeType is compute type of the operation. cusolverDnXgesvdp only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeS	DataTypeU	DataTypeV	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SGESVDP
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DGESVDP
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CGESVDP
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZGESVDP

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$ or $ldu < \max(1, m)$ or $ldv < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.3 cusolverDnXgesvdr()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXgesvdr_bufferSize (
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    signed char jobu,
    signed char jobv,
    int64_t m,
    int64_t n,
    int64_t k,
    int64_t p,
    int64_t niters,
    cudaDataType dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType dataTypeSrand,
    const void *Srand,
    cudaDataType dataTypeUrand,
    const void *Urand,
    int64_t ldUrand,
    cudaDataType dataTypeVrand,
    const void *Vrand,
    int64_t ldVrand,
    cudaDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost )
```

The routine below

```
cusolverStatus_t
cusolverDnXgesvdr(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    signed char jobu,
    signed char jobv,
    int64_t m,
    int64_t n,
    int64_t k,
    int64_t p,
    int64_t niters,
    cudaDataType dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType dataTypeSrand,
    void *Srand,
    cudaDataType dataTypeUrand,
    void *Urand,
    int64_t ldUrand,
    cudaDataType dataTypeVrand,
    void *Vrand,
    int64_t ldVrand,
    cudaDataType computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
```

(continues on next page)

(continued from previous page)

```
size_t workspaceInBytesOnHost,
int *d_info)
```

This function computes the approximated rank- k singular value decomposition (k-SVD) of an $m \times n$ matrix A and the corresponding left and/or right singular vectors. The k-SVD is written as

$$A_k \approx U * \Sigma * V^H$$

where Σ is a $k \times k$ matrix which is zero except for its diagonal elements, U is an $m \times k$ orthonormal matrix, and V is an $k \times n$ orthonormal matrix. The diagonal elements of Σ are the approximated singular values of A ; they are real and non-negative, and are returned in descending order. The columns of U and V are the top- k left and right singular vectors of A .

`cusolverDnXgesvdr` implements randomized methods described in [15] to compute k-SVD that is accurate with high probability if the conditions described in [15] hold. `cusolverDnXgesvdr` is intended to compute a very small portion of the spectrum (meaning that k is very small compared to $\min(m, n)$). of A fast and with good quality, specially when the dimensions of the matrix are large.

The accuracy of the method depends on the spectrum of A , the number of power iterations `niters`, the oversampling parameter `p` and the ratio between `p` and the dimensions of the matrix A . Larger values of oversampling `p` or larger number of iterations `niters` might produce more accurate approximations, but it will also increase the run time of `cusolverDnXgesvdr`.

Our recommendation is to use two iterations and set the oversampling to at least $2k$. Once the solver provides enough accuracy, adjust the values of k and `niters` for better performance.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXgesvdr_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle).

Currently, `cusolverDnXgesvdr` supports only the default algorithm.

Algorithms supported by `cusolverDnXgesvdr`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

Please visit [cuSOLVER Library Samples - Xgesvdr](#) for a code example.

Remark 1: `gesvdr` supports $n \geq m$ as well.

Remark 2: the routine returns V , not V^H

List of input arguments for `cusolverDnXgesvdr_bufferSize` and `cusolverDnXgesvdr`:

Table 50: **API of `cusolverDnXgesvdr`**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .

continues on next page

Table 50 – continued from previous page

Parameter	Memory	In/out	Meaning
jobu	host	input	Specifies options for computing all or part of the matrix U: = 'S': the first k columns of U (the left singular vectors) are returned in the array U; = 'N': no columns of U (no left singular vectors) are computed.
jobv	host	input	Specifies options for computing all or part of the matrix V: = 'S': the first k rows of V (the right singular vectors) are returned in the array V; = 'N': no rows of V (no right singular vectors) are computed.
m	host	input	Number of rows of matrix A.
n	host	input	Number of columns of matrix A.
k	host	input	Rank of the k-SVD decomposition of matrix A. rank is less than $\min(m, n)$.
p	host	input	Oversampling. The size of the subspace will be $(k + p)$. $(k+p)$ is less than $\min(m, n)$.
niters	host	input	Number of iteration of power method.
dataTypeA	host	input	Data type of array A.
A	device	in/out	Array of dimension $lda * n$ with lda is not less than $\max(1, m)$. On exit, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeS	host	input	Data type of array S.
S	device	output	Real array of dimension $\min(m, n)$. The singular values of A, sorted so that $S(i) \geq S(i+1)$.
dataTypeU	host	input	Data type of array U.
U	device	output	Array of dimension $ldu * m$ with ldu is not less than $\max(1, m)$. U contains the $m \times m$ unitary matrix U. if $jobu=S$, only reports first $\min(m, n)$ columns of U.
ldu	host	input	Leading dimension of two-dimensional array used to store matrix U.
dataTypeV	host	input	Data type of array V.
V	device	output	Array of dimension $ldv * n$ with ldv is not less than $\max(1, n)$. V contains the $n \times n$ unitary matrix V. If $jobv=S$, only reports first $\min(m, n)$ columns of V.
ldv	host	input	Leading dimension of two-dimensional array used to store matrix V.

continues on next page

Table 50 – continued from previous page

Parameter	Memory	In/out	Meaning
computeType	host	input	Data type of computation.
bufferOnDe- vice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceIn- BytesOnDe- vice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXgesvdr_bufferSize.
bufferOn- Host	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceIn- BytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXgesvdr_bufferSize.
d_info	device	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle).

The generic API has five different types, dataTypeA is data type of the matrix A, dataTypeS is data type of the vector S and dataTypeU is data type of the matrix U, dataTypeV is data type of the matrix V, computeType is compute type of the operation. cusolverDnXgesvdr only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	DataTypeS	DataTypeU	DataTypeV	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SGESVDR
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DGESVDR
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CGESVDR
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZGESVDR

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n < 0$ or $lda < \max(1, m)$ or $ldu < \max(1, m)$ or $ldv < \max(1, n)$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.4 cusolverDnXsyevd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXsyevd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t dataTypeW,
    const void *W,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t
cusolverDnXsyevd(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType_t dataTypeW,
    void *W,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info)
```

computes eigenvalues and eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A using the generic API interface. The standard symmetric eigenvalue problem is

$$A * V = V * \Lambda$$

where Λ is a real $n \times n$ diagonal matrix. V is an $n \times n$ unitary matrix. The diagonal elements of Λ are the eigenvalues of A in ascending order.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXsyevd_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting `handle`). If `info = i` (greater than zero), i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors of the matrix A . The eigenvectors are computed by a divide and conquer algorithm.

Please visit [cuSOLVER Library Samples - Xsyevd](#) for a code example.

Currently, cusolverDnXsyevd supports only the default algorithm.

Algorithms supported by cusolverDnXsyevd

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for cusolverDnXsyevd_bufferSize and cusolverDnXsyevd:

Table 51: **API of cusolverDnXsyevd**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by cusolverDnSetAdvOptions.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute eigenvalues only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute eigenvalues and eigenvectors.
uplo	host	input	Specifies which part of A is stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A is stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A is stored.
n	host	input	Number of rows (or columns) of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension lda * n with lda is not less than max(1,n). If uplo = CUBLAS_FILL_MODE_UPPER, the leading n-by-n upper triangular part of A contains the upper triangular part of the matrix A. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n-by-n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and info = 0, A contains the orthonormal eigenvectors of the matrix A. If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeW	host	in	Data type of array W.
W	device	output	A real array of dimension n. The eigenvalue values of A, in ascending order, i.e., sorted so that $W(i) \leq W(i+1)$.

continues on next page

Table 51 – continued from previous page

Parameter	Memory	In/out	Meaning
computeType	host	in	Data type of computation.
bufferOnDe- vice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceIn- BytesOnDe- vice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXsyevd_bufferSize.
bufferOn- Host	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceIn- BytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXsyevd_bufferSize.
info	device	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle). If info = i (> 0), info indicates i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

The generic API has three different types, dataTypeA is data type of the matrix A, dataTypeW is data type of the matrix W and computeType is compute type of the operation. cusolverDnXsyevd only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYEVD
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYEVD
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CHEEVD
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZHEEVD

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or jobz is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or uplo is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.5 cusolverDnXsyevdx()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXsyevdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    void *vl,
    void *vu,
    int64_t il,
    int64_t iu,
    int64_t *h_meig,
    cudaDataType_t dataTypeW,
    const void *W,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t cusolverDnXsyevdx(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    void *vl,
    void *vu,
    int64_t il,
    int64_t iu,
    int64_t *meig64,
    cudaDataType_t dataTypeW,
    void *W,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info)
```

computes all or selection of the eigenvalues and optionally eigenvectors of a symmetric (Hermitian) $n \times n$ matrix A using the generic API interface. The standard symmetric eigenvalue problem is

$$A * V = V * \Lambda$$

where Λ is a real $n \times h_meig$ diagonal matrix. V is an $n \times h_meig$ unitary matrix. h_meig is the number of eigenvalues/eigenvectors computed by the routine, h_meig is equal to n when the whole spectrum

(e.g., `range = CUSOLVER_EIG_RANGE_ALL`) is requested. The diagonal elements of Λ are the eigenvalues of A in ascending order.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXsyevdx_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `info = i` (greater than zero), i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

if `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors of the matrix A . The eigenvectors are computed by a divide and conquer algorithm.

Currently, `cusolverDnXsyevdx` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xsyevdx](#) for a code example.

Algorithms supported by `cusolverDnXsyevdx`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXsyevdx_bufferSize` and `cusolverDnXsyevdx`:

Table 52: **API of `cusolverDnXsyevdx`**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the cuSolverDN library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>jobz</code>	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
<code>range</code>	host	input	Specifies options to which selection of eigenvalues and optionally eigenvectors that need to be computed: <code>range = CUSOLVER_EIG_RANGE_ALL</code> : all eigenvalues/eigenvectors will be found, will becomes the classical <code>syevd</code> / <code>heevd</code> routine; <code>range = CUSOLVER_EIG_RANGE_V</code> : all eigenvalues/eigenvectors in the half-open interval $(vl, vu]$ will be found; <code>range = CUSOLVER_EIG_RANGE_I</code> : the il -th through iu -th eigenvalues/eigenvectors will be found;

continues on next page

Table 52 – continued from previous page

Parameter	Memory	In/out	Meaning
uplo	host	input	Specifies which part of A is stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A is stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A is stored.
n	host	input	Number of rows (or columns) of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension lda * n with lda is not less than max(1,n). If uplo = CUBLAS_FILL_MODE_UPPER, the leading n-by-n upper triangular part of A contains the upper triangular part of the matrix A. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n-by-n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and info = 0, A contains the orthonormal eigenvectors of the matrix A. If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda is not less than max(1,n).
vl, vu	host	input	If range = CUSOLVER_EIG_RANGE_V, the lower and upper bounds of the interval to be searched for eigenvalues. vl > vu. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_I. Note that, if eigenvalues are very close to each other, it is well known that two different eigenvalues routines might find slightly different number of eigenvalues inside the same interval. This is due to the fact that different eigenvalue algorithms, or even same algorithm but different run might find eigenvalues within some rounding error close to the machine precision. Thus, if the user want to be sure not to miss any eigenvalue within the interval bound, we suggest that, the user subtract/add epsilon (machine precision) to the interval bound such as (vl=vl-eps, vu=vu+eps]. this suggestion is valid for any selective routine from cuSolver or LAPACK.

continues on next page

Table 52 – continued from previous page

Parameter	Memory	In/out	Meaning
il, iu	host	input	Integer. If range = CUSOLVER_EIG_RANGE_I, the indices (in ascending order) of the smallest and largest eigenvalues to be returned. $1 \leq il \leq iu \leq n$, if $n > 0$; $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_V.
h_meig	host	output	Integer. The total number of eigenvalues found. $0 \leq h_meig \leq n$. If range = CUSOLVER_EIG_RANGE_ALL, $h_meig = n$, and if range = CUSOLVER_EIG_RANGE_I, $h_meig = iu - il + 1$.
dataTypeW	host	in	Data type of array W.
W	device	output	A real array of dimension n. The eigenvalue values of A, in ascending order, i.e., sorted so that $W(i) \leq W(i+1)$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXsyevdx_bufferSize.
bufferOnHost	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceInBytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXsyevdx_bufferSize.
info	device	output	If info = 0, the operation is successful. if info = -i, the i-th parameter is wrong (not counting handle). If info = i (> 0), info indicates i off-diagonal elements of an intermediate tridiagonal form did not converge to zero.

The generic API has three different types, dataTypeA is data type of the matrix A, dataTypeW is data type of the matrix W and computeType is compute type of the operation. cusolverDnXsyevdx only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYEVDX
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYEVDX
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CHEEVDX
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZHEEVDX

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or `jobz` is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or `range` is not `CUSOLVER_EIG_RANGE_ALL` or `CUSOLVER_EIG_RANGE_V` or `CUSOLVER_EIG_RANGE_I`, or `uplo` is not `CUBLAS_FILL_MODE_LOWER` or `CUBLAS_FILL_MODE_UPPER`).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.6 `cusolverDnXstedc()`

The helper function below can calculate the sizes needed for the pre-allocated buffers.

```
cusolverStatus_t
cusolverDnXstedc_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigComp_t compz,
    int64_t n,
    cudaDataType_t dataTypeDE,
    const void *D,
    const void *E,
    cudaDataType_t dataTypeZ,
    const void *Z,
    int64_t ldz,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine

```
cusolverStatus_t
cusolverDnXstedc(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigComp_t compz,
    int64_t n,
    cudaDataType_t dataTypeDE,
    void *D,
    void *E,
    cudaDataType_t dataTypeZ,
    void *Z,
    int64_t ldz,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info)
```

computes all eigenvalues and (optionally) eigenvectors of a real symmetric tridiagonal matrix using the divide and conquer algorithm. This function corresponds to the LAPACK routine STEDC.

The symmetric tridiagonal matrix is defined by the diagonal elements in vector D (length n) and the subdiagonal elements in vector E (length n-1). On successful completion, D contains the eigenvalues in ascending order.

The user has to provide device and host working spaces which are pointed to by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameter `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is the size in bytes of the device (and host) working space, and it is returned by `cusolverDnXstedc_bufferSize()`.

If output parameter `info` = -i (less than zero), the i-th parameter is wrong (not counting handle). If `info` = i (greater than zero), the algorithm did not converge and failed to compute all eigenvalues.

The parameter `compz` specifies the computational mode: `CUSOLVER_EIG_COMP_N` compute only the eigenvalues; `CUSOLVER_EIG_COMP_I` compute eigenvalues and eigenvectors of the tridiagonal matrix; `CUSOLVER_EIG_COMP_V` compute eigenvalues and eigenvectors as the product of the input matrix Z and the unitary transformations applied during STEDC; typically Z is on entry the unitary matrix produced by SYTRD, and then on exit Z holds the eigenvectors of the original matrix.

Currently, `cusolverDnXstedc` supports only the default algorithm.

Algorithms supported by `cusolverDnXstedc`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXstedc_bufferSize` and `cusolverDnXstedc`:

Table 53: **API of `cusolverDnXstedc`**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the <code>cuSolverDN</code> library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>compz</code>	host	input	Computational mode: <code>CUSOLVER_EIG_COMP_N</code> compute only eigenvalues; <code>CUSOLVER_EIG_COMP_I</code> compute eigenvalues and eigenvectors of the tridiagonal matrix; <code>CUSOLVER_EIG_COMP_V</code> compute eigenvalues and eigenvectors as $Z*Q$; Z is typically from SYTRD.
<code>n</code>	host	input	Problem size. Vector D has length n, vector E has length n-1, and matrix Z is n by n.
<code>dataTypeDE</code>	host	input	Data type of vectors D and E.
<code>D</code>	device	in/out	Vector of length n. On input, the diagonal elements of the tridiagonal matrix. On output, if successful, the eigenvalues in ascending order.
<code>E</code>	device	in/out	Vector of length n-1. On input, the subdiagonal elements of the tridiagonal matrix. On output, the contents are destroyed.

continues on next page

Table 53 – continued from previous page

Parameter	Memory	In/out	Meaning
dataTypeZ	host	input	Data type of array Z.
Z	device	in/out	Array of dimension $ldz * n$. If <code>compz = CUSOLVER_EIG_COMP_N</code> , Z is not referenced. If <code>compz = CUSOLVER_EIG_COMP_I</code> , Z holds the eigenvectors of the tridiagonal matrix. If <code>compz = CUSOLVER_EIG_COMP_V</code> , Z is overwritten with $Z*Q$ (Q = unitary from STEDC; Z typically from SYTRD).
ldz	host	input	Leading dimension of two-dimensional array used to store matrix Z. If <code>compz = CUSOLVER_EIG_COMP_I</code> or <code>CUSOLVER_EIG_COMP_V</code> $ldz \geq \max(1, n)$. Otherwise, $ldz \geq 1$.
computeType	host	input	Data type of computation.
bufferOnDe-vice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceIn-BytesOnDe-vice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXstedc_bufferSize</code> .
bufferOn-Host	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceIn-BytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXstedc_bufferSize</code> .
info	device	output	If <code>info = 0</code> , the operation is successful. If <code>info = -i</code> , the i -th parameter is wrong (not counting handle). If <code>info = i</code> ($i > 0$), the algorithm failed to converge and did not compute all eigenvalues.

The generic API uses `dataTypeDE` for the data type of the vectors D and E, `dataTypeZ` for the matrix Z, and `computeType` for the compute type of the operation. `cusolverDnXstedc` only supports the following four combinations:

Valid combination of data type and compute type

DataTypeDE	DataTypeZ	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSTEDC
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSTEDC
CUDA_R_32F	CUDA_C_32F	CUDA_C_32F	CSTEDC
CUDA_R_64F	CUDA_C_64F	CUDA_C_64F	ZSTEDC

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (e.g. $n < 0$, or $ldz < \max(1, n)$ when compz is CUSOLVER_EIG_COMP_I or CUSOLVER_EIG_COMP_V, or compz is not CUSOLVER_EIG_COMP_N, CUSOLVER_EIG_COMP_I, or CUSOLVER_EIG_COMP_V).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.7 cusolverDnXsygvd()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXsygvd_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t dataTypeB,
    const void *B,
    int64_t ldb,
    cudaDataType_t dataTypeW,
    const void *W,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine computes all the eigenvalues, and optionally, the eigenvectors of a generalized symmetric (Hermitian) definite eigenproblem.

```
cusolverStatus_t
cusolverDnXsygvd(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType_t dataTypeB,
    void *B,
    int64_t ldb,
    cudaDataType_t dataTypeW,
    void *W,
    cudaDataType_t computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
```

(continues on next page)

(continued from previous page)

```
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int * info)
```

The generalized symmetric (Hermitian) definite eigenvalue problem is

$$\text{eig}(A, B) = \begin{cases} A * V = B * V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1} \\ A * B * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_2} \\ B * A * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

where the matrix A and B are $n \times n$; A is symmetric/Hermitian and B is symmetric/Hermitian positive definite. The eigenvalues of (A, B) are computed and stored in the W vector in ascending order. V is an $n \times n$ orthogonal matrix. The eigenvectors are normalized as follows:

$$\begin{cases} V^H * B * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1}, \text{CUSOLVER_EIG_TYPE_2} \\ V^H * \text{inv}(B) * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXsygvd_bufferSize()`.

If output parameter `info = -i` (less than zero), the *i*-th parameter is wrong (not counting handle). If `info = i` ($i > 0$ and $i \leq n$) and `jobz = CUSOLVER_EIG_MODE_NOVECTOR`, *i* off-diagonal elements of an intermediate tridiagonal form did not converge to zero. If `info = n + i` ($i > 0$), then the leading minor of order *i* of B is not positive definite. The factorization of B could not be completed and no eigenvalues or eigenvectors were computed.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors of the matrix A. The eigenvectors are computed by a divide and conquer algorithm.

Currently, `cusolverDnXsygvd` supports only the default algorithm.

Algorithms supported by `cusolverDnXsygvd`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXsygvd_bufferSize` and `cusolverDnXsygvd`:

Table 54: **API of `cusolverDnXsygvd`**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the <code>cuSolverDN</code> library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .

continues on next page

Table 54 – continued from previous page

Parameter	Memory	In/out	Meaning
itype	host	input	Specifies the problem type to be solved: ▶ itype=CUSOLVER_EIG_TYPE_1: $A^*x = (\text{lambda})*B^*x$. ▶ itype=CUSOLVER_EIG_TYPE_2: $A^*B^*x = (\text{lambda})*x$. ▶ itype=CUSOLVER_EIG_TYPE_3: $B^*A^*x = (\text{lambda})*x$.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: jobz = CUSOLVER_EIG_MODE_NOVECTOR : Compute eigenvalues only; jobz = CUSOLVER_EIG_MODE_VECTOR : Compute eigenvalues and eigenvectors.
uplo	host	input	Specifies which part of A is stored. uplo = CUBLAS_FILL_MODE_LOWER: Lower triangle of A is stored. uplo = CUBLAS_FILL_MODE_UPPER: Upper triangle of A is stored.
n	host	input	Number of rows (or columns) of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension lda * n with lda is not less than max(1,n). If uplo = CUBLAS_FILL_MODE_UPPER, the leading n-by-n upper triangular part of A contains the upper triangular part of the matrix A. If uplo = CUBLAS_FILL_MODE_LOWER, the leading n-by-n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if jobz = CUSOLVER_EIG_MODE_VECTOR, and info = 0, A contains the orthonormal eigenvectors of the matrix A. If jobz = CUSOLVER_EIG_MODE_NOVECTOR, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeB	host	in	Data type of array B.

continues on next page

Table 54 – continued from previous page

Parameter	Memory	In/out	Meaning
B	device	in/out	Array of dimension $ldb * n$ with ldb is not less than $\max(1, n)$. If $uplo = CUBLAS_FILL_MODE_UPPER$, the leading n -by- n upper triangular part of B contains the upper triangular part of the matrix B. If $uplo = CUBLAS_FILL_MODE_LOWER$, the leading n -by- n lower triangular part of B contains the lower triangular part of the matrix B. On exit, if $devInfo$ is less than n , B is overwritten by triangular factor U or L from the Cholesky factorization of B.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B.
dataTypeW	host	in	Data type of array W.
W	device	output	A real array of dimension n . The eigenvalue values of A, in ascending order, i.e., sorted so that $W(i) \leq W(i+1)$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXsygvd_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceInBytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXsygvd_bufferSize</code> .
info	device	output	If $info = 0$, the operation is successful. If $info = -i$, the i -th parameter is wrong (not counting handle). If $info = i$ (> 0), $info$ indicates either <code>potrf</code> or <code>syevd</code> is wrong.

The generic API has four different data types, `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix B, `dataTypeW` is data type of the matrix W and `computeType` is compute type of the operation. `cusolverDnXsygvd` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	DataTypeB	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYGVD
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYGVD
CUDA_C_32F	CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CHEGVD
CUDA_C_64F	CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZHEGVD

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $ldb < \max(1, n)$, or *itype* is not 1, 2 or 3, or *jobz* is not *CUSOLVER_EIG_MODE_NOVECTOR* or *CUSOLVER_EIG_MODE_VECTOR*, or *uplo* is not *CUBLAS_FILL_MODE_LOWER* or *CUBLAS_FILL_MODE_UPPER*), or the combination of *dataType*{A, B, C} and *computeType* are not supported.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.8 cusolverDnXsygvdx()

The helper function below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXsygvdx_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType dataTypeB,
    const void *B,
    int64_t ldb,
    void *vl,
    void *vu,
    int64_t il,
    int64_t iu,
    int64_t *h_meig,
    cudaDataType dataTypeW,
    const void *W,
    cudaDataType computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine computes all or selection of the eigenvalues, and optionally, the eigenvectors of a generalized symmetric (Hermitian) definite eigenproblem.

```
cusolverStatus_t
cusolverDnXsygvdx(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigType_t itype,
    cusolverEigMode_t jobz,
    cusolverEigRange_t range,
    cublasFillMode_t uplo,
```

(continues on next page)

(continued from previous page)

```

int64_t n,
cudaDataType dataTypeA,
void *A,
int64_t lda,
cudaDataType dataTypeB,
void *d_B,
int64_t ldb,
void *vl,
void *vu,
int64_t il,
int64_t iu,
int64_t *h_meig,
cudaDataType dataTypeW,
void *W,
cudaDataType computeType,
void *bufferOnDevice,
size_t workspaceInBytesOnDevice,
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int *info)

```

The generalized symmetric-definite eigenvalue problem is

$$\text{eig}(A, B) = \begin{cases} A * V = B * V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1} \\ A * B * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_2} \\ B * A * V = V * \Lambda & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

where the matrix A and B are $n \times n$; A is symmetric/Hermitian and B is symmetric/Hermitian positive definite. The eigenvalues of (A, B) are computed and stored in the W vector in ascending order. h_meig represents the number of eigenvalues/eigenvectors computed by the routine, h_meig is equal to n when the whole spectrum (for example, range = CUSOLVER_EIG_RANGE_ALL) is requested. V is an $n \times n$ orthogonal matrix. The eigenvectors are normalized as follows:

$$\begin{cases} V^H * B * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_1}, \text{CUSOLVER_EIG_TYPE_2} \\ V^H * \text{inv}(B) * V = I & \text{if } itype == \text{CUSOLVER_EIG_TYPE_3} \end{cases}$$

The user has to provide device and host working spaces which are pointed by input parameters bufferOnDevice and bufferOnHost. The input parameters workspaceInBytesOnDevice (and workspaceInBytesOnHost) is size in bytes of the device (and host) working space, and it is returned by cusolverDnXsygvdx_bufferSize().

If output parameter info = -i (less than zero), the i-th parameter is wrong (not counting handle). If info = i (i > 0 and i ≤ n) and jobz = CUSOLVER_EIG_MODE_NOVECTOR, i off-diagonal elements of an intermediate tridiagonal form did not converge to zero. If info = n + i (i > 0), then the leading minor of order i of B is not positive definite. The factorization of B could not be completed and no eigenvalues or eigenvectors were computed.

If jobz = CUSOLVER_EIG_MODE_VECTOR, A contains the orthogonal eigenvectors of the matrix A. The eigenvectors are computed by divide and conquer algorithm.

Algorithms supported by cusolverDnXsygvdx

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXsygvdx_bufferSize` and `cusolverDnXsygvdx`:

Table 55: **API of `cusolverDnXsygvdx`**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
itype	host	input	Specifies the problem type to be solved: ▶ <code>itype=CUSOLVER_EIG_TYPE_1</code>: $A*x = (\text{lambda})*B*x$. ▶ <code>itype=CUSOLVER_EIG_TYPE_2</code>: $A*B*x = (\text{lambda})*x$. ▶ <code>itype=CUSOLVER_EIG_TYPE_3</code>: $B*A*x = (\text{lambda})*x$.
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
range	host	input	Specifies options to which selection of eigenvalues and optionally eigenvectors that need to be computed: <code>range = CUSOLVER_EIG_RANGE_ALL</code> : all eigenvalues/eigenvectors will be found, will becomes the classical sygvdx/hegvdx routine; <code>range = CUSOLVER_EIG_RANGE_V</code> : all eigenvalues/eigenvectors in the half-open interval $(vl,vu]$ will be found; <code>range = CUSOLVER_EIG_RANGE_I</code> : the il -th through iu -th eigenvalues/eigenvectors will be found;
uplo	host	input	Specifies which part of A is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A is stored.
n	host	input	Number of rows (or columns) of matrix A.
dataTypeA	host	in	Data type of array A.

continues on next page

Table 55 – continued from previous page

Parameter	Memory	In/out	Meaning
A	device	in/out	Array of dimension $lda * n$ with lda is not less than $\max(1, n)$. If $uplo = CUBLAS_FILL_MODE_UPPER$, the leading n -by- n upper triangular part of A contains the upper triangular part of the matrix A. If $uplo = CUBLAS_FILL_MODE_LOWER$, the leading n -by- n lower triangular part of A contains the lower triangular part of the matrix A. On exit, if $jobz = CUSOLVER_EIG_MODE_VECTOR$, and $info = 0$, A contains the orthonormal eigenvectors of the matrix A. If $jobz = CUSOLVER_EIG_MODE_NOVECTOR$, the contents of A are destroyed.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda is not less than $\max(1, n)$.
dataTypeB	host	in	Data type of array B.
B	device	in/out	Array of dimension $ldb * n$ with ldb is not less than $\max(1, n)$. If $uplo = CUBLAS_FILL_MODE_UPPER$, the leading n -by- n upper triangular part of B contains the upper triangular part of the matrix B. If $uplo = CUBLAS_FILL_MODE_LOWER$, the leading n -by- n lower triangular part of B contains the lower triangular part of the matrix B. On exit, if $devInfo$ is less than n , B is overwritten by triangular factor U or L from the Cholesky factorization of B.
ldb	host	input	Leading dimension of two-dimensional array used to store matrix B.
computeType	host	in	Data type of computation.

continues on next page

Table 55 – continued from previous page

Parameter	Memory	In/out	Meaning
vl, vu	host	input	If range = CUSOLVER_EIG_RANGE_V, the lower and upper bounds of the interval to be searched for eigenvalues. The values are treated as dataTypeW. vl > vu. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_I. Note that, if eigenvalues are very close to each other, it is well known that two different eigenvalues routines might find slightly different number of eigenvalues inside the same interval. This is due to the fact that different eigenvalue algorithms, or even same algorithm but different run might find eigenvalues within some rounding error close to the machine precision. Thus, if the user want to be sure not to miss any eigenvalue within the interval bound, we suggest that, the user subtract/add epsilon (machine precision) to the interval bound such as (vl=vl-eps, vu=vu+eps]. this suggestion is valid for any selective routine from cuSolver or LAPACK.
il, iu	host	input	Integer. If range = CUSOLVER_EIG_RANGE_I, the indices (in ascending order) of the smallest and largest eigenvalues to be returned. $1 \leq il \leq iu \leq n$, if $n > 0$; $il = 1$ and $iu = 0$ if $n = 0$. Not referenced if range = CUSOLVER_EIG_RANGE_ALL or range = CUSOLVER_EIG_RANGE_V.
h_meig	host	output	Integer. The total number of eigenvalues found. $0 \leq h_meig \leq n$. If range = CUSOLVER_EIG_RANGE_ALL, h_meig = n, and if range = CUSOLVER_EIG_RANGE_I, h_meig = iu - il + 1.
dataTypeW	host	in	Data type of array W.
W	device	output	A real array of dimension n. The eigenvalue values of A, in ascending order, i.e., sorted so that $W(i) \leq W(i+1)$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXsygvdx_bufferSize.
bufferOnHost	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.

continues on next page

Table 55 – continued from previous page

Parameter	Memory	In/out	Meaning
workspaceIn-BytesOnHost	host	input	Size in bytes of bufferOnHost, returned by <code>cusolverDnXsygvdx_bufferSize</code> .
info	device	output	If <code>info = 0</code> , the operation is successful. If <code>info = -i</code> , the <i>i</i> -th parameter is wrong (not counting handle). If <code>info = i</code> ($i > 0$), <code>info</code> indicates either <code>potrf</code> or <code>syevd</code> is wrong.

The generic API has four different types, `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix B, `dataTypeW` is data type of the matrix W and `computeType` is compute type of the operation. `cusolverDnXsygvdx` only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeB	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYGVDX
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYGVDX
CUDA_C_32F	CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CHEGVDX
CUDA_C_64F	CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZHEGVDX

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $lda < \max(1, n)$, or $ldb < \max(1, n)$, or `itype` is not 1, 2 or 3, or `jobz` is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or `range` is not `CUSOLVER_EIG_RANGE_ALL` or `CUSOLVER_EIG_RANGE_V` or `CUSOLVER_EIG_RANGE_I`, or `uplo` is not `CUBLAS_FILL_MODE_LOWER` or `CUBLAS_FILL_MODE_UPPER`), or the combination of `dataType{A, B, C}` and `computeType` are not supported.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.9 cusolverDnXsyevBatched()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXsyevBatched_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
```

(continues on next page)

(continued from previous page)

```

cudaDataType dataTypeA,
const void *A,
int64_t lda,
cudaDataType dataTypeW,
const void *W,
cudaDataType computeType,
size_t *workspaceInBytesOnDevice,
size_t *workspaceInBytesOnHost,
int64_t batchSize)

```

The following routine:

```

cusolverStatus_t
cusolverDnXsyevBatched(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int64_t n,
    cudaDataType dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType dataTypeW,
    void *W,
    cudaDataType computeType,
    void *bufferOnDevice,
    size_t workspaceInBytesOnDevice,
    void *bufferOnHost,
    size_t workspaceInBytesOnHost,
    int *info,
    int64_t batchSize)

```

computes eigenvalues and eigenvectors of a sequence of symmetric (Hermitian) $n \times n$ matrices

$$A_j * V_j = V_j * \Lambda_j$$

where Λ_j is a real $n \times n$ diagonal matrix. V_j is an $n \times n$ unitary matrix. The diagonal elements of Λ_j are the eigenvalues of A_j in ascending order.

syevBatched performs an eigendecomposition on each matrix. It requires that all matrices are of the same size n and are packed in a contiguous way,

$$A = \begin{pmatrix} A_0 & A_1 & \dots \end{pmatrix}$$

Each matrix is column-major with leading dimension lda , so the formula for random access is $A_k(i, j) = A[i + lda * j + lda * n * k]$.

The parameter W also contains the eigenvalues of each matrix in a contiguous way,

$$W = \begin{pmatrix} W_0 & W_1 & \dots \end{pmatrix}$$

The formula for random access of W is $W_k(j) = W[j + n * k]$.

The user has to provide device and host working space which are pointed to by the input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` and `workspaceInBytesOnHost` denote the size in bytes of the device and host working space, and returned by `cusolverDnXsyevBatched_bufferSize()`.

The output parameter `info` is an integer array of size `batchSize`. If the function returns `CUSOLVER_STATUS_INVALID_VALUE`, the first element `info[0] = -i` (less than zero) indicates the i -th parameter is wrong (not counting `handle`). Otherwise, if `info[i] > 0`, `syevBatched` does not converge on the i -th matrix.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A_j contains the orthonormal eigenvectors of the matrix A_j .

Note that the problem size is limited by the condition `n*lda*batchSize<=INT32_MAX` primarily due to the current implementation constraints.

Table 56: **Algorithms supported by `cusolverDnXsyevBatched`**

<code>CUSOLVER_ALG_0</code> or <code>NULL</code>	Default. May switch between algorithms for best performance.
<code>CUSOLVER_ALG_1</code>	Uses a single algorithm for consistent accuracy over all n .

List of input arguments for `cusolverDnXsyevBatched_bufferSize` and `cusolverDnXsyevBatched`:

Table 57: **API of `cusolverDnXsyevBatched`**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the <code>cuSolverDN</code> library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>jobz</code>	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only; <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors.
<code>uplo</code>	host	input	Specifies which part of A is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A is stored.
<code>n</code>	host	input	Number of rows (or columns) of matrix A .
<code>dataTypeA</code>	host	in	Data type of array A .
A	device	in/out	Array of dimension <code>lda * n * batchSize</code> with <code>lda</code> is not less than <code>max(1,n)</code> . If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading n -by- n upper triangular part of A_j contains the upper triangular part of the matrix A_j . If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading n -by- n lower triangular part of A_j contains the lower triangular part of the matrix A_j . On exit, if <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> , and <code>info[j] = 0</code> , A_j contains the orthonormal eigenvectors of the matrix A_j . If <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> , the contents of A_j are destroyed.

continues on next page

Table 57 – continued from previous page

Parameter	Memory	In/out	Meaning
lda	host	input	Leading dimension of two-dimensional array used to store matrix A. lda is not less than $\max(1, n)$.
dataTypeW	host	in	Data type of array W.
W	device	output	A real array of dimension $n * batchSize$. The eigenvalue values of A, in ascending order, i.e., sorted so that $W_j(i) \leq W_j(i+1)$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by <code>cusolverDnXsyevBatched_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceInBytesOnHost	host	input	Size in bytes of bufferOnHost, returned by <code>cusolverDnXsyevBatched_bufferSize</code> .
info	device	output	An integer array of dimension <code>batchSize</code> . If <code>CUSOLVER_STATUS_INVALID_VALUE</code> is returned, <code>info[0] = -i</code> (less than zero) indicates <i>i</i> -th parameter is wrong (not counting handle). Otherwise, if <code>info[i] = 0</code> , the operation is successful. If <code>info[i] > 0</code> , <code>syevBatched</code> does not converge on the <i>i</i> -th matrix.
batchSize	host	input	Number of matrices. <code>batchSize</code> is not less than 1.

The generic API has three different types, `dataTypeA` is data type of the matrix A, `dataTypeW` is data type of the array W and `computeType` is compute type of the operation. `cusolverDnXsyevBatched` only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYEV_BATCHED
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYEV_BATCHED
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CSYEV_BATCHED
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZSYEV_BATCHED

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($n < 0$, or $n * lda * batchSize > INT32_MAX$, or $lda < \max(1, n)$, or $jobz$ is not CUSOLVER_EIG_MODE_NOVECTOR or $uplo$ is not CUBLAS_FILL_MODE_LOWER or CUBLAS_FILL_MODE_UPPER or $batchSize < 0$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.10 cusolverDnXgeev()

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverDnXgeev_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobvl,
    cusolverEigMode_t jobvr,
    int64_t n,
    cudaDataType_t dataTypeA,
    const void *A,
    int64_t lda,
    cudaDataType_t dataTypeW,
    const void *W,
    cudaDataType_t dataTypeVL,
    const void *VL,
    int64_t ldvl,
    cudaDataType_t dataTypeVR,
    const void *VR,
    int64_t ldvr,
    cudaDataType_t computeType,
    size_t *workspaceInBytesOnDevice,
    size_t *workspaceInBytesOnHost)
```

The following routine:

```
cusolverStatus_t
cusolverDnXgeev(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cusolverEigMode_t jobvl,
    cusolverEigMode_t jobvr,
    int64_t n,
    cudaDataType_t dataTypeA,
    void *A,
    int64_t lda,
    cudaDataType_t dataTypeW,
    void *W,
    cudaDataType_t dataTypeVL,
    void *VL,
    int64_t ldvl,
    cudaDataType_t dataTypeVR,
    void *VR,
    int64_t ldvr,
```

(continues on next page)

(continued from previous page)

```

cudaDataType computeType,
void *bufferOnDevice,
size_t workspaceInBytesOnDevice,
void *bufferOnHost,
size_t workspaceInBytesOnHost,
int *info)

```

computes for an n -by- n real non-symmetric or complex non-Hermitian matrix A the eigenvalues and, optionally, the left and/or right eigenvectors. The right eigenvector $v(j)$ of A satisfies

$$A * v(j) = w(j) * v(j)$$

where $w(j)$ is its eigenvalue. The left eigenvector $u(j)$ of A satisfies

$$u(j)^H * A = w(j) * v(j)^H$$

where $u(j)^H$ denotes the conjugate-transpose of $u(j)$.

The computed eigenvectors are normalized to have Euclidean norm equal to 1 and largest component real.

If A is real-valued, there are two options to return the eigenvalues in W . The first option sets all data types to real-valued types. Then W holds $2*n$ entries. The first n entries hold the real parts and the last n entries hold the imaginary parts. The LAPACK interface with separate arrays for the real parts WR and the imaginary parts WI can be recovered by settings pointers $WR = W, WI = W+n$. The second option uses a complex data type for W . Then W is n entries long; each real eigenvalue is stored as a complex number and for each complex conjugate pair, both eigenvalues are returned. The computation is still executed fully in real arithmetic.

The user has to provide device and host working space which are pointed to by the input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` and `workspaceInBytesOnHost` denote the size in bytes of the device and host working space, and returned by `cusolverDnXgeev_bufferSize()`.

If the output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle). If `info = 0`, the QR algorithm converged and W contains the computed eigenvalues of A and, if requested, the corresponding left and/or right eigenvectors have been computed. If `info = i` (greater than zero), the QR algorithm failed to compute all the eigenvalues and no eigenvectors have been computed. The elements $i+1:n$ of W contain eigenvalues which have converged.

Remark 1: `geev` only supports the computation of right eigenvectors. So, `jobvl = CUSOLVER_EIG_MODE_NOVECTOR` must be set.

Remark 2: `geev` uses balancing to improve the conditioning of the eigenvalues and eigenvectors.

Remark 3: `geev` is a hybrid CPU-GPU algorithm. Best performance is attained with pinned host memory.

Currently, `cusolverDnXgeev` supports only the default algorithm.

Please visit [cuSOLVER Library Samples - Xgeev](#) for a code example.

Table of algorithms supported by `cusolverDnXgeev`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

List of input arguments for `cusolverDnXgeev_bufferSize` and `cusolverDnXgeev`:

Table 58: API of cusolverDnXgeev

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverDN library context.
params	host	input	Structure with information collected by cusolverDnSetAdvOptions.
jobvl	host	input	Specifies whether or not to compute left eigenvectors. jobvl = CUSOLVER_EIG_MODE_NOVECTOR: Do not compute left eigenvectors of A; jobvl = CUSOLVER_EIG_MODE_VECTOR: Compute left eigenvectors of A.
jobvr	host	input	Specifies whether or not to compute right eigenvectors. jobvl = CUSOLVER_EIG_MODE_NOVECTOR: Do not compute left eigenvectors of A; jobvl = CUSOLVER_EIG_MODE_VECTOR: Compute left eigenvectors of A.
n	host	input	Number of rows (or columns) of matrix A.
dataTypeA	host	in	Data type of array A.
A	device	in/out	Array of dimension lda * n with lda is not less than max(1, n). On entry, the n-by-n matrix A. On exit, A has been overwritten.
lda	host	input	Leading dimension of two-dimensional array used to store matrix A.
dataTypeW	host	in	Data type of array W.
W	device	output	Array holding the computed eigenvalues of A. Its length is 2*n if dataTypeA = CUDA_R_32F and dataTypeW = CUDA_R_32F or dataTypeA = CUDA_R_64F and dataTypeW = CUDA_R_64F and the first n entries of W hold the real parts and the last n entries of W hold the imaginary parts of the eigenvalues. Otherwise, the length is n.
dataTypeVL	host	in	Data type of array VL.

continues on next page

Table 58 – continued from previous page

Parameter	Memory	In/out	Meaning
VL	device	output	Array of dimension $ldvl * n$. If <code>jobvl = CUSOLVER_EIG_MODE_VECTOR</code> , the left eigenvectors $u(j)$ are stored one after another in the columns of VL, in the same order as their eigenvalues. If <code>datatypeVL</code> is complex or the j -th eigenvalue is real, then $u(j) = VL(:, j)$, the j -th column of VL. If <code>dataTypeVL</code> is real and the j -th and $(j+1)$ -st eigenvalues form a complex conjugate pair, then $u(j) = VL(:, j) + i*VL(:, j+1)$ and $u(j+1) = VL(:, j) - i*VL(:, j+1)$. If <code>jobvl = CUSOLVER_EIG_MODE_NOVECTOR</code> , VL is not referenced.
ldvl	host	input	Leading dimension of two-dimensional array used to store matrix VL with $ldvl \geq 1$. If <code>jobvl = CUSOLVER_EIG_MODE_VECTOR</code> , $ldvl \geq n$.
dataTypeVR	host	in	Data type of array VR.
VR	device	output	Array of dimension $ldvr * n$. If <code>jobvr = CUSOLVER_EIG_MODE_VECTOR</code> , the right eigenvectors $v(j)$ are stored one after another in the columns of VR, in the same order as their eigenvalues. If <code>datatypeVR</code> is complex or the j -th eigenvalue is real, then $v(j) = VR(:, j)$, the j -th column of VR. If <code>dataTypeVR</code> is real and the j -th and $(j+1)$ -st eigenvalues form a complex conjugate pair, then $v(j) = VR(:, j) + i*VR(:, j+1)$ and $v(j+1) = VR(:, j) - i*VR(:, j+1)$. If <code>jobvr = CUSOLVER_EIG_MODE_NOVECTOR</code> , VR is not referenced.
ldvr	host	input	Leading dimension of two-dimensional array used to store matrix VR with $ldvr \geq 1$. If <code>jobvr = CUSOLVER_EIG_MODE_VECTOR</code> , $ldvr \geq n$.
computeType	host	in	Data type of computation.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size <code>workspaceInBytesOnDevice</code> bytes.
workspaceIn-BytesOnDevice	host	input	Size in bytes of <code>bufferOnDevice</code> , returned by <code>cusolverDnXgeev_bufferSize</code> .
bufferOnHost	host	in/out	Host workspace. Array of type void of size <code>workspaceInBytesOnHost</code> bytes.
workspaceIn-BytesOnHost	host	input	Size in bytes of <code>bufferOnHost</code> , returned by <code>cusolverDnXgeev_bufferSize</code> .

continues on next page

Table 58 – continued from previous page

Parameter	Memory	In/out	Meaning
info	device	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle). If info = i (greater than zero), the QR algorithm failed to compute all the eigenvalues and no eigenvectors have been computed; elements i+1:n of W contain eigenvalues which have converged.

The generic API has five different types, dataTypeA is the data type of the matrix A, dataTypeW is the data type of the array W, dataTypeVL is the data type of the matrix VL, dataTypeVR is the data type of the matrix VR and computeType is compute type of the operation. cusolverDnXgeev only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeW	DataTypeVL	DataTypeVR	Compute-Type	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SGEEV
CUDA_R_32F	CUDA_C_32F	CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	32F mixed real-complex
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DGEEV
CUDA_R_64F	CUDA_C_64F	CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	64F mixed real-complex
CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CGEEV
CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZGEEV

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (jobvl is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, or jobvr is not CUSOLVER_EIG_MODE_NOVECTOR or CUSOLVER_EIG_MODE_VECTOR, n < 0, or lda < max(1,n), or ldvl < n if jobvl is CUSOLVER_EIG_MODE_VECTOR, or ldvr < n if jobvr is CUSOLVER_EIG_MODE_VECTOR).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.4.5.11 cusolverDnXpolar()

The following helper function calculates the required workspace sizes for `cusolverDnXpolar()`.

```
cusolverStatus_t
cusolverDnXpolar_bufferSize(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasFillMode_t   uplo,
    int64_t             m,
    int64_t             n,
    cudaDataType        dataTypeA,
    const void          *A,
    int64_t             lda,
    cudaDataType        dataTypeH,
    const void          *H,
    int64_t             ldh,
    cudaDataType        computeType,
    size_t              *workspaceInBytesOnDevice,
    size_t              *workspaceInBytesOnHost)
```

The following routine computes the polar decomposition.

```
cusolverStatus_t
cusolverDnXpolar(
    cusolverDnHandle_t handle,
    cusolverDnParams_t params,
    cublasFillMode_t   uplo,
    int64_t             m,
    int64_t             n,
    cudaDataType        dataTypeA,
    void               *A,
    int64_t             lda,
    cudaDataType        dataTypeH,
    void               *H,
    int64_t             ldh,
    cudaDataType        computeType,
    void               *bufferOnDevice,
    size_t              workspaceInBytesOnDevice,
    void               *bufferOnHost,
    size_t              workspaceInBytesOnHost,
    double              *d_res_nrm,
    double              *d_A_nrmF,
    double              *d_rcond,
    int                 *d_info)
```

This function computes the polar decomposition of an $m \times n$ matrix A (where $m \geq n$). The polar decomposition is written

$$A = U_p \cdot H$$

where U_p is an $m \times n$ matrix with orthonormal columns and H is an $n \times n$ Hermitian positive semidefinite matrix.

`cusolverDnXpolar` uses the QDWH (QR-based Dynamically Weighted Halley) iteration [14][16] to compute U_p . The QDWH iteration converges cubically and requires at most ~ 6 iterations. For ill-conditioned or rank-deficient matrices A , the routine adds a small perturbation ξ to ensure the polar decomposition converges correctly. The output parameter `d_res_nrm` reports the magnitude of the residual $\|A - U_p \cdot H\|_2$.

The user has to provide device and host working spaces which are pointed by input parameters `bufferOnDevice` and `bufferOnHost`. The input parameters `workspaceInBytesOnDevice` (and `workspaceInBytesOnHost`) is size in bytes of the device (and host) working space, and it is returned by `cusolverDnXpolar_bufferSize()`.

If output parameter `info = -i` (less than zero), the i -th parameter is wrong (not counting handle).

Currently, `cusolverDnXpolar` supports only the default algorithm.

Algorithms supported by `cusolverDnXpolar`

CUSOLVER_ALG_0 or NULL	Default algorithm.
------------------------	--------------------

Remark 1: If `H` is NULL, computation of `H` is skipped.

Remark 2: `uplo = CUBLAS_FILL_MODE_UPPER` treats `A` as upper triangular, saving one QR decomposition. `CUBLAS_FILL_MODE_LOWER` is not supported.

Remark 3: `d_res_nrm` serves dual purpose. On input, if `*d_res_nrm >= 0`, it overrides the perturbation ξ (skipping automatic search). On output, it reports the residual norm $\|A - U_p \cdot H\|_2$.

List of input arguments for `cusolverDnXpolar_bufferSize` and `cusolverDnXpolar`:

Table 59: **API of `cusolverDnXpolar`**

Parameter	Memory	In/out	Meaning
<code>handle</code>	host	input	Handle to the <code>cuSolverDN</code> library context.
<code>params</code>	host	input	Structure with information collected by <code>cusolverDnSetAdvOptions</code> .
<code>uplo</code>	host	input	Specifies the structure of matrix <code>A</code> . <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : <code>A</code> is upper triangular, saving one QR decomposition. <code>uplo = CUBLAS_FILL_MODE_FULL</code> : <code>A</code> is a general matrix. <code>CUBLAS_FILL_MODE_LOWER</code> is not supported.
<code>m</code>	host	input	Number of rows of matrix <code>A</code> , $m \geq n \geq 0$.
<code>n</code>	host	input	Number of columns of matrix <code>A</code> .
<code>dataTypeA</code>	host	input	Data type of array <code>A</code> .
<code>A</code>	device	in/out	Array of dimension <code>lda * n</code> with <code>lda</code> is not less than $\max(1, m)$. On exit, <code>A</code> is overwritten with the unitary polar factor U_p with orthonormal columns.
<code>lda</code>	host	input	Leading dimension of two-dimensional array used to store matrix <code>A</code> . <code>lda</code> $\geq \max(1, m)$.
<code>dataTypeH</code>	host	input	Data type of array <code>H</code> . Must equal <code>dataTypeA</code> .
<code>H</code>	device	output	Array of dimension <code>ldh * n</code> with <code>ldh</code> is not less than $\max(1, n)$. $n \times n$ Hermitian positive semidefinite factor. If <code>H</code> is NULL, computation of <code>H</code> is skipped.

continues on next page

Table 59 – continued from previous page

Parameter	Memory	In/out	Meaning
ldh	host	input	Leading dimension of two-dimensional array used to store matrix H. $ldh \geq \max(1, n)$ when H is not NULL.
computeType	host	input	Data type of computation. Must equal dataTypeA.
bufferOnDevice	device	in/out	Device workspace. Array of type void of size workspaceInBytesOnDevice bytes.
workspaceInBytesOnDevice	host	input	Size in bytes of bufferOnDevice, returned by cusolverDnXpolar_bufferSize.
bufferOnHost	host	in/out	Host workspace. Array of type void of size workspaceInBytesOnHost bytes.
workspaceInBytesOnHost	host	input	Size in bytes of bufferOnHost, returned by cusolverDnXpolar_bufferSize.
d_res_nrm	device	in/out	On input, if $*d_res_nrm \geq 0$, it overrides the perturbation ξ (skipping automatic search). On output, reports $\ A - U_p \cdot H\ _2$.
d_A_nrmF	device	output	$\ A\ _F$ (Frobenius norm of A). If NULL, the computation is skipped.
d_rcond	device	output	Reciprocal condition number $1/\text{cond}(A, \text{fro})$. If NULL, the computation is skipped. Undefined if a user-supplied $\xi > 0$ was provided via d_res_nrm.
d_info	device	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle). If info = 1, the algorithm failed to find $rcond > 1.e-15$ by adding perturbation. If info = 10, the algorithm failed to converge within 10 iterations.

The generic API has three different types, dataTypeA is data type of the matrix A, dataTypeH is data type of the matrix H, and computeType is compute type of the operation. cusolverDnXpolar only supports the following four combinations:

Valid combination of data type and compute type

DataTypeA	DataTypeH	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SPOLAR
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DPOLAR
CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CPOLAR
CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZPOLAR

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m < 0$, or $n < 0$, or $m < n$, or $lda < \max(1, m)$, or $ldh < \max(1, n)$ when H is not NULL, or $uplo$ is CUBLAS_FILL_MODE_LOWER, or $dataTypeH \neq dataTypeA$, or $computeType \neq dataTypeA$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.5. cuSolverSP: sparse LAPACK Function Reference **[[DEPRECATED]]**

[[DEPRECATED]] cuSolverSP is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

This section describes the API of cuSolverSP, which provides a subset of LAPACK functions for sparse matrices in CSR or CSC format.

8.5.1. Helper Function Reference

8.5.1.1 **cusolverSpCreate()** **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverSpCreate(cusolverSpHandle_t *handle)
```

This function initializes the cuSolverSP library and creates a handle on the cuSolver context. It must be called before any other cuSolverSP API function is invoked. It allocates hardware resources necessary for accessing the GPU.

Output

handle	The pointer to the handle to the cuSolverSP context.
--------	--

Status Returned**CUSOLVER_STATUS_SUCCESS**

The initialization succeeded.

CUSOLVER_STATUS_NOT_INITIALIZED

The CUDA Runtime initialization failed.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

8.5.1.2 `cusolverSpDestroy()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t  
cusolverSpDestroy(cusolverSpHandle_t handle)
```

This function releases CPU-side resources used by the cuSolverSP library.

Input

handle	The handle to the cuSolverSP context.
--------	---------------------------------------

Status Returned**CUSOLVER_STATUS_SUCCESS**

The shutdown succeeded.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.5.1.3 `cusolverSpSetStream()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t  
cusolverSpSetStream(cusolverSpHandle_t handle, cudaStream_t streamId)
```

This function sets the stream to be used by the cuSolverSP library to execute its routines.

Input

handle	The handle to the cuSolverSP context.
streamId	The stream to be used by the library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The stream was set successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.5.1.4 cusolverSpXcsrissym() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrissymHost(cusolverSpHandle_t handle,
    int m,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const int *csrRowPtrA,
    const int *csrEndPtrA,
    const int *csrColIndA,
    int *issym);
```

This function checks if A has symmetric pattern or not. The output parameter `issym` reports 1 if A is symmetric; otherwise, it reports 0.

The matrix A is an $m \times m$ sparse matrix that is defined in CSR storage format by the four arrays `csrValA`, `csrRowPtrA`, `csrEndPtrA` and `csrColIndA`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`.

The `csr1svlu` and `csr1svqr` do not accept non-general matrix. the user has to extend the matrix into its missing upper/lower part, otherwise the result is not expected. The user can use `csrissym` to check if the matrix has symmetric pattern or not.

Remark 1: only CPU path is provided.

Remark 2: the user has to check returned status to get valid information. The function converts A to CSC format and compare CSR and CSC format. If the CSC failed because of insufficient resources, `issym` is undefined, and this state can only be detected by the return status code.

Input

Parameter	MemorySpace	Description
handle	host	Handle to the cuSolverSP library context.
m	host	Number of rows and columns of matrix A.
nnzA	host	Number of nonzeros of matrix A. It is the size of csrValA and csrColIndA.
descrA	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrRowPtrA	host	Integer array of m elements that contains the start of every row.
csrEndPtrA	host	Integer array of m elements that contains the end of the last row plus one.
csrColIndA	host	Integer array of nnzA column indices of the nonzero elements of matrix A.

Output

Parameter	MemorySpace	Description
issym	host	1 if A is symmetric; 0 otherwise.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, nnzA ≤ 0), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2. High Level Function Reference

This section describes high level API of cuSolverSP, including linear solver, least-square solver and eigenvalue solver. The high-level API is designed for ease-of-use, so it allocates any required memory under the hood automatically. If the host or GPU system memory is not enough, an error is returned.

8.5.2.1 cusolverSp<t>csrslvlu() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverSpScsrslvlu[Host](cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const float *b,
    float tol,
    int reorder,
    float *x,
    int *singularity);

cusolverStatus_t
cusolverSpDcsrslvlu[Host](cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const double *b,
    double tol,
    int reorder,
    double *x,
    int *singularity);

cusolverStatus_t
cusolverSpCcsrslvlu[Host](cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const cuComplex *b,
    float tol,
    int reorder,
    cuComplex *x,
    int *singularity);

cusolverStatus_t
```

(continues on next page)

(continued from previous page)

```

cusolverSpZcsrslvlu[Host](cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const cuDoubleComplex *b,
    double tol,
    int reorder,
    cuDoubleComplex *x,
    int *singularity);

```

This function solves the linear system

$$A * x = b$$

where A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. `b` is the right-hand-side vector of size `n`, and `x` is the solution vector of size `n`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. If matrix A is symmetric/Hermitian and only lower/upper part is used or meaningful, the user has to extend the matrix into its missing upper/lower part, otherwise the result would be wrong.

The linear system is solved by sparse LU with partial pivoting:

$$P * A = L * U$$

`cusolver` library provides three reordering schemes, `symrcmsymamd`, and `csrmetisnd` to reduce zero fill-in which dramatically affects the performance of LU factorization. The input parameter `reorder` can enable `symrcm` (`symamd` or `csrmetisnd`) if `reorder` is 1 (2, or 3), otherwise, no reordering is performed.

If `reorder` is nonzero, `csrslvlu` does

$$P * A * Q^T = L * U$$

where $Q = \text{symrcm}(A + A^T)$.

If A is singular under given tolerance ($\max(\text{tol}, 0)$), then some diagonal elements of U is zero, i.e.

$$|U(j, j)| < \text{tol for some } j$$

The output parameter `singularity` is the smallest index of such `j`. If A is non-singular, `singularity` is -1. The index is base-0, independent of base index of A. For example, if 2nd column of A is the same as first column, then A is singular and `singularity` = 1 which means $U(1, 1) \approx 0$.

Remark 1: `csrslvlu` performs traditional LU with partial pivoting, the pivot of k-th column is determined dynamically based on the k-th column of intermediate matrix. `csrslvlu` follows Gilbert and Peierls's algorithm [4] which uses depth-first-search and topological ordering to solve triangular system (Davis also describes this algorithm in detail in his book [1]). Since CUDA 10.1, `csrslvlu` will incrementally reallocate the memory to store L and U. This feature can avoid over-estimate size from QR factorization. In some cases, zero fill-in of QR can be order of magnitude higher than LU.

Remark 2: only CPU (Host) path is provided.

Input

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
handle	host	host	Handle to the cuSolverSP library context.
n	host	host	Number of rows and columns of matrix A.
nnzA	host	host	Number of nonzeros of matrix A.
descra	host	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrValA	device	host	<type> array of nnzA(=csrRowPtrA(n)–csrRowPtrA(0)) nonzero elements of matrix A.
csrRowPtrA	device	host	Integer array of n+1 elements that contains the start of every row and the end of the last row plus one.
csrColIndA	device	host	Integer array of nnzA(=csrRowPtrA(n)–csrRowPtrA(0)) column indices of the nonzero elements of matrix A.
b	device	host	Right hand side vector of size n.
tol	host	host	Tolerance to decide if singular or not.
reorder	host	host	No ordering if reorder=0. Otherwise, symrcm, symamd, or crsmetisnd is used to reduce zero fill-in.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
x	device	host	Solution vector of size n, $x = \text{inv}(A) * b$.
singularity	host	host	-1 if A is invertible. Otherwise, first index j such that $U(j, j) \approx 0$

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n, nnzA ≤ 0, base index is not 0 or 1, reorder is not 0,1,2,3).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2.2 cusolverSp<t>csrsvqr() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpScsrsvqr[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const float *b,
    float tol,
    int reorder,
    float *x,
    int *singularity);
```

```
cusolverStatus_t
cusolverSpDcsrsvqr[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const double *b,
    double tol,
    int reorder,
    double *x,
    int *singularity);
```

```
cusolverStatus_t
cusolverSpCcsrsvqr[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const cuComplex *b,
    float tol,
    int reorder,
    cuComplex *x,
    int *singularity);
```

```
cusolverStatus_t
cusolverSpZcsrsvqr[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
```

(continues on next page)

(continued from previous page)

```

const cuDoubleComplex *csrValA,
const int *csrRowPtrA,
const int *csrColIndA,
const cuDoubleComplex *b,
double tol,
int reorder,
cuDoubleComplex *x,
int *singularity);

```

This function solves the linear system

$$A * x = b$$

A is an $m \times m$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. `b` is the right-hand-side vector of size `m`, and `x` is the solution vector of size `m`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. If matrix A is symmetric/Hermitian and only lower/upper part is used or meaningful, the user has to extend the matrix into its missing upper/lower part, otherwise the result would be wrong.

The linear system is solved by sparse QR factorization,

$$A = Q * R$$

If A is singular under given tolerance ($\max(\text{tol}, 0)$), then some diagonal elements of R is zero, i.e.

$$|R(j, j)| < \text{tol for some } j$$

The output parameter `singularity` is the smallest index of such `j`. If A is non-singular, `singularity` is -1. The `singularity` is base-0, independent of base index of A. For example, if 2nd column of A is the same as first column, then A is singular and `singularity` = 1 which means $R(1, 1) \approx 0$.

`cusolver` library provides three reordering schemes, `symrcmsymamd`, and `csrmetisnd` to reduce zero fill-in which dramatically affects the performance of QR factorization. The input parameter `reorder` can enable `symrcm` (`symamd` or `csrmetisnd`) if `reorder` is 1 (2, or 3), otherwise, no reordering is performed.

Input

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
<code>handle</code>	host	host	Handle to the <code>cuSolverSP</code> library context.
<code>m</code>	host	host	Number of rows and columns of matrix A.
<code>nnz</code>	host	host	Number of nonzeros of matrix A.
<code>descrA</code>	host	host	The descriptor of matrix A. The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .

continues on next page

Table 61 – continued from previous page

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
csrValA	device	host	<type> array of $\text{nnz}(=\text{csrRowPtrA}(m) - \text{csrRowPtrA}(0))$ nonzero elements of matrix A.
csrRowPtrA	device	host	Integer array of $m+1$ elements that contains the start of every row and the end of the last row plus one.
csrColIndA	device	host	Integer array of $\text{nnz}(=\text{csrRowPtrA}(m) - \text{csrRowPtrA}(0))$ column indices of the nonzero elements of matrix A.
b	device	host	Right hand side vector of size m .
tol	host	host	Tolerance to decide if singular or not.
reorder	host	host	No ordering if $\text{reorder}=0$. Otherwise, <code>symrcm</code> , <code>symamd</code> , or <code>csrmetisnd</code> is used to reduce zero fill-in.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
x	device	host	Solution vector of size m , $x = \text{inv}(A)*b$.
singularity	host	host	-1 if A is invertible. Otherwise, first index j such that $R(j, j) \approx 0$

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m , $\text{nnz} \leq 0$, base index is not 0 or 1, reorder is not 0,1,2,3)

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2.3 cusolverSp<t>csrsvchol() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverSpScsrsvchol[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const float *csrVal,
    const int *csrRowPtr,
    const int *csrColInd,
    const float *b,
    float tol,
    int reorder,
    float *x,
    int *singularity);

cusolverStatus_t
cusolverSpDcsrsvchol[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrVal,
    const int *csrRowPtr,
    const int *csrColInd,
    const double *b,
    double tol,
    int reorder,
    double *x,
    int *singularity);

cusolverStatus_t
cusolverSpCcsrsvchol[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrVal,
    const int *csrRowPtr,
    const int *csrColInd,
    const cuComplex *b,
    float tol,
    int reorder,
    cuComplex *x,
    int *singularity);

cusolverStatus_t
cusolverSpZcsrsvchol[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrVal,
    const int *csrRowPtr,
    const int *csrColInd,
    const cuDoubleComplex *b,
    double tol,
```

(continues on next page)

(continued from previous page)

```
int reorder,
cuDoubleComplex *x,
int *singularity);
```

This function solves the linear system

$$A * x = b$$

A is an $m \times m$ symmetric positive definite sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. `b` is the right-hand-side vector of size `m`, and `x` is the solution vector of size `m`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL` and upper triangular part of A is ignored (if parameter `reorder` is zero). In other words, suppose input matrix A is decomposed as $A = L + D + U$, where L is lower triangular, D is diagonal and U is upper triangular. The function would ignore U and regard A as a symmetric matrix with the formula $A = L + D + L^H$. If parameter `reorder` is nonzero, the user has to extend A to a full matrix, otherwise the solution would be wrong.

The linear system is solved by sparse Cholesky factorization,

$$A = G * G^H$$

where G is the Cholesky factor, a lower triangular matrix.

The output parameter `singularity` has two meanings:

- If A is not positive definite, there exists some integer k such that $A(0:k, 0:k)$ is not positive definite. `singularity` is the minimum of such k.
- If A is positive definite but near singular under tolerance ($\max(\text{tol}, 0)$), i.e. there exists some integer k such that $G(k, k) \leq \text{tol}$. `singularity` is the minimum of such k.

`singularity` is base-0. If A is positive definite and not near singular under tolerance, `singularity` is -1. If the user wants to know if A is positive definite or not, `tol=0` is enough.

`cusolver` library provides three reordering schemes, `symrcmsymamd`, and `csrmetisnd` to reduce zero fill-in which dramatically affects the performance of Cholesky factorization. The input parameter `reorder` can enable `symrcm` (`symamd` or `csrmetisnd`) if `reorder` is 1 (2, or 3), otherwise, no reordering is performed.

Remark 1: the function works for in-place (`x` and `b` point to the same memory block) and out-of-place.

Remark 2: the function only works on 32-bit index, if matrix G has large zero fill-in such that number of nonzeros is bigger than 2^{31} , then `CUSOLVER_STATUS_ALLOC_FAILED` is returned.

Input

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
<code>handle</code>	host	host	Handle to the <code>cuSolverSP</code> library context.
<code>m</code>	host	host	Number of rows and columns of matrix A.
<code>nnz</code>	host	host	Number of nonzeros of matrix A.

continues on next page

Table 62 – continued from previous page

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
descra	host	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrValA	device	host	<type> array of nnz(=csrRowPtrA(m)–csrRowPtrA(0)) nonzero elements of matrix A.
csrRowPtrA	device	host	Integer array of m+1 elements that contains the start of every row and the end of the last row plus one.
csrColIndA	device	host	Integer array of nnz(=csrRowPtrA(m)–csrRowPtrA(0)) column indices of the nonzero elements of matrix A.
b	device	host	Right hand side vector of size m.
tol	host	host	Tolerance to decide singularity.
reorder	host	host	No ordering if reorder=0. Otherwise, symrcm, symamd, or crsmetisnd is used to reduce zero fill-in.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
x	device	host	Solution vector of size m, $x = \text{inv}(A) * b$.
singularity	host	host	-1 if A is symmetric positive definite.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, nnz<=0, base index is not 0 or 1, reorder is not 0,1,2,3).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2.4 cusolverSp<t>csrIsqvr() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverSpScsrIsqvr[Host](cusolverSpHandle_t handle,
    int m,
    int n,
    int nnz,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const float *b,
    float tol,
    int *rankA,
    float *x,
    int *p,
    float *min_norm);

cusolverStatus_t
cusolverSpDcsrIsqvr[Host](cusolverSpHandle_t handle,
    int m,
    int n,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const double *b,
    double tol,
    int *rankA,
    double *x,
    int *p,
    double *min_norm);
```

The C and Z data types are complex valued single and double precision, respectively.

```
cusolverStatus_t
cusolverSpCcsrIsqvr[Host](cusolverSpHandle_t handle,
    int m,
    int n,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const cuComplex *b,
    float tol,
    int *rankA,
    cuComplex *x,
    int *p,
    float *min_norm);

cusolverStatus_t
cusolverSpZcsrIsqvr[Host](cusolverSpHandle_t handle,
```

(continues on next page)

(continued from previous page)

```

int m,
int n,
int nnz,
const cusparseMatDescr_t descrA,
const cuDoubleComplex *csrValA,
const int *csrRowPtrA,
const int *csrColIndA,
const cuDoubleComplex *b,
double tol,
int *rankA,
cuDoubleComplex *x,
int *p,
double *min_norm);

```

This function solves the following least-square problem:

$$x = \operatorname{argmin} \|A * z - b\|$$

A is an $m \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. `b` is the right-hand-side vector of size `m`, and `x` is the least-square solution vector of size `n`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. If A is square, symmetric/Hermitian and only lower/upper part is used or meaningful, the user has to extend the matrix into its missing upper/lower part, otherwise the result is wrong.

This function only works if `m` is greater or equal to `n`, in other words, A is a tall matrix.

The least-square problem is solved by sparse QR factorization with column pivoting,

$$A * P^T = Q * R$$

If A is of full rank (i.e. all columns of A are linear independent), then matrix P is an identity. Suppose rank of A is `k`, less than `n`, the permutation matrix P reorders columns of A in the following sense:

$$A * P^T = \begin{pmatrix} A_1 & A_2 \end{pmatrix} = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_{11} & R_{12} \\ & R_{22} \end{pmatrix}$$

where R_{11} and A have the same rank, but R_{22} is almost zero, i.e. every column of A_2 is linear combination of A_1 .

The input parameter `tol` decides numerical rank. The absolute value of every entry in R_{22} is less than or equal to `tolerance=max(tol, 0)`.

The output parameter `rankA` denotes numerical rank of A.

Suppose $y = P * x$ and $c = Q^H * b$, the least square problem can be reformed by

$$\min \|A * x - b\| = \min \|R * y - c\|$$

or in matrix form

$$\begin{pmatrix} R_{11} & R_{12} \\ & R_{22} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$$

The output parameter `min_norm` is $\|c_2\|$, which is minimum value of least-square problem.

If A is not of full rank, above equation does not have a unique solution. The least-square problem is equivalent to

$$\begin{aligned} & \min \|y\| \\ & \text{subject to } R_{11} * y_1 + R_{12} * y_2 = c_1 \end{aligned}$$

Or equivalently another least-square problem

$$\min \left\| \begin{pmatrix} R_{11} \setminus R_{12} \\ I \end{pmatrix} * y_2 - \begin{pmatrix} R_{11} \setminus c_1 \\ O \end{pmatrix} \right\|$$

The output parameter x is $P^T * y$, the solution of least-square problem.

The output parameter p is a vector of size n. It corresponds to a permutation matrix P. $p(i)=j$ means $(P*x)(i) = x(j)$. If A is of full rank, $p=0:n-1$.

Remark 1: p is always base 0, independent of base index of A.

Remark 2: only CPU (Host) path is provided.

Input

Parameter	cu-solver	*Host MemSpace	Description
handle	host	host	Handle to the cuSolver library context.
m	host	host	Number of rows of matrix A.
n	host	host	Number of columns of matrix A.
nnz	host	host	Number of nonzeros of matrix A.
descrA	host	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrValA	device	host	<type> array of $nnz(=csrRowPtrA(m)-csrRowPtrA(0))$ nonzero elements of matrix A.
csrRowPtrA	device	host	Integer array of $m+1$ elements that contains the start of every row and the end of the last row plus one.
csrColIndA	device	host	Integer array of $nnz(=csrRowPtrA(m)-csrRowPtrA(0))$ column indices of the nonzero elements of matrix A.
b	device	host	Right hand side vector of size m.
tol	host	host	Tolerance to decide rank of A.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
rankA	host	host	Numerical rank of A.
x	device	host	Solution vector of size n, $x = \text{pinv}(A) * b$.
p	device	host	A vector of size n, which represents the permutation matrix P satisfying $A * P^T = Q * R$.
min_norm	host	host	$\ A * x - b\ $, $x = \text{pinv}(A) * b$.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($m, n, nnz \leq 0$), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2.5 cusolverSp<t>csreigvsi() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

The S and D data types are real valued single and double precision, respectively.

```
cusolverStatus_t
cusolverSpScsreigvsi[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    float mu0,
    const float *x0,
    int maxite,
    float tol,
    float *mu,
    float *x);
```

(continues on next page)

(continued from previous page)

```

cusolverStatus_t
cusolverSpDcsreigvsi[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    double mu0,
    const double *x0,
    int maxite,
    double tol,
    double *mu,
    double *x);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverSpCcsreigvsi[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuComplex mu0,
    const cuComplex *x0,
    int maxite,
    float tol,
    cuComplex *mu,
    cuComplex *x);

```

```

cusolverStatus_t
cusolverSpZcsreigvsi(cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuDoubleComplex mu0,
    const cuDoubleComplex *x0,
    int maxite,
    double tol,
    cuDoubleComplex *mu,
    cuDoubleComplex *x);

```

This function solves the simple eigenvalue problem $A * x = \lambda * x$ by shift-inverse method.

A is an $m \times m$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. The output parameter `x` is the approximated eigenvector of size `m`,

The following shift-inverse method corrects eigenpair step-by-step until convergence.

It accepts several parameters:

`mu0` is an initial guess of eigenvalue. The shift-inverse method will converge to the eigenvalue `mu` nearest `mu0` if `mu` is a singleton. Otherwise, the shift-inverse method may not converge.

x_0 is an initial eigenvector. If the user has no preference, just chose x_0 randomly. x_0 must be nonzero. It can be non-unit length.

`tol` is the tolerance to decide convergence. If `tol` is less than zero, it would be treated as zero.

`maxite` is maximum number of iterations. It is useful when shift-inverse method does not converge because the tolerance is too small or the desired eigenvalue is not a singleton.

Shift-Inverse Method

Given an initial guess of eigenvalue μ_0 and initial vector x_0

```

 $x(0) = x_0$  of unit length
for  $j = 0 : \text{maxite}$ 
    solve  $(A - \mu_0 * I) * x^{k+1} = x^k$ 
    normalize  $x^{k+1}$  to unit length
    compute approx. eigenvalue  $\mu = x^H * A * x$  where  $x = x^{k+1}$ 
    if  $\|A * x^{k+1} - \mu * x^{k+1}\| < \text{tolerance}$ , then stop
endfor

```

The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. If A is symmetric/Hermitian and only lower/upper part is used or meaningful, the user has to extend the matrix into its missing upper/lower part, otherwise the result is wrong.

Remark 1: `[cu|h]solver[S|D]csreigvsi` only allows μ_0 as a real number. This works if A is symmetric. Otherwise, the non-real eigenvalue has a conjugate counterpart on the complex plan, and shift-inverse method would not converge to such eigenvalue even the eigenvalue is a singleton. The user has to extend A to complex numbers and call `[cu|h]solver[C|Z]csreigvsi` with μ_0 not on real axis.

Remark 2: the tolerance `tol` should not be smaller than $|\mu_0| * \text{eps}$, where `eps` is machine zero. Otherwise, shift-inverse may not converge because of small tolerance.

Input

Parameter	cu-solver	*Host MemSpace MemSpace	Description
handle	host	host	Handle to the cuSolver library context.
m	host	host	Number of rows and columns of matrix A.
nnz	host	host	Number of nonzeros of matrix A.
descra	host	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrValA	device	host	<type> array of $\text{nnz}(\text{csrRowPtrA}(m) - \text{csrRowPtrA}(0))$ nonzero elements of matrix A.

continues on next page

Table 64 – continued from previous page

Parameter	cu-solverSp MemSpace	*Host MemSpace	Description
csr-RowPtrA	device	host	Integer array of $m+1$ elements that contains the start of every row and the end of the last row plus one.
csr-ColIndA	device	host	Integer array of $nnz(=csrRowPtrA(m)-csrRowPtrA(0))$ column indices of the nonzero elements of matrix A.
mu0	host	host	Initial guess of eigenvalue.
x0	device	host	Initial guess of eigenvector, a vector of size m .
max-ite	host	host	Maximum iterations in shift-inverse method.
tol	host	host	Tolerance for convergence.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
mu	device	host	Approximated eigenvalue nearest mu0 under tolerance.
x	device	host	Approximated eigenvector of size m .

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m , $nnz \leq 0$), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.2.6 cusolverSp<t>csreigs() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
solverspScsreigs[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuComplex left_bottom_corner,
    cuComplex right_upper_corner,
    int *num_eigs);

cusolverStatus_t
cusolverSpDcsreigs[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuDoubleComplex left_bottom_corner,
    cuDoubleComplex right_upper_corner,
    int *num_eigs);

cusolverStatus_t
cusolverSpCcsreigs[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuComplex left_bottom_corner,
    cuComplex right_upper_corner,
    int *num_eigs);

cusolverStatus_t
cusolverSpZcsreigs[Host](cusolverSpHandle_t handle,
    int m,
    int nnz,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    cuDoubleComplex left_bottom_corner,
    cuDoubleComplex right_upper_corner,
    int *num_eigs);
```

This function computes number of algebraic eigenvalues in a given box B by contour integral

$$\text{number of algebraic eigenvalues in box B} = \frac{1}{2 * \pi * \sqrt{-1}} \oint_C \frac{P'(z)}{P(z)} dz$$

where closed line C is boundary of the box B which is a rectangle specified by two points, one is left bottom corner (input parameter `left_bottom_corner`) and the other is right upper corner (input parameter `right_upper_corner`). $P(z)=\det(A - z*I)$ is the characteristic polynomial of A.

A is an $m \times m$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`.

The output parameter `num_eigs` is number of algebraic eigenvalues in the box B. This number may not be accurate due to several reasons:

1. The contour C is close to some eigenvalues or even passes through some eigenvalues.
2. The numerical integration is not accurate due to coarse grid size. The default resolution is 1200 grids along contour C uniformly.

Even though `csreigs` may not be accurate, it still can give the user some idea how many eigenvalues in a region where the resolution of disk theorem is bad. For example, standard 3-point stencil of finite difference of Laplacian operator is a tridiagonal matrix, and disk theorem would show “all eigenvalues are in the interval $[0, 4*N^2]$ ” where N is number of grids. In this case, `csreigs` is useful for any interval inside $[0, 4*N^2]$.

Remark 1: if A is symmetric in real or Hermitian in complex, all eigenvalues are real. The user still needs to specify a box, not an interval. The height of the box can be much smaller than the width.

Remark 2: only CPU (Host) path is provided.

Input

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
<code>handle</code>	host	host	Handle to the cuSolverSP library context.
<code>m</code>	host	host	Number of rows and columns of matrix A.
<code>nnz</code>	host	host	Number of nonzeros of matrix A.
<code>descrA</code>	host	host	The descriptor of matrix A. The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
<code>csrValA</code>	device	host	<type> array of <code>nnz(=csrRowPtrA(m)-csrRowPtrA(0))</code> nonzero elements of matrix A.
<code>csrRowPtrA</code>	device	host	Integer array of <code>m+1</code> elements that contains the start of every row and the end of the last row plus one.
<code>csrColIndA</code>	device	host	Integer array of <code>nnz(=csrRowPtrA(m)-csrRowPtrA(0))</code> column indices of the nonzero elements of matrix A.
<code>left_bottom_corner</code>	host	host	Left bottom corner of the box.
<code>right_upper_corner</code>	host	host	Right upper corner of the box.

Output

Parameter	cusolverSp MemSpace	*Host MemSpace	Description
num_eigs	host	host	Number of algebraic eigenvalues in a box.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m , $nnz \leq 0$), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3. Low Level Function Reference

This section describes low level API of cuSolverSP, including symrcm and batched QR.

8.5.3.1 cusolverSpXcsrsmrcm() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrsmrcmHost(cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *p);
```

This function implements Symmetric Reverse Cuthill-McKee permutation. It returns a permutation vector p such that $A(p, p)$ would concentrate nonzeros to diagonal. This is equivalent to `symrcm` in MATLAB, however the result may not be the same because of different heuristics in the pseudoperipheral finder. The cuSolverSP library implements `symrcm` based on the following two papers:

- E. Cuthill and J. McKee, Reducing the bandwidth of sparse symmetric matrices, ACM '69 Proceedings of the 1969 24th national conference, Pages 157-172
- Alan George, Joseph W. H. Liu, An Implementation of a Pseudoperipheral Node Finder, ACM Transactions on Mathematical Software (TOMS) Volume 5 Issue 3, Sept. 1979, Pages 284-295

The output parameter `p` is an integer array of `n` elements. It represents a permutation array and it indexed using the base-0 convention. The permutation array `p` corresponds to a permutation matrix `P`, and satisfies the following relation:

$$A(p, p) = P * A * P^T$$

`A` is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. Internally `rcm` works on $A + A^T$, the user does not need to extend the matrix if the matrix is not symmetric.

Remark 1: only CPU (Host) path is provided.

Input

Parameter	*Host MemSpace	Description
<code>handle</code>	host	Handle to the cuSolverSP library context.
<code>n</code>	host	Number of rows and columns of matrix <code>A</code> .
<code>nnzA</code>	host	Number of nonzeros of matrix <code>A</code> . It is the size of <code>csrValA</code> and <code>csrColIndA</code> .
<code>descrA</code>	host	The descriptor of matrix <code>A</code> . The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
<code>csrRowPtrA</code>	host	Integer array of <code>n+1</code> elements that contains the start of every row and the end of the last row plus one.
<code>csrColIndA</code>	host	Integer array of <code>nnzA</code> column indices of the nonzero elements of matrix <code>A</code> .

Output

Parameter	hsolver	Description
<code>p</code>	host	Permutation vector of size <code>n</code> .

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (`n`, `nnzA` ≤ 0), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.2 cusolverSpXcsrSymmdq() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrSymmdqHost(cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *p);
```

This function implements Symmetric Minimum Degree Algorithm based on Quotient Graph. It returns a permutation vector p such that $A(p, p)$ would have less zero fill-in during Cholesky factorization. The cuSolverSP library implements symmdq based on the following two papers:

Patrick R. Amestoy, Timothy A. Davis, Iain S. Duff, An Approximate Minimum Degree Ordering Algorithm, SIAM J. Matrix Analysis Applic. Vol 17, no 4, pp. 886-905, Dec. 1996.

Alan George, Joseph W. Liu, A Fast Implementation of the Minimum Degree Algorithm Using Quotient Graphs, ACM Transactions on Mathematical Software, Vol 6, No. 3, September 1980, page 337-358.

The output parameter p is an integer array of n elements. It represents a permutation array with base-0 index. The permutation array p corresponds to a permutation matrix P , and satisfies the following relation:

$$A(p, p) = P * A * P^T$$

A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. Internally `mdq` works on $A + A^T$, the user does not need to extend the matrix if the matrix is not symmetric.

Remark 1: only CPU (Host) path is provided.

Input

Parameter	*Host MemSpace	Description
handle	host	Handle to the cuSolverSP library context.
n	host	Number of rows and columns of matrix A.
nnzA	host	Number of nonzeros of matrix A. It is the size of csrValA and csrColIndA.
descrA	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrRowPtrA	host	Integer array of n+1 elements that contains the start of every row and the end of the last row plus one.
csrColIndA	host	Integer array of nnzA column indices of the nonzero elements of matrix A.

Output

Parameter	hsolver	Description
p	host	Permutation vector of size n.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n, nnzA ≤ 0), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.3 cusolverSpXcsrSymamd() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrSymamdHost(cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *p);
```

This function implements Symmetric Approximate Minimum Degree Algorithm based on Quotient Graph. It returns a permutation vector p such that $A(p, p)$ would have less zero fill-in during Cholesky factorization. The cuSolverSP library implements symamd based on the following paper:

Patrick R. Amestoy, Timothy A. Davis, Iain S. Duff, An Approximate Minimum Degree Ordering Algorithm, SIAM J. Matrix Analysis Applic. Vol 17, no 4, pp. 886-905, Dec. 1996.

The output parameter p is an integer array of n elements. It represents a permutation array with base-0 index. The permutation array p corresponds to a permutation matrix P , and satisfies the following relation:

$$A(p, p) = P * A * P^T$$

A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. Internally `amd` works on $A + A^T$, the user does not need to extend the matrix if the matrix is not symmetric.

Remark 1: only CPU (Host) path is provided.

Input

Parameter	*Host MemSpace	Description
<code>handle</code>	host	Handle to the cuSolverSP library context.
<code>n</code>	host	Number of rows and columns of matrix A .
<code>nnzA</code>	host	Number of nonzeros of matrix A . It is the size of <code>csrValA</code> and <code>csrColIndA</code> .
<code>descrA</code>	host	The descriptor of matrix A . The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
<code>csrRowPtrA</code>	host	Integer array of $n+1$ elements that contains the start of every row and the end of the last row plus one.
<code>csrColIndA</code>	host	Integer array of <code>nnzA</code> column indices of the nonzero elements of matrix A .

Output

Parameter	hsolver	Description
p	host	Permutation vector of size n.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n , $nnzA \leq 0$), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.4 cusolverSpXcsrmetisnd() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrmetisndHost(
    cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const int64_t *options,
    int *p);
```

This function is a wrapper of METIS_NodeND. It returns a permutation vector p such that $A(p,p)$ would have less zero fill-in during nested dissection. The cuSolverSP library links `libcusolver_mmetis_static.a` which is 64-bit metis-5.1.0.

The parameter `options` is the configuration of `metis`. For those who do not have experiences of `metis`, set `options = NULL` for default setting.

The output parameter p is an integer array of n elements. It represents a permutation array with base-0 index. The permutation array p corresponds to a permutation matrix P , and satisfies the following relation:

$$A(p,p) = P * A * P^T$$

A is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`.

The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Internally csrmetisnd works on $A + A^T$, the user does not need to extend the matrix if the matrix is not symmetric.

Remark 1: only CPU (Host) path is provided.

Input

Parameter	*Host MemSpace	Description
handle	host	Handle to the cuSolverSP library context.
n	host	Number of rows and columns of matrix A.
nnzA	host	Number of nonzeros of matrix A. It is the size of csrValA and csrColIndA.
descrA	host	The descriptor of matrix A. The supported matrix type is CUSPARSE_MATRIX_TYPE_GENERAL. Also, the supported index bases are CUSPARSE_INDEX_BASE_ZERO and CUSPARSE_INDEX_BASE_ONE.
csrRowPtrA	host	Integer array of n+1 elements that contains the start of every row and the end of the last row plus one.
csrColIndA	host	Integer array of nnzA column indices of the nonzero elements of matrix A.
options	host	Integer array to configure metis.

Output

Parameter	*Host MemSpace	Description
p	host	Permutation vector of size n.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n, nnzA <= 0), base index is not 0 or 1.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.5 `cusolverSpXcsrzdHost()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpScsrzfdHost(
    cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

cusolverStatus_t
cusolverSpDcsrzdHost(
    cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

cusolverStatus_t
cusolverSpCcsrzdHost(
    cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)

cusolverStatus_t
cusolverSpZcsrzdHost(
    cusolverSpHandle_t handle,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int *P,
    int *numnz)
```

This function implements MC21, zero-free diagonal algorithm. It returns a permutation vector `p` such that `A(p, :)` has no zero diagonal.

`A` is an $n \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA`, and `csrColIndA`. The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`.

The output parameter `p` is an integer array of `n` elements. It represents a permutation array with base-

0 index. The permutation array p corresponds to a permutation matrix P , and satisfies the following relation:

$$A(p, :) = P * A$$

The output parameter `numnz` describes number of nonzero diagonal in permuted matrix $A(p, :)$. If `numnz` is less than n , matrix A has structural singularity.

Remark 1: only CPU (Host) path is provided.

Remark 2: this routine does not maximize diagonal value of permuted matrix. The user cannot expect this routine can make “LU without pivoting” stable.

Input

Parameter	*Host MemSpace	Description
<code>handle</code>	host	Handle to the cuSolverSP library context.
<code>n</code>	host	Number of rows and columns of matrix A .
<code>nnzA</code>	host	Number of nonzeros of matrix A . It is the size of <code>csrValA</code> and <code>csrColIndA</code> .
<code>descrA</code>	host	The descriptor of matrix A . The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
<code>csrValA</code>	host	<type> array of <code>nnzA(=csrRowPtrA(m)–csrRowPtrA(0))</code> nonzero elements of matrix A .
<code>csrRowPtrA</code>	host	Integer array of $n+1$ elements that contains the start of every row and the end of the last row plus one.
<code>csrColIndA</code>	host	Integer array of <code>nnzA</code> column indices of the nonzero elements of matrix A .

Output

Parameter	*Host MemSpace	Description
<code>p</code>	host	Permutation vector of size n .
<code>numnz</code>	host	Number of nonzeros on diagonal of permuted matrix.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (n , $nnzA \leq 0$), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.6 cusolverSpXcsrperm() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

```
cusolverStatus_t
cusolverSpXcsrperm_bufferSizeHost(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    int *csrRowPtrA,
    int *csrColIndA,
    const int *p,
    const int *q,
    size_t *bufferSizeInBytes);

cusolverStatus_t
cusolverSpXcsrpermHost(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    int *csrRowPtrA,
    int *csrColIndA,
    const int *p,
    const int *q,
    int *map,
    void *pBuffer);
```

Given a left permutation vector p which corresponds to permutation matrix P and a right permutation vector q which corresponds to permutation matrix Q , this function computes permutation of matrix A by

$$B = P * A * Q^T$$

A is an $m \times n$ sparse matrix that is defined in CSR storage format by the three arrays `csrValA`, `csrRowPtrA` and `csrColIndA`.

The operation is in-place, i.e. the matrix A is overwritten by B .

The permutation vector p and q are base 0. p performs row permutation while q performs column permutation. One can also use MATLAB command $B = A(p, q)$ to permute matrix A .

This function only computes sparsity pattern of B . The user can use parameter `map` to get `csrValB` as well. The parameter `map` is an input/output. If the user sets `map=0:(nnzA-1)` before calling `csrperm`, `csrValB=csrValA(map)`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. If A is symmetric and only lower/upper part is provided, the user has to pass $A + A^T$ into this function.

This function requires a buffer size returned by `csrperm_bufferSize()`. The address of `pBuffer` must be a multiple of 128 bytes. If it is not, `CUSOLVER_STATUS_INVALID_VALUE` is returned.

For example, if matrix A is

$$A = \begin{pmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \\ 7.0 & 8.0 & 9.0 \end{pmatrix}$$

and left permutation vector $p=(0, 2, 1)$, right permutation vector $q=(2, 1, 0)$, then $P * A * Q^T$ is

$$P * A * Q^T = \begin{pmatrix} 3.0 & 2.0 & 1.0 \\ 9.0 & 8.0 & 7.0 \\ 6.0 & 5.0 & 4.0 \end{pmatrix}$$

Remark 1: only CPU (Host) path is provided.

Remark 2: the user can combine `csrsymrcm` and `csrperm` to get $P * A * P^T$ which has less zero fill-in during QR factorization.

Input

Parameter	cusolverSp MemSpace	Description
handle	host	Handle to the cuSolver library context.
m	host	Number of rows of matrix A.
n	host	Number of columns of matrix A.
nnzA	host	Number of nonzeros of matrix A. It is the size of <code>csrValA</code> and <code>csrColIndA</code> .
descrA	host	The descriptor of matrix A. The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
csrRowPtrA	host	Integer array of <code>m+1</code> elements that contains the start of every row and end of last row plus one of matrix A.
csrColIndA	host	Integer array of <code>nnzA</code> column indices of the nonzero elements of matrix A.
p	host	Left permutation vector of size <code>m</code> .
q	host	Right permutation vector of size <code>n</code> .
map	host	Integer array of <code>nnzA</code> indices. If the user wants to get relationship between A and B, <code>map</code> must be set <code>0:(nnzA-1)</code> .
pBuffer	host	Buffer allocated by the user, the size is returned by <code>csrperm_bufferSize()</code> .

Output

Parameter	hsolver	Description
csrRowPtrA	host	Integer array of m+1 elements that contains the start of every row and end of last row plus one of matrix B.
csrColIndA	host	Integer array of nnzAcolumn indices of the nonzero elements of matrix B.
map	host	Integer array of nnzA indices that maps matrix A to matrix B.
pBuffer-SizeInBytes	host	Number of bytes of the buffer.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n, nnzA<=0), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.5.3.7 cusolverSpXcsrqrBatched() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version.

The create and destroy methods start and end the lifetime of a csrqrInfo object.

```
cusolverStatus_t
cusolverSpCreateCsrqrInfo(csrqrInfo_t *info);

cusolverStatus_t
cusolverSpDestroyCsrqrInfo(csrqrInfo_t info);
```

Analysis is the same for all data types, but each data type has a unique buffer size.

```
cusolverStatus_t
cusolverSpXcsrqrAnalysisBatched(cusolverSpHandle_t handle,
                                int m,
                                int n,
                                int nnzA,
                                const cusparseMatDescr_t descrA,
                                const int *csrRowPtrA,
                                const int *csrColIndA,
```

(continues on next page)

(continued from previous page)

```

        csrqrInfo_t info);

cusolverStatus_t
cusolverSpScsrqrBufferInfoBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int batchSize,
    csrqrInfo_t info,
    size_t *internalDataInBytes,
    size_t *workspaceInBytes);

cusolverStatus_t
cusolverSpDcsrqrBufferInfoBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int batchSize,
    csrqrInfo_t info,
    size_t *internalDataInBytes,
    size_t *workspaceInBytes);

```

Calculate buffer sizes for complex valued data types.

```

cusolverStatus_t
cusolverSpCcsrqrBufferInfoBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int batchSize,
    csrqrInfo_t info,
    size_t *internalDataInBytes,
    size_t *workspaceInBytes);

cusolverStatus_t
cusolverSpZcsrqrBufferInfoBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuDoubleComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    int batchSize,
    csrqrInfo_t info,

```

(continues on next page)

(continued from previous page)

```

size_t *internalDataInBytes,
size_t *workspaceInBytes);

```

The S and D data types are real valued single and double precision, respectively.

```

cusolverStatus_t
cusolverSpScsrqrsvBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const float *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const float *b,
    float *x,
    int batchSize,
    csqrInfo_t info,
    void *pBuffer);

cusolverStatus_t
cusolverSpDcsrqrsvBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnz,
    const cusparseMatDescr_t descrA,
    const double *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const double *b,
    double *x,
    int batchSize,
    csqrInfo_t info,
    void *pBuffer);

```

The C and Z data types are complex valued single and double precision, respectively.

```

cusolverStatus_t
cusolverSpCcsrqrsvBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,
    const cusparseMatDescr_t descrA,
    const cuComplex *csrValA,
    const int *csrRowPtrA,
    const int *csrColIndA,
    const cuComplex *b,
    cuComplex *x,
    int batchSize,
    csqrInfo_t info,
    void *pBuffer);

cusolverStatus_t
cusolverSpZcsrqrsvBatched(cusolverSpHandle_t handle,
    int m,
    int n,
    int nnzA,

```

(continues on next page)

(continued from previous page)

```

const cusparseMatDescr_t descrA,
const cuDoubleComplex *csrValA,
const int *csrRowPtrA,
const int *csrColIndA,
const cuDoubleComplex *b,
cuDoubleComplex *x,
int batchSize,
csrqrInfo_t info,
void *pBuffer);

```

The batched sparse QR factorization is used to solve either a set of least-squares problems

$$x_j = \operatorname{argmin} \|A_j * z - b_j\|, j = 1, 2, \dots, \text{batchSize}$$

or a set of linear systems

$$A_j x_j = b_j, j = 1, 2, \dots, \text{batchSize}$$

where each A_j is an $m \times n$ sparse matrix that is defined in CSR storage format by the four arrays `csrValA`, `csrRowPtrA` and `csrColIndA`.

The supported matrix type is `CUSPARSE_MATRIX_TYPE_GENERAL`. If A is symmetric and only lower/upper part is provided, the user has to pass $A + A^H$ into this function.

The prerequisite to use batched sparse QR has two-folds. First all matrices A_j must have the same sparsity pattern. Second, no column pivoting is used in least-square problem, so the solution is valid only if A_j is of full rank for all $j = 1, 2, \dots, \text{batchSize}$. All matrices have the same sparsity pattern, so only one copy of `csrRowPtrA` and `csrColIndA` is used. But the array `csrValA` stores coefficients of A_j one after another. In other words, `csrValA[k*nnzA : (k+1)*nnzA]` is the value of A_k .

The batched QR uses opaque data structure `csrqrInfo` to keep intermediate data, for example, matrix Q and matrix R of QR factorization. The user needs to create `csrqrInfo` first by `cusolverSpCreateCsrqrInfo` before any function in batched QR operation. The `csrqrInfo` would not release internal data until `cusolverSpDestroyCsrqrInfo` is called.

There are three routines in batched sparse QR, `cusolverSpXcsrqrAnalysisBatched`, `cusolverSp[S|D|C|Z]csrqrBufferInfoBatched` and `cusolverSp[S|D|C|Z]csrqrsvBatched`.

First, `cusolverSpXcsrqrAnalysisBatched` is the analysis phase, used to analyze sparsity pattern of matrix Q and matrix R of QR factorization. Also parallelism is extracted during analysis phase. Once analysis phase is done, the size of working space to perform QR is known. However `cusolverSpXcsrqrAnalysisBatched` uses CPU to analyze the structure of matrix A , and this may consume a lot of memory. If host memory is not sufficient to finish the analysis, `CUSOLVER_STATUS_ALLOC_FAILED` is returned. The required memory for analysis is proportional to zero fill-in in QR factorization. The user may need to perform some kind of reordering to minimize zero fill-in, for example, `colamd` or `symrcm` in MATLAB. `cuSolverSP` library provides `symrcm` (`cusolverSpXcsrSymrcm`).

Second, the user needs to choose proper `batchSize` and to prepare working space for sparse QR. There are two memory blocks used in batched sparse QR. One is internal memory block used to store matrix Q and matrix R . The other is working space used to perform numerical factorization. The size of the former is proportional to `batchSize`, and the size is specified by returned parameter `internalDataInBytes` of `cusolverSp[S|D|C|Z]csrqrBufferInfoBatched`. while the size of the latter is almost independent of `batchSize`, and the size is specified by returned parameter `workspaceInBytes` of `cusolverSp[S|D|C|Z]csrqrBufferInfoBatched`. The internal memory block is allocated implicitly during first call of `cusolverSp[S|D|C|Z]csrqrsvBatched`. The user only needs to allocate working space for `cusolverSp[S|D|C|Z]csrqrsvBatched`.

Instead of trying all batched matrices, the user can find maximum batchSize by querying `cusolverSp[S|D|C|Z]csrqrBufferInfoBatched`. For example, the user can increase batchSize till summation of `internalDataInBytes` and `workspaceInBytes` is greater than size of available device memory.

Suppose that the user needs to perform 253 linear solvers and available device memory is 2GB. if `cusolverSp[S|D|C|Z]csrqrsvBatched` can only afford batchSize 100, the user has to call `cusolverSp[S|D|C|Z]csrqrsvBatched` three times to finish all. The user calls `cusolverSp[S|D|C|Z]csrqrBufferInfoBatched` with batchSize 100. The opaque info would remember this batchSize and any subsequent call of `cusolverSp[S|D|C|Z]csrqrsvBatched` cannot exceed this value. In this example, the first two calls of `cusolverSp[S|D|C|Z]csrqrsvBatched` will use batchSize 100, and last call of `cusolverSp[S|D|C|Z]csrqrsvBatched` will use batchSize 53.

Example: suppose that $A_0, A_1, \dots, A_9$ have the same sparsity pattern, the following code solves 10 linear systems $A_j x_j = b_j$, $j = 0, 2, \dots, 9$ by batched sparse QR.

```
// Suppose that A0, A1, ..., A9 are m x m sparse matrix represented by CSR format,
// Each matrix Aj has nonzero nnzA, and shares the same csrRowPtrA and csrColIndA.
// csrValA is aggregation of A0, A1, ..., A9.
int m ; // number of rows and columns of each Aj
int nnzA ; // number of nonzeros of each Aj
int *csrRowPtrA ; // each Aj has the same csrRowPtrA
int *csrColIndA ; // each Aj has the same csrColIndA
double *csrValA ; // aggregation of A0, A1, ..., A9
const int batchSize = 10; // 10 linear systems

cusolverSpHandle_t handle; // handle to cusolver library
csrqrInfo_t info = NULL;
cusparsMatDescr_t descrA = NULL;
void *pBuffer = NULL; // working space for numerical factorization

// step 1: create a descriptor
cusparsCreateMatDescr(&descrA);
cusparsSetMatIndexBase(descrA, CUSPARSE_INDEX_BASE_ONE); // A is base-1
cusparsSetMatType(descrA, CUSPARSE_MATRIX_TYPE_GENERAL); // A is a general matrix

// step 2: create empty info structure
cusolverSpCreateCsrqrInfo(&info);

// step 3: symbolic analysis
cusolverSpXcsrqrAnalysisBatched(
    handle, m, m, nnzA,
    descrA, csrRowPtrA, csrColIndA, info);

// step 4: allocate working space for Aj*xj=bj
cusolverSpDcsrqrBufferInfoBatched(
    handle, m, m, nnzA,
    descrA,
    csrValA, csrRowPtrA, csrColIndA,
    batchSize,
    info,
    &internalDataInBytes,
    &workspaceInBytes);

cudaMalloc(&pBuffer, workspaceInBytes);
```

(continues on next page)

(continued from previous page)

```
// step 5: solve  $A_j \cdot x_j = b_j$ 
cusolverSpDcsrqrsvBatched(
    handle, m, m, nnzA,
    descrA, csrValA, csrRowPtrA, csrColIndA,
    b,
    x,
    batchSize,
    info,
    pBuffer);

// step 7: destroy info
cusolverSpDestroyCsrqrInfo(info);
```

Please refer to [cuSOLVER Library Samples - csrqr](#) for a code example.

Remark 1: only GPU (device) path is provided.

Input

Parameter	cusolverSp MemSpace	Description
handle	host	Handle to the cuSolverSP library context.
m	host	Number of rows of each matrix A_j .
n	host	Number of columns of each matrix A_j .
nnzA	host	Number of nonzeros of each matrix A_j . It is the size <code>csrColIndA</code> .
descrA	host	The descriptor of each matrix A_j . The supported matrix type is <code>CUSPARSE_MATRIX_TYPE_GENERAL</code> . Also, the supported index bases are <code>CUSPARSE_INDEX_BASE_ZERO</code> and <code>CUSPARSE_INDEX_BASE_ONE</code> .
csrValA	device	<type> array of <code>nnzA*batchSize</code> nonzero elements of matrices $A_0, A_1, \dots$. All matrices are aggregated one after another.
csrRowPtrA	device	Integer array of <code>m+1</code> elements that contains the start of every row and the end of the last row plus one.
csrColIndA	device	Integer array of <code>nnzA</code> column indices of the nonzero elements of each matrix A_j .
b	device	<type> array of <code>m*batchSize</code> of right-hand-side vectors $b_0, b_1, \dots$. All vectors are aggregated one after another.
batchSize	host	Number of systems to be solved.
info	host	Opaque structure for QR factorization.
pBuffer	device	Buffer allocated by the user, the size is returned by <code>cusolverSpXcsrqrBufferInfoBatched()</code> .

Output

Parameter	cusolverSp MemSpace	Description
x	device	<type> array of m*batchSize of solution vectors x0, x1, All vectors are aggregated one after another.
internal-DataInBytes	host	Number of bytes of the internal data.
workspaceIn-Bytes	host	Number of bytes of the buffer in numerical factorization.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (m, n, nnzA<=0), base index is not 0 or 1.

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

CUSOLVER_STATUS_MATRIX_TYPE_NOT_SUPPORTED

The matrix type is not supported.

8.6. cuSolverRF: Refactorization Reference [[DEPRECATED]]

[[DEPRECATED]] cuSolverRF is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

This section describes API of cuSolverRF, a library for fast refactorization.

8.6.1. cusolverRfAccessBundledFactorsDevice() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfAccessBundledFactorsDevice(/* Input */
                                     cusolverRfHandle_t handle,
                                     /* Output (in the host memory) */
                                     int* nnzM,
                                     /* Output (in the device memory) */
                                     int** Mp,
                                     int** Mi,
                                     double** Mx);
```

This routine allows direct access to the lower L and upper U triangular factors stored in the cuSolverRF library handle. The factors are compressed into a single matrix $M = (L - I) + U$, where the unitary diagonal of L is not stored. It is assumed that a prior call to the `cusolverRfRefactor()` was done in order to generate these triangular factors.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
nnzM	host	output	The number of non-zero elements of matrix M.
Mp	device	output	The array of offsets corresponding to the start of each row in the arrays Mi and Mx. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix M. The array size is n+1.
Mi	device	output	The array of column indices corresponding to the non-zero elements in the matrix M. It is assumed that this array is sorted by row and by column within each row. The array size is nnzM.
Mx	device	output	The array of values corresponding to the non-zero elements in the matrix M. It is assumed that this array is sorted by row and by column within each row. The array size is nnzM.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.2. cusolverRfAnalyze() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfAnalyze(cusolverRfHandle_t handle);
```

This routine performs the appropriate analysis of parallelism available in the LU re-factorization depending upon the algorithm chosen by the user.

$$A = L * U$$

It is assumed that a prior call to the `cusolverRfSetup[Host|Device]()` was done in order to create internal data structures needed for the analysis.

This routine needs to be called only once for a single linear system

$$A_i x_i = f_i$$

Parameter	MemSpace	In/out	Meaning
handle	host	in/out	The handle to the cuSolverRF library.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

`cusolverRfSetupDevice()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSetupDevice(/* Input (in the device memory) */
    int n,
    int nnzA,
    int* csrRowPtrA,
    int* csrColIndA,
    double* csrValA,
    int nnzL,
    int* csrRowPtrL,
```

(continues on next page)

(continued from previous page)

```

int* csrColIndL,
double* csrValL,
int nnzU,
int* csrRowPtrU,
int* csrColIndU,
double* csrValU,
int* P,
int* Q,
/* Output */
cusolverRfHandle_t handle);

```

This routine assembles the internal data structures of the cuSolverRF library. It is often the first routine to be called after the call to the `cusolverRfCreate()` routine.

This routine accepts as input (on the device) the original matrix A, the lower (L) and upper (U) triangular factors, as well as the left (P) and the right (Q) permutations resulting from the full LU factorization of the first ($i=1$) linear system

$$A_i x_i = f_i$$

The permutations P and Q represent the final composition of all the left and right reorderings applied to the original matrix A, respectively. However, these permutations are often associated with partial pivoting and reordering to minimize fill-in, respectively.

This routine needs to be called only once for a single linear system

$$A_i x_i = f_i$$

Parameter	MemSpace	In/out	Meaning
n	host	input	The number of rows (and columns) of matrix A.
nnzA	host	input	The number of non-zero elements of matrix A.
csrRowPtrA	device	input	The array of offsets corresponding to the start of each row in the arrays <code>csrColIndA</code> and <code>csrValA</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix. The array size is $n+1$.
csrColIndA	device	input	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzA</code> .
csrValA	device	input	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzA</code> .
nnzL	host	input	The number of non-zero elements of matrix L.

continues on next page

Table 68 – continued from previous page

Parameter	MemSpace	In/out	Meaning
csrRowPtrL	device	input	The array of offsets corresponding to the start of each row in the arrays <code>csrColIndL</code> and <code>csrValL</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix L. The array size is $n+1$.
csrColIndL	device	input	The array of column indices corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzL</code> .
csrValL	device	input	The array of values corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzL</code> .
nnzU	host	input	The number of non-zero elements of matrix U.
csrRowPtrU	device	input	The array of offsets corresponding to the start of each row in the arrays <code>csrColIndU</code> and <code>csrValU</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix U. The array size is $n+1$.
csrColIndU	device	input	The array of column indices corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzU</code> .
csrValU	device	input	The array of values corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzU</code> .
P	device	input	The left permutation (often associated with pivoting). The array size is n .
Q	device	input	The right permutation (often associated with re-ordering). The array size is n .
handle	host	output	The handle to the <code>cuSolverRF</code> library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.3. cusolverRfSetupHost() **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSetupHost(/* Input (in the host memory) */
    int n,
    int nnzA,
    int* h_csrRowPtrA,
    int* h_csrColIndA,
    double* h_csrValA,
    int nnzL,
    int* h_csrRowPtrL,
    int* h_csrColIndL,
    double* h_csrValL,
    int nnzU,
    int* h_csrRowPtrU,
    int* h_csrColIndU,
    double* h_csrValU,
    int* h_P,
    int* h_Q,
    /* Output */
    cusolverRfHandle_t handle);
```

This routine assembles the internal data structures of the cuSolverRF library. It is often the first routine to be called after the call to the `cusolverRfCreate()` routine.

This routine accepts as input (on the host) the original matrix A, the lower (L) and upper (U) triangular factors, as well as the left (P) and the right (Q) permutations resulting from the full LU factorization of the first ($i=1$) linear system

$$A_i x_i = f_i$$

The permutations P and Q represent the final composition of all the left and right reorderings applied to the original matrix A, respectively. However, these permutations are often associated with partial pivoting and reordering to minimize fill-in, respectively.

This routine needs to be called only once for a single linear system

$$A_i x_i = f_i$$

Parameter	MemSpace	In/out	Meaning
n	host	input	The number of rows (and columns) of matrix A.

continues on next page

Table 69 – continued from previous page

Parameter	MemSpace	In/out	Meaning
nnzA	host	input	The number of non-zero elements of matrix A.
h_csrRowPtrA	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndA and h_csrValA. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix. The array size is n+1.
h_csrColIndA	host	input	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is nnzA.
h_csrValA	host	input	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is nnzA.
nnzL	host	input	The number of non-zero elements of matrix L.
h_csrRowPtrL	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndL and h_csrValL. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix L. The array size is n+1.
h_csrColIndL	host	input	The array of column indices corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is nnzL.
h_csrValL	host	input	The array of values corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is nnzL.
nnzU	host	input	The number of non-zero elements of matrix U.
h_csrRowPtrU	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndU and h_csrValU. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix U. The array size is n+1.
h_csrColIndU	host	input	The array of column indices corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is nnzU.
h_csrValU	host	input	The array of values corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is nnzU.

continues on next page

Table 69 – continued from previous page

Parameter	MemSpace	In/out	Meaning
h_P	host	input	The left permutation (often associated with pivoting). The array size in n.
h_Q	host	input	The right permutation (often associated with re-ordering). The array size in n.
handle	host	output	The handle to the cuSolverRF library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.4. `cusolverRfCreate()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t cusolverRfCreate(cusolverRfHandle_t *handle);
```

This routine initializes the cuSolverRF library. It allocates required resources and must be called prior to any other cuSolverRF library routine.

Parameter	MemSpace	In/out	Meaning
handle	host	output	The pointer to the cuSolverRF library handle.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.5. cusolverRfExtractBundledFactorsHost() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfExtractBundledFactorsHost(/* Input */
    cusolverRfHandle_t handle,
    /* Output (in the host memory) */
    int* h_nnzM,
    int** h_Mp,
    int** h_Mi,
    double** h_Mx);
```

This routine extracts lower (L) and upper (U) triangular factors from the cuSolverRF library handle into the host memory. The factors are compressed into a single matrix $M = (L - I) + U$, where the unitary diagonal of (L) is not stored. It is assumed that a prior call to the `cusolverRfRefactor()` was done in order to generate these triangular factors.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
h_nnzM	host	output	The number of non-zero elements of matrix M.
h_Mp	host	output	The array of offsets corresponding to the start of each row in the arrays h_Mi and h_Mx. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix M. The array size is n+1.
h_Mi	host	output	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is h_nnzM.
h_Mx	host	output	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is h_nnzM.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.6. cusolverRfExtractSplitFactorsHost() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfExtractSplitFactorsHost(/* Input */
                                   cusolverRfHandle_t handle,
                                   /* Output (in the host memory) */
                                   int* h_nnzL,
                                   int** h_Lp,
                                   int** h_Li,
                                   double** h_Lx,
                                   int* h_nnzU,
                                   int** h_Up,
                                   int** h_Ui,
                                   double** h_Ux);
```

This routine extracts lower (L) and upper (U) triangular factors from the cuSolverRF library handle into the host memory. It is assumed that a prior call to the `cusolverRfRefactor()` was done in order to generate these triangular factors.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
h_nnzL	host	output	The number of non-zero elements of matrix L.
h_Lp	host	output	The array of offsets corresponding to the start of each row in the arrays <code>h_Li</code> and <code>h_Lx</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix L. The array size is <code>n+1</code> .
h_Li	host	output	The array of column indices corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is <code>h_nnzL</code> .
h_Lx	host	output	The array of values corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is <code>h_nnzL</code> .
h_nnzU	host	output	The number of non-zero elements of matrix U.

continues on next page

Table 70 – continued from previous page

Parameter	MemSpace	In/out	Meaning
h_Up	host	output	The array of offsets corresponding to the start of each row in the arrays h_Ui and h_Ux. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix U. The array size is n+1.
h_Ui	host	output	The array of column indices corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is h_nnzU.
h_Ux	host	output	The array of values corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is h_nnzU.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.7. `cusolverRfDestroy()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t cusolverRfDestroy(cusolverRfHandle_t handle);
```

This routine shuts down the cuSolverRF library. It releases acquired resources and must be called after all the cuSolverRF library routines.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The cuSolverRF library handle.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.8. cusolverRfGetMatrixFormat() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfGetMatrixFormat(cusolverRfHandle_t handle,
                          cusolverRfMatrixFormat_t *format,
                          cusolverRfUnitDiagonal_t *diag);
```

This routine gets the matrix format used in the `cusolverRfSetupDevice()`, `cusolverRfSetupHost()`, `cusolverRfResetValues()`, `cusolverRfExtractBundledFactorsHost()` and `cusolverRfExtractSplitFactorsHost()` routines.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
format	host	output	The enumerated matrix format type.
diag	host	output	The enumerated unit diagonal type.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.9. cusolverRfGetNumericProperties() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfGetNumericProperties(cusolverRfHandle_t handle,
                              double *zero,
                              double *boost);
```

This routine gets the numeric values used for checking for “zero” pivot and for boosting it in the `cusolverRfRefactor()` and `cusolverRfSolve()` routines. The numeric boosting will be used only if `boost > 0.0`.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
zero	host	output	The value below which zero pivot is flagged.
boost	host	output	The value which is substituted for zero pivot (if the later is flagged).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.10. `cusolverRfGetNumericBoostReport()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfGetNumericBoostReport(cusolverRfHandle_t handle,
                                cusolverRfNumericBoostReport_t *report);
```

This routine gets the report whether numeric boosting was used in the `cusolverRfRefactor()` and `cusolverRfSolve()` routines.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
report	host	output	The enumerated boosting report type.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.11. `cusolverRfGetResetValuesFastMode()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfGetResetValuesFastMode(cusolverRfHandle_t handle,
                                  cusolverRfResetValuesFastMode_t *fastMode);
```

This routine gets the mode used in the `cusolverRfResetValues` routine.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
fastMode	host	output	The enumerated mode type.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.12. `cusolverRfGetAlgs()`

```
cusolverStatus_t
cusolverRfGetAlgs(cusolverRfHandle_t handle,
                  cusolverRfFactorization_t* fact_alg,
                  cusolverRfTriangularSolve_t* solve_alg);
```

This routine gets the algorithm used for the refactorization in `cusolverRfRefactor()` and the triangular solve in `cusolverRfSolve()`.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
alg	host	output	The enumerated algorithm type.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.13. cusolverRfRefactor() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t cusolverRfRefactor(cusolverRfHandle_t handle);
```

This routine performs the LU re-factorization:

$$A = L * U$$

exploring the available parallelism on the GPU. It is assumed that a prior call to the `cusolverRfAnalyze()` was done in order to find the available parallelism.

This routine may be called multiple times, once for each of the linear systems:

$$A_i x_i = f_i$$

There are some constraints to the combination of algorithms used for refactorization and solving routines, `cusolverRfRefactor()` and `cusolverRfSolve()`. The wrong combination generates the error code `CUSOLVER_STATUS_INVALID_VALUE`. The table below summarizes the supported combinations of algorithms:

Compatible algorithms for solving and refactorization routines.

Factorization	Solving
CUSOLVERRF_FACTORIZATION_ALG0	TRIANGULAR_SOLVE_ALG1
CUSOLVERRF_FACTORIZATION_ALG1	TRIANGULAR_SOLVE_ALG2, TRIANGULAR_SOLVE_ALG3
CUSOLVERRF_FACTORIZATION_ALG2	TRIANGULAR_SOLVE_ALG2, TRIANGULAR_SOLVE_ALG3

Parameter	MemSpace	In/out	Meaning
handle	host	in/out	The handle to the cuSolverRF library.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_ZERO_PIVOT

A zero pivot was encountered during the computation.

8.6.14. cusolverRfResetValues() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfResetValues(/* Input (in the device memory) */
    int n,
    int nnzA,
    int* csrRowPtrA,
    int* csrColIndA,
    double* csrValA,
    int* P,
    int* Q,
    /* Output */
    cusolverRfHandle_t handle);
```

This routine updates internal data structures with the values of the new coefficient matrix. It is assumed that the arrays `csrRowPtrA`, `csrColIndA`, `P` and `Q` have not changed since the last call to the `cusolverRfSetup[Host|Device]` routine. This assumption reflects the fact that the sparsity pattern of coefficient matrices as well as reordering to minimize fill-in and pivoting remain the same in the set of linear systems:

$$A_i x_i = f_i$$

This routine may be called multiple times, once for each of the linear systems:

$$A_i x_i = f_i$$

Parameter	MemSpace	In/out	Meaning
n	host	input	The number of rows (and columns) of matrix A.
nnzA	host	input	The number of non-zero elements of matrix A.
csrRowPtrA	device	input	The array of offsets corresponding to the start of each row in the arrays <code>csrColIndA</code> and <code>csrValA</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix. The array size is <code>n+1</code> .
csrColIndA	device	input	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzA</code> .
csrValA	device	input	The array of values corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzA</code> .
P	device	input	The left permutation (often associated with pivoting). The array size is <code>n</code> .

continues on next page

Table 71 – continued from previous page

Parameter	MemSpace	In/out	Meaning
Q	device	input	The right permutation (often associated with re-ordering). The array size in n.
handle	host	output	The handle to the cuSolverRF library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.15. `cusolverRfSetMatrixFormat()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSetMatrixFormat(cusolverRfHandle_t handle,
                          cusolverRfMatrixFormat_t format,
                          cusolverRfUnitDiagonal_t diag);
```

This routine sets the matrix format used in the `cusolverRfSetupDevice()`, `cusolverRfSetupHost()`, `cusolverRfResetValues()`, `cusolverRfExtractBundledFactorsHost()` and `cusolverRfExtractSplitFactorsHost()` routines. It may be called once prior to `cusolverRfSetupDevice()` and `cusolverRfSetupHost()` routines.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
format	host	input	The enumerated matrix format type.
diag	host	input	The enumerated unit diagonal type.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An enumerated mode parameter is wrong.

8.6.16. `cusolverRfSetNumericProperties()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSetNumericProperties(cusolverRfHandle_t handle,
                             double zero,
                             double boost);
```

This routine sets the numeric values used for checking for “zero” pivot and for boosting it in the `cusolverRfRefactor()` and `cusolverRfSolve()` routines. It may be called multiple times prior to `cusolverRfRefactor()` and `cusolverRfSolve()` routines. The numeric boosting will be used only if `boost > 0.0`.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
zero	host	input	The value below which zero pivot is flagged.
boost	host	input	The value which is substituted for zero pivot (if the later is flagged).

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.17. `cusolverRfSetResetValuesFastMode()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSetResetValuesFastMode(cusolverRfHandle_t handle,
                                 cusolverRfResetValuesFastMode_t fastMode);
```

This routine sets the mode used in the `cusolverRfResetValues` routine. The fast mode requires extra memory and is recommended only if very fast calls to `cusolverRfResetValues()` are needed. It may be called once prior to `cusolverRfAnalyze()` routine.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
fastMode	host	input	The enumerated mode type.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An enumerated mode parameter is wrong.

8.6.18. `cusolverRfSetAlgs()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t  
cusolverRfSetAlgs(cusolverRfHandle_t handle,  
                  cusolverRfFactorization_t fact_alg,  
                  cusolverRfTriangularSolve_t alg);
```

This routine sets the algorithm used for the refactorization in `cusolverRfRefactor()` and the triangular solve in `cusolverRfSolve()`. It may be called once prior to `cusolverRfAnalyze()` routine.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
alg	host	input	The enumerated algorithm type.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

8.6.19. cusolverRfSolve() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfSolve(/* Input (in the device memory) */
               cusolverRfHandle_t handle,
               int *P,
               int *Q,
               int nrhs,
               double *Temp,
               int ldt,
               /* Input/Output (in the device memory) */
               double *XF,
               /* Input */
               int ldxf);
```

This routine performs the forward and backward solve with the lower $L \in R^{n \times n}$ and upper $U \in R^{n \times n}$ triangular factors resulting from the LU re-factorization:

$$A = L * U$$

which is assumed to have been computed by a prior call to the `cusolverRfRefactor()` routine.

The routine can solve linear systems with multiple right-hand-sides (RHS):

$$AX = (LU)X = L(UX) = LY = F \text{ where } UX = Y$$

even though currently only a single RHS is supported.

This routine may be called multiple times, once for each of the linear systems:

$$A_i x_i = f_i$$

Parameter	MemSpace	in/out	Meaning
handle	host	output	The handle to the cuSolverRF library.
P	device	input	The left permutation (often associated with pivoting). The array size in n.
Q	device	input	The right permutation (often associated with reordering). The array size in n.
nrhs	host	input	The number right-hand-sides to be solved.
Temp	device	input	The dense matrix that contains temporary workspace (of size $ldt * nrhs$).
ldt	host	input	The leading dimension of dense matrix Temp ($ldt \geq n$).
XF	device	in/output	The dense matrix that contains the right-hand-sides F and solutions X (of size $ldxf * nrhs$).
ldxf	host	input	The leading dimension of dense matrix XF ($ldxf \geq n$).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.20. `cusolverRfBatchSetupHost()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfBatchSetupHost(/* Input (in the host memory) */
    int batchSize,
    int n,
    int nnzA,
    int* h_csrRowPtrA,
    int* h_csrColIndA,
```

(continues on next page)

(continued from previous page)

```

double *h_csrValA_array[],
int nnzL,
int* h_csrRowPtrL,
int* h_csrColIndL,
double *h_csrValL,
int nnzU,
int* h_csrRowPtrU,
int* h_csrColIndU,
double *h_csrValU,
int* h_P,
int* h_Q,
/* Output */
cusolverRfHandle_t handle);

```

This routine assembles the internal data structures of the cuSolverRF library for batched operation. It is called after the call to the `cusolverRfCreate()` routine, and before any other batched routines.

The batched operation assumes that the user has the following linear systems:

$$A_j x_j = b_j, j = 1, 2, \dots, batchSize$$

where each matrix in the set: $\{A_j\}$ has the same sparsity pattern, and quite similar such that factorization can be done by the same permutation P and Q. In other words, $A_j, j > 1$ is a small perturbation of A_1 .

This routine accepts as input (on the host) the original matrix A (sparsity pattern and batched values), the lower (L) and upper (U) triangular factors, as well as the left (P) and the right (Q) permutations resulting from the full LU factorization of the first ($i=1$) linear system:

$$A_i x_i = f_i$$

The permutations P and Q represent the final composition of all the left and right reorderings applied to the original matrix A, respectively. However, these permutations are often associated with partial pivoting and reordering to minimize fill-in, respectively.

Remark 1: the matrices A, L and U must be CSR format and base-0.

Remark 2: to get best performance, `batchSize` should be multiple of 32 and greater or equal to 32. The algorithm is memory-bound, once bandwidth limit is reached, there is no room to improve performance by large `batchSize`. In practice, `batchSize` of 32 - 128 is often enough to obtain good performance, but in some cases larger `batchSize` might be beneficial.

The following routine needs to be called only once for a single linear system:

$$A_i x_i = f_i$$

Parameter	MemSpace	In/out	Meaning
batchSize	host	input	The number of matrices in the batched mode.
n	host	input	The number of rows (and columns) of matrix A.
nnzA	host	input	The number of non-zero elements of matrix A.

continues on next page

Table 72 – continued from previous page

Parameter	MemSpace	In/out	Meaning
h_csrRowPtrA	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndA and h_csrValA. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix. The array size is n+1.
h_csrColIndA	host	input	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is nnzA.
h_csrValA_array	host	input	Array of pointers of size batchSize, each pointer points to the array of values corresponding to the non-zero elements in the matrix.
nnzL	host	input	The number of non-zero elements of matrix L.
h_csrRowPtrL	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndL and h_csrValL. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix L. The array size is n+1.
h_csrColIndL	host	input	The array of column indices corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is nnzL.
h_csrValL	host	input	The array of values corresponding to the non-zero elements in the matrix L. It is assumed that this array is sorted by row and by column within each row. The array size is nnzL.
nnzU	host	input	The number of non-zero elements of matrix U.
h_csrRowPtrU	host	input	The array of offsets corresponding to the start of each row in the arrays h_csrColIndU and h_csrValU. This array has also an extra entry at the end that stores the number of non-zero elements in the matrix U. The array size is n+1.
h_csrColIndU	host	input	The array of column indices corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is nnzU.
h_csrValU	host	input	The array of values corresponding to the non-zero elements in the matrix U. It is assumed that this array is sorted by row and by column within each row. The array size is nnzU.
h_P	host	input	The left permutation (often associated with pivoting). The array size is n.

continues on next page

Table 72 – continued from previous page

Parameter	MemSpace	In/out	Meaning
h_Q	host	input	The right permutation (often associated with re-ordering). The array size in n.
handle	host	output	The handle to the cuSolverRF library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.21. `cusolverRfBatchAnalyze()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t cusolverRfBatchAnalyze(cusolverRfHandle_t handle);
```

This routine performs the appropriate analysis of parallelism available in the batched LU re-factorization.

It is assumed that a prior call to the `cusolverRfBatchSetup[Host]()` was done in order to create internal data structures needed for the analysis.

The following routine needs to be called only once for a single linear system:

$$A_j x_j = b_j, j = 1, 2, \dots, batchSize$$

Parameter	Memory	In/out	Meaning
handle	host	in/out	The handle to the cuSolverRF library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_ALLOC_FAILED

An allocation of memory failed.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.22. `cusolverRfBatchResetValues()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfBatchResetValues(/* Input (in the device memory) */
    int batchSize,
    int n,
    int nnzA,
    int* csrRowPtrA,
    int* csrColIndA,
    double* csrValA_array[],
    int *P,
    int *Q,
    /* Output */
    cusolverRfHandle_t handle);
```

This routine updates internal data structures with the values of the new coefficient matrix. It is assumed that the arrays `csrRowPtrA`, `csrColIndA`, `P` and `Q` have not changed since the last call to the `cusolverRfbatch_setup_host` routine.

This assumption reflects the fact that the sparsity pattern of coefficient matrices as well as reordering to minimize fill-in and pivoting remain the same in the set of linear systems:

$$A_j x_j = b_j, j = 1, 2, \dots, batchSize$$

The input parameter `csrValA_array` is an array of pointers on device memory. `csrValA_array(j)` points to matrix: A_j which is also on device memory.

Parameter	MemSpace	In/out	Meaning
batchSize	host	input	The number of matrices in batched mode.
n	host	input	The number of rows (and columns) of matrix A.
nnzA	host	input	The number of non-zero elements of matrix A.
csrRowPtrA	device	input	The array of offsets corresponding to the start of each row in the arrays <code>csrColIndA</code> and <code>csrValA</code> . This array has also an extra entry at the end that stores the number of non-zero elements in the matrix. The array size is <code>n+1</code> .
csrColIndA	device	input	The array of column indices corresponding to the non-zero elements in the matrix. It is assumed that this array is sorted by row and by column within each row. The array size is <code>nnzA</code> .
csr-ValA_array	device	input	Array of pointers of size <code>batchSize</code> , each pointer points to the array of values corresponding to the non-zero elements in the matrix.
P	device	input	The left permutation (often associated with pivoting). The array size is <code>n</code> .
Q	device	input	The right permutation (often associated with re-ordering). The array size is <code>n</code> .
handle	host	output	The handle to the <code>cuSolverRF</code> library.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.23. `cusolverRfBatchRefactor()` **[[DEPRECATED]]**

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t cusolverRfBatchRefactor(cusolverRfHandle_t handle);
```

This routine performs the LU re-factorization:

$$M_j = P * A_j * Q^T = L_j * U_j$$

exploring the available parallelism on the GPU. It is assumed that a prior call to the `cusolverRfBatch-Analyze()` was done in order to find the available parallelism.

Remark: `cusolverRfBatchRefactor()` would not report any failure of LU refactorization. The user has to call `cusolverRfBatchZeroPivot()` to know which matrix failed the LU refactorization.

Parameter	Memory	In/out	Meaning
handle	host	in/out	The handle to the cuSolverRF library.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

8.6.24. `cusolverRfBatchSolve()` [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfBatchSolve(/* Input (in the device memory) */
    cusolverRfHandle_t handle,
    int *P,
    int *Q,
    int nrhs,
    double *Temp,
    int ldt,
    /* Input/Output (in the device memory) */
    double *XF_array[],
    /* Input */
    int ldxf);
```

To solve $A_j * x_j = b_j$, first we reform the equation by $M_j * Q * x_j = P * b_j$ where $M_j = P * A_j * Q^T$. Then do refactorization $M_j = L_j * U_j$ by `cusolverRfBatchRefactor()`. Further `cusolverRfBatchSolve()` takes over the remaining steps, including:

$$z_j = P * b_j$$

$$M_j * y_j = z_j$$

$$x_j = Q^T * y_j$$

The input parameter `XF_array` is an array of pointers on device memory. `XF_array(j)` points to matrix x_j which is also on device memory.

Remark 1: only a single rhs is supported.

Remark 2: no singularity is reported during backward solve. If some matrix A_j failed the refactorization and U_j has some zero diagonal, backward solve would compute NAN. The user has to call `cusolverRfBatchZeroPivot` to check if refactorization is successful or not.

Parameter	MemSpace	In/out	Meaning
handle	host	output	The handle to the cuSolverRF library.
P	device	input	The left permutation (often associated with pivoting). The array size in n.
Q	device	input	The right permutation (often associated with re-ordering). The array size in n.
nrhs	host	input	The number right-hand-sides to be solved.
Temp	device	input	The dense matrix that contains temporary workspace (of size ldt*nrhs).
ldt	host	input	The leading dimension of dense matrix Temp (ldt >= n).
XF_array	device	in/out	Array of pointers of size batchSize, each pointer points to the dense matrix that contains the right-hand-sides F and solutions X (of size ldxf*nrhs).
ldxf	host	input	The leading dimension of dense matrix XF (ldxf >= n).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

An unsupported value or parameter was passed.

CUSOLVER_STATUS_EXECUTION_FAILED

A kernel failed to launch on the GPU.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

8.6.25. cusolverRfBatchZeroPivot() [[DEPRECATED]]

[[DEPRECATED]] This function is deprecated and will be removed in a future major version. Please use the [cuDSS](#) library instead for better performance and support. For the transition, please visit [cuDSS Library Samples](#) for a code example.

```
cusolverStatus_t
cusolverRfBatchZeroPivot(/* Input */
    cusolverRfHandle_t handle
    /* Output (in the host memory) */
    int *position);
```

Although A_j is close to each other, it does not mean $M_j = P * A_j * Q^T = L_j * U_j$ exists for every j. The user can query which matrix failed LU refactorization by checking corresponding value in position

array. The input parameter `position` is an integer array of size `batchSize`.

The j -th component denotes the refactorization result of matrix A_j . If `position(j)` is -1, the LU refactorization of matrix A_j is successful. If `position(j)` is $k \geq 0$, matrix A_j is not LU factorizable and its matrix $U_j(j, j)$ is zero.

The return value of `cusolverRfBatch_Zero_Pivot` is `CUSOLVER_STATUS_ZERO_PIVOT` if there exists one A_j which failed LU refactorization. The user can redo LU factorization to get new permutation P and Q if error code `CUSOLVER_STATUS_ZERO_PIVOT` is returned.

Parameter	MemSpace	In/out	Meaning
handle	host	input	The handle to the cuSolverRF library.
position	host	output	Integer array of size <code>batchSize</code> . The value of <code>position(j)</code> reports singularity of matrix A_j , -1 if no structural/numerical zero, $k \geq 0$ if $A_j(k, k)$ is either structural zero or numerical zero.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_ZERO_PIVOT

A zero pivot was encountered during the computation.

Chapter 9. Using the CUSOLVERMG API

Note: cuSolverMG is deprecated. Users are encouraged to use **cuSOLVERMp** for multi-GPU functionality, both single- and multi-node.

9.1. General Description

This section describes how to use the cuSolverMG library API. It is not a reference for the cuSolverMG API data types and functions; that is provided in subsequent chapters.

9.1.1. Thread Safety

The library is thread-safe only if there is one cuSolverMG context per thread.

9.1.2. Determinism

Currently all cuSolverMG API routines from a given toolkit version generate the same bit-wise results when the following conditions are respected:

- ▶ all GPUs participating to the computation have the same compute-capabilities and the same number of SMs.
- ▶ the tiles size is kept the same between run.
- ▶ number of logical GPUs is kept the same. The order of GPUs are not important because all have the same compute-capabilities.

9.1.3. Tile Strategy

The tiling strategy of cuSolverMG is compatible with ScaLAPACK. The current release only supports 1-D column block cyclic, column-major PACKED format.

Figure 1.a shows a partition of the matrix A of dimension M_A by N_A . Each column tile has T_A columns. There are seven columns of tiles, labeled as 0,1,2,3,4,5,6, distributed into three GPUs in a *cyclic* way, i.e. each GPU takes one column tile in turn. For example, GPU 0 has column tile 0, 3, 6 (yellow tiles) and GPU 1 takes column tiles next to GPU 0 (blue tiles). Not all GPUs have the same number of tiles; in this example, GPU 0 has three tiles, others have only two tiles.

Figure 1.b shows two possible formats to store those column tiles locally in each GPU. Left side is called PACKED format and right side is UNPACKED format. PACKED format aggregates three column tiles in a contiguous memory block while UNPACKED format distributes these three column tiles into different memory blocks. The only difference between them is that PACKED format can have a big GEMM call instead of three GEMM calls in UNPACKED format. So theoretically speaking, PACKED format can deliver better performance than UNPACKED format. cuSolverMG only supports PACKED format in the API. In order to achieve maximal performance, the user just needs to choose the proper tile size T_A to partition the matrix, not too small, for example 256 or above is enough.

There is another parameter, called LLD_A , to control the leading dimension of the local matrix in each GPU. LLD_A must be greater or equal to M_A . The purpose of LLD_A is for better performance of GEMM. For small problems, GEMM is faster if LLD_A is power of 2. However for big problems, LLD_A does not show significant improvement. cuSolverMG only supports $LLD_A=M_A$.

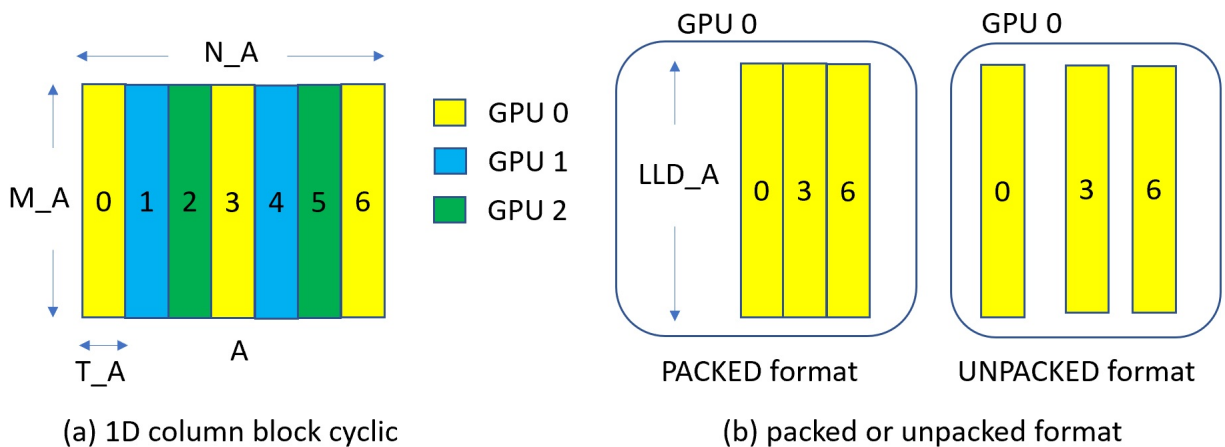


Fig. 1: Example of cuSolverMG tiling for 3 GPUs

The processing grid in cuSolverMG is a list of GPU IDs, similar to the process ID in ScaLAPACK. cuSolverMG only supports 1D column block cyclic, so only 1D grid is supported as well. Suppose `deviceId` is a list of GPU IDs, both `deviceId=1, 1, 1` and `deviceId=2, 1, 0` are valid. The former describes three logical devices that are selected to run cuSolverMG routines, and all have the same physical ID, 0. The latter still uses three logical devices, but each has a different physical ID. The current design only accepts 32 logical devices, that is, the length of `deviceId` is less or equal to 32. Figure 1 uses `deviceId=0, 1, 2`.

In practice, the matrix A is distributed into GPUs listed in `deviceId`. If the user chooses `deviceId=1, 1, 1`, all columns tile are located in GPU 1, this will limit the size of the problem because of memory capacity of one GPU. Besides, multiGPU routine adds extra overhead on data communication through the off-chip bus, which has a big performance impact if NVLINK is not supported or used. It would be

faster to run on a single GPU instead of running multiGPU version with devices of the same GPU ID.

9.1.4. Global Matrix Versus Local Matrix

Operating a submatrix of the matrix A is simple in dense linear algebra, just shift the pointer to the starting point of the submatrix relative to A. For example, `gesvd(10, 10, A)` is SVD of $A(0:9, 0:9)$. `gesvd(10, 10, A + 5 + 2*lda)` is SVD of 10-by-10 submatrix starting at $A(5, 2)$.

However it is not simple to operate on a submatrix of a distributed matrix because different starting point of the submatrix changes the distribution of the layout of that submatrix. ScaLAPACK introduces two parameters, IA and JA, to locate the submatrix. Figure 2 shows (global) matrix A of dimension M_A by N_A . The `sub(A)` is a M by N submatrix of A, starting at IA and JA. Please be aware that IA and JA are base-1.

Given a distributed matrix A, the user can compute eigenvalues of the submatrix `sub(A)` by either calling `syevd(A, IA, JA)` or gathering `sub(A)` to another distributed matrix B and calling `syevd(B, IB=1, JB=1)`.

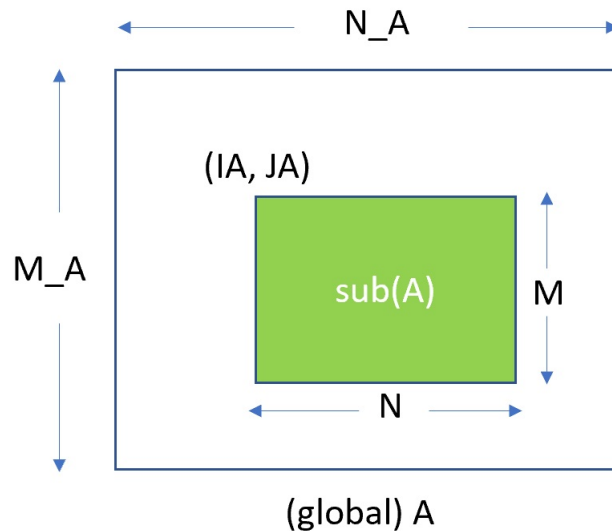


Fig. 2: global matrix and local matrix

9.1.5. Usage of `_bufferSize`

There is no `cudaMalloc` inside `cuSolverMG` library, so the user must allocate the device workspace explicitly. The routine `xyz_bufferSize` is to query the size of workspace of the routine `xyz`, for example `xyz = syevd`. To make the API simple, `xyz_bufferSize` follows almost the same signature of `xyz` even it only depends on some parameters, for example, the device pointer is not used to decide the size of workspace. In most cases, `xyz_bufferSize` is called in the beginning before actual device data (pointing by a device pointer) is prepared or before the device pointer is allocated. In such cases, the user can pass a null pointer to `xyz_bufferSize` without breaking the functionality.

`xyz_bufferSize` returns `bufferSize` for each device. The size is number of elements, not number of bytes.

9.1.6. Synchronization

All routines are in synchronous (blocking call) manner. The data is ready after the routine. However the user has to prepare the distributed data before calling the routine. For example, if the user has multiple streams to set up the matrix, stream synchronization or device synchronization is necessary to guarantee the distributed matrix is ready.

9.1.7. Context Switch

The user does not need to restore the device by `cudaSetDevice()` after each `cuSolverMG` call. All routines set the device back to what the caller has.

9.1.8. NVLINK

The peer-to-peer communication via NVLINK can dramatically reduce the overhead of data exchange among GPUs. `cuSolverMG` does not enable NVLINK implicitly, instead, it gives this option back to the user, not to interfere with other libraries. The example code H.1 shows how to enable peer-to-peer communication.

9.2. cuSolverMG Types Reference

9.2.1. cuSolverMG Types

The `float`, `double`, `cuComplex`, and `cuDoubleComplex` data types are supported. The first two are standard C data types, while the last two are exported from `cuComplex.h`. In addition, `cuSolverMG` uses some familiar types from `cuBLAS`.

9.2.2. `cusolverMgHandle_t`

This is a pointer type to an opaque `cuSolverMG` context, which the user must initialize by calling `cusolverMgCreate()` prior to calling any other library function. An un-initialized handle object will lead to unexpected behavior, including crashes of `cuSolverMG`. The handle created and returned by `cusolverMgCreate()` must be passed to every `cuSolverMG` function.

9.2.3. cusolverMgGridMapping_t

The type indicates layout of grids.

Value	Meaning
CUDALIBMG_GRID_MAPPING_ROW_MAJOR	Row-major ordering.
CUDALIBMG_GRID_MAPPING_COL_MAJOR	Column-major ordering.

9.2.4. cudaLibMgGrid_t

Opaque structure of the distributed grid.

9.2.5. cudaLibMgMatrixDesc_t

Opaque structure of the distributed matrix descriptor.

9.3. Helper Function Reference

9.3.1. cusolverMgCreate() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t
cusolverMgCreate(cusolverMgHandle_t *handle)
```

This function initializes the cuSolverMG library and creates a handle on the cuSolverMG context. It must be called before any other cuSolverMG API function is invoked. It allocates hardware resources necessary for accessing the GPU.

Output

handle	The pointer to the handle to the cuSolverMG context.
--------	--

Status Returned

CUSOLVER_STATUS_SUCCESS

The initialization succeeded.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

9.3.2. cusolverMgDestroy() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t  
cusolverMgDestroy( cusolverMgHandle_t handle)
```

This function releases CPU-side resources used by the cuSolverMG library.

Input

handle	The handle to the cuSolverMG context.
--------	---------------------------------------

Status Returned

CUSOLVER_STATUS_SUCCESS
The shutdown succeeded.

9.3.3. cusolverMgDeviceSelect() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t  
cusolverMgDeviceSelect(  
    cusolverMgHandle_t handle,  
    int nbDevices,  
    int deviceId[] )
```

This function registers a subset of devices (GPUs) to cuSolverMG handle. Such subset of devices is used in subsequent API calls. The array `deviceId` contains a list of logical device ID. The term **logical** means repeated device ID are permitted. For example, suppose the user has only one GPU in the system, say device 0. If the user sets `deviceId=0, 0, 0`, then cuSolverMG treats them as three independent GPUs, one stream each, so concurrent kernel launches still hold. The current design only supports up to 32 logical devices.

Input

handle	The pointer to the handle to the cuSolverMG context.
nbDevices	The number of logical devices.
deviceId	An integer array of size nbDevices.

Status Returned

CUSOLVER_STATUS_SUCCESS
The initialization succeeded.

CUSOLVER_STATUS_INVALID_VALUE
nbDevices must be greater than zero, and less or equal to 32.

CUSOLVER_STATUS_ALLOC_FAILED

The resources could not be allocated.

CUSOLVER_STATUS_INTERNAL_ERROR

Internal error occurred when setting internal streams and events.

9.3.4. cusolverMgCreateDeviceGrid() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t
cusolverMgCreateDeviceGrid(
    cusolverMgGrid_t* grid,
    int32_t numRowsDevices,
    int32_t numColDevices,
    const int32_t deviceId[],
    cusolverMgGridMapping_t mapping)
```

This function sets up a grid of devices.

Only 1-D column block cyclic is supported, so numRowsDevices must be equal to 1.

Warning: cusolverMgCreateDeviceGrid() must be consistent with cusolverMgDeviceSelect(), i.e. numColDevices must be equal to nbDevices in cusolverMgDeviceSelect().

Parameter	MemSpace	In/out	Meaning
grid	host	out-put	The pointer to the opaque structure.
numRowDe- vices	host	input	Number of devices in the row.
numColDe- vices	host	input	Number of devices in the column.
deviceId	host	input	Integer array of size numColDevices, containing device IDs.
mapping	host	input	Row-major or column-major ordering.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

numColDevices is not greater than 0. numRowsDevices is not 1.

9.3.5. cusolverMgDestroyGrid() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t
cusolverMgDestroyGrid(
    cusolverMgGrid_t grid)
```

This function releases resources of a grid.

Parameter	MemSpace	In/out	Meaning
grid	host	input/output	The pointer to the opaque structure.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

9.3.6. cusolverMgCreateMatrixDesc() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t
cusolverMgCreateMatrixDesc(
    cusolverMgMatrixDesc_t * desc,
    int64_t numRows,
    int64_t numCols,
    int64_t rowBlockSize,
    int64_t colBlockSize,
    cudaDataType_t dataType,
    const cusolverMgGrid_t grid)
```

This function sets up the matrix descriptor desc.

Only 1-D column block cyclic is supported, so numRows must be equal to rowBlockSize.

Parameter	Memory	In/out	Meaning
desc	host	output	The matrix descriptor.
numRows	host	input	The number of rows of global A.
numCols	host	input	The number of columns of global A.
rowBlockSize	host	input	The number of rows per tile.
colBlockSize	host	input	The number of columns per tile.
dataType	host	input	Data type of the matrix.
grid	host	input	The pointer to structure of grid.

Status Returned

CUSOLVER_STATUS_SUCCESS

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

numRows, numCols, or rowBlockSize or colBlockSize is less than 0. numRows is not equal to rowBlockSize.

9.3.7. cusolverMgDestroyMatrixDesc() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

```
cusolverStatus_t
cusolverMgDestroyMatrixDesc(
    cusolverMgMatrixDesc_t desc)
```

This function releases the matrix descriptor desc.

Parameter	Memory	In/out	Meaning
desc	host	input/output	The matrix descriptor.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

9.4. Dense Linear Solver Reference

This section describes the linear solver API of cuSolverMG.

9.4.1. cusolverMgPOTrf() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

The following helper function can calculate the sizes needed for pre-allocated buffer for `cusolverMgPOTrf`:

```
cusolverStatus_t
cusolverMgPOTrf_bufferSize(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descra,
```

(continues on next page)

(continued from previous page)

```

cudaDataType computeType,
int64_t *lwork)

```

The following routine:

```

cusolverStatus_t
cusolverMgPotrf(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    cudaDataType computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info)

```

computes the Cholesky factorization of a Hermitian positive-definite matrix using the generic API interface.

A is an $n \times n$ Hermitian matrix; only lower or upper part is meaningful. The input parameter `uplo` indicates which part of the matrix is used. The function would leave other parts untouched.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, only lower triangular part of A is processed, and replaced by lower triangular Cholesky factor L:

$$A = L * L^H$$

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, only upper triangular part of A is processed, and replaced by upper triangular Cholesky factor U:

$$A = U^H * U$$

The user has to provide device working space in `array_d_work`. `array_d_work` is a host pointer array of dimension G, where G is number of devices. `array_d_work[j]` is a device pointer pointing to a device memory in j-th device. The data type of `array_d_work[j]` is `computeType`. The size of `array_d_work[j]` is `lwork` which is the number of elements per device, returned by `cusolverMgPotrf_bufferSize()`.

If Cholesky factorization failed, i.e. some leading minor of A is not positive definite, or equivalently some diagonal elements of L or U is not a real number. The output parameter `info` would indicate smallest leading minor of A which is not positive definite.

If output parameter `info = -i` (less than zero), the i-th parameter is wrong (not counting handle).

The generic API has two different types, `dataTypeA` is data type of the matrix A, and `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA`, so there is no explicit parameter of `dataTypeA`. `cusolverMgPotrf` only supports the following four combinations.

Please visit [cuSOLVER Library Samples - MgPotrf](#) for a code example.

Valid combination of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SPOTRF
CUDA_R_64F	CUDA_R_64F	DPOTRF
CUDA_C_32F	CUDA_C_32F	CPOTRF
CUDA_C_64F	CUDA_C_64F	ZPOTRF

Table 1: **API of potrf**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMg library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced. Only CUBLAS_FILL_MODE_LOWER is supported.
N	host	input	Number of rows and columns of matrix sub(A).
array_d_A	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing sub(A) of dimension N * N. On exit, sub(A) contains the factors L or U.
IA	host	input	The row index in the global array A indicating the first row of sub(A).
JA	host	input	The column index in the global array A indicating the first column of sub(A).
descrA	host	input	Matrix descriptor for the distributed matrix A.
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. array_d_work[j] points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of array_d_work[j], returned by cusolverMgPotrf_bufferSize. lwork denotes number of elements, not number of bytes.

continues on next page

Table 1 – continued from previous page

Parameter	Memory	In/out	Meaning
info	host	output	If <code>info = 0</code>, the Cholesky factorization is successful. If <code>info = -i</code>, the <i>i</i>-th parameter is wrong (not counting handle). If <code>info = i</code>, the leading minor of order <i>i</i> is not positive definite.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($M, N < 0$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

9.4.2. `cusolverMgPotrs()` [DEPRECATED]

[[DEPRECATED]] `cuSolverMG` is deprecated. Users are encouraged to use `cuSOLVERMg` for multi-GPU functionality, both single- and multi-node.

The helper function below can calculate the sizes needed for pre-allocated buffer for `cusolverMgPotrs`.

```
cusolverStatus_t
cusolverMgPotrs_bufferSize(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    void *array_d_B[],
    int IB,
    int JB,
    cudaLibMgMatrixDesc_t descrB,
    cudaDataType computeType,
    int64_t *lwork )
```

The following routine:

```

cusolverStatus_t
cusolverMgPotrs(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int n,
    int nrhs,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    void *array_d_B[],
    int IB,
    int JB,
    cudaLibMgMatrixDesc_t descrB,
    cudaDataType computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info)

```

This function solves a system of linear equations:

$$A * X = B$$

where A is an $n \times n$ Hermitian matrix, only lower or upper part is meaningful using the generic API interface. The input parameter `uplo` indicates which part of the matrix is used. The function would leave other parts untouched.

If input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, the matrix should A contain the lower triangular factor for Cholesky decomposition previously computed by `cusolverMgPotrf` routine.

$$A = L * L^H$$

If input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, the matrix should A contain the upper triangular factor for Cholesky decomposition previously computed by the `cusolverMgPotrf` routine.

$$A = U^H * U$$

The operation is in-place, i.e. matrix B contains the solution of the linear system on exit.

If output parameter `info` = -i (less than zero), the i-th parameter is wrong (not counting handle).

The user has to provide device working space in `array_d_work`. `array_d_work` is a host pointer array of dimension G, where G is the number of devices. `array_d_work[j]` is a device pointer pointing to a device memory in the j-th device. The data type of `array_d_work[j]` is `computeType`. The size of `array_d_work[j]` is `lwork` which is number of elements per device, returned by `cusolverMgPotrs_bufferSize()`.

If output parameter `info` = -i (less than zero), the i-th parameter is wrong (not counting handle).

The generic API has four different types: `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix B, `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA` and `descrB` contains `dataTypeB` and so there is no explicit parameter of `dataTypeA` and `dataTypeB`. `cusolverMgPotrs` only supports the following four combinations.

Please visit [cuSOLVER Library Samples - MgPotrf](#) for a code example.

Valid combination of data type and compute type

DataTypeA	DataTypeB	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SPOTRS
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DPOTRS
CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CPOTRS
CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZPOTRS

Table 2: **API of potrs**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMg library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced. Only CUBLAS_FILL_MODE_LOWER is supported.
N	host	input	Number of rows and columns of matrix sub(A).
NRHS	host	input	Number of columns of matrix sub(A) and sub(B).
array_d_A	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing sub(A) of dimension M * N. On exit, sub(A) contains the factors L and U.
IA	host	input	The row index in the global array A indicating the first row of sub(A).
JA	host	input	The column index in the global array A indicating the first column of sub(A).
descrA	host	input	Matrix descriptor for the distributed matrix A.
array_d_B	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing sub(B) of dimension N * NRHS. On exit, sub(A) contains the solution to the linear system.
IB	host	input	The row index in the global array B indicating the first row of sub(B).
JB	host	input	The column index in the global array B indicating the first column of sub(B).
descrB	host	input	Matrix descriptor for the distributed matrix B.

continues on next page

Table 2 – continued from previous page

Parameter	Memory	In/out	Meaning
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. array_d_work[j] points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of array_d_work[j], returned by cusolverMgPotrs_bufferSize. lwork denotes number of elements, not number of bytes.
info	host	output	If info = 0, the routine successful. If info = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed (M, N < 0).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

9.4.3. cusolverMgPotri() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use cuSOLVERMp for multi-GPU functionality, both single- and multi-node.

The helper function below can calculate the sizes needed for pre-allocated buffer for cusolverMgPotri.

```
cusolverStatus_t
cusolverMgPotri_bufferSize(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    cudaDataType computeType,
    int64_t *lwork)
```

The following routine:

```

cusolverStatus_t
cusolverMgPotri(
    cusolverMgHandle_t handle,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    cudaDataType computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info)

```

This function computes the inverse of a Hermitian positive-definite matrix A using the Cholesky factorization

$$A = L * L^H = U^H * U$$

computed by `cusolverMgPotrf()`.

If the input parameter `uplo` is `CUBLAS_FILL_MODE_LOWER`, on input, matrix A contains the lower triangular factor of A computed by `cusolverMgPotrf`. Only lower triangular part of A is processed, and replaced the by lower triangular part of the inverse of A.

If the input parameter `uplo` is `CUBLAS_FILL_MODE_UPPER`, on input, matrix A contains the upper triangular factor of A computed by `cusolverMgPotrf`. Only upper triangular part of A is processed, and replaced the by upper triangular part of the inverse of A.

The user has to provide device working space in `array_d_work`. `array_d_work` is a host pointer array of dimension G, where G is number of devices. `array_d_work[j]` is a device pointer pointing to a device memory in the j-th device. The data type of `array_d_work[j]` is `computeType`. The size of `array_d_work[j]` is `lwork` which is number of elements per device, returned by `cusolverMgPotri_bufferSize()`.

If the computation of the inverse fails, i.e. some leading minor of L or U, is null, the output parameter `info` would indicate the smallest leading minor of L or U which is not positive definite.

If the output parameter `info = -i` (less than zero), the i-th parameter is wrong (not counting the handle).

The generic API has two different types, `dataTypeA` is data type of the matrix A, `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA`, so there is no explicit parameter of `dataTypeA`. `cusolverMgPotri` only supports the following four combinations.

Please visit [cuSOLVER Library Samples - MgPotrf](#) for a code example.

Valid combinations of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SPOTRI
CUDA_R_64F	CUDA_R_64F	DPOTRI
CUDA_C_32F	CUDA_C_32F	CPOTRI
CUDA_C_64F	CUDA_C_64F	ZPOTRI

Table 3: API of potri

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMg library context.
uplo	host	input	Indicates if matrix A lower or upper part is stored, the other part is not referenced. Only CUBLAS_FILL_MODE_LOWER is supported.
N	host	input	Number of rows and columns of matrix sub(A).
array_d_A	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing sub(A) of dimension N * N. On exit, sub(A) contains the upper or lower triangular part of the inverse of A depending on the value of uplo argument.
IA	host	input	The row index in the global array A indicating the first row of sub(A).
JA	host	input	The column index in the global array A indicating the first column of sub(A).
descrA	host	input	Matrix descriptor for the distributed matrix A.
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. array_d_work[j] points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of array_d_work[j], returned by cusolverMgPotri_bufferSize. lwork denotes number of elements, not number of bytes.
info	host	output	If info = 0, the Cholesky factorization is successful. If info = -i, the i-th parameter is wrong (not counting handle). If info = i, the leading minor of order i is zero.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_NOT_INITIALIZED

The library was not initialized.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($M, N < 0$).

CUSOLVER_STATUS_ARCH_MISMATCH

The device only supports compute capability 5.0 and above.

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

9.4.4. cusolverMgGetrf() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverMgGetrf_bufferSize(
    cusolverMgHandle_t handle,
    int M,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    int *array_d_IPIV[],
    cudaDataType_t computeType,
    int64_t *lwork);
```

```
cusolverStatus_t
cusolverMgGetrf(
    cusolverMgHandle_t handle,
    int M,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    int *array_d_IPIV[],
    cudaDataType_t computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info );
```

This function computes the LU factorization of an $M \times N$ matrix

$$P * A = L * U$$

where A is an $M \times N$ matrix, P is a permutation matrix, L is a lower triangular matrix with unit diagonal, and U is an upper triangular matrix.

The user has to provide device working space in array_d_work. array_d_work is a host pointer array of dimension G, where G is number of devices. array_d_work[j] is a device pointer pointing to a device memory in j-th device. The data type of array_d_work[j] is computeType. The size of array_d_work[j] is lwork which is number of elements per device, returned by cusolverMgGetrf_bufferSize().

If LU factorization failed, i.e. matrix A (U) is singular, The output parameter `info=i` indicates $U(i, i) = 0$.

If output parameter `info = -i` (less than zero), the *i*-th parameter is wrong (not counting handle).

If `array_d_IPIV` is null, no pivoting is performed. The factorization is $A=L*U$, which is not numerically stable.

`array_d_IPIV` must be consistent with `array_d_A`, i.e. JA is the first column of `sub(A)`, also the first column of `sub(IPIV)`.

No matter LU factorization failed or not, the output parameter `array_d_IPIV` contains pivoting sequence, row *i* is interchanged with row `array_d_IPIV(i)`.

The generic API has three different types, `dataTypeA` is data type of the matrix A, `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA`, so there is no explicit parameter of `dataTypeA`. `cusolverMgGetrf` only supports the following four combinations.

Please visit [cuSOLVER Library Samples - MgGetrf](#) for a code example.

Valid combinations of data type and compute type

DataTypeA	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	SGETRF
CUDA_R_64F	CUDA_R_64F	DGETRF
CUDA_C_32F	CUDA_C_32F	CGETRF
CUDA_C_64F	CUDA_C_64F	ZGETRF

Remark 1: tile size TA must be less or equal to 512.

Table 4: **API of getrf**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMg library context.
M	host	input	Number of rows of matrix <code>sub(A)</code> .
N	host	input	Number of columns of matrix <code>sub(A)</code> .
array_d_A	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing <code>sub(A)</code> of dimension M * N. On exit, <code>sub(A)</code> contains the factors L and U.
IA	host	input	The row index in the global array A indicating the first row of <code>sub(A)</code> .
JA	host	input	The column index in the global array A indicating the first column of <code>sub(A)</code> .
descrA	host	input	Matrix descriptor for the distributed matrix A.

continues on next page

Table 4 – continued from previous page

Parameter	Memory	In/out	Meaning
array_d_IPIV	host	output	A host pointer array of dimension G. it contains a distributed integer array containing sub(IPIV) of size $\min(M, N)$. sub(IPIV) contains pivot indices.
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. array_d_work[j] points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of array_d_work[j], returned by cusolverMgGetrf_bufferSize. lwork denotes number of elements, not number of bytes.
info	host	output	If info = 0, the LU factorization is successful. If info = -i, the i-th parameter is wrong (not counting handle). If info = i, the $U(i, i) = 0$.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($M, N < 0$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

9.4.5. cusolverMgGetrs() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use **cuSOLVERMp** for multi-GPU functionality, both single- and multi-node.

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverMgGetrs_bufferSize(
    cusolverMgHandle_t handle,
    cublasOperation_t TRANS,
    int N,
    int NRHS,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    int *array_d_IPIV[],
```

(continues on next page)

(continued from previous page)

```

    void *array_d_B[],
    int IB,
    int JB,
    cudaLibMgMatrixDesc_t descrB,
    cudaDataType_t computeType,
    int64_t *lwork);

cusolverStatus_t
cusolverMgGetrs(
    cusolverMgHandle_t handle,
    cublasOperation_t TRANS,
    int N,
    int NRHS,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    int *array_d_IPIV[],
    void *array_d_B[],
    int IB,
    int JB,
    cudaLibMgMatrixDesc_t descrB,
    cudaDataType_t computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info );

```

This function solves a linear system of multiple right-hand sides

$$op(A) * X = B$$

where A is an $N \times N$ matrix, and was LU-factored by `getrf`, that is, lower triangular part of A is L, and upper triangular part (including diagonal elements) of A is U. B is an $N \times NRHS$ right-hand side matrix. The solution matrix X overwrites the right-hand-side matrix B.

The input parameter TRANS is defined by

$$op(A) = \begin{cases} A & \text{if } trans == \text{CUBLAS_OP_N} \\ A^T & \text{if } trans == \text{CUBLAS_OP_T} \\ A^H & \text{if } trans == \text{CUBLAS_OP_C} \end{cases}$$

The user has to provide device working space in `array_d_work`. `array_d_work` is a host pointer array of dimension G, where G is number of devices. `array_d_work[j]` is a device pointer pointing to a device memory in j-th device. The data type of `array_d_work[j]` is `computeType`. The size of `array_d_work[j]` is `lwork` which is number of elements per device, returned by `cusolverMgGetrs_bufferSize()`.

If `array_d_IPIV` is null, no pivoting is performed. Otherwise, `array_d_IPIV` is an output of `getrf`. It contains pivot indices, which are used to permute right-hand sides.

If output parameter `info = -i` (less than zero), the i-th parameter is wrong (not counting handle).

The generic API has three different types, `dataTypeA` is data type of the matrix A, `dataTypeB` is data type of the matrix B, and `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA`, so there is no explicit parameter of

dataTypeA.descrB contains dataTypeB, so there is no explicit parameter of dataTypeB. cusolverMgGetrs only supports the following four combinations.

Valid combinations of data type and compute type

DataTypeA	DataTypeB	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SGETRS
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DGETRS
CUDA_C_32F	CUDA_C_32F	CUDA_C_32F	CGETRS
CUDA_C_64F	CUDA_C_64F	CUDA_C_64F	ZGETRS

Remark 1: tile size TA must be less or equal to 512.

Remark 2: only support TRANS=CUBLAS_OP_N.

Please visit [cuSOLVER Library Samples - MgGetrf](#) for a code example.

Table 5: **API of getrs**

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMG library context.
TRANS	host	input	Operation op(A) that is non- or (conj.) transpose.
N	host	input	Number of rows and columns of matrix sub(A).
NRHS	host	input	Number of columns of matrix sub(B).
array_d_A	host	input	A host pointer array of dimension G. It contains a distributed <type> array containing sub(A) of dimension M * N. sub(A) contains the factors L and U.
IA	host	input	The row index in the global array A indicating the first row of sub(A).
JA	host	input	The column index in the global array A indicating the first column of sub(A).
descrA	host	input	Matrix descriptor for the distributed matrix A.
array_d_IPIV	host	input	A host pointer array of dimension G. it contains a distributed integer array containing sub(IPIV) of dimension min(M,N). sub(IPIV) contains pivot indices.

continues on next page

Table 5 – continued from previous page

Parameter	Memory	In/out	Meaning
array_d_B	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing sub(B) of dimension N * NRHS.
IB	host	input	The row index in the global array B indicating the first row of sub(B).
JB	host	input	The column index in the global array B indicating the first column of sub(B).
descrB	host	input	Matrix descriptor for the distributed matrix B.
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. array_d_work[j] points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of array_d_work[j], returned by cusolverMgGetrs_bufferSize. lwork denotes number of elements, not number of bytes.
info	host	output	If info = 0, the operation is successful. If info = -i, the i-th parameter is wrong (not counting handle).

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($N < 0$ or $NRHS < 0$).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

9.5. Dense Eigenvalue Solver Reference

This section describes the eigenvalue solver API of cuSolverMG.

9.5.1. cusolverMgSyevd() [DEPRECATED]

[[DEPRECATED]] cuSolverMG is deprecated. Users are encouraged to use [cuSOLVERMp](#) for multi-GPU functionality, both single- and multi-node.

The helper functions below can calculate the sizes needed for pre-allocated buffer.

```
cusolverStatus_t
cusolverMgSyevd_bufferSize(
    cusolverMgHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    void *W,
    cudaDataType_t dataTypeW,
    cudaDataType_t computeType,
    int64_t *lwork
);
```

```
cusolverStatus_t
cusolverMgSyevd(
    cusolverMgHandle_t handle,
    cusolverEigMode_t jobz,
    cublasFillMode_t uplo,
    int N,
    void *array_d_A[],
    int IA,
    int JA,
    cudaLibMgMatrixDesc_t descrA,
    void *W,
    cudaDataType_t dataTypeW,
    cudaDataType_t computeType,
    void *array_d_work[],
    int64_t lwork,
    int *info );
```

This function computes eigenvalues and eigenvectors of a symmetric (Hermitian) $N \times N$ matrix A . The standard symmetric eigenvalue problem is:

$$A * V = V * \Lambda$$

where Λ is a real $N \times N$ diagonal matrix. V is an $N \times N$ unitary matrix. The diagonal elements of Λ are the eigenvalues of A in ascending order.

cusolverMgSyevd returns the eigenvalues in W and overwrites the eigenvectors in A . W is a host $1 \times N$ vector.

The generic API has three different types, `dataTypeA` is data type of the matrix A, `dataTypeW` is data type of the vector W, and `computeType` is compute type of the operation and data type of the workspace (`array_d_work`) `descrA` contains `dataTypeA`, so there is no explicit parameter of `dataTypeA`. `cusolverMgSyevd` only supports the following four combinations.

Valid combination of data type and compute type

DataTypeA	DataTypeW	ComputeType	Meaning
CUDA_R_32F	CUDA_R_32F	CUDA_R_32F	SSYEVD
CUDA_R_64F	CUDA_R_64F	CUDA_R_64F	DSYEVD
CUDA_C_32F	CUDA_R_32F	CUDA_C_32F	CHEEVD
CUDA_C_64F	CUDA_R_64F	CUDA_C_64F	ZHEEVD

The user has to provide device working space in `array_d_work`. `array_d_work` is a host pointer array of dimension G, where G is number of devices. `array_d_work[j]` is a device pointer pointing to a device memory in j-th device. The data type of `array_d_work[j]` is `computeType`. The size of `array_d_work[j]` is `lwork` which is number of elements per device, returned by `cusolverMgSyevd_bufferSize()`.

`array_d_A` is also a host pointer array of dimension G. `array_d_A[j]` is a device pointer pointing to a device memory in j-th device. The data type of `array_d_A[j]` is `dataTypeA`. The size of `array_d_A[j]` is about $N \times TA \times (\text{blocks per device})$. The user has to prepare `array_d_A` manually (see [cuSOLVER Library Samples - MgSyevd](#) for a code example.).

If output parameter `info = -i` (less than zero), the i-th parameter is wrong (not counting handle). If `info = i` (greater than zero), the algorithm failed to converge all eigenvalues.

If `jobz = CUSOLVER_EIG_MODE_VECTOR`, A contains the orthonormal eigenvectors of the matrix A. The eigenvectors are computed by a divide and conquer algorithm.

Remark 1: only `CUBLAS_FILL_MODE_LOWER` is supported, so the user has to prepare lower triangle of A.

Remark 2: only `IA=1` and `JA=1` are supported.

Remark 3: tile size TA must be less or equal to 1024. To achieve best performance, TA should be 256 or 512.

Please visit [cuSOLVER Library Samples - MgSyevd](#) for a code example.

Table 6: API of `syevd`

Parameter	Memory	In/out	Meaning
handle	host	input	Handle to the cuSolverMG library context.

continues on next page

Table 6 – continued from previous page

Parameter	Memory	In/out	Meaning
jobz	host	input	Specifies options to either compute eigenvalue only or compute eigen-pair: <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> : Compute eigenvalues only <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> : Compute eigenvalues and eigenvectors
uplo	host	input	Specifies which part of A is stored. <code>uplo = CUBLAS_FILL_MODE_LOWER</code> : Lower triangle of A is stored. <code>uplo = CUBLAS_FILL_MODE_UPPER</code> : Upper triangle of A is stored. Only <code>CUBLAS_FILL_MODE_LOWER</code> is supported.
N	host	input	Number of rows (or columns) of matrix <code>sub(A)</code> .
array_d_A	host	in/out	A host pointer array of dimension G. It contains a distributed <type> array containing <code>sub(A)</code> of dimension $N * N$. If <code>uplo = CUBLAS_FILL_MODE_UPPER</code> , the leading N-by-N upper triangular part of <code>sub(A)</code> contains the upper triangular part of the matrix <code>sub(A)</code> . If <code>uplo = CUBLAS_FILL_MODE_LOWER</code> , the leading N-by-N lower triangular part of <code>sub(A)</code> contains the lower triangular part of the matrix <code>sub(A)</code> . On exit, if <code>jobz = CUSOLVER_EIG_MODE_VECTOR</code> , and <code>info = 0</code> , <code>sub(A)</code> contains the orthonormal eigenvectors of the matrix <code>sub(A)</code> . If <code>jobz = CUSOLVER_EIG_MODE_NOVECTOR</code> , the contents of A are destroyed.
IA	host	input	The row index in the global array A indicating the first row of <code>sub(A)</code> .
JA	host	input	The column index in the global array A indicating the first column of <code>sub(A)</code> .
descrA	host	input	Matrix descriptor for the distributed matrix A.

continues on next page

Table 6 – continued from previous page

Parameter	Memory	In/out	Meaning
W	host	output	A real array of dimension N. The eigenvalue values of $\text{sub}(A)$, in ascending order, that is, sorted so that $W(i) \leq W(i+1)$.
dataTypeW	host	input	Data type of the vector W.
computeType	host	input	Data type used for computation.
array_d_work	host	in/out	A host pointer array of dimension G. <code>array_d_work[j]</code> points to a device working space in j-th device, <type> array of size lwork.
lwork	host	input	Size of <code>array_d_work[j]</code> , returned by <code>cusolverMgSyevd_bufferSize</code> . lwork denotes number of elements, not number of bytes.
info	host	output	If <code>info = 0</code> , the operation is successful. If <code>info = -i</code> , the i-th parameter is wrong (not counting handle). If <code>info = i (> 0)</code> , not all eigenvalues converged.

Status Returned**CUSOLVER_STATUS_SUCCESS**

The operation completed successfully.

CUSOLVER_STATUS_INVALID_VALUE

Invalid parameters were passed ($N < 0$, or $\text{lda} < \max(1, N)$, or `jobz` is not `CUSOLVER_EIG_MODE_NOVECTOR` or `CUSOLVER_EIG_MODE_VECTOR`, or `uplo` is not `CUBLAS_FILL_MODE_LOWER`, or `IA` and `JA` are not 1, or N is bigger than dimension of global A , or the combination of `dataType` and `computeType` is not valid).

CUSOLVER_STATUS_INTERNAL_ERROR

An internal operation failed.

Chapter 10. Acknowledgements

NVIDIA would like to thank the following individuals and institutions for their contributions:

- ▶ CPU LAPACK routines from netlib, CLAPACK-3.2.1 (<http://www.netlib.org/clapack/>)

The following is license of CLAPACK-3.2.1.

Copyright (c) 1992-2008 The University of Tennessee. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- ▶ Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- ▶ Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- ▶ Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- ▶ METIS-5.1.0 (<http://glaros.dtc.umn.edu/gkhome/metis/metis/overview>)

The following is license of METIS (Apache 2.0 license).

Copyright 1995-2013, Regents of the University of Minnesota

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

- ▶ QD (A C++/fortran-90 double-double and quad-double package) (<http://crd-legacy.lbl.gov/~dhbailey/mpdist/>)

The following is license of QD (modified BSD license).

Copyright (c) 2003-2009, The Regents of the University of California, through Lawrence Berkeley National Laboratory (subject to receipt of any required approvals from U.S. Dept. of Energy) All rights reserved.

1. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
 - (1) Redistributions of source code must retain the copyright notice, this list of conditions and the following disclaimer.
 - (2) Redistributions in binary form must reproduce the copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
 - (3) Neither the name of the University of California, Lawrence Berkeley National Laboratory, U.S. Dept. of Energy nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.
2. THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
3. You are under no obligation whatsoever to provide any bug fixes, patches, or upgrades to the features, functionality or performance of the source code ("Enhancements") to anyone; however, if you choose to make your Enhancements available either publicly, or directly to Lawrence Berkeley National Laboratory, without imposing a separate written license agreement for such Enhancements, then you hereby grant the following license: a non-exclusive, royalty-free perpetual license to install, use, modify, prepare derivative works, incorporate into other computer software, distribute, and sublicense such enhancements or derivative works thereof, in binary and source code form.

Chapter 11. Bibliography

- [1] Timothy A. Davis, *Direct Methods for Sparse Linear Systems*, SIAM, 2006.
- [2] E. Chuthill and J. McKee, reducing the bandwidth of sparse symmetric matrices, *ACM '69 Proceedings of the 1969 24th national conference*, Pages 157-172.
- [3] Alan George, Joseph W. H. Liu, An Implementation of a Pseudoperipheral Node Finder, *ACM Transactions on Mathematical Software (TOMS) Volume 5 Issue 3*, Sept. 1979 Pages 284-295.
- [4] J. R. Gilbert and T. Peierls, Sparse partial pivoting in time proportional to arithmetic operations, *SIAM J. Sci. Statist. Comput.*, 9 (1988), pp. 862-874.
- [5] Alan George and Esmond Ng, An Implementation of Gaussian Elimination with Partial Pivoting for Sparse Systems, *SIAM J. Sci. and Stat. Comput.*, 6(2), 390-409.
- [6] Alan George and Esmond Ng, Symbolic Factorization for Sparse Gaussian Elimination with Partial Pivoting, *SIAM J. Sci. and Stat. Comput.*, 8(6), 877-898.
- [7] John R. Gilbert, Xiaoye S. Li, Esmond G. Ng, Barry W. Peyton, Computing Row and Column Counts for Sparse QR and LU Factorization, *BIT* 2001, Vol. 41, No. 4, pp. 693-711.
- [8] Patrick R. Amestoy, Timothy A. Davis, Iain S. Duff, An Approximate Minimum Degree Ordering Algorithm, *SIAM J. Matrix Analysis Applic.* Vol 17, no 4, pp. 886-905, Dec. 1996.
- [9] Alan George, Joseph W. Liu, A Fast Implementation of the Minimum Degree Algorithm Using Quotient Graphs, *ACM Transactions on Mathematical Software*, Vol 6, No. 3, September 1980, page 337-358.
- [10] Alan George, Joseph W. Liu, *Computer Solution of Large Sparse Positive Definite Systems*, Englewood Cliffs, New Jersey: Prentice-Hall, 1981.
- [11] Iain S. Duff, ALGORITHM 575 Permutations for a Zero-Free Diagonal, *ACM Transactions on Mathematical Software*, Vol 7, No 3, September 1981, Page 387-390
- [12] Iain S. Duff and Jacko Koster, On algorithms for permuting large entries to the diagonal of a sparse matrix, *SIAM Journal on Matrix Analysis and Applications*, 2001, Vol. 22, No. 4 : pp. 973-996
- [13] George Karypis and Vipin Kumar, A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs, *SIAM Journal on Scientific Computing*, Vol. 20, No. 1, pp. 359-392, 1999.
- [14] Yuji Nakatsukasa, Zhaojun Bai, and François Gygi, Optimizing Halley's Iteration for Computing the Matrix Polar Decomposition, *SIAM J. Matrix Anal. Appl.*, 31 (5): 2700-2720, 2010.
- [15] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp, Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions, *SIAM Review*, Vol. 53, No. 2, pp. 217-288, 2011.
- [16] Yuji Nakatsukasa and Nicholas J. Higham, Stable and Efficient Spectral Divide and Conquer Algorithms for the Symmetric Eigenvalue Decomposition and the SVD, *SIAM J. Sci. Comput.*, 35 (3): A1325-A1349, 2013

Chapter 12. Notices

12.1. Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation (“NVIDIA”) makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgement, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer (“Terms of Sale”). NVIDIA hereby expressly objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer’s own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer’s sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer’s product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

No license, either expressed or implied, is granted under any NVIDIA patent right, copyright, or other NVIDIA intellectual property right under this document. Information published by NVIDIA regarding third-party products or services does not constitute a license from NVIDIA to use such products or

services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property rights of the third party, or a license from NVIDIA under the patents or other intellectual property rights of NVIDIA.

Reproduction of information in this document is permissible only if approved in advance by NVIDIA in writing, reproduced without alteration and in full compliance with all applicable export laws and regulations, and accompanied by all associated conditions, limitations, and notices.

THIS DOCUMENT AND ALL NVIDIA DESIGN SPECIFICATIONS, REFERENCE BOARDS, FILES, DRAWINGS, DIAGNOSTICS, LISTS, AND OTHER DOCUMENTS (TOGETHER AND SEPARATELY, "MATERIALS") ARE BEING PROVIDED "AS IS." NVIDIA MAKES NO WARRANTIES, EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE MATERIALS, AND EXPRESSLY DISCLAIMS ALL IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE. TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL NVIDIA BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF NVIDIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. Notwithstanding any damages that customer might incur for any reason whatsoever, NVIDIA's aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms of Sale for the product.

12.2. OpenCL

OpenCL is a trademark of Apple Inc. used under license to the Khronos Group Inc.

12.3. Trademarks

NVIDIA and the NVIDIA logo are trademarks or registered trademarks of NVIDIA Corporation in the U.S. and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

©2014-2026, NVIDIA Corporation & affiliates. All rights reserved